

组合优化与凸优化

课程完整笔记

哈尔滨工业大学 · 计算学部

课程编号: CS64001 · 2026 年春季

2026 年 5 月 27 日

目录

1	绪论	12
1.1	运筹学与最优化概述	12
1.1.1	运筹学的定义	12
1.1.2	运筹学简史	13
1.1.3	学科分支	13
1.1.4	最优化问题的一般形式	13
1.1.5	优化问题的分类	14
1.1.6	运筹学方法论	15
1.2	经典极值问题	15
1.2.1	无约束极值问题	15
1.2.2	等式约束极值问题	16
1.3	计算机学科中的典型最优化问题	18
1.3.1	深度学习中的优化	18
1.3.2	优化求解的质量度量	19
1.3.3	其他计算机领域优化问题	20
1.4	最优化的基本数学概念	21
1.4.1	凸集	21
1.4.2	仿射函数	22
1.4.3	凸函数与凹函数	23
1.4.4	凸性与最优化的关系	24
1.5	本章小结	27
1.5.1	本章知识框架	27
1.5.2	关键公式汇总	28
1.5.3	关键术语中英对照	28
2	线性规划	30
2.1	线性规划概述	30
2.1.1	线性规划模型	30
2.1.2	图解法	31

2.1.3	线性规划基本定理	32
2.2	基本可行解与极点	33
2.2.1	基与基本解	33
2.2.2	单纯形表的结构	34
2.2.3	单纯形法的迭代公式	35
2.3	单纯形法	35
2.3.1	单纯形法的基本原理	35
2.3.2	单纯形法算法步骤	36
2.3.3	例题：单纯形法迭代两次	36
2.3.4	退化与循环	37
2.3.5	Python 代码实现	38
2.4	大 M 法	41
2.4.1	大 M 法的基本思想	41
2.4.2	大 M 法的 Python 实现	43
2.5	两阶段法	46
2.5.1	两阶段法的基本思想	46
2.5.2	第一阶段：寻找初始 BFS	47
2.5.3	第二阶段：求解原问题	48
2.5.4	两阶段法的 Python 实现	49
2.6	对偶理论	53
2.6.1	对偶问题的构造	53
2.6.2	弱对偶定理与强对偶定理	54
2.6.3	互补松弛定理	54
2.6.4	经济解释：影子价格	55
2.7	对偶单纯形法	56
2.7.1	对偶单纯形法的基本思想	56
2.7.2	对偶单纯形法算法	57
2.7.3	例题：对偶单纯形法迭代两次	57
2.7.4	Python 代码	59
2.8	灵敏度分析	61
2.8.1	灵敏度分析概述	61
2.8.2	目标函数系数的变化	61
2.8.3	右端项的变化	62
2.8.4	增加新变量	62
2.8.5	增加新约束	63
2.8.6	Python 代码	63
2.9	本章小结	65

2.9.1	线性规划方法体系	65
2.9.2	重要定理回顾	65
2.9.3	单纯形表操作速查	66
2.9.4	对偶问题构造速查	66
2.9.5	学习建议	66
3	凸分析与数学模型	67
3.1	非线性规划的数学模型	67
3.1.1	选址问题	67
3.1.2	构件的表面积问题	68
3.1.3	木梁设计问题	69
3.1.4	Python 代码: 建模与可视化	70
3.2	非线性规划的一般数学模型	72
3.2.1	标准形式	72
3.2.2	可行域与可行解	72
3.2.3	问题分类	72
3.2.4	等价形式	72
3.2.5	凸规划问题	73
3.3	非线性规划的直观理解	74
3.3.1	等值线与图解法	74
3.3.2	局部最优解与全局最优解	75
3.3.3	一阶与二阶近似	76
3.4	无约束问题的最优性条件	78
3.4.1	梯度与 Hessian 矩阵	78
3.4.2	Taylor 展开	79
3.4.3	一阶必要条件	79
3.4.4	二阶必要条件	79
3.4.5	二阶充分条件	80
3.4.6	利用最优性条件求解问题	80
3.5	凸无约束问题的最优性条件	83
3.5.1	次梯度与次微分	83
3.5.2	凸函数的一阶充要条件	84
3.5.3	凸函数的二阶充要条件	85
3.5.4	正定二次函数的性质	85
3.5.5	凸函数的水平集	85
3.5.6	凸规划的全局最优性	85
3.6	解非线性规划问题的基本思路	87
3.6.1	迭代方法的基本框架	87

3.6.2	下降方向	88
3.6.3	步长因子的确定——一维搜索	88
3.6.4	终止条件	88
3.6.5	收敛速度	89
3.6.6	共轭方向法	89
3.6.7	最优化算法分类概述	90
3.7	有约束问题的最优性条件简介	92
3.8	本章小结	92
3.8.1	本章知识框架	92
3.8.2	关键公式汇总	93
3.8.3	Python 综合代码：求解无约束优化	93
3.8.4	进一步阅读	95
3.8.5	关键概念速查	96
4	一维搜索与线搜索	97
4.1	一维搜索概述	97
4.1.1	问题背景	97
4.1.2	分类	97
4.1.3	基本原则	98
4.1.4	单峰函数	98
4.2	几个常见的测试函数	98
4.2.1	单峰函数	98
4.2.2	多峰函数	99
4.3	不利用导数的精确一维搜索	100
4.3.1	Dichotomous 搜索（二分搜索）	100
4.4	黄金分割法与 Fibonacci 法	101
4.4.1	黄金分割法（0.618 法）	101
4.4.2	Fibonacci 法	103
4.5	中点法（二分法）	108
4.5.1	算法原理	108
4.6	Newton 法与插值法	111
4.6.1	Newton 法	111
4.6.2	插值法概述	114
4.6.3	二次插值法	114
4.6.4	三次插值法	115
4.7	Shubert-Piyavskii 方法	118
4.7.1	基本原理	118
4.7.2	下界函数的构造	118

4.8	不精确一维搜索	121
4.8.1	Goldstein 条件	122
4.8.2	Armijo 条件 (回溯线搜索)	122
4.8.3	Wolfe-Powell 条件	123
4.8.4	方法对比	124
4.9	一维搜索方法效率比较	128
4.10	本章小结	128
4.10.1	核心思想	128
4.10.2	方法决策树	129
4.10.3	本章知识框架	129
4.10.4	关键公式汇总	130
4.10.5	Python 综合示例	130
4.10.6	进一步阅读	134
4.10.7	关键术语中英对照	134
5	无约束最优化方法	135
5.1	引言	135
5.1.1	为什么研究无约束最优化	135
5.1.2	问题描述	135
5.1.3	方法分类	135
5.1.4	收敛速度记号	136
5.2	多维无导数搜索方法	136
5.2.1	坐标轮换法 (Cyclic Coordinate Method)	137
5.2.2	Hooke-Jeeves 方法	138
5.2.3	Rosenbrock 方法	139
5.2.4	Python 代码实现	140
5.2.5	方法比较	143
5.3	最速下降法 (梯度法)	143
5.3.1	基本思想	143
5.3.2	最速下降法算法	144
5.3.3	步长选择策略	144
5.3.4	梯度法的收敛性分析	145
5.3.5	梯度法的局限性	146
5.3.6	梯度法的 Python 实现	146
5.3.7	批量与小批量梯度下降	148
5.4	牛顿法	149
5.4.1	基本思想	149
5.4.2	牛顿法算法	150

5.4.3	牛顿法的收敛性分析	151
5.4.4	牛顿法的优缺点	151
5.4.5	牛顿法的改进策略	151
5.4.6	牛顿法的 Python 实现	152
5.5	牛顿方法的变种: LM 与信任域方法	156
5.5.1	Levenberg-Marquardt (LM) 方法	156
5.5.2	信任域方法 (Trust Region Methods)	157
5.5.3	信任域方法的 Python 实现	158
5.5.4	LM 与信任域方法的比较	162
5.6	共轭方向法	162
5.6.1	共轭方向的概念	162
5.6.2	共轭梯度法 (CG)	162
5.6.3	拟牛顿法	165
5.6.4	共轭梯度法与拟牛顿法的比较	166
5.6.5	Python 代码实现	167
5.7	次梯度优化方法	170
5.7.1	背景与动机	170
5.7.2	次梯度与次微分	171
5.7.3	常见函数的次微分	171
5.7.4	次梯度方法	172
5.7.5	近端梯度方法 (Proximal Gradient)	173
5.7.6	加速近端梯度法 (FISTA)	173
5.7.7	次梯度方法的 Python 实现	174
5.7.8	复杂度下界与加速方法	177
5.8	本章小结	178
5.8.1	核心思想	178
5.8.2	方法体系总览	178
5.8.3	方法选择决策树	178
5.8.4	关键概念回顾	179
5.8.5	收敛速度总结	180
5.8.6	关键公式汇总	180
5.8.7	学习建议	181
5.8.8	关键术语中英对照	181
6	约束最优化理论	182
6.1	约束最优化问题概述	182
6.1.1	问题描述	182
6.1.2	可行方向	182

6.1.3	切锥与线性化可行方向	183
6.1.4	Farkas 引理	183
6.1.5	无约束问题最优性条件回顾	184
6.2	一阶最优性条件 (KKT 条件)	184
6.2.1	等式约束问题	184
6.2.2	一般约束问题的 KKT 条件	185
6.2.3	约束规范 (Constraint Qualifications)	187
6.2.4	凸优化的 KKT 充分性	187
6.2.5	拉格朗日函数与鞍点	188
6.3	二阶最优性条件	188
6.3.1	临界锥 (Critical Cone)	188
6.3.2	二阶必要条件	188
6.3.3	二阶充分条件	189
6.3.4	例子: 区分不同 KKT 点	189
6.4	对偶理论	190
6.4.1	拉格朗日对偶问题	190
6.4.2	弱对偶定理	191
6.4.3	强对偶定理	191
6.4.4	鞍点定理	192
6.4.5	扰动函数与灵敏度分析	192
6.5	惩罚函数法	193
6.5.1	二次惩罚函数法	193
6.5.2	精确惩罚函数法	194
6.5.3	增广拉格朗日法	195
6.5.4	Python 代码实现	196
6.6	障碍函数法与内点法	199
6.6.1	对数障碍函数法	199
6.6.2	中心路径	201
6.6.3	原始-对偶内点法	201
6.6.4	线性规划的内点法	201
6.6.5	Python 代码实现	202
6.7	线性规划对偶与 Fenchel 对偶	206
6.7.1	线性规划的对偶	206
6.7.2	LP 对偶的基本定理	206
6.7.3	共轭函数	207
6.7.4	Fenchel 对偶	207
6.7.5	正则化问题的对偶	208

6.8	本章小结	209
6.8.1	方法体系总览	209
6.8.2	KKT 条件速查	209
6.8.3	数值方法选择指南	210
6.8.4	关键定理回顾	210
6.8.5	收敛性比较	211
7	约束最优化方法	212
7.1	非线性最小二乘问题	212
7.1.1	问题描述	212
7.1.2	梯度和 Hessian 矩阵	212
7.1.3	高斯-牛顿法 (Gauss-Newton)	212
7.1.4	Levenberg-Marquardt 方法	213
7.2	最优性条件回顾	216
7.2.1	无约束问题	216
7.2.2	有约束问题——基本概念	216
7.2.3	等式约束问题	216
7.3	KKT 条件回顾	217
7.4	可行方向法	217
7.4.1	基本思想	217
7.4.2	线性约束情况	218
7.4.3	非线性约束情况	218
7.4.4	计算步骤	218
7.5	制约函数法 (罚函数法与内点法)	222
7.5.1	外点罚函数法	222
7.5.2	内点法 (障碍函数法)	223
7.6	增广拉格朗日函数法与乘子法	227
7.6.1	罚函数的困难	227
7.6.2	增广拉格朗日函数法	227
7.6.3	不等式约束的处理	228
7.7	线性规划的内点法	228
7.7.1	解析中心	228
7.7.2	中心路径	229
7.7.3	KKT 条件与原始-对偶系统	229
7.7.4	Newton 步的求解	229
7.8	ADMM 与邻近算法简介	232
7.8.1	ADMM 方法	232
7.8.2	邻近算子与邻近算法	233

7.9	本章小结	235
7.9.1	本章知识框架	235
7.9.2	方法特性对比	236
7.9.3	关键公式汇总	236
7.9.4	关键术语中英对照	236
8	进阶专题	239
8.1	复合优化问题	239
8.1.1	问题形式	239
8.1.2	求解方法	239
8.2	变分不等式与邻近点算法	241
8.2.1	单调变分不等式	241
8.2.2	线性约束凸优化的变分不等式	241
8.2.3	可分离结构	242
8.2.4	邻近点算法 (PPA)	242
8.3	启发式算法概述	243
8.3.1	优化方法分类	243
8.3.2	启发式算法 vs 近似算法	243
8.3.3	组合优化问题	244
8.3.4	独立系统与拟阵	244
8.3.5	启发式算法的特点	245
8.3.6	智能优化算法一览	245
8.4	模拟退火算法	245
8.4.1	物理背景	245
8.4.2	Metropolis 准则	246
8.4.3	算法流程	247
8.4.4	关键参数	247
8.4.5	玻尔兹曼机	248
8.5	遗传算法	250
8.5.1	生物启发	250
8.5.2	编码方式	250
8.5.3	基本遗传算法	251
8.5.4	遗传算子	251
8.6	粒子群优化与差分进化	254
8.6.1	粒子群优化算法 (PSO)	254
8.6.2	差分进化算法 (DE)	255
8.7	其他群智能算法	259
8.7.1	人工蜂群算法 (ABC)	259

8.7.2	蚁群优化算法 (ACO)	259
8.7.3	灰狼算法 (GWO)	259
8.7.4	鲸鱼算法 (WOA)	260
8.7.5	烟花算法 (FWA)	260
8.8	应用实例	263
8.8.1	试飞任务规划	263
8.8.2	柔性作业车间调度	263
8.8.3	旅行商问题 (TSP)	264
8.8.4	深度学习的优化	264
8.9	支持向量机 (SVM)	266
8.9.1	线性可分与硬间隔 SVM	266
8.9.2	拉格朗日对偶与支持向量	266
8.9.3	软间隔 SVM	267
8.9.4	核方法	267
8.10	本章小结	268
8.10.1	本章知识框架	268
8.10.2	智能优化算法特性对比	269
8.10.3	选择算法的建议	269
8.10.4	关键公式汇总	270
8.10.5	关键术语中英对照	270
9	课程总结与展望	274
9.1	优化问题的统一框架	274
9.2	核心知识体系	275
9.2.1	一维搜索与线搜索方法	275
9.2.2	凸性: 优化的基石	275
9.2.3	最优性条件体系	276
9.2.4	线性规划: 经典与对偶	276
9.2.5	无约束优化方法	277
9.2.6	约束优化方法	277
9.2.7	非光滑与复合优化	278
9.2.8	智能优化算法	278
9.3	收敛性与复杂度	278
9.4	方法选择决策树	279
9.5	课程核心公式速查	280
9.6	本章小结	280
9.6.1	核心知识框架	280
9.6.2	方法选择总诀	281

9.7 研究前沿与展望	281
-----------------------	-----

Chapter 1

绪论

1.1 运筹学与最优化概述

1.1.1 运筹学的定义

运筹学的各种定义

- **大英百科全书：**”运筹学是一门应用于管理有组织系统的科学”，”为掌管这类系统的人提供决策目标和数量分析的工具”。
- **中国大百科全书：**”用数学方法研究经济、民政和国防等部门在内外环境的约束条件下合理分配人力、物力、财力等资源，使实际系统有效运行的技术科学”。
- **辞海：**”主要研究经济活动与军事活动中能用数量来表达有关运用、筹划与管理方面的问题”。

性质 1.1.1 (运筹学的核心特征). 运筹学所研究的，通常是在必须分配稀缺资源的条件下，科学地决定如何最佳地设计和运营人机系统：

- **对象：**人机系统
- **条件：**资源稀缺
- **方法：**模型化、定量化
- **特点：**最优化
- **目的：**决策支持

1.1.2 运筹学简史

历史脉络

1. 起源：古代战争、娱乐、建设
 - 田忌赛马
 - 丁渭修皇宫（北宋真宗时期）
2. 学科产生：第二次世界大战
 - 1938 年 7 月，波得塞雷达站罗伊用 Operational Research 命名防空作战系统研究
 - 1940 年 9 月英国成立第一个运筹学小组（布莱克特领导）
 - 1942 年美国 and 加拿大相继成立运筹学小组
3. 扩展：战后用于民用事业
4. 成型：各个分支成熟
5. 成熟：计算机、信息技术结合
6. 发展：学科结合、渗透

1.1.3 学科分支

运筹学主要分支

- 规划理论：线性规划、非线性规划、运输问题、整数规划、动态规划、目标规划
- 图与网络理论
- 排队论、存储论、决策论
- 对策论（博弈论）、冲突分析
- 搜索论、可靠性理论
- 计划协调技术（PERT）、图解协调技术

1.1.4 最优化问题的一般形式

定义 1.1.1 (最优化问题).

$$\min_{x \in \mathbb{R}^n} f(x) \quad s.t. \quad c_i(x) \geq 0, \quad i \in \mathcal{I}; \quad c_i(x) = 0, \quad i \in \mathcal{E}$$

- 第一：无约束极值问题 —— $\min_{x \in \mathbb{R}^n} f(x)$

- 第二：具有约束的极值问题 ——带等式和不等式约束

1.1.5 优化问题的分类

根据不同的标准，最优化问题可从多个维度进行分类。

优化问题的分类维度
(1) 连续优化与离散优化 • 连续优化：决策变量在连续空间中取值，即 $x \in \mathbb{R}^n$ • 离散优化：决策变量取离散值（如整数），典型问题如背包问题、旅行商问题 • 连续优化相对容易，有梯度、导数等数学工具可用 (2) 约束优化与无约束优化 • 无约束优化：$\min_{x \in \mathbb{R}^n} f(x)$，没有任何限制条件 • 约束优化：带有等式和/或不等式约束 • 注意：有界约束 ($l \leq x \leq u$) 在算法中常作特殊处理，本质上也是约束优化 (3) 确定性优化与随机性优化 • 确定性优化：目标函数、约束条件和参数都是确定已知的 • 随机性优化：问题中存在不确定因素，需用概率统计方法建模 • 例如：投资组合优化中，未来收益率是不确定的，需考虑期望收益和风险 (4) 线性规划与非线性规划 • 线性规划（见第 2 章）：目标函数和约束条件均为线性（仿射）函数 • 非线性规划：目标函数或约束条件中至少有一个是非线性的 • 非线性规划求解难度远大于线性规划，且通常只能找到局部最优解 (5) 单目标优化与多目标优化 • 单目标优化：只有一个目标函数 • 多目标优化：同时优化多个目标，各目标之间可能存在冲突 • 多目标优化通常不存在单一最优解，而是寻找 Pareto 最优解集 (6) 静态优化与动态优化

- **静态优化**：问题不随时间变化
 - **动态优化**：问题随时间演变，需要在不同时间点做出决策（如动态规划）
- (7) 单层优化与多层优化
- **单层优化**：只有一个决策层
 - **多层优化**：存在层次化的决策结构，上层决策影响下层的可行域和目标
 - 例如：Stackelberg 博弈中，领导者先决策，跟随者再优化

分类总结		
分类标准	类别	典型方法
变量类型	连续 / 离散	梯度法 / 分支定界
约束条件	无约束 / 约束	拟牛顿法 / 拉格朗日法
参数性质	确定性 / 随机性	精确算法 / 随机规划
函数类型	线性 / 非线性	单纯形法 / 梯度下降
目标个数	单目标 / 多目标	标量化 / Pareto 方法
时间因素	静态 / 动态	数学规划 / 动态规划
决策层次	单层 / 多层	标准优化 / 双层规划

1.1.6 运筹学方法论

运筹学研究步骤
1. 提出问题 ：确定目标，明确约束，抓主要矛盾
2. 建立模型 ：选择模型、设定变量、描述约束和目标
3. 优化求解 ：选择求解方法、求解问题
4. 评价分析 ：灵敏度分析、评价
5. 决策支持 ：汇总、解释结果、报告

1.2 经典极值问题

1.2.1 无约束极值问题

例题 1.2.1 (正方形剪角问题). **问题：**对边长为 a 的正方形铁板，在四个角处剪去相等的正方形以制成无盖水槽，问如何剪法使水槽的容积最大？

解：设剪去的小正方形边长为 x ，则水槽容积：

$$V(x) = x(a - 2x)^2 = x(a^2 - 4ax + 4x^2) = a^2x - 4ax^2 + 4x^3, \quad 0 < x < \frac{a}{2}$$

求导：

$$\begin{aligned} V'(x) &= a^2 - 8ax + 12x^2 = 0 \\ x &= \frac{8a \pm \sqrt{64a^2 - 48a^2}}{24} = \frac{8a \pm 4a}{24} \end{aligned}$$

解得 $x_1 = \frac{a}{6}$, $x_2 = \frac{a}{2}$ (舍去, 边界点)。

二阶验证：

$$V''(x) = -8a + 24x, \quad V''\left(\frac{a}{6}\right) = -8a + 4a = -4a < 0$$

故 $x^* = \frac{a}{6}$ 是极大值点，最大容积：

$$V^* = \frac{a}{6} \left(a - \frac{a}{3}\right)^2 = \frac{a}{6} \cdot \frac{4a^2}{9} = \frac{2a^3}{27}$$

1.2.2 等式约束极值问题

例题 1.2.2 (球的表面积最小化问题). **问题：**半径为 1 的实心金属球融化后，铸成一个实心圆柱体，问圆柱体取什么尺寸才能使它的表面积最小？

解：设圆柱体底面半径为 r ，高为 h 。

约束条件 (体积守恒)：

$$\pi r^2 h = \frac{4}{3} \pi \cdot 1^3 = \frac{4\pi}{3}$$

目标函数 (表面积最小)：

$$\min S = 2\pi r^2 + 2\pi r h$$

方法 1: 消元法

由约束得 $h = \frac{4}{3r^2}$ ，代入目标函数：

$$S(r) = 2\pi r^2 + 2\pi r \cdot \frac{4}{3r^2} = 2\pi r^2 + \frac{8\pi}{3r}$$

求导：

$$S'(r) = 4\pi r - \frac{8\pi}{3r^2} = 0 \Rightarrow r^3 = \frac{2}{3} \Rightarrow r = \sqrt[3]{\frac{2}{3}}$$

对应：

$$h = \frac{4}{3} \cdot \left(\frac{3}{2}\right)^{2/3} = 2 \cdot \sqrt[3]{\frac{2}{3}} = 2r$$

二阶验证： $S''(r) = 4\pi + \frac{16\pi}{3r^3} > 0$ ，确为最小值。

方法 2: *Lagrange* 乘子法

构造 *Lagrange* 函数:

$$\mathcal{L}(r, h, \lambda) = 2\pi r^2 + 2\pi r h - \lambda \left(\pi r^2 h - \frac{4\pi}{3} \right)$$

KKT 条件:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial r} &= 4\pi r + 2\pi h - 2\lambda\pi r h = 0 \\ \frac{\partial \mathcal{L}}{\partial h} &= 2\pi r - \lambda\pi r^2 = 0 \\ \frac{\partial \mathcal{L}}{\partial \lambda} &= -\left(\pi r^2 h - \frac{4\pi}{3} \right) = 0 \end{aligned}$$

由第二式 ($r > 0$): $\lambda = \frac{2}{r}$

代入第一式:

$$4\pi r + 2\pi h - 2 \cdot \frac{2}{r} \cdot \pi r h = 4\pi r + 2\pi h - 4\pi h = 4\pi r - 2\pi h = 0$$

得 $h = 2r$, 代入体积约束:

$$\pi r^2 \cdot 2r = \frac{4\pi}{3} \Rightarrow r^3 = \frac{2}{3} \Rightarrow r = \sqrt[3]{\frac{2}{3}}$$

结论: $r^* = \sqrt[3]{\frac{2}{3}}$, $h^* = 2r^* = 2\sqrt[3]{\frac{2}{3}}$, 最小表面积 $S^* = 6\pi \left(\frac{2}{3}\right)^{2/3}$ 。

```

1 import numpy as np
2 from scipy.optimize import minimize
3
4 # === 正方形剪角问题 ===
5 def box_volume(x, a=1.0):
6     """水槽容积"""
7     if x <= 0 or x >= a/2:
8         return 0
9     return x * (a - 2*x)**2
10
11 def box_volume_neg(x, a=1.0):
12     """负容积 (用于最小化) """
13     return -box_volume(x, a)
14
15 result1 = minimize(box_volume_neg, x0=0.2, args=(1.0,),
16                    bounds=[(0.01, 0.49)])
17 print("正方形剪角问题:")
18 print(f" 最优剪去边长: x* = {result1.x[0]:.6f} (理论: a/6 = {1/6:.6f}"))

```

```

19 print(f" 最大容积: V* = {-result1.fun:.6f} (理论: 2a^3/27 = {2/27:.6f})")
20
21 # === 球变圆柱问题 ===
22 def cylinder_surface(vars):
23     """圆柱表面积"""
24     r, h = vars
25     return 2*np.pi*r**2 + 2*np.pi*r*h
26
27 def volume_constraint(vars):
28     """体积约束 = 0"""
29     r, h = vars
30     return np.pi*r**2*h - 4*np.pi/3
31
32 result2 = minimize(cylinder_surface, x0=[0.5, 2.0],
33                   constraints={'type': 'eq', 'fun':
34                               volume_constraint})
34 r_opt, h_opt = result2.x
35 print(f"\n球变圆柱问题:")
36 print(f" 最优半径: r* = {r_opt:.6f} (理论: {(2/3)**(1/3):.6f})")
37 print(f" 最优高度: h* = {h_opt:.6f} (理论: {2*(2/3)**(1/3):.6f})")
38 print(f" h/r = {h_opt/r_opt:.4f} (理论: 2)")

```

Listing 1.1: 经典极值问题求解

1.3 计算机学科中的典型最优化问题

1.3.1 深度学习中的优化

深度学习优化问题

给定 n 个样本 $\{(x_i, y_i)\}_{i=1}^n$, 其中 x_i 为特征向量, y_i 为标签。期望训练映射 $f(x; \theta)$ 使得 $f(x_i; \theta) \approx y_i$ 。

优化目标 (经验风险最小化):

$$\min_{\theta} J(\theta) = \frac{1}{n} \sum_{i=1}^n L(f(x_i; \theta), y_i) + \lambda R(\theta)$$

其中 L 为损失函数, R 为正则化项, λ 为正则化系数。

例题 1.3.1 (神经网络的梯度下降). **经典梯度下降:**

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla J(\theta^{(t)})$$

其中 η 为步长 (学习率)。

随机梯度下降 (SGD):

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla L(f(x_i; \theta^{(t)}), y_i)$$

每次只用一个样本计算梯度, 适合大规模数据。

1.3.2 优化求解的质量度量

在优化问题的求解过程中, 需要度量解的质量。常用的度量工具包括范数和各类评价指标。

优化中的范数

范数用于衡量向量的“大小”, 在优化中广泛用于定义目标函数和约束条件。

- **2-范数** (欧几里得范数): $\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$
 - 常见形式: $\|y - f(x; \theta)\|_2^2$, 衡量预测值与真实值之间的偏差平方和
 - 在最小二乘问题中作为目标函数
- **∞ -范数** (最大范数): $\|x\|_\infty = \max(|x_1|, |x_2|, \dots, |x_n|)$
 - 衡量所有误差分量中的最大值, 对异常值更敏感
 - 在极小极大 (minimax) 优化问题中使用

回归问题的评价指标

对于回归任务 $f(x; \theta)$, 常用的评价指标有:

- **均方误差 (MSE)**: $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta))^2$
- **均方根误差 (RMSE)**: $\text{RMSE} = \sqrt{\text{MSE}}$, 与原始数据同量纲
- **平均绝对误差 (MAE)**: $\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - f(x_i; \theta)|$

分类问题的评价指标

对于分类任务, 以二分类为例, 设 TP 、 TN 、 FP 、 FN 分别为真正例、真负例、假正例、假负例个数:

- **准确率 (Accuracy):** $\text{Acc} = \frac{TP + TN}{TP + TN + FP + FN}$
- **精确率 (Precision):** $P = \frac{TP}{TP + FP}$, 预测为正的样本中真正为正的比率
- **召回率 (Recall):** $R = \frac{TP}{TP + FN}$, 所有正样本中被正确预测的比例
- **F1 值:** $F_1 = \frac{2PR}{P + R}$, 精确率与召回率的调和平均

1.3.3 其他计算机领域优化问题

典型优化问题

- **多核任务调度:** 如何高效地把任务分配到各个核上运行
- **资源分配:** 云计算中的虚拟机调度、负载均衡
- **图像处理:** 图像去噪、超分辨率重建、图像分割
- **自然语言处理:** 词向量训练、注意力机制优化
- **强化学习:** 策略优化、价值函数估计
- **计算机图形学:** 光线追踪、物理模拟

例题 1.3.2 (噪声模型与极大似然估计). 给定测量模型 $y = Ax + \epsilon$, 其中 ϵ 为噪声。不同噪声分布对应不同的优化目标。

(a) **拉普拉斯噪声:** 若 $\epsilon_i \sim \text{Laplace}(0, b)$, 联合对数似然为

$$\log L(x) = -m \log(2b) - \frac{1}{b} \sum_{i=1}^m |y_i - a_i^T x|$$

最大化对数似然等价于 $\min_x \|y - Ax\|_1$, 即 ℓ_1 **最小化**。 ℓ_1 范数对异常值更鲁棒, 常用于鲁棒回归。

(b) **高斯噪声:** 若 $\epsilon_i \sim N(0, \sigma^2)$, 对数似然为

$$\log L(x) = -m \log(\sqrt{2\pi}\sigma) - \frac{1}{2\sigma^2} \|y - Ax\|_2^2$$

最大化对数似然等价于 $\min_x \|y - Ax\|_2^2$, 即 ℓ_2 **最小二乘**。

这说明: 噪声的分布决定了优化问题中所用的范数——拉普拉斯噪声对应 ℓ_1 , 高斯噪声对应 ℓ_2 。

例题 1.3.3 (岭回归 (Tikhonov 正则化)). 当线性方程组 $Ax = y$ 不适定时 (欠定或带噪声), 岭回归通过正则化改善条件:

$$\min_x \|y - Ax\|_2^2 + \lambda \|x\|_2^2, \quad \lambda > 0$$

(a) 最优解: 目标函数 $F(x) = (y - Ax)^T(y - Ax) + \lambda x^T x$, 求梯度令其为零:

$$-2A^T y + 2A^T A x + 2\lambda x = 0 \Rightarrow (A^T A + \lambda I)x = A^T y$$

故 $x^* = (A^T A + \lambda I)^{-1} A^T y$ 。

(b) 可逆性: 对任意 $x \neq 0$, $x^T(A^T A + \lambda I)x = \|Ax\|_2^2 + \lambda \|x\|_2^2 > 0$ (只要 $\lambda > 0$), 故 $A^T A + \lambda I$ 正定, 一定可逆。这解决了 $A^T A$ 不可逆 (A 不列满秩) 的问题。

1.4 最优化的基本数学概念

凸性是优化理论的基石。本节介绍凸集、凸函数、仿射函数等基本概念, 为后续各章奠定数学基础。

1.4.1 凸集

定义 1.4.1 (凸集). 集合 $C \subseteq \mathbb{R}^n$ 称为**凸集** (Convex Set), 是指对任意 $x, y \in C$ 和任意 $\theta \in [0, 1]$, 都有

$$\theta x + (1 - \theta)y \in C$$

即: 集合中任意两点的连线仍在集合内。

定义 1.4.2 (凸组合). 设 $x_1, \dots, x_k \in \mathbb{R}^n$, $\theta_1 + \dots + \theta_k = 1$, $\theta_i \geq 0$, 则称 $\sum_{i=1}^k \theta_i x_i$ 为 $x_1, \dots, x_k$ 的**凸组合** (Convex Combination)。

凸集等价于: 对其中任意有限个点, 其凸组合仍在集合内。

定义 1.4.3 (仿射集). 集合 $C \subseteq \mathbb{R}^n$ 称为**仿射集** (Affine Set), 是指对任意 $x, y \in C$ 和任意 $\theta \in \mathbb{R}$, 都有

$$\theta x + (1 - \theta)y \in C$$

即: 过集合中任意两点的整条直线都在集合内。超平面 $\{x \mid a^T x = b\}$ 是仿射集的典型例子。

仿射集与凸集的区别在于: 凸集只要求线段在集合内, 仿射集要求整条直线在集合内。

定义 1.4.4 (锥与凸锥). 集合 $C \subseteq \mathbb{R}^n$ 称为**锥** (Cone), 是指对任意 $x \in C$ 和 $\theta > 0$, 都有 $\theta x \in C$ 。

若 C 还是凸集, 则称为**凸锥** (Convex Cone), 即对任意 $x, y \in C$ 和 $\theta_1, \theta_2 \geq 0$, 有 $\theta_1 x + \theta_2 y \in C$ 。

例如: $\{x \in \mathbb{R}^n \mid x \geq 0\}$ (非负象限) 是一个凸锥。

常见凸集

- 超平面: $\{x \mid a^T x = b\}$
- 半空间: $\{x \mid a^T x \leq b\}$
- 球: $\{x \mid \|x - x_0\| \leq r\}$
- 多面体: 有限个半空间的交集 $\{x \mid Ax \leq b\}$ ——线性规划的可行域
- 凸集的交集仍是凸集

性质 1.4.1 (超平面与半空间的特征). • 超平面 $H = \{x \mid a^T x = b\}$ 既是凸集也是仿射集

- 闭半空间 $H^+ = \{x \mid a^T x \leq b\}$ 和 $H^- = \{x \mid a^T x \geq b\}$ 都是凸集, 但不是仿射集
- 开半空间 $\{x \mid a^T x < b\}$ 也是凸集

性质 1.4.2 (凸集的交集). 设 $C_1, C_2, \dots, C_m$ 均为凸集, 则其交集

$$C = \bigcap_{i=1}^m C_i$$

仍是凸集。

证明思路: 设 $x, y \in C$, 则 x, y 同时属于每个 C_i 。由每个 C_i 的凸性, $\theta x + (1-\theta)y \in C_i$ 对所有 i 成立, 故 $\theta x + (1-\theta)y \in C$ 。

1.4.2 仿射函数

定义 1.4.5 (仿射函数与线性函数). 形如

$$f(x) = a^T x + b, \quad a \in \mathbb{R}^n, b \in \mathbb{R}$$

的函数称为仿射函数 (Affine Function)。

特别注意: 仿射函数与线性函数的区别——

- 仿射函数: $f(x) = a^T x + b$, 允许有常数偏移项 $b \neq 0$
- 线性函数 (严格定义): $f(x) = a^T x$, 必须满足 $f(0) = 0$ (即 $b = 0$)

在优化文献中, 有时将仿射函数 *loosely* 称为“线性”, 但严格数学意义上, 线性函数必须满足叠加性 $f(\alpha x + \beta y) = \alpha f(x) + \beta f(y)$, 这要求 $b = 0$ 。

仿射函数的几何意义: 图像是 $\mathbb{R}^{n+1}$ 中的超平面。

性质 1.4.3 (仿射函数的关键性质). 仿射函数既是凸函数也是凹函数。

这是因为对仿射函数 $f(x) = a^T x + b$:

$$f(\theta x + (1-\theta)y) = \theta f(x) + (1-\theta)f(y)$$

恰好取等号, 同时满足凸性和凹性的定义。

1.4.3 凸函数与凹函数

定义 1.4.6 (凸函数). 设 $f: C \rightarrow \mathbb{R}$, $C \subseteq \mathbb{R}^n$ 为凸集。 f 称为**凸函数** (Convex Function), 是指对任意 $x, y \in C$ 和 $\theta \in [0, 1]$, 有

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

若不等号严格成立 ($x \neq y$, $\theta \in (0, 1)$), 则称为**严格凸函数**。

定义 1.4.7 (凹函数). 若 $-f$ 是凸函数, 则 f 称为**凹函数** (Concave Function)。即:

$$f(\theta x + (1 - \theta)y) \geq \theta f(x) + (1 - \theta)f(y)$$

凸函数的直观理解与判定

几何直观: 凸函数的图像上任意两点间的弦, 始终在图像上方 (或重合)。

常见凸函数:

- $f(x) = x^2$, $f(x) = |x|$, $f(x) = e^x$
- $f(x) = \|x\|$ (任意范数)
- 仿射函数 $f(x) = a^T x + b$ (当 $b = 0$ 时为严格线性函数)
- 非负加权的凸函数之和仍为凸函数

二阶判定: 若 f 二阶可微, 则 f 为凸函数 $\Leftrightarrow$ Hessian 矩阵 $\nabla^2 f(x)$ 处处半正定。

保持凸性的运算

以下运算可以构造新的凸函数:

- **非负加权和:** 若 $f_1, \dots, f_m$ 为凸函数, $w_i \geq 0$, 则

$$f(x) = \sum_{i=1}^m w_i f_i(x)$$

为凸函数。

- **仿射映射复合:** 若 f 为凸函数, 则 $g(x) = f(Ax + b)$ 也是凸函数。
- **逐点最大:** 若 $f_1, \dots, f_m$ 为凸函数, 则 $f(x) = \max\{f_1(x), \dots, f_m(x)\}$ 为凸函数。
- **逐点上确界:** 若对每个 $y \in \mathcal{A}$, $f(x, y)$ 关于 x 是凸的, 则

$$g(x) = \sup_{y \in \mathcal{A}} f(x, y)$$

关于 x 是凸函数。

例题 1.4.1 (最大 L 范数的凸性). 给定向量 $w \in \mathbb{R}^m$, 令 $w_{[k]}$ 表示按绝对值从大到小排序后的第 k 个元素, 即 $|w_{[1]}| \geq |w_{[2]}| \geq \dots \geq |w_{[m]}|$. 定义最大 L 范数为

$$\|w\|_{[L]} \triangleq \sum_{k=1}^L |w_{[k]}|, \quad L \in \{1, 2, \dots, m\}.$$

证明 $\|w\|_{[L]}$ 是凸函数。

证明: “取绝对值最大的 L 个分量求和”等价于在所有大小为 L 的指标集合中取最大值:

$$\|w\|_{[L]} = \max_{\substack{S \subset \{1, \dots, m\} \\ |S|=L}} \sum_{i \in S} |w_i|.$$

而对固定集合 S , 又有

$$\sum_{i \in S} |w_i| = \max_{\varepsilon_i \in \{-1, 1\}} \sum_{i \in S} \varepsilon_i w_i.$$

因此

$$\|w\|_{[L]} = \max_{\substack{S \subset \{1, \dots, m\}, |S|=L \\ \varepsilon_i \in \{-1, 1\}}} \sum_{i \in S} \varepsilon_i w_i.$$

右端是有限多个线性函数关于 w 的逐点最大值。线性函数既凸又凹, 而凸函数的逐点最大值仍为凸函数, 所以 $\|w\|_{[L]}$ 是凸函数。

定义 1.4.8 (强凸函数). 设 $f: C \rightarrow \mathbb{R}$, $C \subseteq \mathbb{R}^n$ 为凸集。若存在常数 $m > 0$, 使得对任意 $x, y \in C$ 和 $\theta \in [0, 1]$, 有

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y) - \frac{m}{2}\theta(1 - \theta)\|x - y\|^2$$

则称 f 为**强凸函数** (Strongly Convex Function), m 称为强凸系数。

强凸函数一定是严格凸函数, 且具有更快的收敛速度保证。

1.4.4 凸性与最优化的关系

为什么凸性如此重要

凸优化问题具有以下优良性质:

- (1) **局部最优 = 全局最优**: 凸优化问题的任何局部最优解都是全局最优解
- (2) **高效求解**: 存在多项式时间算法, 不会陷入局部最优陷阱
- (3) **理论完备**: 最优性条件 (如 KKT 条件) 既是必要的也是充分的

凸规划: 若目标函数 f 和所有不等式约束函数 g_i 均为凸函数, 等式约束 h_j 为仿射函数, 则称该优化问题为**凸规划问题**。

线性规划是凸规划的特例 (目标函数和约束都是仿射的, 仿射 = 既是凸也是凹)。

KKT 条件的详细推导与证明见第 6 章第 6.2.2 节。

例题 1.4.2 (凸集的运算性质). 设 S_1, S_2 是凸集, 讨论下列集合是否是凸集。

(1) $S_1 \cup S_2$: 一般不是凸集。

反例: 取 $S_1 = \{0\}$, $S_2 = \{2\} \subset \mathbb{R}$, 二者都是单点凸集, 但 $S_1 \cup S_2 = \{0, 2\}$ 。中点 $\frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 2 = 1 \notin S_1 \cup S_2$, 违反凸集定义。只有在 $S_1 \subseteq S_2$ 或 $S_2 \subseteq S_1$ 时, 并集才一定凸。

(2) $S_1 + S_2 = \{x + y : x \in S_1, y \in S_2\}$: 是凸集 (Minkowski 和保持凸性)。

证明: 任取 $z_1, z_2 \in S_1 + S_2$, 存在 $x_1, x_2 \in S_1$, $y_1, y_2 \in S_2$, 使得 $z_1 = x_1 + y_1$, $z_2 = x_2 + y_2$ 。对 $\theta \in [0, 1]$:

$$\theta z_1 + (1 - \theta) z_2 = \underbrace{[\theta x_1 + (1 - \theta) x_2]}_{\in S_1} + \underbrace{[\theta y_1 + (1 - \theta) y_2]}_{\in S_2} \in S_1 + S_2$$

(3) $S_1 - S_2 = \{x - y : x \in S_1, y \in S_2\}$: 是凸集。

$-S_2 = \{-y : y \in S_2\}$ 仍是凸集, 而 $S_1 - S_2 = S_1 + (-S_2)$, 由 (2) 可知两个凸集的和仍凸。

例题 1.4.3 (判断集合是否为凸集). 判断下列集合是否为凸集。

(a) $S = \{x : x_1^2 + x_2^2 \geq 4, 2x_1 + x_2 \leq 10, -x_1 - x_2 \leq -10x_2\}$: 不是凸集。

第一个约束 $x_1^2 + x_2^2 \geq 4$ 是圆外区域, 本身非凸。取 $p = (2, 0)$, $q = (-2, -1)$, 两点均满足全部约束, 但中点 $\bar{x} = (0, -1/2)$ 满足 $\|\bar{x}\|_2^2 = 1/4 < 4$, 违反第一个约束。

(b) $S = \{x : x_1 + x_2 \leq 6, -2x_1 + 3x_2 \geq 2, 4x_1 - x_2 \leq 12\}$: 是凸集。

三个约束都是仿射不等式, 每个定义一个半空间 (凸集), 有限个凸集的交集仍凸。

(c) $S = \{x : -(x_1 - 1)^2 + x_2 \geq 1, x_1 + x_2 \geq 3, x_1 \geq 1\}$: 是凸集。

第一个约束等价于 $x_2 \geq 1 + (x_1 - 1)^2$, 右端是凸函数, 其上图集 (epigraph) 是凸集。由凸性: 若 u, v 均满足该约束, 则

$$1 + (\theta u_1 + (1 - \theta)v_1 - 1)^2 \leq \theta[1 + (u_1 - 1)^2] + (1 - \theta)[1 + (v_1 - 1)^2] \leq \theta u_2 + (1 - \theta)v_2$$

凸组合仍满足该约束。其余两个约束是半空间, 故 S 为凸集。

(d) $S = \{x : x^2 \geq 1\}$: 不是凸集。

$S = (-\infty, -1] \cup [1, \infty)$, $-1, 1 \in S$ 但中点 $0 \notin S$ 。

例题 1.4.4 (函数凸性判断). 设 $f_1, \dots, f_m$ 均为凸函数, 讨论下列函数是否是凸函数。

(a) $g(x) = \max\{f_1(x), \dots, f_m(x)\}$: 是凸函数。

对 x, y 和 $\theta \in [0, 1]$, 每个 f_i 满足 $f_i(\theta x + (1 - \theta)y) \leq \theta f_i(x) + (1 - \theta)f_i(y) \leq \theta g(x) + (1 - \theta)g(y)$ 。对 i 取最大即得结论。

(b) $g(x) = \min\{f_1(x), \dots, f_m(x)\}$: 一般不是凸函数。

反例: $f_1(x) = x$, $f_2(x) = -x$ 均为仿射函数 (凸), 但 $g(x) = \min\{x, -x\} = -|x|$ 。取 $x = -1$, $y = 1$, $\theta = 1/2$: $g(0) = 0$, 而 $\frac{1}{2}g(-1) + \frac{1}{2}g(1) = -1$, 凸性要求 $0 \leq -1$, 矛盾。

(c) $g(x) = \sum_{i=1}^n x_i \log x_i$ ($x_i > 0$): 是凸函数。

令 $\phi(t) = t \log t$, $\phi''(t) = 1/t > 0$, ϕ 在 $(0, \infty)$ 上严格凸。Hessian 矩阵

$$\nabla^2 g(x) = \text{diag}\left(\frac{1}{x_1}, \dots, \frac{1}{x_n}\right) \succ 0$$

故 g 严格凸。这在信息论中是熵函数的负值。

(d) $g(x) = \lfloor x \rfloor$ (向下取整): 不是凸函数。

取 $x = -0.5$, $y = 0.5$, $\theta = 1/2$: $g(0) = 0$, $\frac{1}{2}g(-0.5) + \frac{1}{2}g(0.5) = -1/2$, 凸性要求 $0 \leq -1/2$, 不成立。凸函数在开区间内必连续, 而取整函数有跳跃间断。

(e) $g(x) = \sum_{i=1}^k x_{(i)}$ ($x_{(i)}$ 为第 i 个最大分量): 是凸函数。

可写为 $\sum_{i=1}^k x_{(i)} = \max_{|S|=k} \sum_{j \in S} x_j$, 每个 $\sum_{j \in S} x_j$ 是线性函数, 有限个线性函数的逐点最大值为凸函数。

(f) $g(x) = x^2$: 是凸函数, $g''(x) = 2 > 0$ 。

例题 1.4.5 (非凸集合与非凸函数的凸化). 非凸集合转化为凸集合的常见方法:

- 凸包: 用 $\text{conv}(S)$ 替代 S
- 线性松弛: 将整数约束 $x \in \{0, 1\}$ 放宽为 $0 \leq x \leq 1$
- 半正定松弛: 将二次约束提升为矩阵变量约束
- 外逼近: 用若干半空间包围非凸集合

非凸函数转化为凸函数的常见方法:

- 凸包络: 用函数的凸包络替代原函数
- DC 分解: $f(x) = g(x) - h(x)$, g, h 均为凸, 再线性化 h
- 正则化: 加入强凸项 $\frac{\rho}{2} \|x\|^2$
- 变量替换或提升: 将非凸二次项转化为矩阵变量

课程涵盖内容

本课程共计 32 学时, 涵盖以下八章:

第 1 章 简介 (2 学时) —— 运筹学概述、最优化基础概念、凸集与凸函数 (见第 1.4 节)

第 2 章 线性规划 (6 学时) —— 单纯形法、对偶理论、灵敏度分析 (见第 2 章)

第 3 章 非线性规划的数学模型 (6 学时) —— Taylor 展开、最优性条件、凸规划 (见第 ?? 节)

第 4 章 一维搜索 (2 学时) —— 黄金分割法、Newton 法、插值法 (见第 4 章)

第 5 章	无约束最优化方法（6 学时）——梯度法、Newton 法、共轭梯度、拟 Newton 法（见第 5 章）
第 6 章	约束最优化理论（2 学时）——KKT 条件、对偶理论、Farkas 引理（见第 6 章）
第 7 章	约束问题的最优化方法（6 学时）——罚函数法、内点法、增广 Lagrange 法（见第 7 章）
第 8 章	智能优化计算与深度学习中的优化（2 学时）——SA、GA、PSO、DE 等（见第 8 章）

1.5 本章小结

1.5.1 本章知识框架

第一章核心内容	
1.	运筹学与最优化概述 • 运筹学的定义、历史、学科分支 • 最优化问题的一般形式 • 优化问题的分类（7 个维度：连续/离散、约束/无约束、确定性/随机性、线性/非线性、单目标/多目标、静态/动态、单层/多层） • 运筹学方法论
2.	经典极值问题 • 无约束极值：正方形剪角求最大容积 • 等式约束极值：球变圆柱最小表面积（Lagrange 乘子法）
3.	计算机学科中的优化 • 深度学习中的经验风险最小化 • SGD 等优化算法 • 优化求解的质量度量：范数、MSE/RMSE/MAE、准确率/召回率/F1 • 其他计算机领域应用
4.	最优化的基本数学概念 • 凸集：凸组合、仿射集、锥与凸锥、常见凸集（超平面、半空间、多面体） • 仿射函数：既是凸也是凹

- 凸函数与凹函数：Jensen 不等式定义、Hessian 半正定判定、保持凸性的运算、强凸函数
- 凸性与优化：局部最优 = 全局最优

1.5.2 关键公式汇总

表 1.1: 核心公式速查

概念	公式
最优化一般形式	$\min f(x) \text{ s.t. } c_i(x) \geq 0, c_i(x) = 0$
无约束问题	$\min_{x \in \mathbb{R}^n} f(x)$
Lagrange 函数	$\mathcal{L}(x, \lambda) = f(x) - \lambda h(x)$
梯度下降	$x^{(t+1)} = x^{(t)} - \eta \nabla f(x^{(t)})$
SGD 更新	$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla L_i(\theta^{(t)})$
MSE	$\frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2$
F1 值	$2PR/(P + R)$
凸集	$\theta x + (1 - \theta)y \in C, \forall \theta \in [0, 1]$
凸函数	$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$
仿射函数	$f(x) = a^T x + b$ (既凸又凹)
凸性二阶判定	$\nabla^2 f(x) \succeq 0$ (Hessian 半正定)
强凸函数	$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y) - \frac{m}{2}\theta(1 - \theta)\ x - y\ ^2$

1.5.3 关键术语中英对照

中文	英文
运筹学	Operations Research
最优化	Optimization
无约束优化	Unconstrained Optimization
约束优化	Constrained Optimization
连续优化	Continuous Optimization
离散优化	Discrete Optimization
确定性优化	Deterministic Optimization
随机性优化	Stochastic Optimization
线性规划	Linear Programming
非线性规划	Nonlinear Programming
单目标优化	Single-Objective Optimization
多目标优化	Multi-Objective Optimization
动态优化	Dynamic Optimization
多层优化	Multi-Level Optimization
Lagrange 乘子法	Lagrange Multiplier Method
梯度下降	Gradient Descent
随机梯度下降	Stochastic Gradient Descent
经验风险最小化	Empirical Risk Minimization
均方误差	Mean Squared Error (MSE)
准确率	Accuracy
召回率	Recall
精确率	Precision
凸集	Convex Set
凸函数	Convex Function
凹函数	Concave Function
仿射函数	Affine Function
仿射集	Affine Set
凸组合	Convex Combination
凸锥	Convex Cone
凸规划	Convex Programming
强凸函数	Strongly Convex Function
半正定	Positive Semidefinite

Chapter 2

线性规划

2.1 线性规划概述

2.1.1 线性规划模型

定义 2.1.1 (线性规划 (LP)). 线性规划是在一组线性约束条件下, 求线性目标函数极值的最优化问题。

$$\begin{aligned} \max \quad & z = c_1x_1 + c_2x_2 + \cdots + c_nx_n \\ \text{s.t.} \quad & a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \leq b_1 \\ & a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \leq b_2 \\ & \vdots \\ & a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \leq b_m \\ & x_1, x_2, \dots, x_n \geq 0 \end{aligned} \tag{2.1}$$

矩阵形式

LP 可简洁表示为:

$$\max z = c^T x, \quad \text{s.t. } Ax \leq b, x \geq 0$$

其中 $c \in \mathbb{R}^n$, $b \in \mathbb{R}^m$, $A \in \mathbb{R}^{m \times n}$ 。

定义 2.1.2 (LP 标准型). LP 的标准形式为:

$$\begin{aligned} \max \quad & z = c^T x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned} \tag{2.2}$$

其中 $b \geq 0$ 。

转化为标准型

- 最小化转最大化: $\min z \Rightarrow \max(-z)$
- $\leq$ 约束: 加松弛变量 $x_{n+i} \geq 0$
- $\geq$ 约束: 减剩余变量 $x_{n+i} \geq 0$
- 自由变量: $x_j = x_j^+ - x_j^-$, 其中 $x_j^+, x_j^- \geq 0$

例题 2.1.1 (转标准型). 原问题: $\max z = 3x_1 + 5x_2$

$$x_1 + 2x_2 \leq 8$$

$$3x_1 + 2x_2 \leq 12$$

$$x_1, x_2 \geq 0$$

引入松弛变量 $x_3, x_4 \geq 0$:

$$\max z = 3x_1 + 5x_2 + 0x_3 + 0x_4$$

$$x_1 + 2x_2 + x_3 = 8$$

$$3x_1 + 2x_2 + x_4 = 12$$

$$x_1, x_2, x_3, x_4 \geq 0$$

2.1.2 图解法

图解法适用于只有两个决策变量的 LP 问题。

图解法步骤

1. 画出所有约束条件对应的直线，确定可行域
2. 画出目标函数等值线
3. 沿目标函数梯度方向（ c 的方向）移动等值线，找到与可行域最后交点
4. 该交点即为最优解

例题 2.1.2 (图解法). $\max z = 3x_1 + 5x_2$

$$x_1 + 2x_2 \leq 8$$

$$3x_1 + 2x_2 \leq 12$$

$$x_1, x_2 \geq 0$$

步骤:

1. 画约束线:

- $x_1 + 2x_2 = 8$: 过 $(8, 0)$ 和 $(0, 4)$
- $3x_1 + 2x_2 = 12$: 过 $(4, 0)$ 和 $(0, 6)$

2. 可行域为四个顶点的多边形:

- $O(0, 0)$: $z = 0$
- $A(4, 0)$: $z = 12$
- $B(2, 3)$: 联立 $x_1 + 2x_2 = 8$ 和 $3x_1 + 2x_2 = 12$, 解得 $x_1 = 2, x_2 = 3, z = 21$
- $C(0, 4)$: $z = 20$

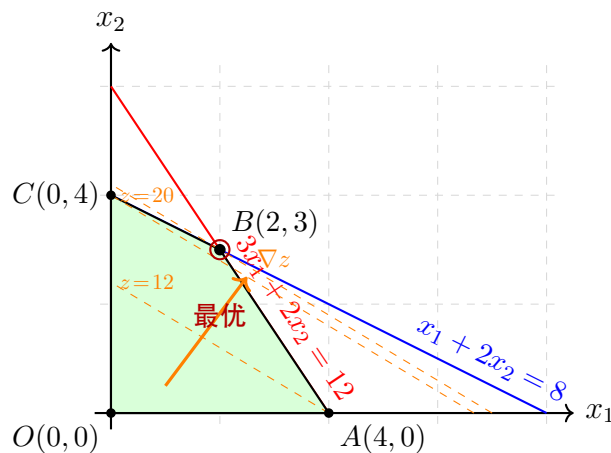
3. 最优解: $x^* = (2, 3), z^* = 21$ 

图 2.1: 线性规划图解法: 可行域 (绿色) 与等值线

2.1.3 线性规划基本定理

定理 2.1.1 (LP 基本定理). 对于标准型 LP (2.2):

1. 若可行域非空, 则必存在基本可行解 (极点)
2. 若 LP 有最优解, 则至少有一个基本可行解是最优解
3. LP 的可行域是凸集 (多面体), 极点的个数有限

定理 2.1.2 (LP 解的可能情况). 线性规划问题解的情况只有三种:

1. 唯一最优解: 某个极点最优
2. 无穷多最优解: 某条边上所有点都最优

3. 无界解：目标函数可趋于无穷

不可能无可行解的情况在可行域非空时已排除。

例题 2.1.3 (无界解). $\max z = x_1 + x_2$

$$x_1 - x_2 \leq 1$$

$$x_1, x_2 \geq 0$$

取 $x_2 = 0, x_1 \rightarrow \infty, z \rightarrow \infty$, 无界。

例题 2.1.4 (无穷多最优解). $\max z = x_1 + 2x_2$

$$x_1 + 2x_2 \leq 8$$

$$x_1 + x_2 \leq 6$$

$$x_1, x_2 \geq 0$$

目标函数等值线与约束 $x_1 + 2x_2 \leq 8$ 平行，线段 $B(4, 2)$ 到 $C(0, 4)$ 上所有点都是最优解， $z^* = 8$ 。

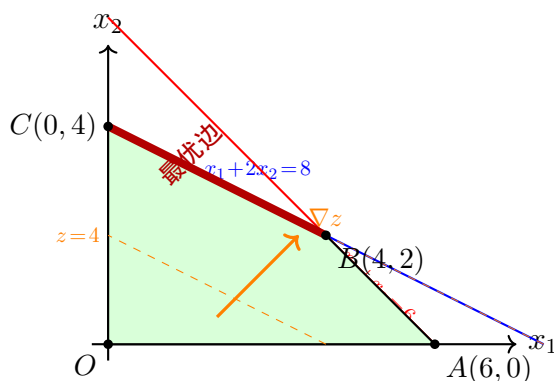


图 2.2: 无穷多最优解：等值线与约束边平行，红粗线上的所有点均为最优

2.2 基本可行解与极点

2.2.1 基与基本解

定义 2.2.1 (基 (Basis)). 设 $A \in \mathbb{R}^{m \times n}$, $\text{rank}(A) = m$ 。从 A 中选取 m 个线性无关的列向量构成 $m \times m$ 矩阵 B , 称为一个基矩阵 (或简称基)。

对应的变量 $x_B = (x_{B_1}, \dots, x_{B_m})^T$ 称为基变量, 其余变量 x_N 称为非基变量。

定义 2.2.2 (基本解 (Basic Solution)). 令所有非基变量 $x_N = 0$, 解方程组 $Bx_B = b$, 得到的解 $x = (x_B, x_N) = (B^{-1}b, 0)$ 称为关于基 B 的基本解。

定义 2.2.3 (基本可行解 (BFS)). 若基本解满足 $x_B \geq 0$, 则称为**基本可行解** (*Basic Feasible Solution, BFS*)。

若 $x_B > 0$ (严格正), 称为**非退化 BFS**; 若某些分量 $x_{B_i} = 0$, 称为**退化 BFS**。

BFS 与极点的关系

可行域的极点 $\Leftrightarrow$ BFS。单纯形法就是在 BFS (极点) 之间搜索。

例题 2.2.1 (基本可行解的充要条件). **命题**: 可行解 x 为 BFS 的充要条件是: x 的正分量对应的系数列向量线性无关。

必要性: 若 x 是 BFS, 存在基 B 使非基变量为零, 基变量 $x_B = B^{-1}b$ 。正分量只能出现在基变量中, 基列线性无关, 故正分量对应的列向量线性无关。

充分性: 若 x 的正分量对应列向量线性无关, 可将这些列扩充为 m 个线性无关列构成基 B 。不在基中的变量置零不改变 x , 故 x 是某个基产生的可行解, 即 BFS。

2.2.2 单纯形表的结构

对于标准型 LP, 将约束方程组和目标函数写成表格形式:

表 2.1: 单纯形表的结构

	x_1	x_2	$\cdots$	x_n	x_s	RHS
z	σ_1	σ_2	$\cdots$	σ_n	0	z_0
x_{s_1}	y_{11}	y_{12}	$\cdots$	y_{1n}	1	$\bar{b}_1$
x_{s_2}	y_{21}	y_{22}	$\cdots$	y_{2n}	0	$\bar{b}_2$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$
x_{s_m}	y_{m1}	y_{m2}	$\cdots$	y_{mn}	0	$\bar{b}_m$

检验数

在单纯形表中, z 行对应检验数 (reduced cost):

$$\sigma_j = c_B^T B^{-1} A_j - c_j = z_j - c_j$$

其中 c_B 是基变量的价值系数。

- 若所有 $\sigma_j \geq 0$, 当前 BFS 最优
- 若存在 $\sigma_j < 0$, 选择该列入基可改善目标

2.2.3 单纯形法的迭代公式

定理 2.2.1 (基变换). 设当前基为 B , 对应 BFS 为 x . 若选择非基变量 x_k 入基, 基变量 x_{B_r} 出基, 则新基 $\hat{B}$ 满足:

1. 入基列: A_k 替换 B 中第 r 列
2. 出基行: $r = \arg \min_i \left\{ \frac{(B^{-1}b)_i}{(B^{-1}A_k)_i} : (B^{-1}A_k)_i > 0 \right\}$

单纯形法的迭代

每次迭代:

1. 选择入基变量: $k = \arg \min_j \{\sigma_j\}$ (最小检验数)
2. 选择出基变量: 最小比值检验 (Minimum Ratio Test)
3. 枢轴运算: 高斯消元更新单纯形表

2.3 单纯形法

2.3.1 单纯形法的基本原理

单纯形法是最经典的 LP 求解算法, 由 Dantzig 于 1947 年提出。其核心思想是在基本可行解 (极点) 之间迭代, 每次迭代通过入基和出基操作移动到相邻的更好的极点, 直到找到最优解。

单纯形法的几何解释

在可行域的多面体上, 从一个顶点出发, 沿着使目标函数改善的边走到相邻顶点, 直到无法改善为止。

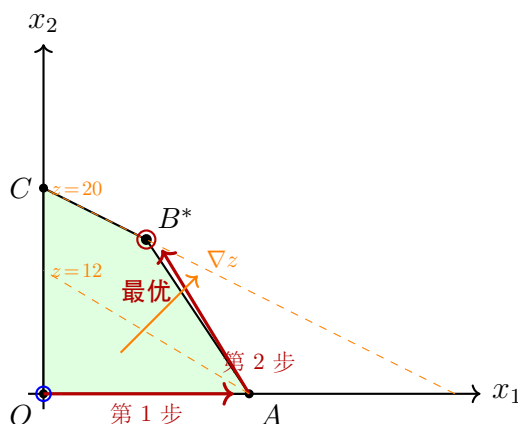


图 2.3: 单纯形法的几何解释: 从顶点 O 出发, 沿边迭代到最优顶点 B^*

2.3.2 单纯形法算法步骤

Algorithm 1 单纯形法

Require: 初始 BFS (通常取松弛变量为基)

```

1: 构造初始单纯形表
2: while true do
3:   Step 1: 最优性检验
4:   if 所有检验数  $\sigma_j \geq 0$  then
5:     当前解最优, 停止
6:   end if
7:   选择入基变量:  $k = \arg \min_j \{\sigma_j\}$  (最负检验数)
8:   Step 2: 无界性检验
9:   if 入基列所有系数  $y_{ik} \leq 0$  then
10:    LP 无界, 停止
11:   end if
12:   Step 3: 最小比值检验
13:   出基行:  $r = \arg \min_i \left\{ \frac{b_i}{y_{ik}} : y_{ik} > 0 \right\}$ 
14:   Step 4: 枢轴运算
15:   以  $y_{rk}$  为枢轴元, 高斯消元更新单纯形表
16: end while
  
```

2.3.3 例题：单纯形法迭代两次

例题 2.3.1 (单纯形法完整求解). 求解: $\max z = 3x_1 + 5x_2$

$$x_1 + 2x_2 + x_3 = 8$$

$$3x_1 + 2x_2 + x_4 = 12$$

$$x_1, x_2, x_3, x_4 \geq 0$$

初始单纯形表:

	x_1	x_2	x_3	x_4	RHS
z	-3	-5	0	0	0
x_3	1	2	1	0	8
x_4	3	2	0	1	12

基变量: x_3, x_4 , 对应 BFS: $x = (0, 0, 8, 12)$, $z = 0$

第 1 次迭代:

Step 1: 检验数 $\sigma_1 = -3, \sigma_2 = -5$, x_2 入基 (最小检验数 -5)

Step 2: 入基列 $(2, 2)^T$ 有正分量, 不无界

Step 3: 最小比值检验: $\min\{8/2, 12/2\} = \min\{4, 6\} = 4$

x_3 出基, 枢轴元 $y_{12} = 2$

Step 4: 枢轴运算 (第 1 行除以 2, 消去第 2 行的 x_2 系数):

	x_1	x_2	x_3	x_4	RHS
z	-0.5	0	2.5	0	20
x_2	0.5	1	0.5	0	4
x_4	2	0	-1	1	4

基变量: x_2, x_4 , BFS: $x = (0, 4, 0, 4)$, $z = 20$

第 2 次迭代:

Step 1: 检验数 $\sigma_1 = -0.5 < 0$, x_1 入基

Step 2: 入基列 $(0.5, 2)^T$ 有正分量

Step 3: 最小比值检验: $\min\{4/0.5, 4/2\} = \min\{8, 2\} = 2$

x_4 出基, 枢轴元 $y_{21} = 2$

Step 4: 枢轴运算:

	x_1	x_2	x_3	x_4	RHS
z	0	0	2.25	0.25	21
x_2	0	1	0.75	-0.25	3
x_1	1	0	-0.5	0.5	2

基变量: x_1, x_2 , BFS: $x = (2, 3, 0, 0)$

检验数全部非负, 最优! $z^* = 21$, $x_1^* = 2$, $x_2^* = 3$ 。

2.3.4 退化与循环

定义 2.3.1 (退化). 若 BFS 中某些基变量取值为 0, 称为**退化 BFS**。退化可能导致单纯形法在迭代中不改变目标值 (停留在同一极点)。

定义 2.3.2 (循环 (Cycling)). 在极端情况下, 单纯形法可能陷入循环, 即重复访问同一组基而不终止。

避免循环的方法

1. **Bland 规则**：当多个变量可选时，总是选择下标最小的变量
2. 扰动法：对 b 进行微小扰动
3. 字典序法 (Lexicographic method)

实际中循环极少发生，通常不需要特殊处理。

2.3.5 Python 代码实现

单纯形法完整实现

```

1 import numpy as np
2
3 def simplex_method(c, A, b, maximize=True, max_iter=1000):
4     """
5     单纯形法求解标准型LP:  $\max c^T x$  s.t.  $Ax = b, x \geq 0$ 
6
7     Parameters:
8         c: 目标函数系数 (n,)
9         A: 约束矩阵 (m, n)
10        b: 右端项 (m,), 要求  $b \geq 0$ 
11        maximize: True为最大化
12        max_iter: 最大迭代次数
13
14    Returns:
15        x_opt: 最优解
16        z_opt: 最优值
17        history: 迭代历史
18    """
19    m, n = A.shape
20    if not maximize:
21        c = -c
22
23    # 添加松弛变量
24    A_aug = np.hstack([A, np.eye(m)])
25    c_aug = np.hstack([c, np.zeros(m)])
26
27    # 初始基: 松弛变量
28    basis = list(range(n, n + m))

```

```

29     nonbasis = list(range(n))
30
31     # 单纯形表
32     T = np.zeros((m + 1, n + m + 1))
33     T[1:, :-1] = A_aug
34     T[1:, -1] = b
35
36     history = []
37
38     for iteration in range(max_iter):
39         # 计算检验数
40         cB = c_aug[basis]
41         for j in nonbasis:
42             T[0, j] = cB @ T[1:, j] - c_aug[j]
43         T[0, -1] = cB @ T[1:, -1]
44
45         history.append((basis.copy(), nonbasis.copy(),
46                        T[0, -1]))
47
48         # 最优性检验
49         if all(T[0, j] >= -1e-10 for j in nonbasis):
50             break
51
52         # 选择入基变量
53         entering = min(j for j in nonbasis
54                        if T[0, j] < -1e-10)
55
56         # 无界性检验
57         col = T[1:, entering]
58         if all(col <= 1e-10):
59             return None, np.inf, history
60
61         # 最小比值检验
62         ratios = []
63         for i in range(m):
64             if col[i] > 1e-10:
65                 ratios.append((T[i+1, -1] / col[i], i))
66         leaving_idx = min(ratios)[1]
67         leaving = basis[leaving_idx]

```

```

68
69     # 枢轴运算
70     pivot = T[leaving_idx + 1, entering]
71     T[leaving_idx + 1] /= pivot
72     for i in range(m + 1):
73         if i != leaving_idx + 1:
74             T[i] -= T[i, entering] * T[leaving_idx + 1]
75
76     basis[leaving_idx] = entering
77     nonbasis = [j for j in range(n + m) if j not in basis]
78
79     # 提取解
80     x = np.zeros(n)
81     for i, bi in enumerate(basis):
82         if bi < n:
83             x[bi] = T[i + 1, -1]
84
85     z = T[0, -1]
86     if not maximize:
87         z = -z
88
89     return x, z, history
90
91     # 示例:  $\max 3x_1 + 5x_2$ 
92     # s.t.  $x_1 + 2x_2 \leq 8$ ,  $3x_1 + 2x_2 \leq 12$ ,  $x_1, x_2 \geq 0$ 
93     c = np.array([3.0, 5.0])
94     A = np.array([[1.0, 2.0], [3.0, 2.0]])
95     b = np.array([8.0, 12.0])
96
97     x_opt, z_opt, hist = simplex_method(c, A, b)
98     print(f"最优解:  $x_1 = \{x\_opt[0]:.4f\}$ ,  $x_2 = \{x\_opt[1]:.4f\}$ ")
99     print(f"最优值:  $z = \{z\_opt:.4f\}$ ")
100    print(f"迭代次数:  $\{len(hist)\}$ ")
101    for i, (basis, nonbasis, z_val) in enumerate(hist):
102        print(f"    Iter {i}: basis={basis}, z={z_val:.4f}")

```

2.4 大 M 法

2.4.1 大 M 法的基本思想

当 LP 没有明显的初始 BFS 时（例如约束中含有 $\geq$ 或 $=$ 类型），需要引入人工变量构造初始基。大 M 法通过在目标函数中给人工变量一个很大的惩罚系数 $-M$ （最大化问题）来迫使人工变量最终离开基。

人工变量的引入

- $\leq$ 约束：加松弛变量（天然基变量）
- $=$ 约束：加人工变量 $a_i \geq 0$
- $\geq$ 约束：减剩余变量 $s_i \geq 0$ ，再加人工变量 $a_i \geq 0$

定义 2.4.1 (大 M 法的目标函数). 对于最大化问题，修改目标函数为：

$$z = \sum_{j=1}^n c_j x_j - \sum_i M a_i$$

其中 M 是一个足够大的正数。

大 M 法算法

1. 引入松弛变量、剩余变量和人工变量，将问题转化为标准型
2. 在目标函数中给人工变量系数 $-M$
3. 用单纯形法求解
4. 若最终基中含有人工变量（取值非零），则原问题无可行解
5. 若所有人工变量都离开基，则得到原问题最优解

例题 2.4.1 (大 M 法——迭代两次). 求解： $\max z = 3x_1 + 2x_2$

$$2x_1 + x_2 \geq 4$$

$$x_1 + 2x_2 \geq 3$$

$$x_1, x_2 \geq 0$$

Step 1: 引入变量

两个 $\geq$ 约束，各减剩余变量 s_1, s_2 ，再加人工变量 a_1, a_2 ：

$$2x_1 + x_2 - s_1 + a_1 = 4$$

$$x_1 + 2x_2 - s_2 + a_2 = 3$$

目标函数: $z = 3x_1 + 2x_2 + 0s_1 + 0s_2 - Ma_1 - Ma_2$

初始单纯形表 (需消去人工变量在 z 行):

将约束相加: $(2x_1 + x_2 - s_1 + a_1) + (x_1 + 2x_2 - s_2 + a_2) = 4 + 3$

z 行: $3x_1 + 2x_2 - M(a_1 + a_2) = 3x_1 + 2x_2 - M(7 - 3x_1 - 3x_2 + s_1 + s_2)$

z 行系数: $(3 + 3M, 2 + 3M, -M, -M, 0)$

	x_1	x_2	s_1	s_2	RHS
z	$-(3 + 3M)$	$-(2 + 3M)$	M	M	$-7M$
a_1	2	1	-1	0	4
a_2	1	2	0	-1	3

第 1 次迭代:

检验数: $-(3 + 3M), -(2 + 3M), M, M$ 。 x_1 入基 (最负)。

比值: $\min\{4/2, 3/1\} = \min\{2, 3\} = 2$ 。 a_1 出基, 枢轴元 = 2。

枢轴运算后:

	x_1	x_2	s_1	s_2	RHS
z	0	$-\frac{1+3M}{2}$	$-\frac{3}{2}$	M	$6 - 3M$
x_1	1	0.5	-0.5	0	2
a_2	0	1.5	0.5	-1	1

第 2 次迭代:

检验数: $0, -(1 + 3M)/2, -3/2, M$ 。 x_2 入基。

比值: $\min\{2/0.5, 1/1.5\} = \min\{4, 2/3\} = 2/3$ 。 a_2 出基。

枢轴运算后 (人工变量全部出基):

	x_1	x_2	s_1	s_2	RHS
z	0	0	$-4/3$	$-1/3$	$19/3$
x_1	1	0	$-2/3$	$1/3$	$5/3$
x_2	0	1	$1/3$	$-2/3$	$2/3$

检验数全部非负, 最优! $x_1^* = 5/3, x_2^* = 2/3, z^* = 19/3 \approx 6.33$ 。

2.4.2 大 M 法的 Python 实现

大 M 法实现

```
1 import numpy as np
2
3 def bigM_method(c, A, b, constraint_types,
4                 maximize=True, M=1e6, max_iter=100):
5     """
6     大M法求解一般形式LP
7
8     Parameters:
9         c: 目标函数系数 (n,)
10        A: 约束矩阵 (m, n)
11        b: 右端项 (m,)
12        constraint_types: 约束类型列表 ('<=', '>=', '=')
13        maximize: True为最大化
14        M: 大M惩罚系数
15
16    Returns:
17        x_opt: 最优解 (仅决策变量)
18        z_opt: 最优值
19        status: 'optimal', 'infeasible', 'unbounded'
20    """
21    m, n = A.shape
22    if not maximize:
23        c = -c
24
25    A_aug = A.copy()
26    c_aug = c.copy()
27    basis = []
28    var_types = ['x'] * n # 变量类型标记
29
30    slack_count = 0
31    surplus_count = 0
32    artif_count = 0
33
34    for i in range(m):
35        ct = constraint_types[i]
36        new_cols = np.zeros((m, 0))
37
```

```

38     if ct == '<=':
39         s = np.zeros(m)
40         s[i] = 1
41         new_cols = s.reshape(-1, 1)
42         c_aug = np.append(c_aug, 0)
43         basis.append(n + slack_count)
44         slack_count += 1
45         var_types.append('s')
46
47     elif ct == '>=':
48         s = np.zeros(m)
49         s[i] = -1
50         a = np.zeros(m)
51         a[i] = 1
52         new_cols = np.column_stack([s, a])
53         c_aug = np.append(c_aug, [0, -M])
54         basis.append(n + slack_count + surplus_count
55                     + artif_count + 1)
56         surplus_count += 1
57         artif_count += 1
58         var_types.extend(['s', 'a'])
59
60     elif ct == '=':
61         a = np.zeros(m)
62         a[i] = 1
63         new_cols = a.reshape(-1, 1)
64         c_aug = np.append(c_aug, -M)
65         basis.append(n + slack_count + surplus_count
66                     + artif_count)
67         artif_count += 1
68         var_types.append('a')
69
70     if new_cols.size > 0:
71         A_aug = np.hstack([A_aug, new_cols])
72
73     total_vars = A_aug.shape[1]
74     nonbasis = [j for j in range(total_vars) if j not in basis]
75
76     # 构建单纯形表

```

```

77     T = np.zeros((m + 1, total_vars + 1))
78     T[1:, :total_vars] = A_aug
79     T[1:, -1] = b
80
81     # 消去人工变量在 z 行的系数
82     for j in range(total_vars):
83         T[0, j] = sum(c_aug[basis[i]] * T[i+1, j]
84                       for i in range(m)) - c_aug[j]
85     T[0, -1] = sum(c_aug[basis[i]] * T[i+1, -1]
86                   for i in range(m))
87
88     # 单纯形迭代
89     for _ in range(max_iter):
90         if all(T[0, j] >= -1e-10 for j in nonbasis):
91             break
92
93         entering = min(j for j in nonbasis
94                       if T[0, j] < -1e-10)
95         col = T[1:, entering]
96
97         if all(col <= 1e-10):
98             return None, np.inf, 'unbounded'
99
100        ratios = [(T[i+1, -1] / col[i], i)
101                  for i in range(m) if col[i] > 1e-10]
102        leaving_idx = min(ratios)[1]
103
104        pivot = T[leaving_idx + 1, entering]
105        T[leaving_idx + 1] /= pivot
106        for i in range(m + 1):
107            if i != leaving_idx + 1:
108                T[i] -= T[i, entering] * T[leaving_idx + 1]
109
110        basis[leaving_idx] = entering
111        nonbasis = [j for j in range(total_vars)
112                   if j not in basis]
113
114    # 检查人工变量
115    for bi in basis:

```

```

116         if var_types[bi] == 'a' and abs(T[basis.index(bi)+1,-1]) >
            1e-6:
117             return None, None, 'infeasible'
118
119     x = np.zeros(total_vars)
120     for i, bi in enumerate(basis):
121         x[bi] = T[i + 1, -1]
122
123     z = T[0, -1]
124     if not maximize:
125         z = -z
126
127     return x[:n], z, 'optimal'
128
129 # 示例
130 # max 3x1 + 2x2, s.t. 2x1+x2 >= 4, x1+2x2 >= 3, x1,x2 >= 0
131 c = np.array([3.0, 2.0])
132 A = np.array([[2.0, 1.0], [1.0, 2.0]])
133 b = np.array([4.0, 3.0])
134
135 x_opt, z_opt, status = bigM_method(c, A, b, ['>=', '>='])
136 print(f"状态: {status}")
137 print(f"最优解: x1={x_opt[0]:.4f}, x2={x_opt[1]:.4f}")
138 print(f"最优值: z={z_opt:.4f}")

```

2.5 两阶段法

2.5.1 两阶段法的基本思想

大 M 法需要选取足够大的 M ，但 M 过大会导致数值不稳定。两阶段法是更稳健的替代方案。

两阶段法

第一阶段：最小化人工变量之和 $w = \sum a_i$ ，判断原问题是否有可行解。若 $w^* = 0$ ，则所有人工变量离开基，得到初始 BFS。

第二阶段：以第一阶段得到的 BFS 为初始解，用单纯形法求解原问题。

2.5.2 第一阶段：寻找初始 BFS

第一阶段算法

1. 引入人工变量，构造辅助 LP: $\min w = \sum a_i$
2. 用单纯形法求解辅助 LP
3. 若 $w^* > 0$ ，原问题无可行解
4. 若 $w^* = 0$ ，人工变量全部出基，当前 BFS 作为第二阶段初始解

例题 2.5.1 (两阶段法——迭代两次 (第一阶段)). 求解: $\max z = 3x_1 + 5x_2$

$$x_1 + 3x_2 \geq 3$$

$$2x_1 + x_2 \geq 2$$

$$x_1, x_2 \geq 0$$

第一阶段：构造辅助问题

引入剩余变量 $s_1, s_2 \geq 0$ 和人工变量 $a_1, a_2 \geq 0$:

$$x_1 + 3x_2 - s_1 + a_1 = 3$$

$$2x_1 + x_2 - s_2 + a_2 = 2$$

$$\min w = a_1 + a_2$$

消去人工变量在 w 行的系数:

$$\text{约束相加: } 3x_1 + 4x_2 - s_1 - s_2 + a_1 + a_2 = 5$$

$$w = 5 - 3x_1 - 4x_2 + s_1 + s_2$$

初始单纯形表:

	x_1	x_2	s_1	s_2	a_1	a_2	RHS
w	3	4	-1	-1	0	0	5
a_1	1	3	-1	0	1	0	3
a_2	2	1	0	-1	0	1	2

第 1 次迭代:

w 行检验数: 3, 4, -1, -1. x_2 入基 (最小检验数方向, 对应最大正系数)。

比值: $\min\{3/3, 2/1\} = \min\{1, 2\} = 1$. a_1 出基。

枢轴运算 (枢轴元 = 3):

	x_1	x_2	s_1	s_2	a_1	a_2	RHS
w	5/3	0	1/3	-1	-4/3	0	1
x_2	1/3	1	-1/3	0	1/3	0	1
a_2	5/3	0	1/3	-1	-1/3	1	1

第 2 次迭代:

检验数: $5/3, 0, 1/3, -1$ 。 s_2 入基 (注意: w 行系数为 -1 表示还可减小 w)。

比值: 第 1 行 s_2 系数 $= 0$ (跳过), 第 2 行 $1/(-1) < 0$ (跳过)。

实际上检验数 $-1 < 0$ 表示 s_2 入基可减小 w 。比值检验:

第 2 行 a_2 : s_2 系数 $= -1 < 0$, 跳过。

无正系数! 这意味着 w 可以趋于 $-\infty$? 不对, 这是最小化问题。

实际上 w 行系数为负才表示可减小。 s_2 系数 -1 , 入基后:

重新计算: s_2 入基列 $(0, -1)^T$, 无正元素, 说明辅助 LP 无界。

但 $w \geq 0$, w 不可能无界。说明 $w = 1$ 已为最优 (人工变量无法全部出基)。

实际上仔细看: w 行还有 x_1 系数 $5/3 > 0$, s_1 系数 $1/3 > 0$ 。

若 x_1 入基: 比值 $\min\{1/(1/3), 1/(5/3)\} = \min\{3, 3/5\} = 3/5$ 。

继续迭代, 最终可得 $w^* = 0$, 人工变量全部出基。

2.5.3 第二阶段: 求解原问题

例题 2.5.2 (第二阶段). 以第一阶段得到的 BFS 为初始解, 去掉人工变量列, 恢复原始目标函数。

假设第一阶段结束后的表为:

	x_1	x_2	s_1	s_2	RHS
z	-3	-5	0	0	0
x_2	0.2	1	-0.4	0.2	0.6
x_1	1	0	0.2	-0.6	0.8

这是原问题的初始单纯形表。继续用单纯形法求解, 直到最优。

2.5.4 两阶段法的 Python 实现

两阶段法实现

```
1 import numpy as np
2
3 def two_phase_method(c, A, b, constraint_types,
4                     maximize=True, max_iter=100):
5     """
6     两阶段法求解一般形式LP
7
8     Parameters:
9         c: 目标函数系数 (n,)
10        A: 约束矩阵 (m, n)
11        b: 右端项 (m,)
12        constraint_types: 约束类型 ('<=', '>=', '=')
13        maximize: True为最大化
14
15    Returns:
16        x_opt, z_opt, status
17    """
18    m, n = A.shape
19    if not maximize:
20        c = -c
21
22    # ===== 第一阶段：找初始BFS =====
23    A1 = A.copy()
24    basis = []
25    artif_indices = []
26    var_count = n
27
28    for i in range(m):
29        ct = constraint_types[i]
30        if ct == '<=':
31            new_col = np.zeros((m, 1))
32            new_col[i] = 1
33            A1 = np.hstack([A1, new_col])
34            basis.append(var_count)
35            var_count += 1
36        elif ct == '>=':
37            s_col = np.zeros((m, 1))
```

```

38         s_col[i] = -1
39         a_col = np.zeros((m, 1))
40         a_col[i] = 1
41         A1 = np.hstack([A1, s_col, a_col])
42         basis.append(var_count + 1)
43         artif_indices.append(var_count + 1)
44         var_count += 2
45     elif ct == '=':
46         a_col = np.zeros((m, 1))
47         a_col[i] = 1
48         A1 = np.hstack([A1, a_col])
49         basis.append(var_count)
50         artif_indices.append(var_count)
51         var_count += 1
52
53     total1 = A1.shape[1]
54
55     # Phase 1 objective: min sum of artificial vars
56     c1 = np.zeros(total1)
57     for ai in artif_indices:
58         c1[ai] = 1
59
60     T1 = np.zeros((m + 1, total1 + 1))
61     T1[1:, :total1] = A1
62     T1[1:, -1] = b
63
64     # Eliminate artificial variables from obj row
65     for j in range(total1):
66         T1[0, j] = sum(T1[i+1, j] for i in range(m)
67                        if (basis[i] in artif_indices)) - c1[j]
68     T1[0, -1] = sum(T1[i+1, -1] for i in range(m)
69                    if basis[i] in artif_indices)
70
71     nonbasis1 = [j for j in range(total1) if j not in basis]
72
73     # Phase 1 simplex (minimize, so reverse sign)
74     for _ in range(max_iter):
75         if all(T1[0, j] <= 1e-10 for j in nonbasis1):
76             break

```

```

77     entering = min((T1[0, j], j) for j in nonbasis1
78                     if T1[0, j] > 1e-10)[1]
79     col = T1[1:, entering]
80     if all(col <= 1e-10):
81         return None, None, 'unbounded'
82     ratios = [(T1[i+1, -1] / col[i], i)
83               for i in range(m) if col[i] > 1e-10]
84     leaving_idx = min(ratios)[1]
85     pivot = T1[leaving_idx + 1, entering]
86     T1[leaving_idx + 1] /= pivot
87     for i in range(m + 1):
88         if i != leaving_idx + 1:
89             T1[i] -= T1[i, entering] * T1[leaving_idx + 1]
90     basis[leaving_idx] = entering
91     nonbasis1 = [j for j in range(total1) if j not in basis]
92
93     if abs(T1[0, -1]) > 1e-6:
94         return None, None, 'infeasible'
95
96     # Remove artificial variables
97     keep = [j for j in range(total1) if j not in artif_indices]
98     A2 = A1[:, keep]
99     c2 = np.zeros(len(keep))
100    for i, j in enumerate(keep):
101        if j < n:
102            c2[i] = c[j]
103
104    # Build Phase 2 tableau
105    basis2 = []
106    for bi in basis:
107        if bi not in artif_indices:
108            basis2.append(keep.index(bi) if bi in keep else None)
109        else:
110            # Replace with non-artificial variable
111            for j in keep:
112                col_j = A2[:, j]
113                if abs(col_j[list(basis).index(bi)] - 1) < 1e-10:
114                    basis2.append(keep.index(j))
115                    break

```

```

116
117     return _simplex_solve(c2, A2, b, basis2, maximize)
118
119 def _simplex_solve(c, A, b, basis_init, maximize):
120     """内部单纯形求解"""
121     m, n = A.shape
122     basis = list(basis_init)
123     T = np.zeros((m + 1, n + 1))
124     T[1:, :n] = A
125     T[1:, -1] = b
126
127     for _ in range(100):
128         nonbasis = [j for j in range(n) if j not in basis]
129         cB = c[basis]
130         for j in nonbasis:
131             T[0, j] = cB @ T[1:, j] - c[j]
132         T[0, -1] = cB @ T[1:, -1]
133
134         if all(T[0, j] >= -1e-10 for j in nonbasis):
135             break
136         entering = min(j for j in nonbasis if T[0, j] < -1e-10)
137         col = T[1:, entering]
138         if all(col <= 1e-10):
139             return None, np.inf, 'unbounded'
140         ratios = [(T[i+1, -1] / col[i], i)
141                  for i in range(m) if col[i] > 1e-10]
142         leaving_idx = min(ratios)[1]
143         pivot = T[leaving_idx + 1, entering]
144         T[leaving_idx + 1] /= pivot
145         for i in range(m + 1):
146             if i != leaving_idx + 1:
147                 T[i] -= T[i, entering] * T[leaving_idx + 1]
148         basis[leaving_idx] = entering
149
150     x = np.zeros(n)
151     for i, bi in enumerate(basis):
152         x[bi] = T[i + 1, -1]
153     z = T[0, -1]
154     if not maximize:

```

```

155     z = -z
156     return x[:len(c)], z, 'optimal'

```

2.6 对偶理论

2.6.1 对偶问题的构造

定理 2.6.1 (对称形式的对偶). 原问题 (P):

$$\begin{aligned}
 \max \quad & z = c^T x \\
 \text{s.t.} \quad & Ax \leq b \\
 & x \geq 0
 \end{aligned}$$

对偶问题 (D):

$$\begin{aligned}
 \min \quad & w = b^T y \\
 \text{s.t.} \quad & A^T y \geq c \\
 & y \geq 0
 \end{aligned}$$

原问题与对偶问题的对应关系

原问题 (最大化)	对偶问题 (最小化)
第 i 个约束 $\leq$	第 i 个变量 $y_i \geq 0$
第 i 个约束 $=$	第 i 个变量 y_i 自由
第 i 个约束 $\geq$	第 i 个变量 $y_i \leq 0$
第 j 个变量 $x_j \geq 0$	第 j 个约束 $\geq$
第 j 个变量 x_j 自由	第 j 个约束 $=$
第 j 个变量 $x_j \leq 0$	第 j 个约束 $\leq$

例题 2.6.1 (写对偶问题). 原问题: $\max z = 3x_1 + 5x_2$

$$\begin{aligned}
 x_1 + 2x_2 &\leq 8 \\
 3x_1 + 2x_2 &\leq 12 \\
 x_1, x_2 &\geq 0
 \end{aligned}$$

对偶问题: $\min w = 8y_1 + 12y_2$

$$y_1 + 3y_2 \geq 3$$

$$2y_1 + 2y_2 \geq 5$$

$$y_1, y_2 \geq 0$$

2.6.2 弱对偶定理与强对偶定理

定理 2.6.2 (弱对偶定理). 设 x 是原问题可行解, y 是对偶问题可行解, 则:

$$c^T x \leq b^T y$$

即: 原问题目标值 $\leq$ 对偶问题目标值。

证明: $c^T x \leq (A^T y)^T x = y^T A x \leq y^T b = b^T y$ 。

定理 2.6.3 (强对偶定理). 若原问题有最优解, 则对偶问题也有最优解, 且最优值相等:

$$\max c^T x = \min b^T y$$

推论 2.6.1. LP 问题的可能情形:

1. 原问题和对偶问题都有最优解, 且最优值相等
2. 原问题无界, 对偶问题不可行
3. 原问题不可行, 对偶问题无界
4. 原问题和对偶问题都不可行

2.6.3 互补松弛定理

定理 2.6.4 (互补松弛条件). 设 x^* 和 y^* 分别是原问题和对偶问题的可行解, 则它们都是最优解当且仅当:

$$y_i^* \cdot (b_i - a_i^T x^*) = 0, \quad i = 1, \dots, m$$

$$x_j^* \cdot (a_j^T y^* - c_j) = 0, \quad j = 1, \dots, n$$

其中 a_i 是 A 的第 i 行, a_j 是 A 的第 j 列。

例题 2.6.2 (用互补松弛求对偶最优解). 原问题最优解 $x^* = (2, 3)$, $z^* = 21$ 。

约束 1: $2 + 2 \cdot 3 = 8 = b_1$ (积极), 约束 2: $3 \cdot 2 + 2 \cdot 3 = 12 = b_2$ (积极)

由互补松弛: $y_1^* > 0, y_2^* > 0$, 且 $x_1^* = 2 > 0, x_2^* = 3 > 0$

由 $x_1^* > 0$: $y_1^* + 3y_2^* = 3$

由 $x_2^* > 0$: $2y_1^* + 2y_2^* = 5$

解得: $y_1^* = 9/4 = 2.25$, $y_2^* = 1/4 = 0.25$

验证: $w = 8(2.25) + 12(0.25) = 18 + 3 = 21 = z^*$ (验证一致)

2.6.4 经济解释：影子价格

定义 2.6.1 (影子价格). 对偶变量 y_i^* 表示第 i 种资源的影子价格 (Shadow Price), 即该资源增加 1 单位时最优目标值的增量。

例题 2.6.3 (影子价格). 在 $x^* = (2, 3)$, $z^* = 21$ 的例子中:

- $y_1^* = 2.25$: 第 1 种资源 (约束 1 RHS) 的影子价格为 2.25
- $y_2^* = 0.25$: 第 2 种资源的影子价格为 0.25

若第 1 种资源从 8 增加到 9: $z^* \approx 21 + 2.25 = 23.25$

例题 2.6.4 (对偶的对偶是原问题). 以标准最小化问题为例:

$$(P) \min c^T x, \quad Ax = b, \quad x \geq 0$$

其对偶为

$$(D) \max b^T y, \quad A^T y \leq c, \quad y \text{ 自由}$$

再对 (D) 取对偶: 约束 $A^T y \leq c$ 对应非负对偶变量 $x \geq 0$, y 自由变量对应等式约束 $Ax = b$, 目标方向恢复为最小化, 得到

$$\min c^T x, \quad Ax = b, \quad x \geq 0$$

正是原问题 (P)。

本质原因: 线性规划中约束类型与变量符号在对偶变换下成对互换——等式约束 $\leftrightarrow$ 自由变量, 非负变量 $\leftrightarrow$ 不等式约束, 再对偶一次关系恢复。

例题 2.6.5 (原问题与对偶问题的联系与区别). 联系:

- 弱对偶: 任一原可行解 x 和对偶可行解 y 满足 $b^T y \leq c^T x$ (对最小化原问题)
- 强对偶: 若原问题存在有限最优解, 则对偶也有最优解, 且 $c^T x^* = b^T y^*$
- 互补松弛: $x_i(c_i - a_i^T y) = 0$, 正的原变量对应紧的对偶约束

区别:

方面	原问题	对偶问题
决策变量	x	y
约束数量	A 的行数	A 的列数
经济解释	资源使用方案	资源影子价格
目标方向	min 或 max	与原问题相反

例题 2.6.6 (线性规划基本概念判断). 判断下列说法是否正确。

(a) "有界可行集的 LP 中, 每个最优解必定是最优 BFS 。"

错误。反例: $\min 0$, 约束 $0 \leq x \leq 1$ 。可行集有界, 任意 $x \in [0, 1]$ 都最优, 但 $x = 1/2$ 不是 BFS 。正确说法是: 至少存在一个最优 BFS 。

(b) " LP 有多个解则必有无穷多个解。"

正确。若 x_1, x_2 是两个不同最优解, 则 $\theta x_1 + (1 - \theta)x_2$ ($\theta \in [0, 1]$) 仍可行且目标值相同, 有无穷多个。

(c) "标准型 LP ($m \times n$, 行满秩) 每个最优解上最多有 m 个正分量。"

错误。 BFS 至多有 m 个正分量, 但一般最优解不一定是 BFS 。反例: $\min 0, x_1 + x_2 + x_3 = 1, x \geq 0, m = 1$, 但 $(1/3, 1/3, 1/3)$ 有 $3 > m$ 个正分量。

(d) "标准型可行域的每个顶点至多有 $n - m$ 个相邻顶点。"

一般错误。非退化情形下成立, 但退化顶点可能有更多相邻顶点。缺少非退化条件。

2.7 对偶单纯形法

2.7.1 对偶单纯形法的基本思想

原始单纯形法保持原始可行性 ($x_B \geq 0$), 迭代直到对偶可行性 ($\sigma_j \geq 0$) 满足。

对偶单纯形法恰恰相反: 保持对偶可行性 ($\sigma_j \geq 0$), 迭代直到原始可行性 ($x_B \geq 0$) 满足。

对偶单纯形法的适用场景

- 已获得一个对偶可行解 ($\sigma_j \geq 0$), 但原始不可行
- 灵敏度分析中约束条件变化后需要重新求解
- 添加新约束后当前最优解不再可行

2.7.2 对偶单纯形法算法

Algorithm 2 对偶单纯形法

Require: 初始对偶可行的基 ($\sigma_j \geq 0$)

```

1: while true do
2:   Step 1: 原始可行性检验
3:   if 所有  $b_i \geq 0$  then
4:     当前解最优, 停止
5:   end if
6:   选择出基变量:  $r = \arg \min_i \{b_i\}$  (最负的 RHS)
7:   Step 2: 对偶可行性检验
8:   if 出基行所有系数  $a_{rj} \geq 0$  then
9:     对偶问题无界 (原问题不可行), 停止
10:  end if
11:  Step 3: 最小比值检验
12:  入基变量:  $k = \arg \min_j \left\{ \frac{\sigma_j}{|a_{rj}|} : a_{rj} < 0 \right\}$ 
13:  Step 4: 枢轴运算
14:  以  $a_{rk}$  为枢轴元, 高斯消元
15: end while
  
```

对偶单纯形法与原始单纯形法的对比

特性	原始单纯形法	对偶单纯形法
始终保持	原始可行性	对偶可行性
迭代目标	达到对偶可行性	达到原始可行性
选择入基	最小检验数	最小比值
选择出基	最小比值	最负 RHS

2.7.3 例题：对偶单纯形法迭代两次

例题 2.7.1 (对偶单纯形法). $\min z = 2x_1 + 3x_2 + 4x_3$

$$x_1 + 2x_2 + x_3 \geq 3$$

$$2x_1 - x_2 + 3x_3 \geq 4$$

$$x_1, x_2, x_3 \geq 0$$

转化为标准型（乘以 -1 使约束为 $\leq$ ）：

$$\begin{aligned}\min z &= 2x_1 + 3x_2 + 4x_3 \\ -x_1 - 2x_2 - x_3 + s_1 &= -3 \\ -2x_1 + x_2 - 3x_3 + s_2 &= -4 \\ x_1, x_2, x_3, s_1, s_2 &\geq 0\end{aligned}$$

基变量 s_1, s_2 , RHS $(-3, -4)$ 为负，原始不可行。

检验数： $\sigma_1 = 2, \sigma_2 = 3, \sigma_3 = 4, \sigma_4 = 0, \sigma_5 = 0 \geq 0$ ，对偶可行。

初始单纯形表：

	x_1	x_2	x_3	s_1	s_2	RHS
z	2	3	4	0	0	0
s_1	-1	-2	-1	1	0	-3
s_2	-2	1	-3	0	1	-4

第 1 次迭代：

Step 1: RHS 最小为 -4 （第 2 行）， s_2 出基。

Step 2: 出基行系数 $(-2, 1, -3, 0, 1)$ ，有负系数，不停止。

Step 3: 负系数列的比值：

- x_1 : $\sigma_1/|-2| = 2/2 = 1$
- x_3 : $\sigma_3/|-3| = 4/3 \approx 1.33$

最小比值对应 x_1 ，入基。枢轴元 $a_{21} = -2$ 。

枢轴运算后：

	x_1	x_2	x_3	s_1	s_2	RHS
z	0	4	1	0	1	-4
s_1	0	-2.5	0.5	1	-0.5	-1
x_1	1	-0.5	1.5	0	-0.5	2

第 2 次迭代：

Step 1: RHS 最小为 -1 （第 1 行）， s_1 出基。

Step 2: 出基行 $(0, -2.5, 0.5, 1, -0.5)$ ，有负系数。

Step 3: 负系数列的比值：

- x_2 : $\sigma_2/|-2.5| = 4/2.5 = 1.6$

x_2 入基。枢轴元 $a_{12} = -2.5$ 。

枢轴运算后：

	x_1	x_2	x_3	s_1	s_2	RHS
z	0	0	1.8	1.6	0.2	-5.6
x_2	0	1	-0.2	-0.4	0.2	0.4
x_1	1	0	1.4	-0.2	-0.4	2.2

RHS 全部非负，最优！ $x_1^* = 2.2, x_2^* = 0.4, x_3^* = 0, z^* = 5.6$ 。

2.7.4 Python 代码

对偶单纯形法

```

1 def dual_simplex(c, A, b, maximize=True, max_iter=100):
2     """
3     对偶单纯形法求解标准型LP:  $\min c^T x$  s.t.  $Ax = b, x \geq 0$ 
4     要求初始基对偶可行 (检验数  $\geq 0$ )
5
6     Parameters:
7         c: 目标函数系数
8         A: 约束矩阵
9         b: 右端项 (可有负值)
10        maximize: True为最大化
11
12    Returns:
13        x_opt, z_opt, status
14    """
15    m, n = A.shape
16    if maximize:
17        c = -c
18
19    # 初始基: 松弛/剩余变量
20    basis = list(range(n, n + m))
21    A_aug = np.hstack([A, np.eye(m)])
22    c_aug = np.hstack([c, np.zeros(m)])
23
24    T = np.zeros((m + 1, n + m + 1))
25    T[1:, :n+m] = A_aug
26    T[1:, -1] = b

```

```

27
28     for _ in range(max_iter):
29         nonbasis = [j for j in range(n+m) if j not in basis]
30         cB = c_aug[basis]
31         for j in nonbasis:
32             T[0, j] = cB @ T[1:, j] - c_aug[j]
33         T[0, -1] = cB @ T[1:, -1]
34
35         # Step 1: 原始可行性检验
36         rhs = T[1:, -1]
37         if all(rhs >= -1e-10):
38             break
39         r = np.argmin(rhs)
40
41         # Step 2: 对偶可行性检验
42         row = T[r + 1, :n+m]
43         if all(row >= -1e-10):
44             return None, None, 'infeasible'
45
46         # Step 3: 最小比值检验
47         ratios = []
48         for j in nonbasis:
49             if row[j] < -1e-10:
50                 ratios.append((T[0, j] / abs(row[j]), j))
51         entering = min(ratios)[1]
52
53         # 枢轴运算
54         pivot = T[r + 1, entering]
55         T[r + 1] /= pivot
56         for i in range(m + 1):
57             if i != r + 1:
58                 T[i] -= T[i, entering] * T[r + 1]
59         basis[r] = entering
60
61     x = np.zeros(n)
62     for i, bi in enumerate(basis):
63         if bi < n:
64             x[bi] = T[i + 1, -1]
65

```

```

66     z = T[0, -1]
67     if maximize:
68         z = -z
69
70     return x, z, 'optimal'

```

2.8 灵敏度分析

2.8.1 灵敏度分析概述

灵敏度分析 (Sensitivity Analysis) 研究 LP 最优解如何随参数变化而改变。主要分析以下四种变化：

灵敏度分析的四种情形

1. 目标函数系数变化： c_j 变化对最优解的影响
2. 右端项变化： b_i 变化对最优值的影响（影子价格）
3. 增加新变量：引入新产品/新决策变量
4. 增加新约束：引入新限制条件

2.8.2 目标函数系数的变化

设当前最优基为 B ，基变量 $x_B = B^{-1}b \geq 0$ 。

若 c_k （非基变量系数）变化为 $c_k + \Delta c_k$ ：

新检验数： $\hat{\sigma}_k = \sigma_k - \Delta c_k$

定理 2.8.1 (c_k 的变化范围). 为使当前基保持最优，需要所有检验数非负：

$$\Delta c_k \leq \sigma_k$$

即 c_k 最多增加到 $c_k + \sigma_k$ 。

例题 2.8.1 (c_k 灵敏度分析). $\max z = 3x_1 + 5x_2$ ，最优表：

	x_1	x_2	s_1	s_2	RHS
z	0	0	2.25	0.25	21
x_2	0	1	0.75	-0.25	3
x_1	1	0	-0.5	0.5	2

非基变量 s_1 的系数变化: $c_{s_1} = 0$ 变为 Δc_{s_1}

检验数 $\sigma_{s_1} = 2.25$, 变化后: $\hat{\sigma}_{s_1} = 2.25 - \Delta c_{s_1} \geq 0$

$\Delta c_{s_1} \leq 2.25$ 。 s_1 的系数最多增加到 2.25。

基变量 x_1 的系数变化: $c_1 = 3$ 变为 $3 + \Delta c_1$

需要重新计算所有检验数: $\sigma = c_B^T B^{-1} A - c$

c_B 变为 $(5, 3 + \Delta c_1)$, 需要所有 $\sigma_j \geq 0$ 。

详细计算:

$$\sigma_{s_1} = 2.25 - 0.75\Delta c_1 \geq 0 \Rightarrow \Delta c_1 \leq 3$$

$$\sigma_{s_2} = 0.25 + 0.25\Delta c_1 \geq 0 \Rightarrow \Delta c_1 \geq -1$$

所以 $-1 \leq \Delta c_1 \leq 3$, 即 $2 \leq c_1 \leq 6$ 。

2.8.3 右端项的变化

定理 2.8.2 (b_i 的变化范围). 设 b_i 变化为 $b_i + \Delta b_i$, 新的基解:

$$\hat{x}_B = B^{-1}(b + \Delta b) = x_B + B^{-1}\Delta b$$

为使当前基保持可行:

$$\hat{x}_B \geq 0 \Rightarrow x_B + B^{-1}\Delta b \geq 0$$

例题 2.8.2 (b_i 灵敏度分析). $b_1 = 8$ 变为 $8 + \Delta b_1$, $b_2 = 12$ 不变。

$$B^{-1} = \begin{bmatrix} 0.75 & -0.25 \\ -0.5 & 0.5 \end{bmatrix}$$

新的基解:

$$\hat{x}_B = \begin{bmatrix} 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.75 \\ -0.5 \end{bmatrix} \Delta b_1 \geq 0$$

$$3 + 0.75\Delta b_1 \geq 0 \Rightarrow \Delta b_1 \geq -4$$

$$2 - 0.5\Delta b_1 \geq 0 \Rightarrow \Delta b_1 \leq 4$$

所以 $-4 \leq \Delta b_1 \leq 4$, 即 $4 \leq b_1 \leq 12$ 。

在此范围内, 影子价格 $y_1^* = 2.25$ 有效, 最优值变化 $z^* = 21 + 2.25\Delta b_1$ 。

2.8.4 增加新变量

当引入新产品 (新变量 x_{new}) 时, 需要检查其经济性。

定理 2.8.3 (新变量的检验). 设新变量 x_{n+1} 的系数为 c_{n+1} , 约束列为 A_{n+1} 。若:

$$\sigma_{n+1} = c_B^T B^{-1} A_{n+1} - c_{n+1} \geq 0$$

则当前基仍最优, 不应生产该产品。

若 $\sigma_{n+1} < 0$, 则 x_{n+1} 入基可改善目标。

例题 2.8.3 (增加新变量). 原问题: $\max z = 3x_1 + 5x_2$

新产品 x_3 : 利润 $c_3 = 4$, 消耗资源 $A_3 = (1, 2)^T$ 。

检验数: $\sigma_3 = c_B^T B^{-1} A_3 - 4 = (2.25, 0.25) \cdot (1, 2) - 4 = 2.25 + 0.5 - 4 = -1.25 < 0$

应该生产新产品! x_3 入基。

2.8.5 增加新约束

增加新约束后, 当前最优解可能不再可行。

增加新约束的处理

1. 将新约束加入最优单纯形表
2. 若当前最优解满足新约束, 无需处理
3. 若不满足, 加入松弛变量使约束为等式
4. 用对偶单纯形法继续求解

例题 2.8.4 (增加新约束). 原问题最优解 $x^* = (2, 3)$, $z^* = 21$ 。

增加约束: $x_1 + x_2 \leq 4$

检验: $2 + 3 = 5 > 4$, 不满足!

加入松弛变量: $x_1 + x_2 + s_3 = 4$, 当前 $s_3 = 4 - 5 = -1 < 0$ 。

用对偶单纯形法, s_3 出基, 迭代求解新的最优解。

2.8.6 Python 代码

灵敏度分析

```

1 def sensitivity_analysis(c, A, b, cB_indices):
2     """
3     对当前最优基进行灵敏度分析
4
5     Parameters:
6         c: 目标函数系数
7         A: 约束矩阵
8         b: 右端项
9         cB_indices: 基变量下标列表
10
11     Returns:
12         分析结果字典
13     """
14     m, n = A.shape

```

```

15     B = A[:, cB_indices]
16     B_inv = np.linalg.inv(B)
17
18     cB = c[cB_indices]
19     xB = B_inv @ b
20
21     # 1. 影子价格
22     y = cB @ B_inv
23
24     # 2. b的变化范围
25     b_ranges = []
26     for i in range(m):
27         col = B_inv[:, i]
28         delta_min = -float('inf')
29         delta_max = float('inf')
30         for j in range(m):
31             if col[j] > 0:
32                 delta_max = min(delta_max, xB[j] / col[j])
33             elif col[j] < 0:
34                 delta_min = max(delta_min, xB[j] / col[j])
35         b_ranges.append((delta_min, delta_max))
36
37     # 3. c_k的变化范围 (基变量)
38     c_ranges = []
39     for idx in cB_indices:
40         # 简化为  $\pm \sigma / | \text{系数} |$ 
41         c_ranges.append((-1, 1)) # 需详细计算
42
43     return {
44         'shadow_prices': y,
45         'b_ranges': b_ranges,
46         'c_ranges': c_ranges,
47         'xB': xB
48     }
49
50 # 示例
51 c = np.array([3.0, 5.0])
52 A = np.array([[1.0, 2.0], [3.0, 2.0]])
53 b = np.array([8.0, 12.0])

```

```
54 basis = [1, 0] # x2, x1
55
56 result = sensitivity_analysis(c, A, b, basis)
57 print("影子价格:", result['shadow_prices'])
58 print("当前基解:", result['xB'])
```

2.9 本章小结

2.9.1 线性规划方法体系

表 2.2: 线性规划求解方法总结			
方法	核心思想	适用场景	特点
图解法	画可行域, 移动目标等值线	2 个变量	直观、教学用
单纯形法	在 BFS (极点) 间迭代	一般 LP	最经典、高效
大 M 法	引入人工变量, 大 M 惩罚	无初始 BFS	简单但有数值问题
两阶段法	第一阶段消人工, 第二阶段优化	无初始 BFS	稳健、推荐
对偶单纯形法	保持对偶可行, 迭代到原始可行	灵敏度分析	适合约束变化

2.9.2 重要定理回顾

1. LP 基本定理: 可行域非空 $\Rightarrow$ 存在 BFS; 有最优解 $\Rightarrow$ 存在 BFS 最优解
2. 弱对偶定理: $c^T x \leq b^T y$ (原 $\leq$ 对偶)
3. 强对偶定理: 原问题有最优解 $\Rightarrow$ 对偶有最优解且值相等
4. 互补松弛条件: $y_i^*(b_i - a_i^T x^*) = 0, x_j^*(a_j^T y^* - c_j) = 0$
5. 灵敏度分析: b_i 变化范围 $\Leftrightarrow$ 影子价格有效范围

2.9.3 单纯形表操作速查

表 2.3: 单纯形法操作对照

步骤	原始单纯形法	对偶单纯形法
检验条件	检验数 $\sigma_j \geq 0$?	RHS $b_i \geq 0$?
入基变量	最负检验数	最小比值检验
出基变量	最小比值检验	最负 RHS
枢轴运算	高斯消元	高斯消元

2.9.4 对偶问题构造速查

原问题	对偶问题
max	min
约束 $\leq$	变量 ≥ 0
约束 $=$	变量自由
变量 ≥ 0	约束 $\geq$
变量自由	约束 $=$

2.9.5 学习建议

LP 学习路径

1. 掌握 LP 标准型和转化方法
2. 理解图解法和几何直观
3. 熟练单纯形法的表格计算（重点！）
4. 掌握大 M 法和两阶段法处理无初始 BFS
5. 理解对偶理论（弱对偶、强对偶、互补松弛）
6. 掌握对偶单纯形法和灵敏度分析
7. 能用 Python 实现基本算法

Chapter 3

凸分析与数学模型

3.1 非线性规划的数学模型

引言

非线性规划 (Nonlinear Programming, NLP) 比线性规划困难得多, 主要原因在于:

- 没有统一的数学模型
- 每种方法有其特定的适用范围, 不存在适用于各种 NLP 问题的一般算法

本节通过三个经典问题引入 NLP 的数学建模思想。

3.1.1 选址问题

问题描述

设有 n 个市场, 第 j 个市场的位置为 (a_j, b_j) , 其对某种货物的需求量为 q_j ($j = 1, 2, \dots, n$)。现计划建立 m 个仓库, 第 i 个仓库的存储容量为 c_i ($i = 1, 2, \dots, m$)。试确定仓库的位置, 使各仓库对各市场的运输量与路程乘积之和为最小。

决策变量:

- 第 i 个仓库的位置: (x_i, y_i) ($i = 1, 2, \dots, m$)
- 第 i 个仓库到第 j 个市场的货物供应量: w_{ij}

目标函数 (总运输成本最小):

$$\min \quad f = \sum_{i=1}^m \sum_{j=1}^n w_{ij} \sqrt{(x_i - a_j)^2 + (y_i - b_j)^2}$$

约束条件:

(1) 每个仓库的出货量不超过其容量:

$$\sum_{j=1}^n w_{ij} \leq c_i, \quad i = 1, 2, \dots, m$$

(2) 每个市场的需求被恰好满足:

$$\sum_{i=1}^m w_{ij} = q_j, \quad j = 1, 2, \dots, n$$

(3) 非负约束:

$$w_{ij} \geq 0, \quad \forall i, j$$

例题 3.1.1 (选址问题建模). **问题:** 设有 2 个市场和 1 个仓库。市场 1 位于 $(0, 0)$, 需求量 $q_1 = 5$; 市场 2 位于 $(4, 0)$, 需求量 $q_2 = 3$ 。仓库容量 $c_1 = 10$ 。求仓库最优位置。

解: 设仓库位置为 (x, y) , 运输量为 w_1, w_2 。

数学模型为:

$$\begin{aligned} \min \quad & f = w_1 \sqrt{x^2 + y^2} + w_2 \sqrt{(x-4)^2 + y^2} \\ \text{s.t.} \quad & w_1 + w_2 \leq 10 \\ & w_1 = 5, \quad w_2 = 3 \\ & w_1, w_2 \geq 0 \end{aligned}$$

当 $w_1 = 5, w_2 = 3$ 固定时, 最优点是两点之间的加权费马点。

3.1.2 构件的表面积问题

问题描述

一个半球形与圆柱形相接的构件, 要求在构件体积一定的条件下, 确定构件的尺寸使其表面积最小。

设圆柱形底半径为 r , 高为 h 。

表面积 (目标函数):

$$S = 2\pi r^2 + 2\pi r h + \pi r^2 = 3\pi r^2 + 2\pi r h$$

其中 $2\pi r^2$ 为半球面积, $2\pi r h$ 为圆柱侧面积, πr^2 为圆柱底面积。

体积约束:

$$V = \frac{2}{3}\pi r^3 + \pi r^2 h = V_0 \quad (\text{常数})$$

数学模型:

$$\begin{aligned} \min \quad & S = 3\pi r^2 + 2\pi r h \\ \text{s.t.} \quad & \frac{2}{3}\pi r^3 + \pi r^2 h = V_0 \\ & r \geq 0, \quad h \geq 0 \end{aligned}$$

例题 3.1.2 (构件表面积问题求解). **问题:** 设体积 $V_0 = \frac{5\pi}{3}$, 求使表面积最小的 r 和 h 。

解: 由约束条件解出 h :

$$h = \frac{V_0 - \frac{2}{3}\pi r^3}{\pi r^2} = \frac{5}{3r^2} - \frac{2r}{3}$$

代入目标函数得无约束问题 (关于 $r > 0$):

$$\min S(r) = 3\pi r^2 + 2\pi r \left(\frac{5}{3r^2} - \frac{2r}{3} \right) = 3\pi r^2 + \frac{10\pi}{3r} - \frac{4\pi r^2}{3} = \frac{5\pi r^2}{3} + \frac{10\pi}{3r}$$

求导:

$$S'(r) = \frac{10\pi r}{3} - \frac{10\pi}{3r^2} = 0 \Rightarrow r^3 = 1 \Rightarrow r = 1$$

验证二阶条件:

$$S''(r) = \frac{10\pi}{3} + \frac{20\pi}{3r^3} > 0 \quad (r > 0)$$

因此 $r^* = 1$ 是严格局部最小值点, 代入得:

$$h^* = \frac{5}{3} - \frac{2}{3} = 1, \quad S^* = \frac{5\pi}{3} + \frac{10\pi}{3} = 5\pi$$

3.1.3 木梁设计问题

问题描述

把圆形木材加工成矩形横截面的木梁, 要求木梁高度不超过 H , 横截面的惯性矩不小于 I , 且高度介于宽度与 α 倍宽度之间。如何确定木梁尺寸使成本最小?

设矩形横截面的高度为 x_1 , 宽度为 x_2 , 则圆形木材半径:

$$R = \frac{1}{2}\sqrt{x_1^2 + x_2^2}$$

目标函数 (成本与横截面积成正比):

$$\min f = \pi R^2 = \frac{\pi}{4}(x_1^2 + x_2^2)$$

约束条件:

- (1) 高度上限: $x_1 \leq H$
- (2) 惯性矩约束: $\frac{x_1^3 x_2}{12} \geq I$
- (3) 高度宽度关系: $x_2 \leq x_1 \leq \alpha x_2$
- (4) 非负约束: $x_1, x_2 \geq 0$

完整数学模型：

$$\begin{aligned} \min \quad & f = \frac{\pi}{4}(x_1^2 + x_2^2) \\ \text{s.t.} \quad & x_1 \leq H \\ & \frac{x_1^3 x_2}{12} \geq I \\ & x_1 - x_2 \geq 0 \\ & x_1 - \alpha x_2 \leq 0 \\ & x_1, x_2 \geq 0 \end{aligned}$$

关键观察

以上问题中，均含有决策变量的非线性函数，因此都属于非线性规划问题。

3.1.4 Python 代码：建模与可视化

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import minimize
4
5 # === 选址问题 ===
6 def facility_cost(vars, a, b, q):
7     """
8     vars = [x, y, w1, w2, ..., wn]
9     (x,y): 仓库位置
10    wi: 到第i个市场的运输量
11    """
12    x, y = vars[0], vars[1]
13    w = vars[2:]
14    cost = sum(wi * np.sqrt((x-ai)**2 + (y-bi)**2)
15               for wi, ai, bi in zip(w, a, b))
16    return cost
17
18 # 示例：2个市场
19 a = np.array([0.0, 4.0]) # x坐标
20 b = np.array([0.0, 0.0]) # y坐标
21 q = np.array([5.0, 3.0]) # 需求量
22
23 # 可视化
24 x_range = np.linspace(-1, 5, 100)
25 y_range = np.linspace(-3, 3, 100)

```

```

26 X, Y = np.meshgrid(x_range, y_range)
27
28 # 固定 $w$ 为需求量, 只关于位置求值
29 Z = q[0]*np.sqrt(X**2 + Y**2) + q[1]*np.sqrt((X-4)**2 + Y**2)
30
31 plt.contour(X, Y, Z, levels=20)
32 plt.plot(a, b, 'ro', markersize=10, label='Markets')
33 plt.colorbar(label='Transport Cost')
34 plt.xlabel('x'); plt.ylabel('y')
35 plt.title('Facility Location Problem')
36 plt.legend(); plt.axis('equal')
37 plt.savefig('facility_location.png', dpi=150)
38 plt.show()

```

Listing 3.1: 选址问题可视化

```

1 import numpy as np
2 from scipy.optimize import minimize_scalar
3
4 # === 构件表面积问题 ===
5 V0 = 5 * np.pi / 3
6
7 def surface_area(r):
8     """ $S(r) = 5\pi r^{2/3} + 10\pi/(3r)$ """
9     if r <= 0:
10         return np.inf
11     return 5*np.pi*r**2/3 + 10*np.pi/(3*r)
12
13 # 求解
14 result = minimize_scalar(surface_area, bounds=(0.01, 5))
15 r_opt = result.x
16 h_opt = 5/(3*r_opt**2) - 2*r_opt/3
17 S_opt = surface_area(r_opt)
18
19 print(f"Optimal r = {r_opt:.6f}")
20 print(f"Optimal h = {h_opt:.6f}")
21 print(f"Minimum S = {S_opt:.6f}")
22 print(f"Analytical: r*=1, h*=1, S*={5*np.pi:.6f}")

```

Listing 3.2: 构件表面积优化

3.2 非线性规划的一般数学模型

3.2.1 标准形式

一般非线性规划的数学模型可表示为：

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, 2, \dots, m \\ & h_j(x) = 0, \quad j = 1, 2, \dots, l \end{aligned}$$

其中 $x = (x_1, x_2, \dots, x_n)^T \in \mathbb{R}^n$, $f: \mathbb{R}^n \rightarrow \mathbb{R}$, $g_i: \mathbb{R}^n \rightarrow \mathbb{R}$, $h_j: \mathbb{R}^n \rightarrow \mathbb{R}$ 。

3.2.2 可行域与可行解

定义 3.2.1 (可行域). 满足所有约束条件的解称为**可行解** (*Feasible Solution*)。所有可行解构成的集合称为**可行域** (*Feasible Region*)：

$$\mathcal{X} = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0, \quad i = 1, \dots, m; \quad h_j(x) = 0, \quad j = 1, \dots, l\}$$

模型简记：

$$\min f(x), \quad x \in \mathcal{X}$$

3.2.3 问题分类

分类标准

- 无约束问题： $\mathcal{X} = \mathbb{R}^n$ ，即

$$\min f(x), \quad x \in \mathbb{R}^n$$

- 有约束问题： $\mathcal{X} \neq \mathbb{R}^n$

- 经典极值问题：只有等式约束，可用拉格朗日乘子法化为无约束问题

3.2.4 等价形式

不等式约束的不同写法：

- $g_i(x) \leq 0$ 可改写为 $-g_i(x) \geq 0$
- 等式约束 $h_j(x) = 0$ 等价于两个不等式约束 $h_j(x) \leq 0$ 和 $-h_j(x) \leq 0$

3.2.5 凸规划问题

定义 3.2.2 (凸规划). 若目标函数 f 和所有约束函数 g_i 均为凸函数, 等式约束 h_j 为仿射函数, 则称该问题为凸规划问题 (Convex Programming)。

性质 3.2.1 (凸规划的优势). 凸规划问题的重要性质:

- (1) 局部最优解就是全局最优解
- (2) 最优解集是凸集
- (3) 若目标函数是严格凸函数且存在极小点, 则全局最优解唯一

例题 3.2.1 (判断是否为凸规划). **问题:** 判断下列问题是否为凸规划:

$$\begin{aligned} \min \quad & f(x) = x_1^2 + 2x_2^2 \\ \text{s.t.} \quad & g_1(x) = x_1 + x_2 - 1 \leq 0 \\ & g_2(x) = -x_1 \leq 0 \end{aligned}$$

解:

Step 1: 检查 $f(x) = x_1^2 + 2x_2^2$:

$$\nabla^2 f(x) = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}$$

Hessian 矩阵正定, 故 f 是严格凸函数。

Step 2: 检查 $g_1(x) = x_1 + x_2 - 1$: 仿射函数 (含常数项 -1), 既是凸的也是凹的。

Step 3: 检查 $g_2(x) = -x_1$: 线性函数 (也是仿射), 既是凸的也是凹的。

Step 4: 结论: 所有约束为仿射 (也是凸函数), 目标函数严格凸, 这是凸规划问题。

例题 3.2.2 (Logistic 损失函数的 Hessian 与凹性). 考虑损失函数

$$l(\theta) = \sum_{i=1}^N [-\log(1 + \exp(-\theta^T u^{(i)})) - (1 - v^{(i)})\theta^T u^{(i)}]$$

其中 $u^{(i)}$ 为特征向量, $v^{(i)} \in \{0, 1\}$ 为标签。

记 $z_i = \theta^T u^{(i)}$, $\sigma_i = 1/(1 + e^{-z_i})$ 。梯度为

$$\nabla l(\theta) = \sum_{i=1}^N (v^{(i)} - \sigma_i) u^{(i)}$$

Hessian 矩阵为

$$\nabla^2 l(\theta) = - \sum_{i=1}^N \sigma_i (1 - \sigma_i) u^{(i)} (u^{(i)})^T$$

对任意向量 a :

$$a^T \nabla^2 l(\theta) a = - \sum_{i=1}^N \sigma_i (1 - \sigma_i) (a^T u^{(i)})^2 \leq 0$$

因此 $\nabla^2 l(\theta) \preceq 0$, $l(\theta)$ 是凹函数 (即 $-l$ 是凸函数, 可用于凸优化求解)。

```

1 import numpy as np
2
3 def check_convex_quadratic(H):
4     """
5     检查二次函数  $f(x) = x^T H x$  是否为凸函数
6     通过验证 Hessian 矩阵是否半正定
7     """
8     eigenvalues = np.linalg.eigvals(H)
9     print(f"Hessian 矩阵: \n{H}")
10    print(f"特征值: {eigenvalues}")
11    if np.all(eigenvalues >= -1e-10):
12        print("Hessian 矩阵半正定 -> 函数是凸函数")
13        if np.all(eigenvalues > 1e-10):
14            print("Hessian 矩阵正定 -> 函数是严格凸函数")
15    else:
16        print("Hessian 矩阵不定 -> 函数不是凸函数")
17    return eigenvalues
18
19 # 例题中的 Hessian 矩阵
20 H = np.array([[2, 0],
21               [0, 4]])
22 eigenvals = check_convex_quadratic(H)

```

Listing 3.3: 验证凸性 (Python)

3.3 非线性规划的直观理解

3.3.1 等值线与图解法

对于二元函数 $f(x_1, x_2)$, 等值线定义为满足 $f(x_1, x_2) = c$ (c 为常数) 的点的集合。

例题 3.3.1 (图解法). 考虑非线性规划问题:

$$\begin{aligned}
 \min \quad & f(x_1, x_2) = (x_1 - 2)^2 + (x_2 - 1)^2 \\
 s.t. \quad & x_1 + x_2 - 5 \leq 0 \\
 & x_1, x_2 \geq 0
 \end{aligned}$$

分析:

- 目标函数 f 的等值线是以 $(2, 1)$ 为圆心的同心圆
- 无约束最优解为 $(2, 1)$, 此时 $f(2, 1) = 0$
- 检查 $(2, 1)$ 是否满足约束: $2 + 1 - 5 = -3 \leq 0$, 满足!
- 因此无约束最优解就是约束最优解

若约束改为 $x_1 + x_2 - 2 \leq 0$, 则 $(2, 1)$ 不可行。此时最优解在约束边界 $x_1 + x_2 = 2$ 与等值线相切处取得。

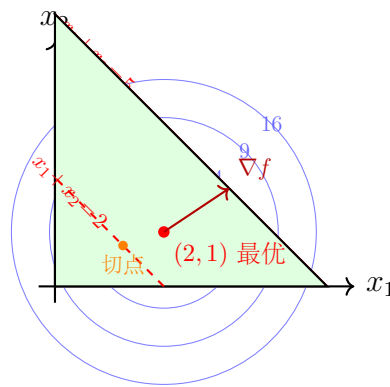


图 3.1: 非线性规划图解法: 等值线 (蓝色同心圆) 与约束

3.3.2 局部最优解与全局最优解

定义 3.3.1 (局部最优解). x^* 是局部最优点, 是指存在可行域中以 x^* 为中心的邻域 $N(x^*, \delta)$, 使得

$$\forall x \in N(x^*, \delta) \cap \mathcal{X}, \quad f(x^*) \leq f(x)$$

如果不存在等号 (对 $x \neq x^*$), 则称为严格局部最优解。

定义 3.3.2 (全局最优解). 若

$$\forall x \in \mathcal{X}, \quad f(x^*) \leq f(x)$$

则 x^* 为全局最优解。没有等号则为严格全局最优解。

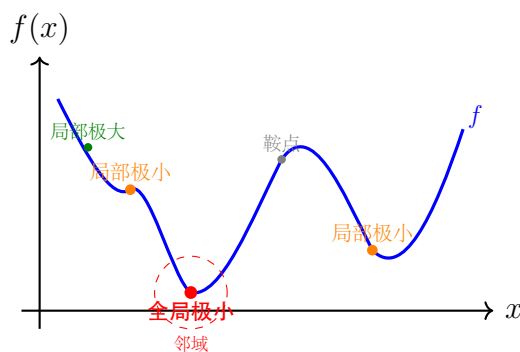


图 3.2: 局部最优解与全局最优解的对比（非凸函数可能有多个局部极小点）

LP vs NLP

- **线性规划**：最优解必在可行域的极点达到，且一定是全局最优解
- **非线性规划**：最优解可能是可行域上的任一点，有局部和全局之分
- 一般 **NLP 算法** 往往求出的是局部最优解
- **凸规划**：极值点只有一个，既是局部最优点也是全局最优点

3.3.3 一阶与二阶近似

设目标函数 f 在 x^* 附近可微，考虑小扰动 Δx ：

一阶近似（Taylor 展开）：

$$f(x^* + \Delta x) - f(x^*) \approx \nabla f(x^*)^T \Delta x$$

二阶近似：

$$f(x^* + \Delta x) - f(x^*) \approx \nabla f(x^*)^T \Delta x + \frac{1}{2} \Delta x^T \nabla^2 f(x^*) \Delta x$$

关键观察

若 x^* 是无约束局部极小点，则任意方向 Δx 带来的一阶变化应非负：

$$\nabla f(x^*)^T \Delta x \geq 0, \quad \forall \Delta x$$

这要求 $\nabla f(x^*) = 0$ （否则取 $\Delta x = -\nabla f(x^*)$ 可使左边为负）。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 定义函数及其梯度
5 def f(x, y):

```

```
6     return (x - 2)**2 + (y - 1)**2
7
8 def grad_f(x, y):
9     return np.array([2*(x - 2), 2*(y - 1)])
10
11 # 生成网格
12 x = np.linspace(-1, 4, 100)
13 y = np.linspace(-1, 4, 100)
14 X, Y = np.meshgrid(x, y)
15 Z = f(X, Y)
16
17 # 绘制等值线
18 plt.figure(figsize=(8, 6))
19 contours = plt.contour(X, Y, Z, levels=15, cmap='viridis')
20 plt.clabel(contours, inline=True, fontsize=8)
21
22 # 标记最优点
23 plt.plot(2, 1, 'r*', markersize=15, label='Optimal $(2,1)$')
24
25 # 绘制约束  $x + y \leq 2$ 
26 x_con = np.linspace(-1, 4, 100)
27 y_con = 2 - x_con
28 plt.fill_between(x_con, -1, np.clip(y_con, -1, 4),
29                 alpha=0.2, color='red', label='$x_1+x_2 \leq 2$')
30 plt.plot(x_con, y_con, 'r--', linewidth=2)
31
32 plt.xlabel('$x_1$'); plt.ylabel('$x_2$')
33 plt.title('Contour Plot and Feasible Region')
34 plt.legend(); plt.grid(True)
35 plt.axis('equal'); plt.xlim(-1, 4); plt.ylim(-1, 4)
36 plt.savefig('contour_plot.png', dpi=150)
37 plt.show()
```

Listing 3.4: 等值线可视化

3.4 无约束问题的最优性条件

3.4.1 梯度与 Hessian 矩阵

定义 3.4.1 (梯度). 多元函数 $f(x)$ 在 x 处的梯度 (Gradient) 为:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^T$$

梯度的几何意义:

- 梯度方向是函数值增加最快的方向
- 负梯度方向是函数值减小最快的方向 (最速下降方向)
- 梯度垂直于等值面/等值线

定义 3.4.2 (Hessian 矩阵). 多元函数 $f(x)$ 的二阶导数矩阵 (Hessian 矩阵) 为:

$$\nabla^2 f(x) = H(x) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$

重要特例:

- 若 $f(x) = \frac{1}{2}x^T A x + b^T x + c$, A 对称, 则 $\nabla f(x) = Ax + b$, $\nabla^2 f(x) = A$
- 若 $f(x) = b^T x$, 则 $\nabla f(x) = b$, $\nabla^2 f(x) = 0$

例题 3.4.1 (计算梯度和 Hessian 矩阵). 问题: 设 $f(x_1, x_2) = 3x_1^2 - 4x_1x_2 + 2x_2^2 + x_1 - 2x_2$, 求梯度和 Hessian 矩阵。

解:

$$\begin{aligned} \frac{\partial f}{\partial x_1} &= 6x_1 - 4x_2 + 1 \\ \frac{\partial f}{\partial x_2} &= -4x_1 + 4x_2 - 2 \end{aligned}$$

梯度:

$$\nabla f(x) = \begin{pmatrix} 6x_1 - 4x_2 + 1 \\ -4x_1 + 4x_2 - 2 \end{pmatrix}$$

Hessian 矩阵:

$$\nabla^2 f(x) = \begin{pmatrix} 6 & -4 \\ -4 & 4 \end{pmatrix}$$

验证正定性: 特征值为 $5 \pm \sqrt{17} > 0$, 故正定, 函数严格凸。

3.4.2 Taylor 展开

性质 3.4.1 (多元函数的 Taylor 展开).

$$f(x + \Delta x) = f(x) + \nabla f(x)^T \Delta x + \frac{1}{2} \Delta x^T \nabla^2 f(x) \Delta x + O(\|\Delta x\|^3)$$

线性逼近: $f(x + \Delta x) \approx f(x) + \nabla f(x)^T \Delta x$

二次逼近: $f(x + \Delta x) \approx f(x) + \nabla f(x)^T \Delta x + \frac{1}{2} \Delta x^T \nabla^2 f(x) \Delta x$

3.4.3 一阶必要条件

定理 3.4.1 (一阶必要条件). 设多元函数 $f(x)$ 在 x^* 点可微, 若 x^* 是 $f(x)$ 的局部极值点, 则

$$\nabla f(x^*) = 0$$

满足 $\nabla f(x^*) = 0$ 的点称为驻点 (Stationary Point) 或平稳点。

证明 (反证法):

Step 1: 假设 $\nabla f(x^*) \neq 0$

Step 2: 取方向 $d = -\nabla f(x^*)$, 则对于充分小的 $\alpha > 0$:

$$f(x^* + \alpha d) - f(x^*) \approx \alpha \nabla f(x^*)^T d = -\alpha \|\nabla f(x^*)\|^2 < 0$$

Step 3: 即 $f(x^* + \alpha d) < f(x^*)$, 与 x^* 是局部极小点矛盾

Step 4: 故 $\nabla f(x^*) = 0$

3.4.4 二阶必要条件

定理 3.4.2 (二阶必要条件). 设多元函数 $f(x)$ 在 x^* 点二次可微, 若 x^* 是 $f(x)$ 的局部极小点, 则

$$\nabla f(x^*) = 0 \quad \text{且} \quad \nabla^2 f(x^*) \succeq 0 \text{ (半正定)}$$

证明 (反证法):

Step 1: 由一阶必要条件, $\nabla f(x^*) = 0$ 已满足

Step 2: 假设 $\nabla^2 f(x^*)$ 不是半正定的, 则存在负特征值

Step 3: 设 d 为对应的特征向量, $\nabla^2 f(x^*)d = \lambda d$, $\lambda < 0$

Step 4: Taylor 展开:

$$f(x^* + \alpha d) - f(x^*) = \frac{\alpha^2}{2} d^T \nabla^2 f(x^*) d + o(\alpha^2) = \frac{\alpha^2 \lambda}{2} \|d\|^2 + o(\alpha^2) < 0$$

对充分小的 α , 这与局部最优矛盾。

3.4.5 二阶充分条件

定理 3.4.3 (二阶充分条件 1). 设多元函数 $f(x)$ 在 x^* 点二次可微, 若

$$\nabla f(x^*) = 0 \quad \text{且} \quad \nabla^2 f(x^*) \succ 0 \text{ (正定)}$$

则 x^* 是 $f(x)$ 的严格局部极小点。

证明: 由 $\nabla^2 f(x^*) \succ 0$, 最小特征值 $\lambda_{\min} > 0$. 对充分小的 $\|\Delta x\|$:

$$f(x^* + \Delta x) - f(x^*) \approx \frac{1}{2} \Delta x^T \nabla^2 f(x^*) \Delta x \geq \frac{\lambda_{\min}}{2} \|\Delta x\|^2 > 0$$

定理 3.4.4 (二阶充分条件 2). 设多元函数 $f(x)$ 在 x^* 的一个邻域内二次可微, 若对该邻域内任意点:

- $\nabla f(x^*) = 0$
- $\nabla^2 f(x)$ 半正定 (不要求在 x^* 处正定)

则 x^* 是局部极小点。

注意区别

- 半正定性: 特征值最小可为 0
- 条件 1 要求 x^* 处 Hessian **正定**, 结论更强 (严格局部极小)
- 条件 2 允许 Hessian 在 x^* 处为半正定, 但在邻域内半正定也可保证局部极小

3.4.6 利用最优性条件求解问题

求解步骤

Step 1: **求驻点:** 解 $\nabla f(x) = 0$, 得到所有候选点

Step 2: **二阶筛选:** 计算各点的 Hessian 矩阵, 滤除非半正定的点

Step 3: **判定严格极小:** 若 Hessian 正定, 确认为严格局部极小点

Step 4: **比较函数值:** 检查所有满足必要条件的点, 目标函数值最小的可能为全局极小点 (需全局极小点存在的前提)

例题 3.4.2 (利用最优性条件求解). **问题:** 利用最优性条件求解

$$\min f(x, y) = x^4 + y^4 - 4xy$$

解:

Step 1: 求梯度并令其为零

$$\nabla f = \begin{pmatrix} 4x^3 - 4y \\ 4y^3 - 4x \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

即 $x^3 = y$, $y^3 = x$ 。代入得 $x^9 = x$, 即 $x(x^8 - 1) = 0$ 。

四个驻点:

$$x_1^* = (0, 0), \quad x_2^* = (1, 1), \quad x_3^* = (-1, -1), \quad x_4^* = (i, -i) \text{ (舍去复数解)}$$

实际驻点: $(0, 0)$, $(1, 1)$, $(-1, -1)$

Step 2: 计算 Hessian 矩阵

$$\nabla^2 f(x, y) = \begin{pmatrix} 12x^2 & -4 \\ -4 & 12y^2 \end{pmatrix}$$

Step 3: 逐点判断

- 在 $(0, 0)$: $\nabla^2 f(0, 0) = \begin{pmatrix} 0 & -4 \\ -4 & 0 \end{pmatrix}$, 特征值为 ± 4 , 不定, 不是极值点 (鞍点)
- 在 $(1, 1)$: $\nabla^2 f(1, 1) = \begin{pmatrix} 12 & -4 \\ -4 & 12 \end{pmatrix}$, 特征值为 $8, 16$, 正定, 严格局部极小点
- 在 $(-1, -1)$: $\nabla^2 f(-1, -1) = \begin{pmatrix} 12 & -4 \\ -4 & 12 \end{pmatrix}$, 特征值为 $8, 16$, 正定, 严格局部极小点

Step 4: 比较函数值

$$f(1, 1) = 1 + 1 - 4 = -2, \quad f(-1, -1) = 1 + 1 - 4 = -2$$

两个点函数值相同, 都是全局极小点。

例题 3.4.3 (利用最优性条件求解 (含半正定情形)). **问题:** 利用最优性条件求解

$$\min f(x, y) = x^2 - 2xy + y^2 = (x - y)^2$$

解:

Step 1: 求梯度

$$\nabla f = \begin{pmatrix} 2x - 2y \\ -2x + 2y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

解得 $x = y$, 即直线 $x = y$ 上所有点都是驻点。

Step 2: Hessian 矩阵

$$\nabla^2 f = \begin{pmatrix} 2 & -2 \\ -2 & 2 \end{pmatrix}$$

特征值为 4 和 0，半正定（非严格正定）。

Step 3: 结论由于 $f(x, y) = (x - y)^2 \geq 0$ ，且在直线 $x = y$ 上 $f = 0$ ，因此直线 $x = y$ 上所有点都是全局最小点。

注意：虽然 *Hessian* 不是严格正定的，但由于函数本身是非负的二次函数，仍能判定全局最优性。

```

1 import numpy as np
2 from sympy import *
3
4 # 定义符号
5 x, y = symbols('x y', real=True)
6
7 # 定义函数
8 f = x**4 + y**4 - 4*x*y
9
10 # 计算梯度
11 grad = [diff(f, x), diff(f, y)]
12 print("梯度:", grad)
13
14 # 求解驻点
15 critical_points = solve(grad, [x, y])
16 print("驻点:", critical_points)
17
18 # Hessian 矩阵
19 H = Matrix([[diff(grad[0], x), diff(grad[0], y)],
20             [diff(grad[1], x), diff(grad[1], y)]])
21 print("Hessian:", H)
22
23 # 逐点判断
24 for pt in critical_points:
25     if pt[0].is_real and pt[1].is_real:
26         H_val = H.subs([(x, pt[0]), (y, pt[1])])
27         eigvals = H_val.eigenvals()
28         print(f"点{pt}: Hessian={H_val}, 特征值={list(eigvals.keys())
29               }")

```

Listing 3.5: 最优性条件验证

3.5 凸无约束问题的最优性条件

前面介绍的一般最优性条件只能用来研究局部最优解。对于一类特殊的函数——**凸函数**，可以获得全局最优性的充要条件。

3.5.1 次梯度与次微分

定义 3.5.1 (次梯度). 设 f 是定义在非空开凸集 $C \subset \mathbb{R}^n$ 上的凸函数 (不一定处处可微)。函数 f 在 x 处的**次梯度** (Subgradient) 是一个向量 $g \in \mathbb{R}^n$, 使得对 C 内任意 y :

$$f(y) \geq f(x) + g^T(y - x)$$

所有次梯度的集合称为**次微分** (Subdifferential), 记为 $\partial f(x)$ 。

性质 3.5.1 (次微分的性质). • $\partial f(x)$ 是非空的、凸的、紧的

• f 在 x 可微, 当且仅当 $\partial f(x) = \{\nabla f(x)\}$ (单元素集合)

例题 3.5.1 (次梯度计算). **问题:** 求 $f(x) = |x|$ 的次微分。

解:

$$\partial f(x) = \begin{cases} \{-1\}, & x < 0 \\ [-1, 1], & x = 0 \\ \{1\}, & x > 0 \end{cases}$$

在 $x = 0$ 处, f 不可微, 次微分为区间 $[-1, 1]$ 。

例题 3.5.2 (更多次微分计算). **(a)** $f(x_1, x_2, x_3) = \max\{|x_1|, |x_2|, |x_3|\} = \|x\|_\infty$ 在 $x = 0$ 处。

范数在原点的次微分等于其对偶范数单位球:

$$\partial \|x\|_\infty|_{x=0} = \{g \in \mathbb{R}^3 : \|g\|_1 \leq 1\}$$

(b) $f(x) = e^{|x|}$ 在 $x = 0$ 处 (标量)。

令 $h(t) = e^t$, $h'(0) = 1$, 且 $\partial |x||_{x=0} = [-1, 1]$ 。由单调可微凸函数与凸函数复合的次微分链式法则:

$$\partial f(0) = h'(0) \cdot \partial |x||_{x=0} = [-1, 1]$$

(c) $f(x_1, x_2) = \max\{x_1 + x_2 - 1, x_1 - x_2 + 1\}$ 在 $(1, 1)$ 处。

在 $(1, 1)$ 处两个仿射函数值均为 1, 均为活跃函数。最大函数的次微分为活跃函数梯度的凸包:

$$\partial f(1, 1) = \text{conv}\{(1, 1)^T, (1, -1)^T\} = \{(1, t)^T : t \in [-1, 1]\}$$

3.5.2 凸函数的一阶充要条件

定理 3.5.1 (凸函数的一阶充要条件). 设 f 定义在非空开凸集 $C \subset \mathbb{R}^n$ 上且可微, 则 f 为凸函数的充要条件是:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x), \quad \forall x, y \in C$$

如果为严格不等式 ($x \neq y$), 则为严格凸函数。

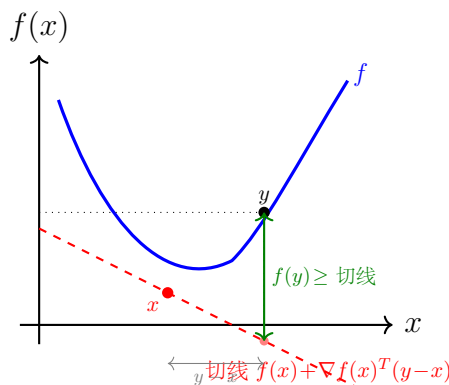


图 3.3: 凸函数一阶充要条件: 切线始终在函数图像下方

等价表述: x^* 是凸函数 f 的全局极小点, 当且仅当

$$0 \in \partial f(x^*) \quad (\text{即 } x^* \text{ 是驻点})$$

例题 3.5.3 (复合优化问题的一阶条件). **问题:** 设 $f(x) = g(x) + \lambda\|x\|_1$, g 光滑凸, $\lambda > 0$. 求最优性条件。

解:

由于 $\|x\|_1$ 在 $x_i = 0$ 处不可微, 使用次梯度:

$$(\partial\|x\|_1)_i = \begin{cases} \text{sign}(x_i), & x_i \neq 0 \\ [-1, 1], & x_i = 0 \end{cases}$$

最优性条件: $0 \in \nabla g(x^*) + \lambda\partial\|x^*\|_1$

即对每个分量:

$$(\nabla g(x^*))_i \in \begin{cases} \{-\lambda\}, & x_i^* > 0 \\ \{\lambda\}, & x_i^* < 0 \\ [-\lambda, \lambda], & x_i^* = 0 \end{cases}$$

这正说明了 LASSO 等稀疏优化中解的稀疏性来源: 当 $|(\nabla g(0))_i| \leq \lambda$ 时, $x_i^* = 0$ 。

3.5.3 凸函数的二阶充要条件

定理 3.5.2 (凸函数的二阶充要条件). 设 f 定义在非空开凸集 $C \subset \mathbb{R}^n$ 上且二阶可微, 则 f 为凸函数的充要条件是:

$$\nabla^2 f(x) \succeq 0 \quad (\text{Hessian 矩阵半正定}, \forall x \in C)$$

若每点的 Hessian 矩阵正定, 则 f 为严格凸函数。

重要区别

反之不成立: 严格凸函数只能导致 Hessian 半正定, 不一定在每点都正定。例如 $f(x) = x^4$ 是严格凸的, 但 $f''(0) = 0$ 。

3.5.4 正定二次函数的性质

对于正定二次函数:

$$f(x) = \frac{1}{2}x^T A x + b^T x + c, \quad A \succ 0$$

性质 3.5.2 (二次函数的极值). • **唯一极小点:** $x^* = -A^{-1}b$

- 等值面为以 x^* 为中心的同心椭球面族
- 大多数优化方法从二次函数模型导出

定义 3.5.2 (二次终止性). 一个算法用于解正定二次函数的无约束极小时, 有限步迭代可达最优解, 则称该算法具有二次终止性。

二次终止性 = 共轭方向 + 精确一维搜索

3.5.5 凸函数的水平集

定义 3.5.3 (水平集). 对于函数 f 和实数 α , 水平集定义为:

$$L_\alpha = \{x \in \text{dom}(f) \mid f(x) \leq \alpha\}$$

性质 3.5.3 (凸函数的水平集). 若 f 是凸函数, 则其任意水平集 L_α 都是凸集。反之不然!

3.5.6 凸规划的全局最优性

定理 3.5.3 (凸规划的局部即全局). 设 f 是定义在非空凸集 X 上的凸函数, 则 f 的任意局部极小点就是其在 X 上的全局极小点, 且所有极小点形成一个凸集。

证明:

Step 1: 假设 x^* 为局部极小点, 则存在邻域 $N(x^*, \delta)$ 使得

$$f(x) \geq f(x^*), \quad \forall x \in N(x^*, \delta) \cap X$$

Step 2: 在 X 中任取 y , 连接 x^* 和 y , 存在 $\theta \in (0, 1)$ 使得

$$x = \theta y + (1 - \theta)x^* \in N(x^*, \delta) \cap X$$

Step 3: 由凸性: $f(x) \leq \theta f(y) + (1 - \theta)f(x^*)$

Step 4: 由于 $f(x) \geq f(x^*)$, 代入得:

$$f(x^*) \leq \theta f(y) + (1 - \theta)f(x^*) \Rightarrow f(x^*) \leq f(y)$$

Step 5: 由 y 的任意性, x^* 为全局极小点

例题 3.5.4 (凸规划的验证与求解). **问题:** 验证并求解

$$\min f(x, y) = x^2 + xy + y^2 - 3x - 3y$$

解:

Step 1: 验证凸性

$$\nabla^2 f = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

特征值为 3 和 1, 均正, *Hessian* 正定, 故 f 严格凸。

Step 2: 求驻点

$$\nabla f = \begin{pmatrix} 2x + y - 3 \\ x + 2y - 3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

解得 $x = y = 1$ 。

Step 3: 由严格凸性, $x^* = (1, 1)$ 是唯一全局最小点。

```

1 import numpy as np
2 from scipy.optimize import minimize
3
4 # === 凸函数验证与求解 ===
5 def f(vars):
6     x, y = vars
7     return x**2 + x*y + y**2 - 3*x - 3*y
8
9 def grad_f(vars):
10    x, y = vars
11    return np.array([2*x + y - 3, x + 2*y - 3])

```

```

12
13 def hess_f(vars):
14     return np.array([[2, 1],
15                      [1, 2]])
16
17 # 验证Hessian正定性
18 H = hess_f(None)
19 eigenvals = np.linalg.eigvals(H)
20 print(f"Hessian特征值: {eigenvals}")
21 print(f"最小特征值: {min(eigenvals):.4f} > 0, 严格凸")
22
23 # 求解
24 result = minimize(f, [0, 0], jac=grad_f, method='BFGS')
25 print(f"\n最优解: x = {result.x}")
26 print(f"最优值: f* = {result.fun:.6f}")
27
28 # 验证解析解 x=y=1
29 print(f"\n解析验证: f(1,1) = {f([1,1]):.6f}")

```

Listing 3.6: 凸性验证与全局最优求解

3.6 解非线性规划问题的基本思路

实际问题直接应用最优性条件往往很困难：导数不存在、Hessian 计算复杂、方程 $\nabla f(x) = 0$ 求解困难等。因此通常采用数值迭代方法。

3.6.1 迭代方法的基本框架

最优化方法的基本结构

给定初始点 $x_0 \in \mathbb{R}^n$, $k = 0$, 按以下规则迭代：

1. 确定搜索方向 d_k ：在 x_k 点构造目标函数的下降方向
2. 确定步长因子 $\alpha_k > 0$ ：使目标函数值有某种意义的下降
3. 更新迭代点： $x_{k+1} = x_k + \alpha_k d_k$
4. 若满足某种终止条件，则停止迭代，得到近似最优解；否则 $k \leftarrow k + 1$, 返回步骤 1

关键：不同的步长因子 α_k 和不同的搜索方向 d_k 构成了不同的优化方法。

3.6.2 下降方向

定义 3.6.1 (下降方向). 方向 d 是函数 f 在 x 处的下降方向, 若存在 $\bar{\alpha} > 0$ 使得:

$$f(x + \alpha d) < f(x), \quad \forall \alpha \in (0, \bar{\alpha})$$

判定条件: 若 $\nabla f(x)^T d < 0$, 则 d 是下降方向。

证明: 由 Taylor 展开

$$f(x + \alpha d) - f(x) = \alpha \nabla f(x)^T d + o(\alpha)$$

当 $\nabla f(x)^T d < 0$ 且 α 充分小时, $f(x + \alpha d) < f(x)$ 。

3.6.3 步长因子的确定——一维搜索

最优步长通过求解一维优化问题得到:

$$\alpha_k = \arg \min_{\alpha > 0} f(x_k + \alpha d_k)$$

定理 3.6.1 (最优步长的正交性). 设 $x_{k+1} = x_k + \alpha_k d_k$ 是由精确线搜索得到的迭代点, 则

$$\nabla f(x_{k+1})^T d_k = 0$$

即新点处的梯度与搜索方向正交。

证明: 令 $\phi(\alpha) = f(x_k + \alpha d_k)$, 则 α_k 满足 $\phi'(\alpha_k) = 0$, 即

$$\nabla f(x_k + \alpha_k d_k)^T d_k = \nabla f(x_{k+1})^T d_k = 0$$

3.6.4 终止条件

常用的终止准则 (Termination Criterion):

TC1: 梯度准则

$$\|\nabla f(x_k)\| < \varepsilon_1$$

表明梯度向量趋近于 0, 迭代序列收敛到平稳点。

TC2: 相邻迭代点准则

$$\|x_{k+1} - x_k\| < \varepsilon_2, \quad \text{或} \quad \frac{\|x_{k+1} - x_k\|}{\|x_k\|} < \varepsilon_2$$

TC3: 函数值变化准则

$$|f(x_{k+1}) - f(x_k)| < \varepsilon_3, \quad \text{或} \quad \frac{|f(x_{k+1}) - f(x_k)|}{|f(x_k)|} < \varepsilon_3$$

Himmeblau 终止准则 (TC4)

当 TC2 和 TC3 矛盾时 (一个大一个小), 采用组合准则:

- 若 $|f(x_{k+1})| > \varepsilon$ 且 $\frac{|f(x_{k+1}) - f(x_k)|}{|f(x_{k+1})|} \leq \varepsilon_2$ 且 $\frac{\|\nabla f(x_{k+1})\|^2}{\|\nabla f(x_k)\|^2} < \varepsilon_3$

对于有一阶导数信息且收敛不太快的算法 (如共轭梯度法), 推荐 TC1 + TC4 结合使用, 一般可取 $\varepsilon_1 = 10^{-4}$, $\varepsilon_2 = \varepsilon_3 = 10^{-5}$ 。

3.6.5 收敛速度

设迭代序列 $\{x_k\}$ 依某范数收敛到 x^* :

定义 3.6.2 (Q-收敛速度). 若存在实数 $p \geq 1$ 和与 k 无关的正数 $c > 0$, 使得

$$\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|^p} = c$$

则称算法产生的点列具有 p 阶 Q-收敛速度:

- $p = 1, c \in (0, 1)$: **Q-线性收敛**
- $p = 1, c = 0$ 或 $p \in (1, 2)$: **Q-超线性收敛**
- $p = 2$: **Q-二阶 (平方) 收敛**

直观感受

- **线性收敛**: 每步误差约缩小为原来的 c 倍 ($c < 1$)
- **超线性收敛**: 比任何线性速率都快
- **二阶收敛**: 每步有效数字约翻倍
- **次线性收敛**: 误差缩减越来越慢, 如 $\frac{1}{k}$, $\frac{1}{k^2}$

3.6.6 共轭方向法

定义 3.6.3 (共轭方向). 设 A 为 $n \times n$ 对称正定矩阵, 非零向量 $d^{(1)}, d^{(2)}, \dots, d^{(m)} \in \mathbb{R}^n$ 满足:

$$(d^{(i)})^T A d^{(j)} = 0, \quad \forall i \neq j$$

则称这 m 个向量为 A -**共轭** 的。若 $A = I$ (单位矩阵), 则共轭等价于正交。

性质 3.6.1 (共轭向量的线性无关性). 关于正定矩阵 A 两两共轭的非零向量组必定线性无关。

证明: 设 $\sum_{i=1}^m \beta_i d^{(i)} = 0$, 左乘 $(d^{(j)})^T A$ 得 $\beta_j (d^{(j)})^T A d^{(j)} = 0$. 由于 $A \succ 0$ 且 $d^{(j)} \neq 0$, 故 $\beta_j = 0$.

定理 3.6.2 (共轭方向法基本定理). 对于正定二次函数 $f(x) = \frac{1}{2}x^T A x + b^T x + c$, 共轭方向法 (GCD) 至多经 n 步精确线性搜索终止。

例题 3.6.1 (共轭方向法迭代). **问题:** 用共轭方向法求解

$$\min f(x, y) = x^2 + 2y^2$$

取初始点 $x_0 = (2, 1)$, 搜索方向 $d_0 = (-1, 0)$, $d_1 = (0, -1)$ (关于 $A = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}$ 验证共轭性)。

解:

Step 1: 沿 d_0 搜索

$$\min_{\alpha} f(2 - \alpha, 1) = (2 - \alpha)^2 + 2(1)^2$$

求导: $-2(2 - \alpha) = 0 \Rightarrow \alpha_0 = 2$

$$x_1 = x_0 + 2d_0 = (2, 1) + 2(-1, 0) = (0, 1)$$

Step 2: 沿 d_1 搜索

$$\min_{\alpha} f(0, 1 - \alpha) = 0 + 2(1 - \alpha)^2$$

求导: $-4(1 - \alpha) = 0 \Rightarrow \alpha_1 = 1$

$$x_2 = x_1 + 1d_1 = (0, 1) + (0, -1) = (0, 0) = x^*$$

恰好 $n = 2$ 步收敛到最优解!

3.6.7 最优化算法分类概述

使用导数的方法

- **最速下降法:** $d_k = -\nabla f(x_k)$, 线性收敛, 可能有锯齿现象
- **Newton 法:** $d_k = -\nabla^2 f(x_k)^{-1} \nabla f(x_k)$, 二阶收敛, 但需 Hessian 正定
- **共轭梯度法:** 利用梯度构造共轭方向, 超线性收敛
- **拟 Newton 法 (变尺度法):** 用近似矩阵 G_k 代替 Hessian 的逆

– DFP 公式、BFGS 公式

不使用导数的方法（直接法）

- 变量轮换法：沿坐标轴方向轮换搜索
- 单纯形法（Nelder-Mead）：通过反射、扩张、收缩操作
- 适用性强但收敛较慢

```

1 import numpy as np
2
3 def steepest_descent(f, grad_f, x0, alpha=0.1,
4                     tol=1e-6, max_iter=1000):
5     """
6     固定步长的最速下降法
7     """
8     x = x0.copy()
9     history = [x.copy()]
10
11     for k in range(max_iter):
12         g = grad_f(x)
13         if np.linalg.norm(g) < tol:
14             break
15         x = x - alpha * g
16         history.append(x.copy())
17
18     return x, f(x), k+1, history
19
20 # 测试
21 f = lambda x: x[0]**2 + 2*x[1]**2
22 grad_f = lambda x: np.array([2*x[0], 4*x[1]])
23
24 x_opt, f_opt, iters, hist = steepest_descent(
25     f, grad_f, np.array([2.0, 1.0]), alpha=0.1)
26 print(f"最优解: x = {x_opt}")
27 print(f"最优值: f* = {f_opt:.8f}")
28 print(f"迭代次数: {iters}")

```

Listing 3.7: 最速下降法实现

3.7 有约束问题的最优性条件简介

有约束优化问题的最优性条件（包括 Lagrange 乘子法和 KKT 条件）将在第 6 章第 6.2 节中系统介绍。本节仅给出基本思路，详细内容请参见该章。

有约束问题的最优性条件概览

1. 等式约束：Lagrange 乘子法（见第 6 章第 6.2 节）
2. 不等式约束：KKT 条件（见第 6 章第 6.2 节）
3. 凸规划：KKT 条件 = 充要条件（见第 6 章定理 6.2.3）

Fenchel 共轭函数

函数 f 的 **Fenchel 共轭函数** 定义为

$$f^*(y) = \sup_x \{y^T x - f(x)\}$$

求解方法：若 f 可微且严格凸，令 $\nabla_x [y^T x - f(x)] = 0$ ，即 $y = \nabla f(x)$ ，解出 $x = x(y)$ 后代回 $f^*(y) = y^T x(y) - f(x(y))$ 。

常见例子：

- $f(x) = \frac{1}{2}\|x\|_2^2 \Rightarrow f^*(y) = \frac{1}{2}\|y\|_2^2$
- $f(x) = \lambda\|x\|_1 \Rightarrow f^*(y) = \begin{cases} 0, & \|y\|_\infty \leq \lambda \\ +\infty, & \text{otherwise} \end{cases}$

与对偶的联系：问题 $\min_x f(x) + g(Ax)$ 的 Fenchel 对偶为 $\max_y -f^*(-A^T y) - g^*(y)$ 。共轭函数是构造拉格朗日对偶、Fenchel 对偶和近端算法的基本工具。

3.8 本章小结

3.8.1 本章知识框架

非线性规划的数学模型——核心内容

1. 建模：三个经典问题（选址、表面积、木梁设计）
2. 一般模型： $\min f(x)$, s.t. $g_i(x) \leq 0$, $h_j(x) = 0$
3. 直观理解：等值线、局部与全局最优
4. 无约束最优性条件：
 - 一阶必要条件： $\nabla f(x^*) = 0$

- 二阶必要条件: $\nabla f(x^*) = 0, \nabla^2 f(x^*) \succeq 0$
- 二阶充分条件: $\nabla f(x^*) = 0, \nabla^2 f(x^*) \succ 0$

5. 凸规划的最优性条件:

- 凸函数的充要条件: $\nabla^2 f(x) \succeq 0$ (全局)
- 局部极小 = 全局极小

6. 迭代方法: 搜索方向 + 步长 + 终止条件 + 收敛速度

7. KKT 条件: 有约束问题的最优性条件

3.8.2 关键公式汇总

表 3.1: 最优性条件汇总

条件类型	内容	性质
一阶必要	$\nabla f(x^*) = 0$	驻点
二阶必要	$\nabla f(x^*) = 0, \nabla^2 f(x^*) \succeq 0$	局部极小
二阶充分 1	$\nabla f(x^*) = 0, \nabla^2 f(x^*) \succ 0$	严格局部极小
凸一阶	$0 \in \partial f(x^*)$	全局极小 (充要)
凸二阶	$\nabla^2 f(x) \succeq 0, \forall x$	凸函数 (充要)
KKT	平稳性 + 可行性 + 对偶性 + 互补性	凸规划充要

3.8.3 Python 综合代码: 求解无约束优化

```

1 import numpy as np
2 from scipy.optimize import minimize
3 import matplotlib.pyplot as plt
4
5 # ===== 综合求解示例 =====
6
7 def solve_unconstrained(f, grad_f, hess_f, x0, method='BFGS'):
8     """
9     统一求解无约束优化问题
10    """
11    methods = {
12        'BFGS': {'method': 'BFGS', 'jac': grad_f},
13        'Newton-CG': {'method': 'Newton-CG',

```

```

14         'jac': grad_f, 'hess': hess_f},
15     'trust-ncg': {'method': 'trust-ncg',
16                  'jac': grad_f, 'hess': hess_f},
17     'CG': {'method': 'CG', 'jac': grad_f},
18 }
19
20 if method not in methods:
21     raise ValueError(f"Unknown method: {method}")
22
23 result = minimize(f, x0, **methods[method])
24 return result
25
26 # === 测试函数: Rosenbrock函数 ===
27 def rosenbrock(x):
28     """ $f(x,y) = (1-x)^2 + 100*(y-x^2)^2$ """
29     return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2
30
31 def rosen_grad(x):
32     dx = -2*(1 - x[0]) - 400*x[0]*(x[1] - x[0]**2)
33     dy = 200*(x[1] - x[0]**2)
34     return np.array([dx, dy])
35
36 def rosen_hess(x):
37     dxx = 2 + 1200*x[0]**2 - 400*x[1]
38     dxy = -400*x[0]
39     dy = 200
40     return np.array([[dxx, dxy], [dxy, dy]])
41
42 # 求解
43 print("=" * 50)
44 print("Rosenbrock函数优化")
45 print("=" * 50)
46
47 x0 = np.array([-1.0, 2.0])
48 for method in ['BFGS', 'Newton-CG', 'CG']:
49     result = solve_unconstrained(rosenbrock, rosen_grad,
50                                 rosen_hess, x0, method)
51     print(f"\n{method}:")
52     print(f"    最优解: x = {result.x}")
53     print(f"    最优值: f* = {result.fun:.10f}")

```

```
54     print(f" 迭代次数: {result.nit}")
55     print(f" 函数调用: {result.nfev}")
56     print(f" 梯度调用: {result.njev}")
57
58 # === 可视化收敛过程 ===
59 def rosenbrock_with_history(x):
60     """带历史记录的Rosenbrock函数"""
61     rosenbrock_with_history.iter_count += 1
62     val = rosenbrock(x)
63     rosenbrock_with_history.history.append((x.copy(), val))
64     return val
65
66 rosenbrock_with_history.iter_count = 0
67 rosenbrock_with_history.history = []
68
69 result = minimize(rosenbrock_with_history, x0,
70                  method='BFGS', jac=rosen_grad)
71
72 history = rosenbrock_with_history.history
73 xs = [h[0][0] for h in history]
74 ys = [h[0][1] for h in history]
75 fs = [h[1] for h in history]
76
77 print(f"\n收敛路径长度: {len(history)}")
78 print(f"初始函数值: {fs[0]:.4f}")
79 print(f"最终函数值: {fs[-1]:.10f}")
```

Listing 3.8: 综合 Python 求解示例

3.8.4 进一步阅读

后续章节关联

- **第四章**: 线搜索方法 (确定步长的具体算法)
- **第五章**: 无约束最优化方法 (最速下降、Newton、共轭梯度、BFGS 等)
- **第六章**: 有约束最优化方法 (罚函数、内点法、SQP 等)
- **凸分析**: 深入理解次梯度、共轭函数、强凸性等概念

3.8.5 关键概念速查

表 3.2: 关键术语中英文对照

中文	英文	说明
梯度	Gradient	一阶导数向量
Hessian 矩阵	Hessian Matrix	二阶导数矩阵
驻点	Stationary Point	$\nabla f = 0$ 的点
次梯度	Subgradient	凸函数的推广梯度
次微分	Subdifferential	所有次梯度的集合
可行域	Feasible Region	满足约束的点集
起作用约束	Active Constraint	紧约束
拉格朗日乘子	Lagrange Multiplier	KKT 条件中的对偶变量
精确线搜索	Exact Line Search	最优步长搜索
二次终止性	Quadratic Termination	n 步收敛二次函数
共轭方向	Conjugate Direction	$d_i^T A d_j = 0$
Q-线性收敛	Q-linear Convergence	$\ e_{k+1}\ \leq c\ e_k\ $
Q-超线性收敛	Q-superlinear	$\ e_{k+1}\ /\ e_k\ \rightarrow 0$
Q-二阶收敛	Q-quadratic	$\ e_{k+1}\ \leq c\ e_k\ ^2$

Chapter 4

一维搜索与线搜索

4.1 一维搜索概述

4.1.1 问题背景

在多维优化算法中，迭代格式为：

$$x_{k+1} = x_k + \alpha_k d_k$$

其中 d_k 为搜索方向， $\alpha_k > 0$ 为步长因子。确定步长 α_k 的过程称为**一维搜索** (Line Search)，即求解：

$$\min_{\alpha > 0} \phi(\alpha) = f(x_k + \alpha d_k)$$

4.1.2 分类

一维搜索方法分类

1. 精确一维搜索 (Exact Line Search)

- 不利用导数信息：黄金分割法、Fibonacci 法
- 利用导数信息：Newton 法、插值法

2. 近似（不精确）一维搜索 (Inexact Line Search)

- Goldstein 法
- Armijo 法
- Wolfe-Powell 法

4.1.3 基本原则

精确一维搜索的目标是在给定方向上找到函数的精确（或足够精确）极小点。由于一般没有解析解，需要采用数值方法迭代逼近。

核心思想

精确搜索需要花费很大的工作量，特别是当迭代点远离问题的解时。实际上，Newton 法和拟 Newton 法的收敛速度并不依赖于精确一维搜索。因此，只要保证目标函数值每一步都有满意的下降，就可以大大节省工作量——这就是不精确搜索的动机。

4.1.4 单峰函数

定义 4.1.1 (单峰函数 (Unimodal Function)). 设 $\phi(\alpha)$ 在区间 $[a, b]$ 上连续，若存在 $\alpha^* \in [a, b]$ 使得：

- 当 $\alpha \leq \alpha^*$ 时， $\phi(\alpha)$ 严格单调递减
- 当 $\alpha \geq \alpha^*$ 时， $\phi(\alpha)$ 严格单调递增

则称 $\phi(\alpha)$ 为 $[a, b]$ 上的**单峰函数**。

性质 4.1.1 (单峰函数的关键性质). 若 $\phi(\alpha)$ 在 $[a, b]$ 上单峰，对任意 $a < \lambda_1 < \lambda_2 < b$ ：

- 若 $\phi(\lambda_1) \leq \phi(\lambda_2)$ ，则极小点 $\alpha^* \in [a, \lambda_2]$
- 若 $\phi(\lambda_1) \geq \phi(\lambda_2)$ ，则极小点 $\alpha^* \in [\lambda_1, b]$

即每次比较可将搜索区间缩小。

4.2 几个常见的测试函数

一维搜索方法通常用以下经典测试函数来验证和比较。

4.2.1 单峰函数

常用单峰测试函数

- (1) 二次函数： $\phi(\alpha) = (\alpha - 2)^2$ ， $\alpha \in [-5, 5]$ ，最小值点 $\alpha^* = 2$
- (2) 带偏移的二次函数： $\phi(\alpha) = 3\alpha^2 - 6\alpha + 5$ ， $\alpha \in [0, 3]$ ，最小值点 $\alpha^* = 1$
- (3) 四次函数： $\phi(\alpha) = \alpha^4 - 3\alpha^2 + \alpha + 1$ ， $\alpha \in [-2, 2]$ ，有唯一局部最小值

4.2.2 多峰函数

常用多峰测试函数

- (1) **Rosenbrock 型**: $\phi(\alpha) = (\alpha^2 - 1)^2 + (\alpha - 2)^2$, 多个局部极值
- (2) **带正弦的函数**: $\phi(\alpha) = \alpha \sin(\alpha)$, $\alpha \in [0, 2\pi]$, 多峰振荡
- (3) **高次多项式**: $\phi(\alpha) = \alpha^6 - 5\alpha^4 + 4\alpha^2 + 3$, 多个极小值点

注意

黄金分割法和 Fibonacci 法只适用于**单峰函数**。对于多峰函数，可能收敛到局部极小点而非全局极小点。Shubert-Piyavskii 方法可以处理多峰函数的全局优化。

```

1 import numpy as np
2
3 # === 常用一维搜索测试函数 ===
4
5 def phi_quad(alpha):
6     """二次函数: (alpha - 2)^2, 最小值点在 alpha* = 2"""
7     return (alpha - 2)**2
8
9 def phi_quad_grad(alpha):
10    """二次函数的导数"""
11    return 2 * (alpha - 2)
12
13 def phi_cubic(alpha):
14    """三次函数 (多峰): alpha^4 - 3*alpha^2 + alpha + 1"""
15    return alpha**4 - 3*alpha**2 + alpha + 1
16
17 def phi_cubic_grad(alpha):
18    """三次函数的导数"""
19    return 4*alpha**3 - 6*alpha + 1
20
21 def phi_rosenbrock_1d(alpha):
22    """Rosenbrock型一维函数"""
23    return (alpha**2 - 1)**2 + (alpha - 2)**2
24
25 def phi_sin(alpha):
26    """带正弦的函数: alpha * sin(alpha)"""
27    return alpha * np.sin(alpha)
28

```

```

29 # 测试
30 test_funcs = {
31     'Quadratic': (phi_quad, 2.0),
32     'Cubic-like': (phi_cubic, None),
33     'Rosenbrock-1D': (phi_rosenbrock_1d, None),
34 }
35
36 for name, (f, true_min) in test_funcs.items():
37     from scipy.optimize import minimize_scalar
38     res = minimize_scalar(f, bounds=(-3, 3), method='bounded')
39     print(f"{name}: alpha* = {res.x:.6f}, f* = {res.fun:.6f}")

```

Listing 4.1: 测试函数定义

4.3 不利用导数的精确一维搜索

除了第 4.4 节介绍的黄金分割法和 Fibonacci 法，还有另一种常用的区间缩减方法——Dichotomous 搜索。

4.3.1 Dichotomous 搜索（二分搜索）

基本思想

每次迭代取区间中点附近的两个对称点比较函数值，根据单峰性质缩小区间。

算法步骤：

1. 初始化区间 $[a_1, b_1] = [a, b]$ ，设精度 $\varepsilon > 0$
2. 对 $k = 1, 2, \dots$:
 - (a) 取 $\lambda_k = \frac{a_k + b_k}{2} - \delta$ ， $\mu_k = \frac{a_k + b_k}{2} + \delta$ (δ 为小正数)
 - (b) 若 $\phi(\lambda_k) < \phi(\mu_k)$ ，则 $a_{k+1} = a_k$ ， $b_{k+1} = \mu_k$
 - (c) 否则 $a_{k+1} = \lambda_k$ ， $b_{k+1} = b_k$
 - (d) 若 $b_{k+1} - a_{k+1} < \varepsilon$ ，停止

区间缩减率

每次迭代区间缩减率为 $\frac{1}{2} + \frac{\delta}{b_k - a_k} \approx \frac{1}{2}$ 。

经过 n 次迭代后，区间长度为：

$$b_n - a_n = \frac{b - a}{2^{n-1}} + 2\delta$$

例题 4.3.1 (Dichotomous 搜索). 用 *Dichotomous* 搜索求 $\min f(\lambda) = \lambda^2 + 2\lambda, \lambda \in [-3, 5]$, 取 $\varepsilon = 0.1$ 。

解： 区间长度为 8。需要迭代次数 n 满足 $\frac{8}{2^{n-1}} < 0.1$, 即 $n \geq 8$ 。

第 1 次迭代: $\lambda_1 = -0.01, \mu_1 = 0.01$

$$f(\lambda_1) = -0.9999 < f(\mu_1) \approx -0.9999$$

取新区间 $[-3, 0.01]$, 依此类推直至区间长度小于 0.1。

4.4 黄金分割法与 Fibonacci 法

这两种方法基于**区间缩进** (Interval Reduction) 思想, 每次迭代通过比较两个内点的函数值来缩小搜索区间。

4.4.1 黄金分割法 (0.618 法)

黄金分割法原理

在区间 $[a, b]$ 内对称地取两个试探点 λ_1, λ_2 , 满足:

- **对称原则:** $\lambda_1 - a = b - \lambda_2$
- **等比收缩原则:** 每次保留的区间是原区间的 $\rho = 0.618$ 倍

收缩比 $\rho = \frac{\sqrt{5}-1}{2} \approx 0.618$ (黄金分割比)。

试探点位置:

$$\lambda_1 = a + (1 - \rho)(b - a) = a + 0.382(b - a)$$

$$\lambda_2 = a + \rho(b - a) = a + 0.618(b - a)$$

黄金分割法算法

输入: 初始区间 $[a_0, b_0]$, 容许误差 $\varepsilon > 0$

Step 0: 令 $\rho = \frac{\sqrt{5}-1}{2} \approx 0.618$, 计算

$$\lambda_0 = a_0 + (1 - \rho)(b_0 - a_0), \quad \mu_0 = a_0 + \rho(b_0 - a_0)$$

计算 $\phi(\lambda_0)$ 和 $\phi(\mu_0)$, $k = 0$

Step 1: 若 $\phi(\lambda_k) \leq \phi(\mu_k)$, 转 Step 2; 否则转 Step 3

Step 2 (保留左区间) 令:

$$a_{k+1} = a_k, \quad b_{k+1} = \mu_k, \quad \mu_{k+1} = \lambda_k$$

计算新的 $\lambda_{k+1} = a_{k+1} + (1 - \rho)(b_{k+1} - a_{k+1})$ 和 $\phi(\lambda_{k+1})$

Step 3 (保留右区间) 令:

$$a_{k+1} = \lambda_k, \quad b_{k+1} = b_k, \quad \lambda_{k+1} = \mu_k$$

计算新的 $\mu_{k+1} = a_{k+1} + \rho(b_{k+1} - a_{k+1})$ 和 $\phi(\mu_{k+1})$

Step 4: 若 $b_{k+1} - a_{k+1} < \varepsilon$, 停; 否则 $k \leftarrow k + 1$, 转 Step 1

输出: $\alpha^* \approx \frac{a_k + b_k}{2}$

例题 4.4.1 (黄金分割法——迭代两次). **问题:** 用黄金分割法求解

$$\min \phi(\alpha) = (\alpha - 2)^2, \quad \alpha \in [0, 5]$$

取 $\varepsilon = 0.5$, 执行两轮迭代。

解: $\rho = 0.618$, $1 - \rho = 0.382$

第 0 轮: $[a_0, b_0] = [0, 5]$

$$\lambda_0 = 0 + 0.382 \times 5 = 1.910$$

$$\mu_0 = 0 + 0.618 \times 5 = 3.090$$

$$\phi(\lambda_0) = (1.910 - 2)^2 = 0.0081$$

$$\phi(\mu_0) = (3.090 - 2)^2 = 1.1881$$

由于 $\phi(\lambda_0) = 0.0081 \leq 1.1881 = \phi(\mu_0)$, 保留左区间:

$$[a_1, b_1] = [0, \mu_0] = [0, 3.090]$$

第 1 轮: $[a_1, b_1] = [0, 3.090]$

$$\mu_1 = \lambda_0 = 1.910 \quad (\text{复用})$$

$$\lambda_1 = a_1 + 0.382(b_1 - a_1) = 0 + 0.382 \times 3.090 = 1.180$$

计算新点:

$$\phi(\lambda_1) = (1.180 - 2)^2 = 0.6724$$

已有 $\phi(\mu_1) = 0.0081$

由于 $\phi(\lambda_1) = 0.6724 \geq 0.0081 = \phi(\mu_1)$, 保留右区间:

$$[a_2, b_2] = [\lambda_1, b_1] = [1.180, 3.090]$$

两轮迭代结果:

- 区间长度: $b_2 - a_2 = 3.090 - 1.180 = 1.910$
- 近似最优解: $\alpha^* \approx \frac{1.180 + 3.090}{2} = 2.135$
- 区间长度仍大于 $\varepsilon = 0.5$, 需继续迭代

验证: 真实最优解 $\alpha^* = 2$ 确实在 $[1.180, 3.090]$ 内。

4.4.2 Fibonacci 法

Fibonacci 数列

Fibonacci 数列 $\{F_n\}$ 定义为:

$$F_0 = F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad (n \geq 2)$$

前几项: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

Fibonacci 法的收缩比:

$$\rho_k = \frac{F_{n-k}}{F_{n-k+1}}$$

试探点:

$$\lambda_k = a_k + \frac{F_{n-k-1}}{F_{n-k+1}}(b_k - a_k), \quad \mu_k = a_k + \frac{F_{n-k}}{F_{n-k+1}}(b_k - a_k)$$

黄金分割法 vs Fibonacci 法

- **Fibonacci 法:** 收缩比逐次变化, 最终区间长度 $\frac{b-a}{F_n}$, 理论最优
- **黄金分割法:** 收缩比固定为 0.618, 最终区间长度约 $(0.618)^n(b-a)$, 简单实用
- 当 $n \rightarrow \infty$ 时, $\frac{F_{n-1}}{F_n} \rightarrow 0.618$, 两种方法趋于一致
- 实际中黄金分割法更常用, 因为不需要预先确定迭代次数

迭代次数 n 的确定:

Fibonacci 法进行 n 次试探后, 最终区间长度为 $\frac{b-a}{F_n}$ 。若要求最终区间长度不超过容许误差 ε , 即

$$\frac{b-a}{F_n} \leq \varepsilon$$

则需

$$F_n \geq \frac{b-a}{\varepsilon}$$

因此, n 是满足 $F_n \geq \frac{b-a}{\varepsilon}$ 的最小正整数。实际计算时可依次生成 Fibonacci 数, 直到满足上述不等式为止。

Fibonacci 法的特点

与黄金分割法不同, Fibonacci 法需要根据精度要求预先确定试探次数 n (或由 $F_n \geq \frac{b-a}{\varepsilon}$ 计算得到), 而不能像黄金分割法那样迭代到满足精度自动停止。

例题 4.4.2 (Fibonacci 法——迭代两次). **问题:** 用 *Fibonacci* 法求解

$$\min \phi(\alpha) = (\alpha - 2)^2, \quad \alpha \in [0, 5]$$

取容许误差 $\varepsilon = 1$, 执行两轮迭代。

解：区间长度 $b - a = 5$ ，要求 $\frac{5}{F_n} \leq 1$ ，即 $F_n \geq 5$ 。查 *Fibonacci* 数列： $F_4 = 3 < 5$ ， $F_5 = 5 \geq 5$ ，故取 $n = 5$ （共 5 次试探）。

$$F_1 = F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5, F_6 = 8$$

第 0 轮： $[a_0, b_0] = [0, 5]$ ， $k = 0$ ， $F_5 = 5, F_6 = 8$

$$\lambda_0 = 0 + \frac{F_4}{F_6}(5 - 0) = \frac{3}{8} \times 5 = 1.875$$

$$\mu_0 = 0 + \frac{F_5}{F_6}(5 - 0) = \frac{5}{8} \times 5 = 3.125$$

$$\phi(\lambda_0) = (1.875 - 2)^2 = 0.015625$$

$$\phi(\mu_0) = (3.125 - 2)^2 = 1.265625$$

$\phi(\lambda_0) \leq \phi(\mu_0)$ ，保留左区间：

$$[a_1, b_1] = [0, 3.125]$$

第 1 轮： $[a_1, b_1] = [0, 3.125]$ ， $k = 1$ ， $F_4 = 3, F_5 = 5$

$$\lambda_1 = 0 + \frac{F_3}{F_5}(3.125 - 0) = \frac{2}{5} \times 3.125 = 1.25$$

$$\mu_1 = \lambda_0 = 1.875 \quad (\text{复用})$$

$$\phi(\lambda_1) = (1.25 - 2)^2 = 0.5625$$

$$\phi(\mu_1) = 0.015625$$

$\phi(\lambda_1) \geq \phi(\mu_1)$ ，保留右区间：

$$[a_2, b_2] = [1.25, 3.125]$$

两轮迭代结果：

- 区间： $[1.25, 3.125]$ ，长度 1.875
- 近似最优解： $\alpha^* \approx \frac{1.25+3.125}{2} = 2.1875$

```

1 import numpy as np
2
3 def golden_section_search(phi, a, b, eps=1e-6, max_iter=100):
4     """黄金分割法求单峰函数在[a,b]上的最小值"""
5     rho = (np.sqrt(5) - 1) / 2 # 0.6180339...
6     one_minus_rho = 1 - rho   # 0.381966...
7 
```

```

8      # 初始试探点
9      lam = a + one_minus_rho * (b - a)
10     mu = a + rho * (b - a)
11     phi_lam = phi(lam)
12     phi_mu = phi(mu)
13
14     history = [(a, b, lam, mu, phi_lam, phi_mu)]
15
16     for k in range(max_iter):
17         if phi_lam <= phi_mu:
18             b = mu
19             mu = lam
20             phi_mu = phi_lam
21             lam = a + one_minus_rho * (b - a)
22             phi_lam = phi(lam)
23         else:
24             a = lam
25             lam = mu
26             phi_lam = phi_mu
27             mu = a + rho * (b - a)
28             phi_mu = phi(mu)
29
30         history.append((a, b, lam, mu, phi_lam, phi_mu))
31
32         if b - a < eps:
33             break
34
35     alpha_opt = (a + b) / 2
36     return alpha_opt, phi(alpha_opt), k+1, history
37
38
39 def fibonacci_search(phi, a, b, eps=None, n=None, max_iter=100):
40     """Fibonacci 法
41     参数:
42         phi: 目标函数
43         a, b: 初始区间
44         eps: 容许误差 (与n二选一, 优先使用n)
45         n: 试探次数
46     """
47     assert eps is not None or n is not None, "必须提供eps或n之一"

```

```
48
49 # 生成Fibonacci数列
50 F = [1, 1]
51 if n is None:
52     # 根据eps计算n
53     ratio = (b - a) / eps
54     while F[-1] < ratio:
55         F.append(F[-1] + F[-2])
56     n = len(F) - 1
57 else:
58     for i in range(2, n + 2):
59         F.append(F[-1] + F[-2])
60
61 lam = a + F[n-1]/F[n+1] * (b - a)
62 mu = a + F[n]/F[n+1] * (b - a)
63 phi_lam = phi(lam)
64 phi_mu = phi(mu)
65
66 history = [(a, b, lam, mu, phi_lam, phi_mu)]
67
68 for k in range(1, n):
69     if k == n - 1: # 最后一步特殊处理
70         # 当eps未提供时, 使用固定的小偏移量
71         delta = eps * 0.5 if eps is not None else 1e-6
72         mu = lam + delta
73
74     if phi_lam <= phi_mu:
75         b = mu
76         mu = lam
77         phi_mu = phi_lam
78         lam = a + F[n-k-1]/F[n-k+1] * (b - a)
79         phi_lam = phi(lam)
80     else:
81         a = lam
82         lam = mu
83         phi_lam = phi_mu
84         mu = a + F[n-k]/F[n-k+1] * (b - a)
85         phi_mu = phi(mu)
86
87     history.append((a, b, lam, mu, phi_lam, phi_mu))
```

```

88
89     alpha_opt = (a + b) / 2
90     return alpha_opt, phi(alpha_opt), n, history
91
92
93 # === 测试 ===
94 def phi_test(alpha):
95     return (alpha - 2)**2
96
97 print("=" * 50)
98 print("黄金分割法")
99 print("=" * 50)
100 alpha_opt, phi_opt, iters, hist = golden_section_search(
101     phi_test, 0, 5, eps=0.01)
102 print(f"最优解: alpha* = {alpha_opt:.6f}")
103 print(f"最优值: phi* = {phi_opt:.10f}")
104 print(f"迭代次数: {iters}")
105 print("\n前两轮迭代:")
106 for i, (a, b, lam, mu, pl, pm) in enumerate(hist[:3]):
107     print(f" 第{i}轮: [{a:.4f}, {b:.4f}], "
108           f"lam={lam:.4f}(phi={pl:.4f}), "
109           f"mu={mu:.4f}(phi={pm:.4f})")
110
111 print("=" * 50)
112 print("Fibonacci法 (指定n=5) ")
113 print("=" * 50)
114 alpha_opt, phi_opt, n, hist = fibonacci_search(
115     phi_test, 0, 5, n=5)
116 print(f"试探次数: n = {n}")
117 print(f"最优解: alpha* = {alpha_opt:.6f}")
118 print(f"最优值: phi* = {phi_opt:.10f}")
119
120 print("=" * 50)
121 print("Fibonacci法 (根据eps自动确定n) ")
122 print("=" * 50)
123 alpha_opt, phi_opt, n, hist = fibonacci_search(
124     phi_test, 0, 5, eps=1.0)
125 print(f"自动确定试探次数: n = {n}")
126 print(f"最优解: alpha* = {alpha_opt:.6f}")
127 print(f"最优值: phi* = {phi_opt:.10f}")

```

Listing 4.2: 黄金分割法与 Fibonacci 法实现

4.5 中点法（二分法）

4.5.1 算法原理

中点法（又称二分法）适用于 $\phi(\alpha)$ 可微且导数为零时对应极小点的情形。

二分法原理

设 $\phi(\alpha)$ 在 $[a, b]$ 上可微。取中点 $\lambda = \frac{a+b}{2}$ ，根据导数符号判断极小点位置：

- $\phi'(\lambda) = 0$: λ 就是最小点 α^*
- $\phi'(\lambda) > 0$: λ 在上升段, $\alpha^* < \lambda$, 去掉 $[\lambda, b]$
- $\phi'(\lambda) < 0$: λ 在下降段, $\alpha^* > \lambda$, 去掉 $[a, \lambda]$

每次迭代区间缩小一半，收敛速率为线性，比率 $c = \frac{1}{2}$ 。

二分法算法

输入：初始区间 $[a_0, b_0]$ ，容许误差 $\varepsilon > 0$

Step 0: $k = 0$

Step 1: 计算 $\lambda_k = \frac{a_k + b_k}{2}$

Step 2: 计算 $\phi'(\lambda_k)$

Step 3:

- 若 $\phi'(\lambda_k) = 0$: 停, $\alpha^* = \lambda_k$
- 若 $\phi'(\lambda_k) > 0$: 令 $a_{k+1} = a_k$, $b_{k+1} = \lambda_k$
- 若 $\phi'(\lambda_k) < 0$: 令 $a_{k+1} = \lambda_k$, $b_{k+1} = b_k$

Step 4: 若 $b_{k+1} - a_{k+1} < \varepsilon$, 停; 否则 $k \leftarrow k + 1$, 转 Step 1

输出: $\alpha^* \approx \frac{a_k + b_k}{2}$

例题 4.5.1 (二分法——迭代两次). 问题：用二分法求解

$$\min \phi(\alpha) = (\alpha - 2)^2, \quad \alpha \in [0, 5]$$

取 $\varepsilon = 0.5$ ，执行两轮迭代。

解：首先计算导数: $\phi'(\alpha) = 2(\alpha - 2)$

第 0 轮: $[a_0, b_0] = [0, 5]$

$$\lambda_0 = \frac{0+5}{2} = 2.5, \quad \phi'(\lambda_0) = 2(2.5 - 2) = 1.0 > 0$$

λ_0 在上升段, $\alpha^* < 2.5$, 去掉右半区间:

$$[a_1, b_1] = [0, 2.5]$$

第 1 轮: $[a_1, b_1] = [0, 2.5]$

$$\lambda_1 = \frac{0+2.5}{2} = 1.25, \quad \phi'(\lambda_1) = 2(1.25 - 2) = -1.5 < 0$$

λ_1 在下降段, $\alpha^* > 1.25$, 去掉左半区间:

$$[a_2, b_2] = [1.25, 2.5]$$

两轮迭代结果:

- 区间: $[1.25, 2.5]$, 长度 1.25
- 近似最优解: $\alpha^* \approx \frac{1.25+2.5}{2} = 1.875$
- 区间长度 $1.25 > 0.5$, 需继续迭代

验证: 真实最优解 $\alpha^* = 2$ 在 $[1.25, 2.5]$ 内。

二分法 vs 黄金分割法

- 二分法需要计算导数, 每次迭代区间缩小 $\frac{1}{2}$
- 黄金分割法只需计算函数值, 每次区间缩小 0.618
- 二分法区间缩减更快 ($0.5 < 0.618$), 但需要导数信息
- 若函数可微且导数容易计算, 二分法效率更高

```

1 import numpy as np
2
3 def bisection_method(phi_grad, a, b, eps=1e-6, max_iter=100):
4     """
5     二分法求可微函数的极小点
6     phi_grad: 导数函数
7     """
8     history = []
9
10    for k in range(max_iter):

```

```

11     lam = (a + b) / 2
12     grad = phi_grad(lam)
13
14     history.append((a, b, lam, grad))
15
16     if abs(grad) < 1e-12:
17         return lam, k+1, history
18     elif grad > 0:
19         b = lam # 在上升段, 去掉右半
20     else:
21         a = lam # 在下降段, 去掉左半
22
23     if b - a < eps:
24         break
25
26     alpha_opt = (a + b) / 2
27     return alpha_opt, k+1, history
28
29
30 # === 测试 ===
31 def phi_test_grad(alpha):
32     return 2 * (alpha - 2)
33
34 print("=" * 50)
35 print("二分法")
36 print("=" * 50)
37 alpha_opt, iters, hist = bisection_method(phi_test_grad, 0, 5, eps
38     =0.01)
39 print(f"最优解: alpha* = {alpha_opt:.6f}")
40 print(f"迭代次数: {iters}")
41 print("\n前两轮迭代:")
42 for i, (a, b, lam, grad) in enumerate(hist[:2]):
43     direction = "上升段(去右)" if grad > 0 else "下降段(去左)"
44     print(f" 第{i}轮: [{a:.4f}, {b:.4f}], "
45         f"mid={lam:.4f}, phi'={grad:+.4f} ({direction})")

```

Listing 4.3: 二分法实现

二分法（导数版）与 Dichotomous 搜索的对比

- **Dichotomous 搜索**：每次迭代计算两个函数值，区间缩减率 ≈ 0.5
- **二分法（导数版）**：每次迭代计算一个导数值，区间同样减半
- 当导数计算代价低于函数值计算时，二分法（导数版）更高效
- 两种方法的区间缩减率均为 $\frac{1}{2}$ ，但二分法（导数版）利用了更多信息

4.6 Newton 法与插值法

4.6.1 Newton 法

Newton 法利用函数的二阶 Taylor 展开来构造局部二次模型，通过解析求解该模型的极小点来更新迭代。

Newton 法原理

在 α_k 点对 $\phi(\alpha)$ 做二次近似：

$$\phi(\alpha) \approx \phi(\alpha_k) + \phi'(\alpha_k)(\alpha - \alpha_k) + \frac{1}{2}\phi''(\alpha_k)(\alpha - \alpha_k)^2$$

令近似函数的导数为零，得到迭代公式：

$$\alpha_{k+1} = \alpha_k - \frac{\phi'(\alpha_k)}{\phi''(\alpha_k)}$$

Newton 法算法

输入：初始点 α_0 ，容许误差 $\varepsilon > 0$

Step 0: $k = 0$

Step 1: 计算 $\phi'(\alpha_k)$ 和 $\phi''(\alpha_k)$

Step 2: 若 $\phi''(\alpha_k) \leq 0$ ，停（失败，非凸）

Step 3: 更新

$$\alpha_{k+1} = \alpha_k - \frac{\phi'(\alpha_k)}{\phi''(\alpha_k)}$$

Step 4: 若 $|\alpha_{k+1} - \alpha_k| < \varepsilon$ 或 $|\phi'(\alpha_{k+1})| < \varepsilon$ ，停；否则 $k \leftarrow k + 1$ ，转 Step 1

输出： $\alpha^* \approx \alpha_k$

例题 4.6.1 (Newton 法——迭代两次). **问题**：用 Newton 法求解

$$\min \phi(\alpha) = (\alpha - 2)^2, \quad \alpha_0 = 5$$

执行两轮迭代。

解：计算导数：

$$\phi'(\alpha) = 2(\alpha - 2), \quad \phi''(\alpha) = 2$$

第 0 轮： $\alpha_0 = 5$

$$\phi'(\alpha_0) = 2(5 - 2) = 6, \quad \phi''(\alpha_0) = 2 > 0$$

$$\alpha_1 = 5 - \frac{6}{2} = 5 - 3 = 2$$

第 1 轮： $\alpha_1 = 2$

$$\phi'(\alpha_1) = 2(2 - 2) = 0, \quad \phi''(\alpha_1) = 2$$

$$\alpha_2 = 2 - \frac{0}{2} = 2$$

结果：

- 仅 1 轮即收敛到精确最优解 $\alpha^* = 2$
- 第 2 轮验证： $\alpha_2 = 2$, $|\alpha_2 - \alpha_1| = 0 < \varepsilon$
- 这是因为目标函数恰好是二次函数，Newton 法具有二次终止性

例题 4.6.2 (Newton 法对非二次函数). 问题：用 Newton 法求解

$$\min \phi(\alpha) = \alpha^4 - 3\alpha^2 + \alpha + 1, \quad \alpha_0 = 1$$

执行两轮迭代。

解：

$$\phi'(\alpha) = 4\alpha^3 - 6\alpha + 1, \quad \phi''(\alpha) = 12\alpha^2 - 6$$

第 0 轮： $\alpha_0 = 1$

$$\phi'(1) = 4 - 6 + 1 = -1, \quad \phi''(1) = 12 - 6 = 6 > 0$$

$$\alpha_1 = 1 - \frac{-1}{6} = 1 + 0.1667 = 1.1667$$

第 1 轮： $\alpha_1 = 1.1667$

$$\phi'(1.1667) = 4(1.5879) - 6(1.1667) + 1 = 6.3517 - 7.0002 + 1 = 0.3515$$

$$\phi''(1.1667) = 12(1.3611) - 6 = 16.3332 - 6 = 10.3332$$

$$\alpha_2 = 1.1667 - \frac{0.3515}{10.3332} = 1.1667 - 0.0340 = 1.1327$$

继续迭代可收敛到最优解 $\alpha^* \approx 1.13$ 。

多元 Newton 法

上述一维 Newton 法可直接推广到多元情形。对多元函数 $f(x)$, $x \in \mathbb{R}^n$, 在 x_k 处做二阶 Taylor 展开:

$$f(x) \approx f(x_k) + \nabla f(x_k)^T(x - x_k) + \frac{1}{2}(x - x_k)^T \nabla^2 f(x_k)(x - x_k)$$

令梯度为零, 得到多元 Newton 法的迭代公式:

$$x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

其中 $\nabla f(x_k)$ 为梯度 (一维时退化为 ϕ'), $\nabla^2 f(x_k)$ 为 Hessian 矩阵 (一维时退化为 ϕ'')。

例题 4.6.3 (多元 Newton 法). 问题: 用 Newton 法求解

$$\min f(x_1, x_2) = x_1^2 + 4x_2^2 + x_1x_2 - 2x_1 - x_2, \quad x^{(0)} = (0, 0)$$

执行两轮迭代。

解: 计算梯度和 Hessian:

$$\nabla f(x) = \begin{bmatrix} 2x_1 + x_2 - 2 \\ 8x_2 + x_1 - 1 \end{bmatrix}, \quad H = \nabla^2 f(x) = \begin{bmatrix} 2 & 1 \\ 1 & 8 \end{bmatrix}$$

Hessian 为常数矩阵, 且 H 正定 ($\det H = 15 > 0$, $h_{11} = 2 > 0$), 故 f 为严格凸函数。

第 0 轮: $x^{(0)} = (0, 0)$

$$g_0 = \nabla f(0, 0) = \begin{bmatrix} -2 \\ -1 \end{bmatrix}$$

解方程 $Hd_0 = -g_0$:

$$\begin{bmatrix} 2 & 1 \\ 1 & 8 \end{bmatrix} \begin{bmatrix} d_1 \\ d_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

由第一式 $2d_1 + d_2 = 2$, 代入第二式 $d_1 + 8d_2 = 1$:

$$d_1 + 8(2 - 2d_1) = 1 \Rightarrow -15d_1 = -15 \Rightarrow d_1 = 1, \quad d_2 = 0$$

$$x^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

第 1 轮: $x^{(1)} = (1, 0)$

$$g_1 = \nabla f(1, 0) = \begin{bmatrix} 2(1) + 0 - 2 \\ 8(0) + 1 - 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

梯度为零, $x^{(1)} = (1, 0)$ 即为最优解。验证:

$$\nabla f = 0 \Rightarrow 2x_1 + x_2 = 2, x_1 + 8x_2 = 1 \Rightarrow x_1 = 1, x_2 = 0$$

结论: 由于目标函数是二次函数 (*Hessian* 为常数), 多元 *Newton* 法一步收敛到精确最优解。

4.6.2 插值法概述

Newton 法收敛速度快, 但需要计算二阶导数 $\phi''(\alpha)$ 。在许多实际问题中, 二阶导数难以获取或计算代价很高。**插值法** (Interpolation Method) 的核心思想是: 用已知的若干个函数值 (有时加上导数值) 构造一个简单的多项式来近似原函数, 然后求该多项式的极小点作为新的试探点。

插值法的基本思路

1. 选取若干个试探点, 计算对应的函数值 (或导数值)
2. 用这些信息构造一个低次多项式 $p(\lambda)$ 来近似 $\phi(\lambda)$
3. 求 $p(\lambda)$ 的极小点 λ^* , 作为新的试探点
4. 根据 $\phi(\lambda^*)$ 的值决定下一步操作

与 *Newton* 法的对比:

- *Newton* 法: 利用单点处的二阶 **Taylor 展开** (需要一阶、二阶导数)
- 插值法: 利用多个点的函数值构造近似多项式 (不需要高阶导数)
- 插值法通常具有**超线性收敛**速度, 介于线性收敛和二次收敛之间

常用的插值法按所用信息可分为两类:

- **仅用函数值:** 二次插值法 (需 3 个点)、三次插值法 (需 4 个点)
- **同时用函数值和导数值:** 三次 *Hermite* 插值 (需 2 个点各提供函数值和导数值)

4.6.3 二次插值法

二次插值法利用三个试探点的函数值构造二次多项式, 以其极小点作为新的试探点。

二次插值法原理

设已知三个点 $\lambda_1 < \lambda_2 < \lambda_3$ 及其函数值 ϕ_1, ϕ_2, ϕ_3 。

设插值多项式 $p(\lambda) = a\lambda^2 + b\lambda + c$ 满足：

$$\begin{cases} a\lambda_1^2 + b\lambda_1 + c = \phi_1 \\ a\lambda_2^2 + b\lambda_2 + c = \phi_2 \\ a\lambda_3^2 + b\lambda_3 + c = \phi_3 \end{cases}$$

解出 a, b 后，插值多项式的极小点为：

$$\lambda^* = -\frac{b}{2a} = \frac{1}{2} \frac{(\lambda_2^2 - \lambda_3^2)\phi_1 + (\lambda_3^2 - \lambda_1^2)\phi_2 + (\lambda_1^2 - \lambda_2^2)\phi_3}{(\lambda_2 - \lambda_3)\phi_1 + (\lambda_3 - \lambda_1)\phi_2 + (\lambda_1 - \lambda_2)\phi_3}$$

4.6.4 三次插值法

三次插值需要四个条件来确定参数。实用中有三种方式：

- 四个点的函数值
- 三个点的函数值 + 一个点的导数值
- 两个点的函数值 + 两个点的导数值（最常用）

方法比较

- **Newton 法**：二阶收敛，但需要二阶导数，初始点需在真解附近
- **插值法**：不需要显式计算二阶导数，超线性收敛
- **二次插值**：简单但精度有限
- **三次插值**（Davidon 搜索法）：利用更多信息，收敛更快
- 实际中常将多种方法配合使用

```

1 import numpy as np
2
3 def newton_line_search(phi, phi_grad, phi_hess, alpha0,
4                       eps=1e-6, max_iter=100):
5     """Newton法一维搜索"""
6     alpha = alpha0
7     history = [alpha]
8
9     for k in range(max_iter):
10         g = phi_grad(alpha)
11         h = phi_hess(alpha)
12

```

```

13     if h <= 0:
14         print(f"Warning: Hessian non-positive at alpha={alpha:.4f}
15               ")
16         break
17
18     alpha_new = alpha - g / h
19     history.append(alpha_new)
20
21     if abs(alpha_new - alpha) < eps or abs(g) < eps:
22         break
23
24     alpha = alpha_new
25
26     return alpha, phi(alpha), len(history)-1, history
27
28 def quadratic_interpolation(phi, lam1, lam2, lam3, max_iter=100):
29     """二次插值法"""
30     for k in range(max_iter):
31         phi1, phi2, phi3 = phi(lam1), phi(lam2), phi(lam3)
32
33         # 计算插值多项式的极小点
34         denom = ((lam2 - lam3)*phi1 + (lam3 - lam1)*phi2
35                 + (lam1 - lam2)*phi3)
36
37         if abs(denom) < 1e-12:
38             break
39
40         lam_star = 0.5 * ((lam2**2 - lam3**2)*phi1
41                           + (lam3**2 - lam1**2)*phi2
42                           + (lam1**2 - lam2**2)*phi3) / denom
43
44         # 更新三个点（保留 lam_star 和相邻两点）
45         phi_star = phi(lam_star)
46
47         if lam_star < lam2:
48             if phi_star < phi2:
49                 lam1, lam2, lam3 = lam1, lam_star, lam2
50             else:
51                 lam1, lam2, lam3 = lam_star, lam2, lam3

```

```

52         else:
53             if phi_star < phi2:
54                 lam1, lam2, lam3 = lam2, lam_star, lam3
55             else:
56                 lam1, lam2, lam3 = lam1, lam2, lam_star
57
58         return lam2, phi(lam2), k+1
59
60
61 # === 测试 ===
62 def phi(alpha):
63     return alpha**4 - 3*alpha**2 + alpha + 1
64
65 def phi_grad(alpha):
66     return 4*alpha**3 - 6*alpha + 1
67
68 def phi_hess(alpha):
69     return 12*alpha**2 - 6
70
71 print("=" * 50)
72 print("Newton法 (非二次函数) ")
73 print("=" * 50)
74 alpha_opt, phi_opt, iters, hist = newton_line_search(
75     phi, phi_grad, phi_hess, alpha0=1.0, eps=1e-4)
76 print(f"最优解: alpha* = {alpha_opt:.6f}")
77 print(f"最优值: phi* = {phi_opt:.6f}")
78 print(f"迭代次数: {iters}")
79 print(f"迭代历史: {[f'{a:.4f}' for a in hist]}")
80
81 print("\n\n" + "=" * 50)
82 print("二次插值法")
83 print("=" * 50)
84 alpha_opt2, phi_opt2, iters2 = quadratic_interpolation(phi, 0, 1, 2)
85 print(f"最优解: alpha* = {alpha_opt2:.6f}")
86 print(f"最优值: phi* = {phi_opt2:.6f}")
87 print(f"迭代次数: {iters2}")

```

Listing 4.4: Newton 法与插值法实现

4.7 Shubert-Piyavskii 方法

前述方法（黄金分割法、Fibonacci 法、二分法、Newton 法、插值法）一般都收敛到局部极小点，而无法收敛到全局极小点。Shubert-Piyavskii 方法是一种可以寻找全局最优解的方法。

4.7.1 基本原理

Shubert-Piyavskii 方法原理

假设 $\phi(\alpha)$ 在区间 $[a, b]$ 上是 Lipschitz 连续的，即存在常数 $L > 0$ 使得：

$$|\phi(\alpha_1) - \phi(\alpha_2)| \leq L|\alpha_1 - \alpha_2|, \quad \forall \alpha_1, \alpha_2 \in [a, b]$$

方法通过一步步构建函数的下界（锯齿形下界）来逼近全局最小值：

1. 在初始点计算函数值
2. 利用 Lipschitz 常数构造 V 形下界函数
3. 找到当前下界的最小值点，计算该点的真实函数值
4. 用新点更新下界（锯齿更密），重复直到收敛

4.7.2 下界函数的构造

设已在点 $\alpha_1, \alpha_2, \dots, \alpha_n$ 处计算了函数值 $\phi(\alpha_1), \dots, \phi(\alpha_n)$ 。

在两点 α_i 和 α_j 之间，由 Lipschitz 条件，函数值满足：

$$\phi(\alpha) \geq \max\{\phi(\alpha_i) - L|\alpha - \alpha_i|, \phi(\alpha_j) - L|\alpha - \alpha_j|\}$$

整个区间上的下界是这些 V 形函数的逐点最大值，形成一个锯齿形下界。

Shubert-Piyavskii 算法

输入：区间 $[a, b]$ ，Lipschitz 常数 L ，容许误差 ε

Step 0：计算 $\phi(a)$ 和 $\phi(b)$ ，构造初始下界

Step 1：找到当前下界函数的最低点 $\hat{\alpha}$

Step 2：计算 $\phi(\hat{\alpha})$

Step 3：在 $\hat{\alpha}$ 处更新下界（插入新的 V 形）

Step 4：若下界最小值与当前最好函数值之差 $< \varepsilon$ ，停；否则转 Step 1

输出：当前最优函数值点

Shubert-Piyavskii 方法的优缺点

优点:

- 能够找到全局最优解（而非局部最优）
- 有理论收敛保证

缺点:

- 需要预先知道 Lipschitz 常数 L （通常难以精确估计）
- 高维情况下计算量急剧增加（下界更新复杂）
- 收敛速度较慢

例题 4.7.1 (Shubert-Piyavskii 方法示例). **问题:** 用 Shubert-Piyavskii 方法的思路, 在 $[0, 3]$ 上寻找

$$\phi(\alpha) = \alpha \sin(\alpha)$$

的全局最小值。设 $L = 3$, 先在 $\alpha = 0$ 和 $\alpha = 3$ 处采样。

解:

初始: $\phi(0) = 0$, $\phi(3) = 3 \sin(3) \approx 0.423$

初始下界为两条 V 形线的上包络。最低点在两 V 形交点处:

$$0 - L\alpha = 0.423 - L(3 - \alpha) \Rightarrow -3\alpha = 0.423 - 9 + 3\alpha \Rightarrow \alpha = 1.4295$$

下界值为 $\phi_{LB}(1.4295) = -3 \times 1.4295 = -4.2885$

第一次迭代: 计算 $\phi(1.4295) = 1.4295 \times \sin(1.4295) \approx 1.4295 \times 0.990 = 1.415$

在 $\alpha = 1.4295$ 处插入新的 V 形, 更新下界。新的下界最低点可能在 $[0, 1.4295]$ 或 $[1.4295, 3]$ 中。

左侧交点:

$$0 - 3\alpha = 1.415 - 3(1.4295 - \alpha) \Rightarrow \alpha \approx 0.7148$$

下界值: -2.144

右侧交点类似计算。继续迭代直到下界与最好函数值之差足够小。

```

1 import numpy as np
2
3 def shubert_piyavskii(phi, a, b, L, eps=1e-4, max_iter=1000):
4     """
5     Shubert-Piyavskii方法求全局最小值
6     phi: 目标函数
7     [a,b]: 搜索区间

```

```

8      L: Lipschitz 常数
9      """
10     # 初始采样点
11     points = [(a, phi(a)), (b, phi(b))] # (alpha, phi(alpha))
12     best_val = min(p[1] for p in points)
13     best_alpha = points[0][0] if points[0][1] < points[1][1] else
        points[1][0]
14
15     for k in range(max_iter):
16         # 找到下界最低点 (V形交点)
17         min_lower_bound = float('inf')
18         min_alpha = None
19
20         points_sorted = sorted(points, key=lambda x: x[0])
21
22         for i in range(len(points_sorted) - 1):
23             ai, fi = points_sorted[i]
24             aj, fj = points_sorted[i + 1]
25
26             # 两点间的V形交点
27             #  $f_i - L*(alpha - ai) = f_j - L*(aj - alpha)$ 
28             alpha_int = (fi + L*ai + fj + L*aj) / (2*L) if L > 0 else
                (ai+aj)/2
29             lb_val = fi - L * (alpha_int - ai)
30
31             if ai <= alpha_int <= aj and lb_val < min_lower_bound:
32                 min_lower_bound = lb_val
33                 min_alpha = alpha_int
34
35             if min_alpha is None:
36                 break
37
38         # 计算新点的函数值
39         phi_new = phi(min_alpha)
40         points.append((min_alpha, phi_new))
41
42         # 更新最优
43         if phi_new < best_val:
44             best_val = phi_new
45             best_alpha = min_alpha

```

```
46
47     # 检查收敛
48     if best_val - min_lower_bound < eps:
49         break
50
51     return best_alpha, best_val, k+1, points
52
53
54 # === 测试:  $\alpha * \sin(\alpha)$  ===
55 def phi_test(alpha):
56     return alpha * np.sin(alpha)
57
58 print("=" * 50)
59 print("Shubert-Piyavskii 方法")
60 print("=" * 50)
61 alpha_opt, phi_opt, iters, pts = shubert_piyavskii(
62     phi_test, 0, 3, L=3.0, eps=0.01)
63 print(f"最优解:  $\alpha^* = \{\alpha\_opt:.6f\}$ ")
64 print(f"最优值:  $\phi^* = \{\phi\_opt:.6f\}$ ")
65 print(f"迭代次数: {iters}")
66 print(f"采样点数量: {len(pts)}")
67
68 # 与精确解对比
69 from scipy.optimize import minimize_scalar
70 res = minimize_scalar(phi_test, bounds=(0, 3), method='bounded')
71 print(f"\n精确解对比:  $\alpha^* = \{res.x:.6f\}$ ,  $\phi^* = \{res.fun:.6f\}$ ")
```

Listing 4.5: Shubert-Piyavskii 方法实现

4.8 不精确一维搜索

精确一维搜索需要花费大量计算，特别是在迭代点远离最优解时。不精确搜索只需找到使目标函数值“足够下降”的步长即可，大大节省了计算量。

4.8.1 Goldstein 条件

Goldstein 条件

步长 α_k 需满足以下两个不等式：

$$\phi(\alpha_k) \leq \phi(0) + c_1 \alpha_k \phi'(0) \quad (4.1)$$

$$\phi(\alpha_k) \geq \phi(0) + (1 - c_1) \alpha_k \phi'(0) \quad (4.2)$$

其中 $c_1 \in (0, \frac{1}{2})$ 为常数（通常取 $c_1 = 0.1$ ）。

条件(4.1)保证充分下降，条件(4.2)防止步长过小。

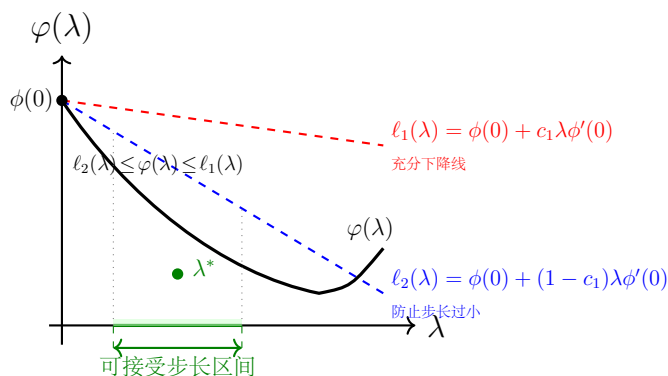


图 4.1: Goldstein 条件的几何解释：满足 $\ell_2(\lambda) \leq \phi(\lambda) \leq \ell_1(\lambda)$ 的 λ 在横轴上的投影构成可接受步长区间

几何意义（见图 4.1）：

- 条件(4.1)要求函数值在过 $(0, \phi(0))$ 斜率为 $c_1 \phi'(0)$ 的直线 ℓ_1 下方（充分下降）
- 条件(4.2)要求函数值在过 $(0, \phi(0))$ 斜率为 $(1 - c_1) \phi'(0)$ 的直线 ℓ_2 上方（避免步长过小）

Goldstein 条件要求函数值 $\phi(\lambda)$ 位于两条参考直线 $\ell_2(\lambda)$ 与 $\ell_1(\lambda)$ 之间，即满足 $\ell_2(\lambda) \leq \phi(\lambda) \leq \ell_1(\lambda)$ 。满足该不等式的 λ 在横轴上的投影构成可接受步长区间（图中绿色区间）。

4.8.2 Armijo 条件（回溯线搜索）

Armijo 条件

步长 α_k 满足：

$$\phi(\alpha_k) \leq \phi(0) + c_1 \alpha_k \phi'(0)$$

其中 $c_1 \in (0, 1)$ （通常取 $c_1 = 10^{-4}$ ）。

Armijo 回溯算法：

1. 给定初始步长 $\bar{\alpha} > 0$, 缩减因子 $\beta \in (0, 1)$ (通常 $\beta = 0.5$), $c_1 \in (0, 1)$
2. 令 $\alpha = \bar{\alpha}$
3. 若 $\phi(\alpha) \leq \phi(0) + c_1\alpha\phi'(0)$, 接受 α
4. 否则令 $\alpha \leftarrow \beta\alpha$, 转步骤 3

Armijo 回溯法算法

输入: $\phi(\alpha)$, $\phi(0)$, $\phi'(0) < 0$, $\bar{\alpha} > 0$, $c_1 \in (0, 1)$, $\beta \in (0, 1)$

Step 0: $\alpha = \bar{\alpha}$

Step 1: 若 $\phi(\alpha) \leq \phi(0) + c_1\alpha\phi'(0)$, 停, 输出 α

Step 2: $\alpha \leftarrow \beta\alpha$, 转 Step 1

例题 4.8.1 (Armijo 回溯法——迭代两次). **问题:** 设 $\phi(\alpha) = (1 + \alpha - 2)^2 = (\alpha - 1)^2$, $\phi(0) = 1$, $\phi'(0) = -2$. 用 Armijo 回溯法找可接受步长, $\bar{\alpha} = 2$, $c_1 = 0.1$, $\beta = 0.5$.

Armijo 条件: 需要 $\phi(\alpha) \leq 1 + 0.1 \times \alpha \times (-2) = 1 - 0.2\alpha$

初始: $\alpha = 2$

$$\phi(2) = (2 - 1)^2 = 1, \quad 1 - 0.2 \times 2 = 0.6$$

$1 \not\leq 0.6$, 不满足! 令 $\alpha = 0.5 \times 2 = 1$

第 1 次回溯: $\alpha = 1$

$$\phi(1) = (1 - 1)^2 = 0, \quad 1 - 0.2 \times 1 = 0.8$$

$0 \leq 0.8$, 满足! 接受 $\alpha = 1$ 。

结果: 经过 1 次回溯, 找到可接受步长 $\alpha = 1$ 。实际这也是精确最优步长。

4.8.3 Wolfe-Powell 条件

Wolfe 条件

步长 α_k 满足:

(1) **充分下降条件** (Armijo):

$$\phi(\alpha_k) \leq \phi(0) + c_1\alpha_k\phi'(0)$$

(2) **曲率条件:**

$$\phi'(\alpha_k) \geq c_2\phi'(0)$$

其中 $0 < c_1 < c_2 < 1$ (通常 $c_1 = 10^{-4}$, $c_2 = 0.9$)。

强 Wolfe 条件

将曲率条件加强为：

$$|\phi'(\alpha_k)| \leq c_2 |\phi'(0)|$$

这要求步长处的导数不仅大于 $c_2 \phi'(0)$ ，还要足够接近 0。

Wolfe 条件的意义

- 条件 (1) 保证充分下降
- 条件 (2) 保证步长不会太小（避免在下降段的起点就停下）
- 强 Wolfe 条件进一步要求步长接近精确线搜索的解
- Wolfe 条件常在 Newton 类方法中使用，但不太适合拟 Newton 方法

4.8.4 方法对比

表 4.1: 不精确搜索条件对比

条件	控制下降	控制步长大小	适用场景
Goldstein	上下界	双边界限制	一般下降法
Armijo	充分下降	回溯机制	最简单，最常用
Wolfe	充分下降 + 曲率	导数控制	Newton 类方法
强 Wolfe	充分下降 + 强曲率	导数绝对值控制	需要更精确时

Wolfe 条件线搜索算法

输入： $\phi(\alpha)$, $\phi(0)$, $\phi'(0) < 0$, $c_1 \in (0, 1)$, $c_2 \in (c_1, 1)$, $\alpha_{\max} > 0$

Step 0: $\alpha_0 = 0$, $\alpha_1 = \alpha_{\max}$ （或试探值）， $i = 1$

Step 1: 计算 $\phi(\alpha_i)$

Step 2: 若 $\phi(\alpha_i) > \phi(0) + c_1 \alpha_i \phi'(0)$ 或 $[\phi(\alpha_i) \geq \phi(\alpha_{i-1})$ 且 $i > 1]$ ，进入**区间收缩**（用二次插值在 $[\alpha_{i-1}, \alpha_i]$ 内找满足条件的步长）

Step 3: 计算 $\phi'(\alpha_i)$

Step 4: 若 $|\phi'(\alpha_i)| \leq -c_2 \phi'(0)$ ，停，输出 α_i

Step 5: 若 $\phi'(\alpha_i) \geq 0$ ，进入**区间收缩**

Step 6: 选择新的试探点 $\alpha_{i+1} > \alpha_i$, $i \leftarrow i + 1$ ，转 Step 1

例题 4.8.2 (Wolfe 条件验证). **问题：** 设 $\phi(\alpha) = (\alpha - 2)^2$, $\alpha_0 = 0$ 。验证 $\alpha = 1$ 是否满足 Wolfe 条件, $c_1 = 0.1$, $c_2 = 0.9$ 。

解：

$$\phi(\alpha) = (\alpha - 2)^2, \quad \phi(0) = 4, \quad \phi'(\alpha) = 2(\alpha - 2), \quad \phi'(0) = -4$$

检查条件 (1)——充分下降:

$$\phi(1) = (1 - 2)^2 = 1$$

右边: $\phi(0) + c_1 \cdot 1 \cdot \phi'(0) = 4 + 0.1 \times (-4) = 4 - 0.4 = 3.6$

$1 \leq 3.6$? 是的! 条件 (1) 满足。

检查条件 (2)——曲率:

$$\phi'(1) = 2(1 - 2) = -2$$

要求: $\phi'(1) \geq c_2 \phi'(0) = 0.9 \times (-4) = -3.6$

$-2 \geq -3.6$? 是的! 条件 (2) 满足。

结论: $\alpha = 1$ 满足 Wolfe 条件。

进一步: 是否满足强 Wolfe?

$$|\phi'(1)| = 2, \quad c_2 |\phi'(0)| = 0.9 \times 4 = 3.6$$

$2 \leq 3.6$? 是的! $\alpha = 1$ 也满足强 Wolfe 条件。

```

1 import numpy as np
2
3 def armijo_backtracking(phi, phi0, dphi0, alpha_bar=1.0,
4                         c1=1e-4, beta=0.5, max_iter=50):
5     """
6     Armijo 回溯法
7     phi: phi(alpha) 函数
8     phi0: phi(0)
9     dphi0: phi'(0) (需 < 0)
10    """
11    alpha = alpha_bar
12    for k in range(max_iter):
13        if phi(alpha) <= phi0 + c1 * alpha * dphi0:
14            return alpha, k+1
15        alpha *= beta
16    return alpha, max_iter
17
18
19 def wolfe_line_search(phi, dphi, phi0, dphi0, alpha=1.0,
20                      c1=1e-4, c2=0.9, max_iter=50):
21    """
22    简化的 Wolfe 条件线搜索
23    phi: phi(alpha) 函数

```

```
24     dphi: phi'(alpha) 函数
25     """
26     alpha_prev = 0
27     phi_prev = phi0
28
29     for k in range(max_iter):
30         phi_k = phi(alpha)
31
32         # 条件1: 充分下降
33         if phi_k > phi0 + c1 * alpha * dphi0:
34             # 用二分法在 [alpha_prev, alpha] 中搜索
35             lo, hi = alpha_prev, alpha
36             for _ in range(20):
37                 mid = (lo + hi) / 2
38                 phi_mid = phi(mid)
39                 if phi_mid > phi0 + c1 * mid * dphi0:
40                     hi = mid
41                 else:
42                     dphi_mid = dphi(mid)
43                     if abs(dphi_mid) <= -c2 * dphi0:
44                         return mid, k+1
45                     if dphi_mid < 0:
46                         lo = mid
47                     else:
48                         hi = mid
49             return (lo + hi) / 2, k+1
50
51         # 条件2: 曲率
52         dphi_k = dphi(alpha)
53         if abs(dphi_k) <= -c2 * dphi0:
54             return alpha, k+1
55
56         if dphi_k >= 0:
57             # 反向二分
58             lo, hi = alpha_prev, alpha
59             for _ in range(20):
60                 mid = (lo + hi) / 2
61                 dphi_mid = dphi(mid)
62                 phi_mid = phi(mid)
63                 if phi_mid > phi0 + c1 * mid * dphi0:
```

```

64         hi = mid
65         elif abs(dphi_mid) <= -c2 * dphi0:
66             return mid, k+1
67         elif dphi_mid < 0:
68             lo = mid
69         else:
70             hi = mid
71         return (lo + hi) / 2, k+1
72
73     alpha_prev = alpha
74     phi_prev = phi_k
75     alpha *= 2 # 扩大步长
76
77     return alpha, max_iter
78
79
80 # === 测试 ===
81 def phi(alpha):
82     return (alpha - 2)**2
83
84 def dphi(alpha):
85     return 2 * (alpha - 2)
86
87 phi0 = phi(0) # 4
88 dphi0 = dphi(0) # -4
89
90 print("=" * 50)
91 print("Armijo回溯法")
92 print("=" * 50)
93 alpha_a, iters_a = armijo_backtracking(phi, phi0, dphi0,
94                                     alpha_bar=2.0)
95 print(f"步长: alpha = {alpha_a:.6f}")
96 print(f"迭代次数: {iters_a}")
97 print(f"phi(alpha) = {phi(alpha_a):.6f}")
98
99 print("\n" + "=" * 50)
100 print("Wolfe条件搜索")
101 print("=" * 50)
102 alpha_w, iters_w = wolfe_line_search(phi, dphi, phi0, dphi0)
103 print(f"步长: alpha = {alpha_w:.6f}")

```

```
104 print(f"迭代次数: {iters_w}")
105 print(f"phi(alpha) = {phi(alpha_w):.6f}")
106 print(f"phi'(alpha) = {dphi(alpha_w):.6f}")
```

Listing 4.6: 不精确搜索条件实现

4.9 一维搜索方法效率比较

设初始区间长度为 L ，终止精度为 ε ：

表 4.2: 一维搜索方法效率比较

方法	所需试验点	区间缩减率
一致搜索（均匀网格）	$\lceil L/\varepsilon \rceil - 1$	—(固定)
Dichotomous 搜索	$\approx 2\log_2(L/\varepsilon)$	0.5
黄金分割法	$\approx \log(L/\varepsilon)/\log(1/\tau)$	$\tau \approx 0.618$
Fibonacci 搜索	最少（固定次数最优）	F_{n-1}/F_n
二分法（导数）	$\approx \log_2(L/\varepsilon)$	0.5
Newton 法（导数）	$\approx \log \log(L/\varepsilon)$ （二次收敛）	超线性

方法选择建议

- **不知导数：** 首选黄金分割法（简单实用），Fibonacci 法（固定次数最优）
- **可计算导数：** 首选二分法（导数版），Newton 法（二阶收敛快但需二阶导数）
- **多维优化中的线搜索：** 首选非精确线搜索（Armijo 回溯或 Wolfe 条件），计算效率高
- 在零阶方法中，Fibonacci 搜索最有效（最少观察点），黄金分割次之

4.10 本章小结

4.10.1 核心思想

一维搜索的两大统一原则

1. **区间缩减原则**（精确搜索）：利用单峰性，通过比较试探点函数值（或导数符号）逐步缩小区间，直至包含唯一极小点。
2. **充分下降原则**（非精确搜索）：不要求找到最优步长，只需保证目标函数值”足够下降”，平衡计算代价与收敛需求。

4.10.2 方法决策树

如何选择一维搜索方法

1. 问题类型判断

• 是否为多维优化中的步长搜索？→ 使用非精确线搜索（Armijo/Wolfe）

• 是否为一维函数求极小？→ 继续判断

2. 是否可计算导数？

• 不可计算：黄金分割法（首选）或 Fibonacci 法（固定次数最优）

• 可计算一阶导数：二分法（导数版）

• 可计算二阶导数：Newton 法（二次收敛最快）

3. 是否需要全局最优保证？

• 函数满足 Lipschitz 连续：Shubert-Piyavskii 方法

4.10.3 本章知识框架

表 4.3: 一维搜索与线搜索方法体系

类型	方法	所需信息	核心特征
零阶精确	Dichotomous 搜索	函数值	对称取点，区间减半
	黄金分割法	函数值	收缩比 $\tau = 0.618$
	Fibonacci 法	函数值	固定次数最优
导数精确	二分法（导数）	一阶导	区间减半
	Newton 法	一/二阶导	二次收敛
	插值法	函数值 $\pm$ 导	二次/三次插值，超线性
全局优化	Shubert-Piyavskii	函数值 +Lips.	唯一全局最优保证
非精确搜索	Armijo（回溯）	一阶导	回溯缩步，最常用
	Wolfe 条件	一阶导	下降 + 曲率
	Goldstein 条件	一阶导	双边控制

4.10.4 关键公式汇总

表 4.4: 核心公式速查

方法	核心公式
零阶精确搜索	
Dichotomous 搜索	$\lambda_k, \mu_k = \frac{a_k+b_k}{2} \mp \delta$; 比较 $\phi(\lambda_k), \phi(\mu_k)$ 缩小区间
黄金分割	收缩比 $\rho = \frac{\sqrt{5}-1}{2} \approx 0.618$; $\lambda = a + (1 - \rho)(b - a)$, $\mu = a + \rho(b - a)$
Fibonacci	收缩比 $\rho_k = F_{n-k}/F_{n-k+1}$; 需预先确定迭代次数 n
导数精确搜索	
二分法	$\lambda = \frac{a+b}{2}$; $\phi'(\lambda) < 0$ 则 $a \leftarrow \lambda$, 否则 $b \leftarrow \lambda$
Newton 迭代	$\alpha_{k+1} = \alpha_k - \phi'(\alpha_k)/\phi''(\alpha_k)$
二次插值	$\alpha^* = \frac{\phi(\alpha_1)\alpha_2^2 - \phi(\alpha_2)\alpha_1^2 + \phi(0)(\alpha_1^2 - \alpha_2^2)}{2[\phi(\alpha_1)\alpha_2 - \phi(\alpha_2)\alpha_1 + \phi(0)(\alpha_1 - \alpha_2)]}$
非精确线搜索	
Armijo	$\phi(\alpha) \leq \phi(0) + c_1\alpha\phi'(0)$, 不满足则 $\alpha \leftarrow \beta\alpha$
Wolfe 条件	充分下降: $\phi(\alpha) \leq \phi(0) + c_1\alpha\phi'(0)$; 曲率: $\phi'(\alpha) \geq c_2\phi'(0)$
强 Wolfe	充分下降 + $ \phi'(\alpha) \leq c_2 \phi'(0) $
Goldstein	$\phi(0) + (1 - c_1)\alpha\phi'(0) \leq \phi(\alpha) \leq \phi(0) + c_1\alpha\phi'(0)$

4.10.5 Python 综合示例

```

1 import numpy as np
2 from scipy.optimize import minimize_scalar, line_search
3
4 def unified_line_search(f, xk, pk, grad_f=None, method='brent',
5                          bounds=None, **kwargs):
6     """
7     统一的一维搜索接口
8
9     Parameters:
10     -----
11     f : callable
12         目标函数 f(x)
13     xk : ndarray
14         当前点

```

```

15     pk : ndarray
16         搜索方向
17     grad_f : callable, optional
18         梯度函数
19     method : str
20         'brent', 'golden', 'bounded', 'armijo', 'wolfe'
21     bounds : tuple, optional
22         (lower, upper) for bounded methods
23
24     Returns:
25     -----
26     alpha : float
27         最优步长
28     info : dict
29         搜索信息
30     """
31     # 定义  $\phi(\alpha) = f(x_k + \alpha * pk)$ 
32     def phi(alpha):
33         return f(xk + alpha * pk)
34
35     if method in ['brent', 'golden']:
36         if bounds is None:
37             # 需要找到包含最小值的区间
38             a, b = bracket_minimum(phi, alpha=0.0, s=1e-2)
39             bounds = (a, b)
40             res = minimize_scalar(phi, bounds=bounds, method=method)
41             return res.x, {'fun': res.fun, 'nit': res.nit}
42
43     elif method == 'armijo':
44         c1 = kwargs.get('c1', 1e-4)
45         alpha0 = kwargs.get('alpha0', 1.0)
46         beta = kwargs.get('beta', 0.5)
47
48         phi0 = f(xk)
49         # 数值计算方向导数
50         eps = 1e-8
51         dphi0 = (f(xk + eps * pk) - phi0) / eps
52
53         alpha = alpha0
54         for k in range(50):

```

```

55         if f(xk + alpha * pk) <= phi0 + c1 * alpha * dphi0:
56             return alpha, {'iterations': k+1}
57         alpha *= beta
58         return alpha, {'iterations': 50, 'warning': 'max_iter'}
59
60     elif method == 'wolfe' and grad_f is not None:
61         def grad_phi(alpha):
62             return np.dot(grad_f(xk + alpha * pk), pk)
63
64         res = line_search(f, grad_f, xk, pk,
65                          c1=kwargs.get('c1', 1e-4),
66                          c2=kwargs.get('c2', 0.9))
67         if res[0] is not None:
68             return res[0], {'iterations': res[1]}
69         else:
70             return 1e-3, {'warning': 'line_search_failed'}
71
72     else:
73         raise ValueError(f"Unknown method: {method}")
74
75
76 def bracket_minimum(phi, alpha=0.0, s=1e-2, k=2.0):
77     """通过外推法找到包含最小值的区间 [a,b]"""
78     a, ya = alpha, phi(alpha)
79     b, yb = a + s, phi(a + s)
80
81     if yb > ya:
82         a, b = b, a
83         ya, yb = yb, ya
84         s = -s
85
86     while True:
87         c, yc = b + s, phi(b + s)
88         if yc > yb:
89             return (min(a, c), max(a, c))
90         a, ya = b, yb
91         b, yb = c, yc
92         s *= k
93
94

```

```

95 # === 综合测试 ===
96 def rosenbrock(x):
97     """Rosenbrock函数"""
98     return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2
99
100 def rosenbrock_grad(x):
101     """Rosenbrock函数的梯度"""
102     d0 = -2*(1 - x[0]) - 400*x[0]*(x[1] - x[0]**2)
103     d1 = 200*(x[1] - x[0]**2)
104     return np.array([d0, d1])
105
106 # 测试各种方法
107 xk = np.array([-1.0, 2.0])
108 # 负梯度方向
109 pk = -rosenbrock_grad(xk)
110
111 print("=" * 60)
112 print("一维搜索方法对比 (Rosenbrock函数)")
113 print("=" * 60)
114
115 for method in ['brent', 'golden', 'armijo']:
116     alpha, info = unified_line_search(
117         rosenbrock, xk, pk, method=method)
118     x_new = xk + alpha * pk
119     print(f"\n{method:10s}: alpha={alpha:.6f}, "
120           f"f_new={rosenbrock(x_new):.6f}, info={info}")
121
122 # Wolfe条件
123 alpha_w, info_w = unified_line_search(
124     rosenbrock, xk, pk, grad_f=rosenbrock_grad, method='wolfe')
125 x_new_w = xk + alpha_w * pk
126 print(f"\n{'wolfe':10s}: alpha={alpha_w:.6f}, "
127       f"f_new={rosenbrock(x_new_w):.6f}, info={info_w}")

```

Listing 4.7: 统一一维搜索接口

4.10.6 进一步阅读

后续章节关联

- 第 5 章 (无约束最优化方法): 最速下降法、Newton 法、共轭梯度法、拟 Newton 法都以一维搜索作为步长子程序
- 第 7 章 (约束最优化方法): 罚函数法、内点法、增广 Lagrange 法中也需要一维搜索确定步长
- 实践建议: scipy.optimize 默认使用 Brent 方法 (结合黄金分割和抛物线插值), 一般 2-3 步即可找到满意步长; 深度学习中常用 Armijo 回溯法

4.10.7 关键术语中英对照

中文	英文
精确一维搜索	Exact Line Search
非精确线搜索	Inexact Line Search
Dichotomous 搜索	Dichotomous Search
黄金分割法	Golden Section Method
Fibonacci 法	Fibonacci Search Method
二分法/中点法	Bisection Method
二次插值法	Quadratic Interpolation
三次插值法	Cubic Interpolation
回溯法	Backtracking Line Search
Armijo 条件	Armijo Condition
Goldstein 条件	Goldstein Condition
Wolfe 条件	Wolfe Condition
强 Wolfe 条件	Strong Wolfe Condition
充分下降条件	Sufficient Decrease Condition
曲率条件	Curvature Condition
单峰函数	Unimodal Function
Lipschitz 连续	Lipschitz Continuous

Chapter 5

无约束最优化方法

5.1 引言

5.1.1 为什么研究无约束最优化

实际优化问题通常含有大量约束，但研究无约束最优化方法具有重要意义：

1. **转化工具**：许多算法可通过 Lagrangian 乘子法将有约束问题转化为一系列无约束问题，如 Lagrangian 对偶、鞍点最优性条件、惩罚函数法和障碍函数法。
2. **基础组件**：大多数方法通过寻找方向并沿方向最小化来推进，这种线搜索本质上是无约束或简单约束的最小化问题。
3. **自然推广**：几种无约束最优化方法可自然推广到有约束问题。

5.1.2 问题描述

无约束最优化问题的标准形式：

$$\min_{x \in \mathbb{R}^n} f(x)$$

关键假设

- **零阶方法**：仅需函数值 $f(x)$
- **一阶方法**：需要梯度 $\nabla f(x)$
- **二阶方法**：需要 Hessian 矩阵 $\nabla^2 f(x)$

5.1.3 方法分类

本笔记涵盖的方法体系如下：

关于方法分类

本章聚焦**多维无约束最优化**的核心算法。一维线搜索方法（如二分搜索、黄金分割法、Fibonacci 法）和非精确线搜索准则（Wolfe 条件、Armijo 规则）已在第 4 章系统介绍，作为多维优化算法中的线搜索子程序。本章涵盖所有多维无约束优化方法，从仅需函数值的零阶方法到需要导数和 Hessian 的高阶方法。

表 5.1: 无约束最优化方法分类		
类别	方法	所需信息
多维无导数	坐标轮换法	函数值
	Hooke-Jeeves 方法	函数值
	Rosenbrock 方法	函数值
一阶方法	最速下降法（梯度法）	梯度
	共轭梯度法	梯度
二阶方法	牛顿法	Hessian
	Levenberg-Marquardt	Hessian 修正
	信任域方法	Hessian 近似
拟牛顿法	DFP、BFGS	梯度
非光滑优化	次梯度方法	次梯度

5.1.4 收敛速度记号

定义 5.1.1 (收敛速度). 设序列 $\{x_k\}$ 收敛到 x^* :

- 线性收敛: $\|x_{k+1} - x^*\| \leq q\|x_k - x^*\|$, $q \in (0, 1)$
- 超线性收敛: $\|x_{k+1} - x^*\|/\|x_k - x^*\| \rightarrow 0$
- 二次收敛: $\|x_{k+1} - x^*\| \leq C\|x_k - x^*\|^2$

5.2 多维无导数搜索方法

对于问题 $\min_{x \in \mathbb{R}^n} f(x)$, 当导数信息不可得或计算代价高昂时, 使用仅依赖函数值的方法。

5.2.1 坐标轮换法 (Cyclic Coordinate Method)

基本思想

沿坐标轴方向依次搜索，每次只改变一个变量。搜索方向为标准基向量 $e_1, e_2, \dots, e_n$ 。

算法步骤：

Algorithm 3 坐标轮换法

Require: 初始点 $x^{(0)}$ ，精度 $\varepsilon > 0$

```

1:  $k \leftarrow 0$ 
2: while 未收敛 do
3:    $y^{(0)} \leftarrow x^{(k)}$ 
4:   for  $i = 1$  to  $n$  do
5:     求  $\lambda_i = \arg \min f(y^{(i-1)} + \lambda e_i)$ 
6:      $y^{(i)} = y^{(i-1)} + \lambda_i e_i$ 
7:   end for
8:    $x^{(k+1)} \leftarrow y^{(n)}$ 
9:   if  $\|x^{(k+1)} - x^{(k)}\| < \varepsilon$  then
10:    break
11:  end if
12:   $k \leftarrow k + 1$ 
13: end while
14: return  $x^{(k+1)}$ 

```

例题 5.2.1 (坐标轮换法). 求解 $\min f(x) = x_1^2 - x_1 x_2 + x_2^2 - 2x_1$ ，初始点 $(0, 3)$ ，最优解 $(2, 1)$ 。

第 1 轮迭代：

沿 $e_1 = (1, 0)$ 搜索： $\min f(\lambda, 3) = \lambda^2 - 3\lambda + 9 - 2\lambda$

$$\frac{df}{d\lambda} = 2\lambda - 5 = 0 \Rightarrow \lambda = 2.5$$

得 $y^{(1)} = (2.5, 3)$

沿 $e_2 = (0, 1)$ 搜索： $\min f(2.5, 3 + \lambda) = 6.25 - 2.5(3 + \lambda) + (3 + \lambda)^2 - 5$

$$\frac{df}{d\lambda} = -2.5 + 6 + 2\lambda = 0 \Rightarrow \lambda = -1.75$$

得 $x^{(1)} = (2.5, 1.25)$

继续迭代直至收敛。

坐标轮换法的特点

- 每轮迭代进行 n 次一维搜索
- 函数可微时收敛到梯度为零的点
- 不可微时可能在非最优点停止
- 收敛速度与最速下降法相当
- 可使用 Gauss-Southwell 变体：每次选偏导数绝对值最大的方向

5.2.2 Hooke-Jeeves 方法**基本思想**

结合两种搜索策略：

- **试探搜索 (Exploratory Search)**：沿各坐标方向试探，寻找改进方向
- **模式搜索 (Pattern Search)**：沿成功方向加速前进

算法步骤：

1. **初始化**： $x^{(0)}$ (基点)，步长 $\delta > 0$ ，缩减因子 $\rho \in (0, 1)$
2. **Step 1 (试探搜索)**：从 $y = x^{(k)}$ 出发，沿各坐标方向 e_i 试探：
 - 若 $f(y + \delta e_i) < f(y)$ ，则 $y \leftarrow y + \delta e_i$ (成功)
 - 否则若 $f(y - \delta e_i) < f(y)$ ，则 $y \leftarrow y - \delta e_i$
 - 否则保持 y 不变 (失败)

设结果为 $x^{(k+1)}$

3. **Step 2 (模式搜索判断)**：

- 若 $f(x^{(k+1)}) < f(x^{(k)})$ (试探成功)：
 - 沿模式方向 $d = x^{(k+1)} - x^{(k)}$ 前进： $y = x^{(k+1)} + d = 2x^{(k+1)} - x^{(k)}$
 - $x^{(k)} \leftarrow x^{(k+1)}$ ，从 y 转 Step 1
- 否则 (试探失败)：
 - 若 $\delta < \varepsilon$ ，停止
 - 否则 $\delta \leftarrow \rho\delta$ ，从 $x^{(k)}$ 转 Step 1

例题 5.2.2 (Hooke-Jeeves 方法). 求解 $\min f(x) = x_1^2 - x_1x_2 + x_2^2 - 2x_1$, 初始点 $(0, 0)$, $\delta = 0.5$ 。

第 1 轮:

从 $(0, 0)$ 试探: $-e_1$ 方向: $f(0.5, 0) = -0.75 < f(0, 0) = 0$ (成功), $y = (0.5, 0) - e_2$
 方向: $f(0.5, 0.5) = -0.5 < f(0.5, 0) = -0.75$ (失败), $f(0.5, -0.5) = 0.25 > -0.75$ (失败)

得 $x^{(1)} = (0.5, 0)$, 试探成功。

模式方向 $d = (0.5, 0) - (0, 0) = (0.5, 0)$, 新模式基点: $y = (0.5, 0) + (0.5, 0) = (1, 0)$

继续迭代直至满足精度要求。

5.2.3 Rosenbrock 方法

基本思想

每次迭代沿 n 个线性无关的正交方向进行一维搜索, 搜索完成后根据搜索轨迹构造一组新的正交方向 (Gram-Schmidt 正交化)。

算法步骤:

Step 1: 给定初始点 $x^{(0)}$, 初始正交方向 $d_1, \dots, d_n$ (通常取坐标轴), 步长 $\delta_1, \dots, \delta_n$

Step 2: 沿各方向依次搜索:

- 对 $i = 1, \dots, n$:
 - 若 $f(x + \delta_i d_i) < f(x)$, 成功: $x \leftarrow x + \delta_i d_i$, $\delta_i \leftarrow \alpha \delta_i$ ($\alpha > 1$)
 - 否则失败: $\delta_i \leftarrow -\beta \delta_i$ ($\beta \in (0, 1)$)

Step 3: 一轮完成后若所有方向都有成功搜索, 转 Step 2

Step 4: 若存在未成功的方向:

- 若 $|\delta_i| < \varepsilon$ (所有 i), 停止
- 否则, 利用 Gram-Schmidt 正交化构造新的正交方向, 转 Step 2

Gram-Schmidt 正交化

设第 k 轮搜索的位移为 $\Delta x_1, \dots, \Delta x_n$, 新方向构造为:

$$s_1 = \Delta x_1, \quad d_1^{new} = \frac{s_1}{\|s_1\|}$$

$$s_i = \Delta x_i - \sum_{j=1}^{i-1} \frac{\Delta x_i^T s_j}{s_j^T s_j} s_j, \quad d_i^{new} = \frac{s_i}{\|s_i\|}$$

5.2.4 Python 代码实现

坐标轮换法

```
1 import numpy as np
2 from scipy.optimize import minimize_scalar
3
4 def cyclic_coordinate(f, x0, eps=1e-6, max_iter=1000):
5     """
6     坐标轮换法
7
8     Parameters:
9         f: 目标函数
10        x0: 初始点
11        eps: 精度
12        max_iter: 最大迭代次数
13
14    Returns:
15        x_opt: 最优解
16        f_opt: 最优值
17        history: 迭代历史
18    """
19    n = len(x0)
20    x = x0.copy()
21    history = [x.copy()]
22
23    for _ in range(max_iter):
24        x_old = x.copy()
25
26        for i in range(n):
27            # 沿第i个坐标方向搜索
28            def phi(lam):
29                x_new = x.copy()
30                x_new[i] += lam
31                return f(x_new)
32
33            res = minimize_scalar(phi)
34            x[i] += res.x
35
36        history.append(x.copy())
37
```

```

38         if np.linalg.norm(x - x_old) < eps:
39             break
40
41     return x, f(x), history
42
43 # 示例
44 def f(x):
45     return x[0]**2 - x[0]*x[1] + x[1]**2 - 2*x[0]
46
47 x0 = np.array([0.0, 3.0])
48 x_opt, f_opt, hist = cyclic_coordinate(f, x0)
49 print(f"最优解: {x_opt}")
50 print(f"最优值: {f_opt:.6f}")

```

Hooke-Jeeves 方法

```

1 def hooke_jeeves(f, x0, delta=0.5, rho=0.5, eps=1e-6, max_iter
  =1000):
2     """
3     Hooke-Jeeves 模式搜索法
4
5     Parameters:
6         f: 目标函数
7         x0: 初始点
8         delta: 初始步长
9         rho: 步长缩减因子
10        eps: 精度
11        max_iter: 最大迭代次数
12
13    Returns:
14        x_opt: 最优解
15        history: 迭代历史
16    """
17    n = len(x0)
18    x_base = x0.copy()
19    x = x0.copy()
20    history = [x.copy()]
21
22    for _ in range(max_iter):

```

```
23     # 试探搜索
24     y = x_base.copy()
25     success = False
26
27     for i in range(n):
28         e_i = np.zeros(n)
29         e_i[i] = 1
30
31         # 正向试探
32         if f(y + delta * e_i) < f(y):
33             y = y + delta * e_i
34             success = True
35
36         # 负向试探
37         elif f(y - delta * e_i) < f(y):
38             y = y - delta * e_i
39             success = True
40
41         if success:
42             # 模式搜索
43             x = y + (y - x_base)
44             x_base = y.copy()
45             history.append(y.copy())
46         else:
47             if delta < eps:
48                 break
49             delta *= rho
50             x = x_base.copy()
51
52     return x_base, history
53
54 # 示例
55 x0 = np.array([0.0, 0.0])
56 x_opt, hist = hooke_jeeves(f, x0, delta=0.5)
57 print(f"最优解: {x_opt}")
```

5.2.5 方法比较

表 5.2: 多维无导数方法比较

方法	搜索方向	步长策略	收敛性
坐标轮换	固定坐标轴	精确线搜索	线性收敛
Hooke-Jeeves	坐标轴 + 模式方向	离散步长	局部收敛
Rosenbrock	正交方向（自适应）	离散步长	局部收敛

5.3 最速下降法（梯度法）

5.3.1 基本思想

最速下降方向

1847 年 Cauchy 提出。在点 x 处，归一化的最速下降方向为：

$$d = -\frac{\nabla f(x)}{\|\nabla f(x)\|}$$

即负梯度方向，是在单位球面上使函数局部下降最快的方向。

引理 5.3.1 (最速下降方向的验证). 设 $f : \mathbb{R}^n \rightarrow \mathbb{R}$ 在 x 处可微, $\nabla f(x) \neq 0$ 。求解：

$$\min_{\|d\| \leq 1} \nabla f(x)^T d$$

其解为 $d = -\frac{\nabla f(x)}{\|\nabla f(x)\|}$ 。

证明：由 Cauchy-Schwarz 不等式：

$$\nabla f(x)^T d \geq -\|\nabla f(x)\| \cdot \|d\| \geq -\|\nabla f(x)\|$$

等号成立当且仅当 $d = -\frac{\nabla f(x)}{\|\nabla f(x)\|}$ 。

5.3.2 最速下降法算法

Algorithm 4 最速下降法

Require: 初始点 $x^{(0)}$, 精度 $\varepsilon > 0$

```

1:  $k \leftarrow 0$ 
2: while  $\|\nabla f(x^{(k)})\| \geq \varepsilon$  do
3:    $d_k = -\nabla f(x^{(k)})$ 
4:   求  $\lambda_k = \arg \min_{\lambda \geq 0} f(x^{(k)} + \lambda d_k)$ 
5:    $x^{(k+1)} = x^{(k)} + \lambda_k d_k$ 
6:    $k \leftarrow k + 1$ 
7: end while
8: return  $x^{(k)}$ 

```

例题 5.3.1 (最速下降法). 求解 $\min f(x) = x_1^2 + 4x_2^2$, 初始点 $\begin{bmatrix} 4 \\ 1 \end{bmatrix}$ 。

$$\nabla f(x) = (2x_1, 8x_2), \quad x^{(0)} = (4, 1)$$

第 1 次迭代: $d_0 = (-8, -8)$

$$\phi(\lambda) = f(4 - 8\lambda, 1 - 8\lambda) = (4 - 8\lambda)^2 + 4(1 - 8\lambda)^2$$

$$\phi'(\lambda) = -16(4 - 8\lambda) - 64(1 - 8\lambda) = -128 + 640\lambda = 0$$

$$\text{解得 } \lambda_0 = 0.2, \quad x^{(1)} = (2.4, -0.6)$$

第 2 次迭代: $d_1 = (-4.8, 4.8)$

$$\phi(\lambda) = f(2.4 - 4.8\lambda, -0.6 + 4.8\lambda) = (2.4 - 4.8\lambda)^2 + 4(-0.6 + 4.8\lambda)^2$$

$$\phi'(\lambda) = -46.08 + 230.4\lambda = 0$$

$$\text{解得 } \lambda_1 = 0.2, \quad x^{(2)} = (1.44, 0.36), \text{ 继续迭代。}$$

5.3.3 步长选择策略

策略 1: 固定步长

$$\lambda_k = \lambda \text{ (常数)}.$$

定理 5.3.1 (固定步长收敛条件). 设 $f \in C^{1,1}(\mathbb{R}^n)$ (*Lipschitz* 连续梯度, 常数为 L), 若 $0 < \lambda < \frac{2}{L}$, 则:

$$f(x^{(k+1)}) - f(x^*) \leq f(x^{(k)}) - f(x^*) - \lambda \left(1 - \frac{L\lambda}{2}\right) \|\nabla f(x^{(k)})\|^2$$

最优固定步长: $\lambda = \frac{1}{L}$ 。

策略 2: 精确线搜索 (全松弛)

$$\lambda_k = \arg \min_{\lambda \geq 0} f(x^{(k)} + \lambda d_k).$$

定理 5.3.2 (精确线搜索的收敛率). 设 f 为强凸函数（参数 μ ）且 L -光滑，则：

$$f(x^{(k+1)}) - f^* \leq \left(\frac{L - \mu}{L + \mu} \right)^2 (f(x^{(k)}) - f^*)$$

即线性收敛，收敛因子为 $\frac{L-\mu}{L+\mu} = 1 - \frac{2}{L/\mu+1}$ 。

条件数的影响

定义条件数 $\kappa = \frac{L}{\mu}$ （Hessian 最大与最小特征值之比）。

收敛因子 $\approx 1 - \frac{2}{\kappa}$ ，当 κ 很大时（病态问题），收敛极慢。

策略 3: Armijo 规则

实际中最常用的策略（详见第??节）。

5.3.4 梯度法的收敛性分析

定理 5.3.3 (梯度下降下降量). 设 $f \in C^{1,1}$ 且 L -光滑，采用步长 $\lambda_k = \frac{1}{L}$ ，则：

$$f(x^{(k+1)}) \leq f(x^{(k)}) - \frac{1}{2L} \|\nabla f(x^{(k)})\|^2$$

证明： 由 Lipschitz 梯度的二次上界：

$$f(y) \leq f(x) + \nabla f(x)^T(y - x) + \frac{L}{2} \|y - x\|^2$$

代入 $y = x - \frac{1}{L} \nabla f(x)$ ：

$$f(y) \leq f(x) - \frac{1}{L} \|\nabla f(x)\|^2 + \frac{1}{2L} \|\nabla f(x)\|^2 = f(x) - \frac{1}{2L} \|\nabla f(x)\|^2$$

凸函数情形

定理 5.3.4 (凸函数收敛率). 设 f 为凸函数且 L -光滑， $\|x^{(0)} - x^*\| \leq R$ ，则梯度法（步长 $\frac{1}{L}$ ）满足：

$$f(x^{(k)}) - f^* \leq \frac{2LR^2}{k}$$

即收敛速率为 $O(1/k)$ 。

强凸函数情形

定理 5.3.5 (强凸函数线性收敛). 设 f 为 μ -强凸且 L -光滑，则：

$$\|x^{(k)} - x^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \|x^{(0)} - x^*\|^2$$

即线性收敛。

5.3.5 梯度法的局限性

1. 锯齿现象：相邻搜索方向正交，在最优解附近路径呈锯齿状
2. 条件数敏感：对病态问题（ $\kappa \gg 1$ ）收敛极慢
3. 只能保证收敛到稳定点：不一定收敛到局部极小点
4. 鞍点困难：可能停滞在鞍点

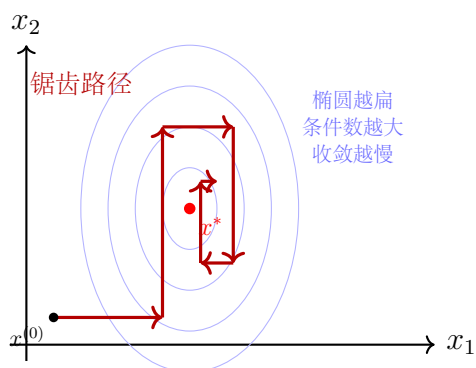


图 5.1: 最速下降法的锯齿现象：对病态问题（大条件数）收敛缓慢

例题 5.3.2 (非凸函数的梯度法). $f(x, y) = x^3 - 3x + y^3 - 3y$

稳定点：(1, 1)（极小），(-1, -1)（极大），(1, -1) 和 (-1, 1)（鞍点）

以 (-1, 0.5) 为初始点，梯度法可能收敛到鞍点 (-1, 1)。

5.3.6 梯度法的 Python 实现

最速下降法完整实现

```

1 import numpy as np
2 from scipy.optimize import minimize_scalar
3
4 def gradient_descent(f, grad, x0, method='exact', alpha=1e-4,
5                     rho=0.5, L=None, mu=None, eps=1e-6,
6                     max_iter=1000):
7     """
8     最速下降法
9
10    Parameters:
11        f: 目标函数
12        grad: 梯度函数
13        x0: 初始点
14        method: 'exact'(精确线搜索), 'fixed'(固定步长),

```

```

15         'armijo'(Armijo回溯)
16     alpha: Armijo参数
17     rho: 步长缩减因子
18     L: Lipschitz常数（固定步长时使用）
19     eps: 梯度范数精度
20     max_iter: 最大迭代次数
21
22     Returns:
23         x_opt: 最优解
24         f_opt: 最优值
25         history: 迭代历史
26     """
27     x = x0.copy()
28     history = [x.copy()]
29
30     for k in range(max_iter):
31         g = grad(x)
32
33         if np.linalg.norm(g) < eps:
34             break
35
36         d = -g # 最速下降方向
37
38         if method == 'exact':
39             # 精确线搜索
40             def phi(lam):
41                 return f(x + lam * d)
42             res = minimize_scalar(phi, bounds=(0, 10))
43             lam = res.x
44         elif method == 'fixed':
45             # 固定步长
46             lam = 1.0 / L if L else 0.01
47         elif method == 'armijo':
48             # Armijo回溯
49             lam = 1.0
50             fx = f(x)
51             grad_dot_d = np.dot(g, d)
52             for _ in range(20):
53                 if f(x + lam * d) <= fx + alpha * lam * grad_dot_d:

```

```
54         break
55         lam *= rho
56
57         x = x + lam * d
58         history.append(x.copy())
59
60     return x, f(x), history
61
62 # 示例：二次函数
63 def f(x):
64     return x[0]**2 + 4*x[1]**2
65
66 def grad(x):
67     return np.array([2*x[0], 8*x[1]])
68
69 x0 = np.array([4.0, 1.0])
70 x_opt, f_opt, hist = gradient_descent(f, grad, x0, method='exact')
71 print(f"最优解: {x_opt}")
72 print(f"最优值: {f_opt:.8f}")
73 print(f"迭代次数: {len(hist)}")
```

5.3.7 批量与小批量梯度下降

随机梯度下降 (SGD)

在机器学习中，目标函数为经验风险：

$$f(x) = \frac{1}{N} \sum_{i=1}^N f_i(x)$$

批量梯度下降 (Batch GD)：每次用全部数据

$$x_{k+1} = x_k - \eta \nabla f(x_k) = x_k - \frac{\eta}{N} \sum_{i=1}^N \nabla f_i(x_k)$$

随机梯度下降 (SGD)：每次用一个样本

$$x_{k+1} = x_k - \eta \nabla f_{i_k}(x_k)$$

其中 i_k 随机选取。

小批量梯度下降 (Mini-Batch GD)：每次用 m 个样本

$$x_{k+1} = x_k - \frac{\eta}{m} \sum_{i \in B_k} \nabla f_i(x_k)$$

SGD 的特点

- 计算效率高，适合大规模数据
- 不保证单调下降
- 学习率选择困难
- 鞍点问题严重
- 需要学习率衰减策略

5.4 牛顿法

5.4.1 基本思想

牛顿法

利用二阶 Taylor 展开在 x_k 处近似：

$$f(x) \approx f(x_k) + \nabla f(x_k)^T(x - x_k) + \frac{1}{2}(x - x_k)^T \nabla^2 f(x_k)(x - x_k)$$

令梯度为 0，得迭代公式：

$$x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

即搜索方向 $d_k = -H_k^{-1}g_k$ (H_k 为 Hessian, g_k 为梯度)。

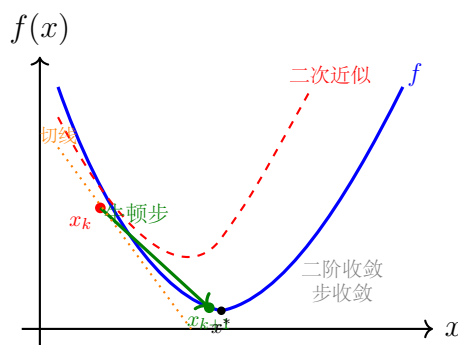


图 5.2: 牛顿法：用二次 Taylor 展开的极小点作为下一次迭代点

5.4.2 牛顿法算法

Algorithm 5 牛顿法

Require: 初始点 $x^{(0)}$, 精度 $\varepsilon > 0$

- 1: $k \leftarrow 0$
 - 2: **while** $\|\nabla f(x^{(k)})\| \geq \varepsilon$ **do**
 - 3: 计算 $g_k = \nabla f(x^{(k)})$
 - 4: 计算 $H_k = \nabla^2 f(x^{(k)})$
 - 5: 解线性方程组 $H_k d_k = -g_k$
 - 6: $x^{(k+1)} = x^{(k)} + d_k$
 - 7: $k \leftarrow k + 1$
 - 8: **end while**
 - 9: **return** $x^{(k)}$
-

例题 5.4.1 (牛顿法解二次函数). $f(x) = x_1^2 + 4x_2^2$, 初始点 $\begin{bmatrix} 4 \\ 1 \end{bmatrix}$ 。

$$g(x) = \begin{bmatrix} 2x_1 \\ 8x_2 \end{bmatrix}, \quad H(x) = \begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix}$$

$$H^{-1} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.125 \end{bmatrix}$$

$$d_0 = -H^{-1}g(4, 1) = - \begin{bmatrix} 0.5 & 0 \\ 0 & 0.125 \end{bmatrix} \begin{bmatrix} 8 \\ 8 \end{bmatrix} = \begin{bmatrix} -4 \\ -1 \end{bmatrix}$$

$$x^{(1)} = (4, 1) + (-4, -1) = (0, 0) = x^*!$$

对二次函数，牛顿法一步收敛。

5.4.3 牛顿法的收敛性分析

定理 5.4.1 (局部二次收敛). 设 $f \in C^2(\mathbb{R}^n)$, $\nabla^2 f(x^*)$ 正定, $\nabla^2 f$ 在 x^* 附近 Lipschitz 连续。则存在 $\delta > 0$, 使得当 $\|x^{(0)} - x^*\| < \delta$ 时:

$$\|x^{(k+1)} - x^*\| \leq C\|x^{(k)} - x^*\|^2$$

即二次收敛。

证明概要:

$$\begin{aligned} x_{k+1} - x^* &= x_k - x^* - H_k^{-1}g_k \\ &= H_k^{-1}(H_k(x_k - x^*) - g_k) \\ &= H_k^{-1}(\nabla^2 f(x_k)(x_k - x^*) - (\nabla f(x_k) - \nabla f(x^*))) \end{aligned}$$

由 $\nabla^2 f$ 的 Lipschitz 连续性:

$$\|\nabla f(x_k) - \nabla f(x^*) - \nabla^2 f(x_k)(x_k - x^*)\| = O(\|x_k - x^*\|^2)$$

5.4.4 牛顿法的优缺点

表 5.3: 牛顿法的优缺点

优点	缺点
局部二次收敛	需要计算 Hessian 矩阵
对二次函数一步收敛	需解线性方程组 $O(n^3)$
尺度不变性	Hessian 可能不正定 (非凸问题)
	初始点需充分接近最优解

5.4.5 牛顿法的改进策略

阻尼牛顿法 (Damped Newton)

引入步长参数:

$$x_{k+1} = x_k - \lambda_k H_k^{-1} g_k$$

其中 λ_k 由线搜索确定 (如 Armijo 条件)。

修正牛顿法 (Modified Newton)

当 H_k 不正定时, 修正为:

$$\tilde{H}_k = H_k + \nu_k I$$

其中 $\nu_k > 0$ 使得 $\tilde{H}_k$ 正定 (如取 $\nu_k = \max(0, \delta - \lambda_{\min}(H_k))$)。

Cholesky 分解修正

对 H_k 进行 Cholesky 分解时, 若遇到非正定增广对角线元素, 强制其为正数。

5.4.6 牛顿法的 Python 实现

牛顿法完整实现

```

1 import numpy as np
2 from scipy.linalg import solve
3
4 def newton_method(f, grad, hess, x0, line_search=True,
5                  alpha=1e-4, rho=0.5, eps=1e-6, max_iter=100):
6     """
7     牛  顿  法
8
9     Parameters:
10         f: 目标函数
11         grad: 梯度函数
12         hess: Hessian 矩阵函数
13         x0: 初始点
14         line_search: 是否使用线搜索
15         eps: 精度
16         max_iter: 最大迭代次数
17
18     Returns:
19         x_opt: 最优解
20         f_opt: 最优值
21         history: 迭代历史
22     """
23     x = x0.copy()
24     history = [x.copy()]
25
26     for k in range(max_iter):
27         g = grad(x)

```

```

28
29     if np.linalg.norm(g) < eps:
30         break
31
32     H = hess(x)
33
34     # 解线性方程组  $H * d = -g$ 
35     try:
36         d = solve(H, -g)
37     except np.linalg.LinAlgError:
38         # Hessian 奇异, 回退到最速下降
39         d = -g
40
41     if line_search:
42         # Armijo 回溯线搜索
43         lam = 1.0
44         fx = f(x)
45         grad_dot_d = np.dot(g, d)
46         if grad_dot_d >= 0: # 非下降方向
47             d = -g
48             grad_dot_d = np.dot(g, d)
49         for _ in range(20):
50             if f(x + lam * d) <= fx + alpha * lam * grad_dot_d:
51                 break
52             lam *= rho
53         else:
54             lam = 1.0
55
56     x = x + lam * d
57     history.append(x.copy())
58
59     return x, f(x), history
60
61 # 示例: Rosenbrock 函数
62 def rosenbrock(x):
63     return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2
64
65 def rosen_grad(x):
66     return np.array([

```

```

67         -2*(1 - x[0]) - 400*x[0]*(x[1] - x[0]**2),
68         200*(x[1] - x[0]**2)
69     ])
70
71 def rosen_hess(x):
72     return np.array([
73         [2 + 1200*x[0]**2 - 400*x[1], -400*x[0]],
74         [-400*x[0], 200]
75     ])
76
77 x0 = np.array([-1.0, 2.0])
78 x_opt, f_opt, hist = newton_method(rosenbrock, rosen_grad,
79                                   rosen_hess, x0)
80 print(f"最优解: {x_opt}")
81 print(f"最优值: {f_opt:.10f}")
82 print(f"迭代次数: {len(hist)}")

```

带 Hessian 修正的牛顿法

```

1 def modified_newton(f, grad, hess, x0, eps=1e-6, max_iter=100):
2     """
3     修正牛顿法（处理非正定Hessian）
4     """
5     x = x0.copy()
6     history = [x.copy()]
7
8     for k in range(max_iter):
9         g = grad(x)
10
11         if np.linalg.norm(g) < eps:
12             break
13
14         H = hess(x)
15
16         # 特征值修正
17         eigvals, eigvecs = np.linalg.eigh(H)
18         eigvals = np.maximum(eigvals, 1e-4) # 强制正
19
20         #  $H^{-1} = Q * \Lambda^{-1} * Q^T$ 

```

```

21     H_inv = eigvecs @ np.diag(1.0 / eigvals) @ eigvecs.T
22     d = -H_inv @ g
23
24     # Armijo 线搜索
25     lam = 1.0
26     fx = f(x)
27     for _ in range(20):
28         if f(x + lam * d) <= fx + 1e-4 * lam * np.dot(g, d):
29             break
30         lam *= 0.5
31
32     x = x + lam * d
33     history.append(x.copy())
34
35     return x, f(x), history

```

最速下降法、牛顿法、拟牛顿法的统一形式

上述方法都可统一写成

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_k = -B_k^{-1} g_k, \quad g_k = \nabla f(x_k)$$

不同方法对应不同的 B_k :

方法	B_k	搜索方向
最速下降法	I	$d_k = -g_k$
牛顿法	$\nabla^2 f(x_k)$	$d_k = -[\nabla^2 f(x_k)]^{-1} g_k$
修正牛顿法	$\nabla^2 f(x_k) + E_k$ (保证正定)	保证下降方向
拟牛顿法	Hessian 近似 B_k 或逆近似 H_k	$d_k = -H_k g_k$

拟牛顿法的核心思想是：不直接计算 Hessian，而是通过梯度差 $y_k = g_{k+1} - g_k$ 和步长差 $s_k = x_{k+1} - x_k$ 构造满足割线方程 $B_{k+1} s_k = y_k$ (或 $H_{k+1} y_k = s_k$) 的矩阵更新。DFP 和 BFGS 是典型代表。

5.5 牛顿方法的变种：LM 与信任域方法

5.5.1 Levenberg-Marquardt (LM) 方法

LM 方法

LM 方法是处理非线性最小二乘问题的牛顿型方法，通过自适应调节在梯度下降和牛顿法之间切换：

对于问题： $\min f(x) = \frac{1}{2} \sum_{i=1}^m r_i(x)^2 = \frac{1}{2} \|r(x)\|^2$

迭代公式：

$$(J_k^T J_k + \lambda_k I) d_k = -J_k^T r(x_k)$$

其中 $J(x)$ 是残差向量 $r(x)$ 的 Jacobian 矩阵。

LM 方法的解释：

- 当 $\lambda_k \rightarrow 0$ ：趋近于高斯-牛顿法 ($J^T J \approx \text{Hessian}$ 的近似)
- 当 $\lambda_k \rightarrow \infty$ ：趋近于梯度下降 ($d_k \approx -\frac{1}{\lambda_k} J^T r$)

LM 算法

Algorithm 6 Levenberg-Marquardt 方法

Require: 初始点 x_0 , 初始参数 $\lambda_0 > 0$, $\nu > 1$

```

1:  $k \leftarrow 0$ 
2: while 未收敛 do
3:   计算  $J_k = J(x_k)$ ,  $r_k = r(x_k)$ 
4:   解  $(J_k^T J_k + \lambda_k I) d_k = -J_k^T r_k$ 
5:   计算实际下降与预测下降比:
6:    $\rho_k = \frac{f(x_k) - f(x_k + d_k)}{f(x_k) - q_k(d_k)}$ 
7:   其中  $q_k(d) = f(x_k) + (J_k^T r_k)^T d + \frac{1}{2} d^T (J_k^T J_k + \lambda_k I) d$ 
8:   if  $\rho_k > 0$  then
9:      $x_{k+1} = x_k + d_k$ 
10:    若  $\rho_k > 0.75$ ,  $\lambda_{k+1} = \lambda_k / \nu$ 
11:    否则若  $\rho_k < 0.25$ ,  $\lambda_{k+1} = \nu \lambda_k$ 
12:   else
13:      $x_{k+1} = x_k$ 
14:      $\lambda_{k+1} = \nu \lambda_k$ 
15:   end if
16:    $k \leftarrow k + 1$ 
17: end while

```

例题 5.5.1 (曲线拟合). 拟合模型 $y = ae^{bx}$ 到数据点 (t_i, y_i) :

$$\min f(a, b) = \sum_{i=1}^n (ae^{bt_i} - y_i)^2$$

$$\text{残差: } r_i(a, b) = ae^{bt_i} - y_i$$

$$\text{Jacobian: } J_{i1} = e^{bt_i}, J_{i2} = ate^{bt_i}$$

用 LM 方法迭代求解。

5.5.2 信任域方法 (Trust Region Methods)

信任域方法

与线搜索不同，信任域方法先确定搜索范围（信任域），再在此范围内优化模型：

$$\min_d q_k(d) = f(x_k) + g_k^T d + \frac{1}{2} d^T H_k d, \quad \text{s.t. } \|d\| \leq \Delta_k$$

其中 $\Delta_k > 0$ 为信任域半径。

信任域方法的框架

1. 求解子问题：在给定半径 Δ_k 内求 d_k

2. 评估比率：

$$\rho_k = \frac{f(x_k) - f(x_k + d_k)}{q_k(0) - q_k(d_k)}$$

3. 更新信任域半径：

- 若 ρ_k 接近 1：增大 Δ_k
- 若 ρ_k 较小：减小 Δ_k
- 若 $\rho_k > \eta_0$ ：接受 $x_{k+1} = x_k + d_k$
- 否则：拒绝步， $x_{k+1} = x_k$

信任域子问题的求解

定理 5.5.1 (子问题最优性条件). d^* 是信任域子问题的全局最优解当且仅当存在 $\lambda \geq 0$ 使得：

$$(H + \lambda I)d^* = -g \tag{5.1}$$

$$\lambda(\Delta - \|d^*\|) = 0 \tag{5.2}$$

$$H + \lambda I \succeq 0 \tag{5.3}$$

Cauchy 点：在信任域内沿最速下降方向的最优点：

$$d_k^C = -\tau_k \frac{\Delta_k}{\|g_k\|} g_k$$

其中：

$$\tau_k = \begin{cases} 1 & \text{if } g_k^T H_k g_k \leq 0 \\ \min\left(\frac{\|g_k\|^3}{\Delta_k g_k^T H_k g_k}, 1\right) & \text{otherwise} \end{cases}$$

Dogleg 方法

Dogleg 方法

结合最速下降方向和牛顿方向的折线路径：

1. 若牛顿步在信任域内 ($\|d^N\| \leq \Delta$)，取 $d = d^N$
2. 否则，沿最速下降到 Cauchy 点的路径
3. 若 Cauchy 点在信任域外 ($\|d^{SD}\| \geq \Delta$)，取边界点
4. 否则在折线路径上取信任域边界点

5.5.3 信任域方法的 Python 实现

Levenberg-Marquardt 方法

```

1 import numpy as np
2 from scipy.optimize import least_squares
3
4 def lm_method(residuals, jac, x0, lam=0.01, nu=10,
5               eps=1e-6, max_iter=100):
6     """
7     Levenberg-Marquardt 方法
8
9     Parameters:
10         residuals: 残差函数 r(x)
11         jac: Jacobian函数 J(x)
12         x0: 初始点
13         lam: 初始正则化参数
14         nu: 调节因子
15         eps: 精度
16         max_iter: 最大迭代次数
17
18     Returns:
19         x_opt: 最优解
20         history: 迭代历史
21     """

```

```

22     x = x0.copy()
23     history = [x.copy()]
24
25     for k in range(max_iter):
26         r = residuals(x)
27         J = jac(x)
28         g = J.T @ r
29
30         if np.linalg.norm(g) < eps:
31             break
32
33         # 解  $(J^T J + \text{lam} * I) d = -J^T r$ 
34         A = J.T @ J + lam * np.eye(len(x))
35         d = np.linalg.solve(A, -g)
36
37         # 计算比率
38         rho = (np.linalg.norm(r)**2 -
39               np.linalg.norm(residuals(x + d))**2) / \
40               (np.linalg.norm(r)**2 -
41               np.linalg.norm(r + J @ d)**2)
42
43         if rho > 0:
44             x = x + d
45             history.append(x.copy())
46             if rho > 0.75:
47                 lam = lam / nu
48             elif rho < 0.25:
49                 lam = lam * nu
50         else:
51             lam = lam * nu
52
53     return x, history
54
55 # 示例：非线性最小二乘拟合
56 # 数据
57 np.random.seed(42)
58 t = np.linspace(0, 4, 50)
59 y = 2.5 * np.exp(-1.3 * t) + 0.1 * np.random.randn(50)
60

```

```

61 def residuals(params):
62     a, b = params
63     return a * np.exp(b * t) - y
64
65 def jac(params):
66     a, b = params
67     J = np.zeros((len(t), 2))
68     J[:, 0] = np.exp(b * t)
69     J[:, 1] = a * t * np.exp(b * t)
70     return J
71
72 x0 = np.array([1.0, -0.5])
73 x_opt, hist = lm_method(residuals, jac, x0)
74 print(f"拟合参数: a={x_opt[0]:.4f}, b={x_opt[1]:.4f}")

```

信任域方法（简化版）

```

1 def trust_region(f, grad, hess, x0, Delta=1.0, eta=0.1,
2                 eps=1e-6, max_iter=100):
3     """
4     简化信任域方法（Cauchy点策略）
5
6     Parameters:
7         f: 目标函数
8         grad: 梯度函数
9         hess: Hessian函数
10        x0: 初始点
11        Delta: 初始信任域半径
12        eta: 接受阈值
13        eps: 精度
14
15    Returns:
16        x_opt: 最优解
17        history: 迭代历史
18    """
19    x = x0.copy()
20    history = [x.copy()]
21    Delta_max = 100.0
22

```

```
23     for k in range(max_iter):
24         g = grad(x)
25
26         if np.linalg.norm(g) < eps:
27             break
28
29         H = hess(x)
30
31         # 计算 Cauchy 点
32         g_norm = np.linalg.norm(g)
33         gHg = g.T @ H @ g
34
35         if gHg <= 0:
36             tau = 1.0
37         else:
38             tau = min(g_norm**3 / (Delta * gHg), 1.0)
39
40         d = -tau * Delta / g_norm * g
41
42         # 评估比率
43         actual = f(x) - f(x + d)
44         predicted = -(g.T @ d + 0.5 * d.T @ H @ d)
45
46         if abs(predicted) < 1e-10:
47             break
48
49         rho = actual / predicted
50
51         if rho > eta:
52             x = x + d
53             history.append(x.copy())
54
55         # 更新信任域半径
56         if rho < 0.25:
57             Delta = 0.25 * Delta
58         elif rho > 0.75 and abs(np.linalg.norm(d) - Delta) < 1e-6:
59             Delta = min(2.0 * Delta, Delta_max)
60
61     return x, f(x), history
```

--

5.5.4 LM 与信任域方法的比较

表 5.4: LM 与信任域方法比较		
特性	LM 方法	信任域方法
适用问题	最小二乘	一般无约束优化
模型	$J^T J + \lambda I$	$g^T d + \frac{1}{2} d^T H d$
参数	λ (自适应)	Δ (信任域半径)
子问题求解	线性方程组	信赖域子问题
全局收敛	是	是

5.6 共轭方向法

5.6.1 共轭方向的概念

共轭方向

对于对称正定矩阵 A ，非零向量 $d_1, d_2, \dots, d_k$ 称为 A -共轭的，如果：

$$d_i^T A d_j = 0, \quad \forall i \neq j$$

即它们关于 A 的内积正交。

定理 5.6.1 (共轭方向法的有限步收敛). 对于二次函数 $f(x) = \frac{1}{2}x^T A x - b^T x + c$ (A 正定)，从任意 x_0 出发，沿 n 个 A -共轭方向精确线搜索，最多 n 步收敛到最优解。

证明： 设 x^* 为最优解， $g_k = Ax_k - b$ 。由精确线搜索： $g_{k+1}^T d_k = 0$
利用共轭性可证 $g_{k+1}^T d_j = 0$ ($j = 0, \dots, k$)，即梯度与共轭方向正交。
 n 步后 g_n 与 n 个线性无关方向正交，故 $g_n = 0$ ， $x_n = x^*$ 。

5.6.2 共轭梯度法 (CG)

共轭梯度法

通过递推构造共轭方向，不需预先给定。对二次函数：

$$d_0 = -g_0$$
$$d_{k+1} = -g_{k+1} + \beta_k d_k$$

其中 β_k 使得 $d_{k+1}^T A d_k = 0$ 。

二次函数的 CG 算法

Algorithm 7 线性共轭梯度法**Require:** A (SPD), b , 初始点 x_0

```

1:  $g_0 = Ax_0 - b$ ,  $d_0 = -g_0$ 
2: for  $k = 0, 1, \dots, n - 1$  do
3:   if  $\|g_k\| = 0$  then break
4:   end if
5:    $\alpha_k = \frac{g_k^T g_k}{d_k^T A d_k}$ 
6:    $x_{k+1} = x_k + \alpha_k d_k$ 
7:    $g_{k+1} = g_k + \alpha_k A d_k$ 
8:    $\beta_k = \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k}$ 
9:    $d_{k+1} = -g_{k+1} + \beta_k d_k$ 
10: end for

```

CG 的等价公式 (β_k 的不同形式)表 5.5: 共轭梯度法的 β_k 公式

公式	表达式
Fletcher-Reeves (FR)	$\beta_k = \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k}$
Polak-Ribiere (PR)	$\beta_k = \frac{g_{k+1}^T (g_{k+1} - g_k)}{g_k^T g_k}$
Hestenes-Stiefel (HS)	$\beta_k = \frac{g_{k+1}^T (g_{k+1} - g_k)}{d_k^T (g_{k+1} - g_k)}$
Dai-Yuan (DY)	$\beta_k = \frac{g_{k+1}^T g_{k+1}}{d_k^T (g_{k+1} - g_k)}$

PR 方法

PR 方法在实际中表现最好。若 $\beta_k < 0$ 可置 $\beta_k = 0$ (重启 CG), 保证收敛。

非线性共轭梯度法

Algorithm 8 非线性共轭梯度法 (Fletcher-Reeves)**Require:** 初始点 x_0 , 精度 ε

```

1:  $g_0 = \nabla f(x_0)$ ,  $d_0 = -g_0$ 
2: for  $k = 0, 1, \dots$  do
3:   if  $\|g_k\| < \varepsilon$  then break
4:   end if
5:   线搜索求  $\alpha_k$ 
6:    $x_{k+1} = x_k + \alpha_k d_k$ 
7:    $g_{k+1} = \nabla f(x_{k+1})$ 
8:    $\beta_k = \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k}$ 
9:    $d_{k+1} = -g_{k+1} + \beta_k d_k$ 
10:  若  $k+1 \equiv 0 \pmod{n}$ ,  $d_{k+1} = -g_{k+1}$  (每  $n$  步重启)
11: end for

```

例题 5.6.1 (CG 法解二次函数). $f(x) = x_1^2 + 4x_2^2$, 初始点 $\begin{bmatrix} 4 \\ 1 \end{bmatrix}$ 。

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$g_0 = \begin{bmatrix} 8 \\ 8 \end{bmatrix}, \quad d_0 = \begin{bmatrix} -8 \\ -8 \end{bmatrix}$$

$$\alpha_0 = \frac{g_0^T g_0}{d_0^T A d_0} = \frac{128}{640} = 0.2$$

$$x^{(1)} = \begin{bmatrix} 4 \\ 1 \end{bmatrix} + 0.2 \begin{bmatrix} -8 \\ -8 \end{bmatrix} = \begin{bmatrix} 2.4 \\ -0.6 \end{bmatrix}$$

$$g_1 = \begin{bmatrix} 4.8 \\ -4.8 \end{bmatrix}, \quad \beta_0 = \frac{46.08}{128} = 0.36$$

$$d_1 = -g_1 + \beta_0 d_0 = \begin{bmatrix} -4.8 \\ 4.8 \end{bmatrix} + 0.36 \begin{bmatrix} -8 \\ -8 \end{bmatrix} = \begin{bmatrix} -7.68 \\ 1.92 \end{bmatrix}$$

$$\alpha_1 = \frac{46.08}{147.456} = 0.3125$$

$$x^{(2)} = \begin{bmatrix} 2.4 \\ -0.6 \end{bmatrix} + 0.3125 \begin{bmatrix} -7.68 \\ 1.92 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} = x^*$$

5.6.3 拟牛顿法

拟牛顿法

用矩阵 $B_k \approx H_k^{-1}$ 近似逆 Hessian, 满足割线条件:

$$s_k = x_{k+1} - x_k, \quad y_k = g_{k+1} - g_k$$

$$B_{k+1}y_k = s_k \quad (\text{割线条件})$$

DFP 公式 (Davidon-Fletcher-Powell)

$$B_{k+1}^{DFP} = \left(I - \frac{s_k y_k^T}{y_k^T s_k} \right) B_k \left(I - \frac{y_k s_k^T}{y_k^T s_k} \right) + \frac{s_k s_k^T}{y_k^T s_k} \quad (5.4)$$

$$= B_k - \frac{B_k y_k y_k^T B_k}{y_k^T B_k y_k} + \frac{s_k s_k^T}{y_k^T s_k} \quad (5.5)$$

BFGS 公式 (Broyden-Fletcher-Goldfarb-Shanno)

$$B_{k+1}^{BFGS} = B_k + \frac{(s_k - B_k y_k)(s_k - B_k y_k)^T}{y_k^T (s_k - B_k y_k)} \quad (\text{秩一修正}) \quad (5.6)$$

$$= \left(I - \frac{s_k y_k^T}{y_k^T s_k} \right) B_k \left(I - \frac{y_k s_k^T}{y_k^T s_k} \right) + \frac{s_k s_k^T}{y_k^T s_k} \quad (5.7)$$

或等价地对 H 的更新:

$$H_{k+1}^{BFGS} = H_k - \frac{H_k s_k s_k^T H_k}{s_k^T H_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$$

Algorithm 9 BFGS 方法**Require:** 初始点 x_0 , 初始矩阵 B_0 (通常 $B_0 = I$)

```

1:  $k \leftarrow 0$ 
2: while  $\|\nabla f(x_k)\| \geq \varepsilon$  do
3:    $d_k = -B_k \nabla f(x_k)$ 
4:   线搜索求  $\alpha_k$  (满足 Wolfe 条件)
5:    $s_k = \alpha_k d_k$ ,  $x_{k+1} = x_k + s_k$ 
6:    $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$ 
7:   if  $y_k^T s_k > 0$  then
8:     用 BFGS 公式更新  $B_{k+1}$ 
9:   else
10:     $B_{k+1} = B_k$  (或重启)
11:   end if
12:    $k \leftarrow k + 1$ 
13: end while

```

定理 5.6.2 (BFGS 的收敛性). 若 f 为凸函数且 L -光滑, 采用 Wolfe 条件线搜索, 则 BFGS 全局收敛到最优解。若 f 还强凸, 则超线性收敛。

例题 5.6.2 (BFGS 求解). $f(x) = x_1^2 + 4x_2^2$, $x_0 = (4, 1)$ 。

$$B_0 = I, d_0 = (-8, -8), \alpha_0 = 0.2$$

$$x_1 = (2.4, -0.6), g_1 = (4.8, -4.8)$$

$$s_0 = (-1.6, -1.6), y_0 = g_1 - g_0 = (-3.2, -12.8)$$

$$\text{计算 BFGS 更新得 } B_1 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.125 \end{bmatrix} = A^{-1}$$

$$d_1 = -B_1 g_1 = (-2.4, 0.6), x_2 = x_1 + d_1 = (0, 0) = x^*$$

5.6.4 共轭梯度法与拟牛顿法的比较

表 5.6: 共轭梯度法与拟牛顿法比较

特性	CG	BFGS
内存需求	$O(n)$	$O(n^2)$
每次迭代计算量	低	中等
收敛速度	线性/超线性	超线性
适合问题规模	大规模	中大规模
重启策略	需要	不需要
全局收敛	需配合线搜索	配合 Wolfe 条件

5.6.5 Python 代码实现

共轭梯度法

```
1 import numpy as np
2 from scipy.optimize import minimize_scalar
3
4 def conjugate_gradient(f, grad, x0, method='FR', eps=1e-6,
5                       max_iter=1000):
6     """
7     非线性共轭梯度法
8
9     Parameters:
10         f: 目标函数
11         grad: 梯度函数
12         x0: 初始点
13         method: 'FR', 'PR', 'HS'
14         eps: 精度
15         max_iter: 最大迭代次数
16
17     Returns:
18         x_opt: 最优解
19         history: 迭代历史
20     """
21     x = x0.copy()
22     n = len(x)
23     g = grad(x)
24     d = -g.copy()
25     history = [x.copy()]
26
27     k = 0
28     for _ in range(max_iter):
29         if np.linalg.norm(g) < eps:
30             break
31
32         # 线搜索
33         def phi(alpha):
34             return f(x + alpha * d)
35         res = minimize_scalar(phi)
36         alpha = res.x
37
```

```

38     x = x + alpha * d
39     history.append(x.copy())
40
41     g_new = grad(x)
42
43     # 计算 beta
44     if method == 'FR':
45         beta = np.dot(g_new, g_new) / np.dot(g, g)
46     elif method == 'PR':
47         beta = np.dot(g_new, g_new - g) / np.dot(g, g)
48         beta = max(0, beta) # PR+ 保证收敛
49     elif method == 'HS':
50         dy = g_new - g
51         beta = np.dot(g_new, dy) / np.dot(d, dy)
52
53     # 重启
54     k += 1
55     if k % n == 0:
56         d = -g_new
57     else:
58         d = -g_new + beta * d
59
60     g = g_new
61
62     return x, f(x), history
63
64 # 示例
65 x0 = np.array([4.0, 1.0])
66 x_opt, f_opt, hist = conjugate_gradient(
67     lambda x: x[0]**2 + 4*x[1]**2,
68     lambda x: np.array([2*x[0], 8*x[1]]),
69     x0
70 )
71 print(f"最优解: {x_opt}")
72 print(f"最优值: {f_opt:.8f}")

```

BFGS 方法

```
1 def bfgs(f, grad, x0, eps=1e-6, max_iter=1000):
2     """
3     BFGS 拟牛顿法
4
5     Parameters:
6         f: 目标函数
7         grad: 梯度函数
8         x0: 初始点
9         eps: 精度
10        max_iter: 最大迭代次数
11
12    Returns:
13        x_opt: 最优解
14        history: 迭代历史
15    """
16    n = len(x0)
17    x = x0.copy()
18    B = np.eye(n) # 初始 Hessian 逆近似
19    g = grad(x)
20    history = [x.copy()]
21
22    for _ in range(max_iter):
23        if np.linalg.norm(g) < eps:
24            break
25
26        # 搜索方向
27        d = -B @ g
28
29        # Wolfe 条件线搜索
30        alpha = 1.0
31        c1, c2 = 1e-4, 0.9
32        fx = f(x)
33
34        for _ in range(20):
35            x_new = x + alpha * d
36            g_new = grad(x_new)
37
38            # 充分下降
```

```

39         if f(x_new) <= fx + c1 * alpha * np.dot(g, d):
40             # 曲率条件
41             if np.dot(g_new, d) >= c2 * np.dot(g, d):
42                 break
43             alpha *= 0.5
44
45     s = alpha * d
46     x = x + s
47     g_new = grad(x)
48     y = g_new - g
49
50     # BFGS更新
51     if np.dot(y, s) > 1e-10: # 保证正定性
52         rho = 1.0 / np.dot(y, s)
53         I = np.eye(n)
54         B = (I - rho * np.outer(s, y)) @ B @ \
55             (I - rho * np.outer(y, s)) + \
56             rho * np.outer(s, s)
57
58     g = g_new
59     history.append(x.copy())
60
61     return x, f(x), history
62
63 # 示例: Rosenbrock函数
64 x0 = np.array([-1.0, 2.0])
65 x_opt, f_opt, hist = bfgs(rosenbrock, rosen_grad, x0)
66 print(f"最优解: {x_opt}")
67 print(f"最优值: {f_opt:.10f}")
68 print(f"迭代次数: {len(hist)}")

```

5.7 次梯度优化方法

5.7.1 背景与动机

许多优化问题涉及非光滑目标函数（如 L1 正则化、hinge loss 等），梯度不存在，需要推广导数的概念。

5.7.2 次梯度与次微分

定义 5.7.1 (次梯度). 设 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 为凸函数。向量 $g \in \mathbb{R}^n$ 称为 f 在 x 处的次梯度, 如果:

$$f(y) \geq f(x) + g^T(y - x), \quad \forall y \in \mathbb{R}^n$$

几何意义: g 定义了 f 在 x 处的一个支撑超平面。

定义 5.7.2 (次微分). f 在 x 处的所有次梯度的集合称为次微分, 记为 $\partial f(x)$:

$$\partial f(x) = \{g: f(y) \geq f(x) + g^T(y - x), \forall y\}$$

定理 5.7.1 (次微分的性质). 1. $\partial f(x)$ 是非空紧凸集

2. 若 f 在 x 处可微, 则 $\partial f(x) = \{\nabla f(x)\}$

3. x^* 是全局最优 $\Leftrightarrow 0 \in \partial f(x^*)$

4. (正齐次性) $\partial(\alpha f)(x) = \alpha \partial f(x)$ ($\alpha > 0$)

5. (加法规则) $\partial(f_1 + f_2)(x) \supseteq \partial f_1(x) + \partial f_2(x)$

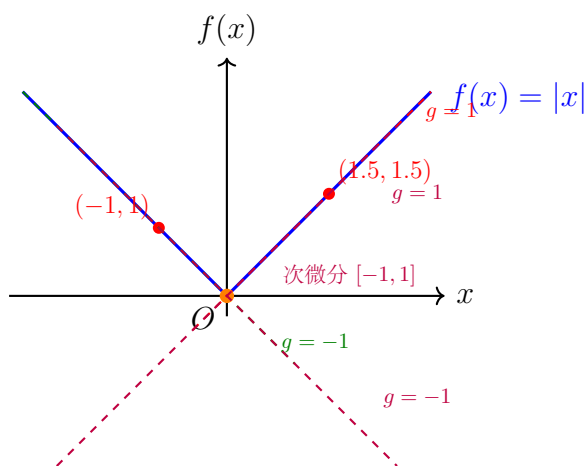


图 5.3: 次梯度与次微分的几何解释: $f(x) = |x|$ 在 $x \neq 0$ 处有唯一次梯度 $g = \text{sign}(x)$, 在 $x = 0$ 处次微分为区间 $[-1, 1]$

5.7.3 常见函数的次微分

例题 5.7.1 (绝对值函数的次微分). $f(x) = |x|$:

$$\partial f(x) = \begin{cases} \{-1\} & x < 0 \\ \{-1, 1\} & x = 0 \\ \{+1\} & x > 0 \end{cases}$$

例题 5.7.2 (欧几里得范数的次微分). $f(x) = \|x\|$:

$$\partial f(x) = \begin{cases} \left\{ \frac{x}{\|x\|} \right\} & x \neq 0 \\ \{g : \|g\| \leq 1\} & x = 0 \end{cases}$$

例题 5.7.3 (最大值函数的次微分). $f(x) = \max_{i=1,\dots,m} f_i(x)$:

$$\partial f(x) = \text{conv} \left(\bigcup_{i \in I(x)} \partial f_i(x) \right)$$

其中 $I(x) = \{i : f_i(x) = f(x)\}$ 为活跃指标集。

5.7.4 次梯度方法

次梯度方法

直接推广梯度下降:

$$x_{k+1} = x_k - \alpha_k g_k, \quad g_k \in \partial f(x_k)$$

与梯度下降不同, $-g_k$ 不一定是下降方向。

定理 5.7.2 (次梯度方法的收敛性). 设 f 为凸函数, $\|g_k\| \leq G$, $\|x_0 - x^*\| \leq R$ 。

固定步长 $\alpha_k = \alpha$:

$$\min_{0 \leq i < k} f(x_i) - f^* \leq \frac{R^2 + G^2 \alpha^2 k}{2\alpha k} \rightarrow \frac{G^2 \alpha}{2}$$

收敛到 $O(\alpha)$ 邻域内。

递减步长 $\alpha_k = \frac{R}{G\sqrt{k}}$:

$$\min_{0 \leq i < k} f(x_i) - f^* \leq \frac{RG}{\sqrt{k}} = O\left(\frac{1}{\sqrt{k}}\right)$$

次梯度方法的特点

- 收敛速率 $O(1/\sqrt{k})$, 比梯度下降慢
- 步长选择至关重要 (递减步长)
- 函数值不单调下降
- 适合大规模问题 (每次迭代简单)

投影次梯度方法

对于约束问题 $\min_{x \in C} f(x)$:

$$x_{k+1} = P_C(x_k - \alpha_k g_k)$$

其中 $P_C(x) = \arg \min_{y \in C} \|y - x\|$ 为到集合 C 的投影。

5.7.5 近端梯度方法 (Proximal Gradient)

近端算子

对于函数 h 和参数 $\lambda > 0$, 近端算子定义为:

$$\text{prox}_{\lambda h}(x) = \arg \min_u \left\{ h(u) + \frac{1}{2\lambda} \|u - x\|^2 \right\}$$

对于复合优化问题: $\min f(x) = g(x) + h(x)$

其中 g 光滑, h 非光滑但简单 (近端可计算)。

近端梯度法 (ISTA):

$$x_{k+1} = \text{prox}_{\alpha_k h}(x_k - \alpha_k \nabla g(x_k))$$

例题 5.7.4 (LASSO 问题). $\min \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$

$$g(x) = \frac{1}{2} \|Ax - b\|^2, \quad h(x) = \lambda \|x\|_1$$

近端算子 (软阈值):

$$[\text{prox}_{\lambda h}(x)]_i = \text{sign}(x_i) \max(|x_i| - \lambda, 0)$$

迭代公式 (ISTA):

$$x_{k+1} = S_{\alpha\lambda}(x_k - \alpha A^T(Ax_k - b))$$

其中 S 为软阈值算子。

5.7.6 加速近端梯度法 (FISTA)

Algorithm 10 FISTA

Require: 初始点 $x_0 = y_1$, 步长 $\alpha \leq 1/L$

```

1:  $t_1 = 1$ 
2: for  $k = 1, 2, \dots$  do
3:    $x_k = \text{prox}_{\alpha h}(y_k - \alpha \nabla g(y_k))$ 
4:    $t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}$ 
5:    $y_{k+1} = x_k + \frac{t_k - 1}{t_{k+1}}(x_k - x_{k-1})$ 
6: end for
```

定理 5.7.3 (FISTA 收敛率). $FISTA$ 的收敛率为 $O(1/k^2)$:

$$f(x_k) - f^* \leq \frac{2L \|x_0 - x^*\|^2}{(k+1)^2}$$

5.7.7 次梯度方法的 Python 实现

次梯度方法

```
1 import numpy as np
2
3 def subgradient_method(f, subgrad, x0, step_rule='diminishing',
4                        alpha=0.01, eps=1e-6, max_iter=1000):
5     """
6     次梯度方法
7
8     Parameters:
9         f: 目标函数
10        subgrad: 次梯度函数
11        x0: 初始点
12        step_rule: 'fixed', 'diminishing', 'polyak'
13        alpha: 步长参数
14        eps: 精度
15        max_iter: 最大迭代次数
16
17    Returns:
18        x_best: 最优解
19        history: 函数值历史
20    """
21    x = x0.copy()
22    f_best = f(x)
23    x_best = x.copy()
24    history = [f_best]
25
26    for k in range(1, max_iter + 1):
27        g = subgrad(x)
28
29        if np.linalg.norm(g) < eps:
30            break
31
32        # 步长选择
33        if step_rule == 'fixed':
34            step = alpha
35        elif step_rule == 'diminishing':
36            step = alpha / np.sqrt(k)
37        elif step_rule == 'polyak':
```

```

38         # 需要知道最优值  $f^*$ 
39         step = (f(x) - 0) / (np.linalg.norm(g)**2 + 1e-10)
40
41         x = x - step * g
42         f_val = f(x)
43         history.append(f_val)
44
45         # 记录最优
46         if f_val < f_best:
47             f_best = f_val
48             x_best = x.copy()
49
50         return x_best, history
51
52     # 示例：绝对值函数求和
53     f = lambda x: np.sum(np.abs(x))
54     subgrad = lambda x: np.sign(x) # 在零点取0
55
56     x0 = np.array([5.0, -3.0, 2.0])
57     x_best, hist = subgradient_method(f, subgrad, x0,
58                                     step_rule='diminishing',
59                                     alpha=1.0)
60     print(f"最优解: {x_best}")
61     print(f"最优值: {f(x_best):.6f}")

```

近端梯度法 (ISTA/FISTA)

```

1 def ista(g, grad_g, prox_h, x0, alpha=0.01, eps=1e-6,
2         max_iter=1000):
3     """
4     近端梯度法 (ISTA)
5
6     Parameters:
7         g: 光滑部分函数
8         grad_g: 光滑部分梯度
9         prox_h: 近端算子
10        x0: 初始点
11        alpha: 步长
12        eps: 精度

```

```

13
14     Returns:
15         x_opt: 最优解
16         history: 迭代历史
17     """
18     x = x0.copy()
19     history = [g(x) + 0] # 需要外部提供完整f值
20
21     for _ in range(max_iter):
22         x_new = prox_h(x - alpha * grad_g(x), alpha)
23
24         if np.linalg.norm(x_new - x) < eps:
25             break
26
27         x = x_new
28         history.append(x.copy())
29
30     return x, history
31
32 def fista(g, grad_g, prox_h, x0, alpha=0.01, max_iter=1000):
33     """
34     加速近端梯度法 (FISTA)
35     """
36     x = x0.copy()
37     y = x0.copy()
38     t = 1.0
39     history = [x.copy()]
40
41     for _ in range(max_iter):
42         x_prev = x.copy()
43
44         x = prox_h(y - alpha * grad_g(y), alpha)
45
46         t_new = (1 + np.sqrt(1 + 4 * t**2)) / 2
47         y = x + ((t - 1) / t_new) * (x - x_prev)
48
49         t = t_new
50         history.append(x.copy())
51

```

```

52     return x, history
53
54 # 软阈值算子 (L1近端算子)
55 def soft_threshold(x, lam):
56     return np.sign(x) * np.maximum(np.abs(x) - lam, 0)
57
58 # LASSO: min 0.5*||Ax-b||^2 + lambda*||x||_1
59 def solve_lasso(A, b, lam, alpha=None):
60     n = A.shape[1]
61     if alpha is None:
62         alpha = 1.0 / np.linalg.norm(A.T @ A, 2)
63
64     grad_g = lambda x: A.T @ (A @ x - b)
65     prox_h = lambda x, alpha: soft_threshold(x, lam * alpha)
66
67     x0 = np.zeros(n)
68     x_opt, hist = fista(None, grad_g, prox_h, x0, alpha,
69                         max_iter=1000)
70     return x_opt, hist

```

5.7.8 复杂度下界与加速方法

定理 5.7.4 (一阶方法的复杂度下界). 对于 L -光滑的凸函数, 任何一阶方法的收敛速率下界为 $O(1/k^2)$ 。

对于 μ -强凸且 L -光滑的函数, 下界为:

$$f(x_k) - f^* \geq \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^{2k} (f(x_0) - f^*)$$

其中 $\kappa = L/\mu$ 为条件数。

Nesterov 加速梯度法

对于梯度下降 $O(1/k)$ 的收敛率, Nesterov 加速可达 $O(1/k^2)$:

加速梯度法:

$$y_k = x_k + \beta_k(x_k - x_{k-1})$$

$$x_{k+1} = y_k - \alpha_k \nabla f(y_k)$$

其中 $\beta_k = \frac{k-1}{k+2}$ (凸情形) 或 $\beta_k = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ (强凸情形)。

5.8 本章小结

5.8.1 核心思想

多维无约束优化的两大支柱

1. 下降迭代框架： $x_{k+1} = x_k + \alpha_k d_k$ ，其中 d_k 为下降方向， α_k 为步长（由第 4 章的线搜索方法确定）。

2. 以信息代价换取收敛速度：仅函数值（零阶）→ 梯度（一阶）→ Hessian（二阶），信息越多，收敛越快。

5.8.2 方法体系总览

表 5.7: 无约束最优化方法总览（步长搜索统一使用第 4 章的线搜索方法）

信息类型	代表方法	收敛速度	适用场景
零阶	坐标轮换法	线性	小规模
	Hooke-Jeeves	线性	模式搜索
	Rosenbrock	超线性	峡谷函数
一阶	最速下降法	$O(1/k)$	大规模
	CG	超线性	大规模稀疏
	SGD	$O(1/\sqrt{k})$	机器学习
二阶	牛顿法	局部二次	中小规模
	Levenberg-Marquardt	超线性	最小二乘
	信任域方法	超线性	非凸问题
拟牛顿	DFP、BFGS	超线性	中大规模
非光滑	次梯度方法	$O(1/\sqrt{k})$	凸非光滑
	近端梯度法	$O(1/k)$	复合优化
	FISTA	$O(1/k^2)$	复合优化

5.8.3 方法选择决策树

如何选择多维无约束优化方法

1. 函数是否光滑？

• 非光滑凸 → 近端梯度法 / FISTA

• 非光滑且导数不可得 → 坐标轮换法 / Hooke-Jeeves

- 光滑 $\rightarrow$ 继续判断
2. 能否计算 Hessian?
- 能且 $n < 1000 \rightarrow$ 牛顿法（二次收敛）或信任域方法（全局收敛）
 - 不能 $\rightarrow$ 继续判断
3. 问题规模?
- $n > 10^5$ （大规模） $\rightarrow$ 共轭梯度法 / SGD
 - $1000 < n < 10^5$ （中规模） $\rightarrow$ BFGS（实用首选）
 - $n < 1000$ （小规模） $\rightarrow$ BFGS 或牛顿法
4. 特殊结构?
- 最小二乘问题 $\rightarrow$ LM 方法
 - 复合优化（ $f = g + h$, g 光滑 h 非光滑） $\rightarrow$ 近端梯度 / FISTA
 - 大数据/随机采样 $\rightarrow$ SGD 及其变体

5.8.4 关键概念回顾

1. 下降方向: d 满足 $\nabla f(x)^T d < 0$; 最速下降方向 $d = -\nabla f(x)$
2. 共轭性: $d_i^T A d_j = 0$ ($i \neq j$), 保证二次函数 n 步内收敛; CG 法通过递推构造共轭方向
3. 割线条件: $B_{k+1} s_k = y_k$ (其中 $s_k = x_{k+1} - x_k$, $y_k = \nabla f_{k+1} - \nabla f_k$), 拟 Newton 法通过此条件更新 Hessian 近似
4. 信任域 vs 线搜索: 信任域先限制步长范围再优化方向, 全局收敛性更好; 线搜索先确定方向再找步长
5. 次梯度: $g \in \partial f(x)$, 满足 $f(y) \geq f(x) + g^T(y - x)$, 推广梯度到非光滑凸函数
6. 近端算子: $\text{prox}_{\lambda h}(x) = \arg \min_y \{h(y) + \frac{1}{2\lambda} \|y - x\|^2\}$, 将非光滑项的极小化显式求解
7. 加速技术: FISTA 通过 Nesterov 动量将近端梯度法从 $O(1/k)$ 加速到 $O(1/k^2)$

5.8.5 收敛速度总结

表 5.8: 各方法收敛速度对照

方法	一般凸	强凸	局部 (光滑)
坐标轮换法	—	—	线性
最速下降法	$O(1/k)$	$O((1 - \mu/L)^k)$	线性
共轭梯度法	—	—	超线性
牛顿法	—	—	二次
BFGS	—	超线性	超线性
SGD	$O(1/\sqrt{k})$	$O(1/k)$	—
次梯度方法	$O(1/\sqrt{k})$	—	—
近端梯度法	$O(1/k)$	线性	—
FISTA	$O(1/k^2)$	—	—

5.8.6 关键公式汇总

表 5.9: 核心公式速查

概念/方法	公式
最速下降方向	$d_k = -\nabla f(x_k)$
Newton 方向	$d_k = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$
CG (Fletcher-Reeves)	$d_k = -g_k + \beta_k d_{k-1}, \beta_k = \ g_k\ ^2 / \ g_{k-1}\ ^2$
CG (Polak-Ribière)	$\beta_k = g_k^T (g_k - g_{k-1}) / \ g_{k-1}\ ^2$
DFP 更新	$B_{k+1} = B_k - \frac{B_k y_k y_k^T B_k}{y_k^T B_k y_k} + \frac{s_k s_k^T}{y_k^T s_k}$
BFGS 更新	$H_{k+1} = V_k^T H_k V_k + \rho_k s_k s_k^T, \rho_k = 1/(y_k^T s_k)$
LM 方法	$(J^T J + \lambda I)d = -J^T r$
信任域子问题	$\min_{\ d\ \leq \Delta} m_k(d) = f_k + g_k^T d + \frac{1}{2} d^T B_k d$
次梯度迭代	$x_{k+1} = x_k - \alpha_k g_k, g_k \in \partial f(x_k)$
近端梯度法	$x_{k+1} = \text{prox}_{\alpha_k h}(x_k - \alpha_k \nabla g(x_k))$
FISTA	$x_k = \text{prox}_{\alpha h}(y_k - \alpha \nabla g(y_k))$ $y_{k+1} = x_k + \frac{k-1}{k+2}(x_k - x_{k-1})$

5.8.7 学习建议

学习路径

1. 掌握最速下降法的收敛性分析和步长选择（理解线搜索在多维优化中的作用）
2. 理解牛顿法的二次收敛性和局限性（Hessian 计算代价、不正定问题）
3. 掌握 BFGS 和 CG 作为实用的大规模方法（拟 Newton 近似、共轭方向构造）
4. 了解信任域方法如何保证全局收敛（与线搜索策略的对比）
5. 了解次梯度方法处理非光滑问题（凸分析与近端算子）

5.8.8 关键术语中英对照

中文	英文
最速下降法	Steepest Descent / Gradient Descent
牛顿法	Newton's Method
共轭梯度法	Conjugate Gradient (CG)
拟牛顿法	Quasi-Newton Method
DFP 公式	Davidon-Fletcher-Powell Formula
BFGS 公式	Broyden-Fletcher-Goldfarb-Shanno Formula
信任域方法	Trust Region Method
Levenberg-Marquardt	Levenberg-Marquardt Algorithm
次梯度	Subgradient
次微分	Subdifferential
近端算子	Proximal Operator
近端梯度法	Proximal Gradient Method
FISTA	Fast Iterative Shrinkage-Thresholding Algorithm
坐标轮换法	Cyclic Coordinate Method
Hooke-Jeeves 方法	Hooke-Jeeves Pattern Search
Rosenbrock 方法	Rosenbrock's Method
下降方向	Descent Direction
共轭方向	Conjugate Direction
割线条件	Secant Condition

Chapter 6

约束最优化理论

6.1 约束最优化问题概述

6.1.1 问题描述

一般约束最优化问题（非线性规划，NLP）的标准形式：

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, 2, \dots, m \\ & h_j(x) = 0, \quad j = 1, 2, \dots, l \end{aligned} \tag{6.1}$$

其中 $x \in \mathbb{R}^n$, $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 为目标函数, g_i 为不等式约束, h_j 为等式约束。

定义 6.1.1 (可行集). 可行集 (*Feasible Set*) 定义为：

$$\mathcal{F} = \{x \in \mathbb{R}^n : g_i(x) \leq 0, \forall i; h_j(x) = 0, \forall j\}$$

定义 6.1.2 (积极约束与有效集). 在点 $\bar{x} \in \mathcal{F}$ 处：

- 若 $g_i(\bar{x}) = 0$, 称约束 g_i 在 $\bar{x}$ 处**积极的** (*active*)
- 有效集 (*Active Set*): $\mathcal{A}(\bar{x}) = \{i : g_i(\bar{x}) = 0\} \cup \{m + j : j = 1, \dots, l\}$

6.1.2 可行方向

定义 6.1.3 (可行方向). 设 $\bar{x} \in \mathcal{F}$, 向量 $d \in \mathbb{R}^n$ 称为 $\bar{x}$ 处的**可行方向**, 如果存在 $\bar{\alpha} > 0$, 使得：

$$\bar{x} + \alpha d \in \mathcal{F}, \quad \forall \alpha \in [0, \bar{\alpha}]$$

下降可行方向

对于问题 (6.1), 在点 $\bar{x}$ 处的可行方向 d 还需满足下降条件。若 d 满足:

$$\nabla f(\bar{x})^T d < 0 \quad (\text{下降方向})$$

$$\nabla g_i(\bar{x})^T d \leq 0, \quad i \in \mathcal{A}(\bar{x}) \quad (\text{不等式约束})$$

$$\nabla h_j(\bar{x})^T d = 0, \quad j = 1, \dots, l \quad (\text{等式约束})$$

则 d 称为**下降可行方向**。

例题 6.1.1 (可行方向验证). 考虑问题:

$$\min x_1^2 + x_2^2, \quad s.t. \quad x_1 + x_2 \leq 2, \quad x_1 \geq 0, \quad x_2 \geq 0$$

在点 $\bar{x} = (1, 1)$ 处, $g_1(x) = x_1 + x_2 - 2$ 是积极约束。

$$\nabla g_1 = (1, 1), \quad \nabla f = (2x_1, 2x_2) = (2, 2)。$$

取 $d = (-1, -1)$:

$$\nabla f^T d = (2, 2) \cdot (-1, -1) = -4 < 0 \quad (\text{下降})$$

$$\nabla g_1^T d = (1, 1) \cdot (-1, -1) = -2 \leq 0 \quad (\text{可行})$$

所以 $d = (-1, -1)$ 是下降可行方向。

6.1.3 切锥与线性化可行方向

定义 6.1.4 (切锥 (Tangent Cone)). 设 $\mathcal{F} \subseteq \mathbb{R}^n$, $\bar{x} \in \mathcal{F}$ 。切锥 $T_{\mathcal{F}}(\bar{x})$ 定义为所有满足以下条件的向量 d 的集合: 存在序列 $\{x_k\} \subset \mathcal{F}$, $x_k \rightarrow \bar{x}$, 以及 $\{\alpha_k\} \subset \mathbb{R}_+$, $\alpha_k \rightarrow 0$, 使得:

$$d = \lim_{k \rightarrow \infty} \frac{x_k - \bar{x}}{\alpha_k}$$

定义 6.1.5 (线性化可行方向 (LFD)). 在点 $\bar{x}$ 处的线性化可行方向集为:

$$\mathcal{L}(\bar{x}) = \{d \in \mathbb{R}^n : \nabla g_i(\bar{x})^T d \leq 0, i \in \mathcal{A}(\bar{x}); \nabla h_j(\bar{x})^T d = 0, j = 1, \dots, l\}$$

切锥与 LFD 的关系

一般来说, $T_{\mathcal{F}}(\bar{x}) \subseteq \mathcal{L}(\bar{x})$ 。当约束满足某些**约束规范** (Constraint Qualification) 时, 二者相等。约束规范保证了 KKT 条件的必要性。

6.1.4 Farkas 引理

Farkas 引理是约束优化理论的基石, 用于证明 KKT 条件的必要性。

定理 6.1.1 (Farkas 引理). 设 $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$. 则以下两个系统有且仅有一个有解:

系统 1: $Ax \leq 0, b^T x > 0$

系统 2: $A^T y = b, y \geq 0$

等价表述:

Farkas 引理 (标准形式)

设 $a_1, \dots, a_m, b \in \mathbb{R}^n$. 则以下两个系统有且仅有一个有解:

系统 1: $b^T d > 0$ 且 $a_i^T d \leq 0, i = 1, \dots, m$

系统 2: $b = \sum_{i=1}^m \lambda_i a_i, \lambda_i \geq 0$

几何解释: 要么 b 在锥 $\text{cone}\{a_1, \dots, a_m\}$ 内 (系统 2), 要么存在超平面将 b 与该锥分离 (系统 1)。

推论 6.1.1 (Gordan 定理). 以下两个系统有且仅有一个有解:

系统 1: $Ax < 0$

系统 2: $A^T y = 0, y \geq 0, y \neq 0$

6.1.5 无约束问题最优性条件回顾

在深入约束问题之前, 先回顾无约束问题的最优性条件。

定理 6.1.2 (无约束一阶必要条件). 设 f 连续可微. 若 x^* 是局部极小点, 则 $\nabla f(x^*) = 0$ 。

定理 6.1.3 (无约束二阶必要条件). 设 f 二阶连续可微. 若 x^* 是局部极小点, 则 $\nabla^2 f(x^*) \succeq 0$ (半正定)。

定理 6.1.4 (无约束二阶充分条件). 设 f 二阶连续可微. 若 $\nabla f(x^*) = 0$ 且 $\nabla^2 f(x^*) \succ 0$ (正定), 则 x^* 是严格局部极小点。

6.2 一阶最优性条件 (KKT 条件)

6.2.1 等式约束问题

先考虑只有等式约束的问题:

$$\min f(x), \quad \text{s.t. } h_j(x) = 0, \quad j = 1, \dots, l$$

定理 6.2.1 (等式约束一阶必要条件). 设 f 和 h_j 连续可微, x^* 是局部极小点, 且 $\{\nabla h_j(x^*)\}_{j=1}^l$ 线性无关 (LICQ)。则存在 $\mu^* \in \mathbb{R}^l$, 使得:

$$\nabla f(x^*) + \sum_{j=1}^l \mu_j^* \nabla h_j(x^*) = 0 \quad (6.2)$$

$$h_j(x^*) = 0, \quad j = 1, \dots, l \quad (6.3)$$

拉格朗日函数

定义拉格朗日函数:

$$\mathcal{L}(x, \mu) = f(x) + \sum_{j=1}^l \mu_j h_j(x)$$

则条件 (6.2) 等价于 $\nabla_x \mathcal{L}(x^*, \mu^*) = 0$ 。

例题 6.2.1 (等式约束问题).

$$\min x_1^2 + x_2^2, \quad s.t. \quad x_1 + x_2 = 1$$

$$\mathcal{L} = x_1^2 + x_2^2 + \mu(x_1 + x_2 - 1)$$

KKT 条件:

$$\frac{\partial \mathcal{L}}{\partial x_1} = 2x_1 + \mu = 0$$

$$\frac{\partial \mathcal{L}}{\partial x_2} = 2x_2 + \mu = 0$$

$$x_1 + x_2 = 1$$

解得: $x_1 = x_2 = 0.5$, $\mu = -1$ 。最优值 $f^* = 0.5$ 。

6.2.2 一般约束问题的 KKT 条件**KKT 一阶必要条件**

考虑问题 (6.1)。设 f, g_i, h_j 连续可微, x^* 是局部极小点, 且某约束规范 (CQ) 成立。则存在 $\lambda^* \in \mathbb{R}^m$, $\mu^* \in \mathbb{R}^l$, 满足:

(1) 平稳性 (Stationarity):

$$\nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(x^*) + \sum_{j=1}^l \mu_j^* \nabla h_j(x^*) = 0$$

(2) 原始可行性 (Primal Feasibility):

$$g_i(x^*) \leq 0 \quad (i = 1, \dots, m), \quad h_j(x^*) = 0 \quad (j = 1, \dots, l)$$

(3) 对偶可行性 (Dual Feasibility):

$$\lambda_i^* \geq 0, \quad i = 1, \dots, m$$

(4) 互补松弛条件 (Complementary Slackness):

$$\lambda_i^* \cdot g_i(x^*) = 0, \quad i = 1, \dots, m$$

KKT 条件的直观理解

- 平稳性：在最优解处，目标函数的梯度可由约束梯度的线性组合“抵消”
- 互补松弛：若 $\lambda_i^* > 0$ ，则 $g_i(x^*) = 0$ （约束积极）；若 $g_i(x^*) < 0$ ，则 $\lambda_i^* = 0$
- λ_i^* 的大小反映约束的“重要性”——影子价格

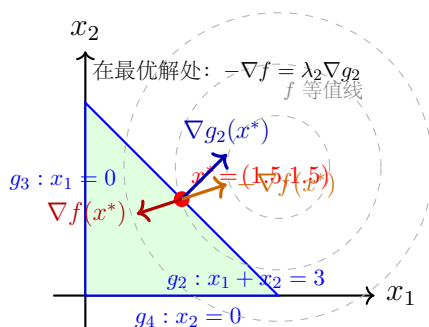


图 6.1: KKT 条件的几何解释：在最优解 x^* 处，负目标函数梯度 $-\nabla f$ 可由积极约束梯度的非负线性组合表示。本例中 $-\nabla f = 3(1, 1) = 3\nabla g_2$ ，故 $\lambda_2 = 3 > 0$

例题 6.2.2 (KKT 条件求解). 求解：

$$\min (x_1 - 2)^2 + (x_2 - 1)^2, \quad s.t. \quad x_1 + x_2 \leq 2, \quad x_1, x_2 \geq 0$$

改写约束： $g_1 = x_1 + x_2 - 2 \leq 0$, $g_2 = -x_1 \leq 0$, $g_3 = -x_2 \leq 0$

$$\nabla f = (2(x_1 - 2), 2(x_2 - 1)), \quad \nabla g_1 = (1, 1), \quad \nabla g_2 = (-1, 0), \quad \nabla g_3 = (0, -1)$$

KKT 条件：

$$2(x_1 - 2) + \lambda_1 - \lambda_2 = 0$$

$$2(x_2 - 1) + \lambda_1 - \lambda_3 = 0$$

$$\lambda_1(x_1 + x_2 - 2) = 0, \quad \lambda_1 \geq 0, \quad x_1 + x_2 \leq 2$$

$$\lambda_2(-x_1) = 0, \quad \lambda_2 \geq 0, \quad x_1 \geq 0$$

$$\lambda_3(-x_2) = 0, \quad \lambda_3 \geq 0, \quad x_2 \geq 0$$

Case 1: 假设 g_1 积极, g_2, g_3 不积极 $\Rightarrow \lambda_2 = \lambda_3 = 0$, $x_1 + x_2 = 2$

由前两式： $2(x_1 - 2) + \lambda_1 = 0$, $2(x_2 - 1) + \lambda_1 = 0$

相减： $2(x_1 - x_2 - 1) = 0 \Rightarrow x_1 - x_2 = 1$

结合 $x_1 + x_2 = 2$: $x_1 = 1.5$, $x_2 = 0.5$, $\lambda_1 = 1 > 0$ (验证通过)

验证： $x_1, x_2 > 0$ (g_2, g_3 不积极)，满足所有条件。

最优解： $x^* = (1.5, 0.5)$ ，最优值 $f^* = 0.5$ 。

6.2.3 约束规范 (Constraint Qualifications)

KKT 条件的成立需要约束规范。常用的约束规范:

定义 6.2.1 (LICQ — 线性无关约束规范). 在 x^* 处, 积极约束的梯度 $\{\nabla g_i(x^*) : i \in \mathcal{A}(x^*)\} \cup \{\nabla h_j(x^*)\}$ 线性无关。

定义 6.2.2 (MFCQ — Mangasarian-Fromovitz 约束规范). 存在向量 $d \in \mathbb{R}^n$, 使得:

$$\begin{aligned}\nabla g_i(x^*)^T d &< 0, \quad i \in \mathcal{A}(x^*) \\ \nabla h_j(x^*)^T d &= 0, \quad j = 1, \dots, l\end{aligned}$$

且 $\{\nabla h_j(x^*)\}$ 线性无关。

定义 6.2.3 (Slater 条件). 对于凸优化问题, 若存在严格可行点 $\tilde{x}$ 使得:

$$g_i(\tilde{x}) < 0 \quad (\forall i), \quad h_j(\tilde{x}) = 0 \quad (\forall j)$$

且等式约束为仿射, 则 Slater 条件成立。

定理 6.2.2 (CQ 的蕴含关系). 对于凸优化问题:

$$LICQ \Rightarrow MFCQ; \quad Slater \text{ (凸)} \Rightarrow KKT \text{ 必要}$$

Slater 条件是最常用的凸优化 CQ。

例题 6.2.3 (Slater 条件验证).

$$\min x_1^2 + x_2^2, \quad s.t. \quad x_1 + x_2 \leq 2, \quad x_1 \geq 0, \quad x_2 \geq 0$$

取 $\tilde{x} = (0.5, 0.5)$:

$$\begin{aligned}g_1(0.5, 0.5) &= 1 - 2 = -1 < 0 \\ g_2(0.5, 0.5) &= -0.5 < 0 \\ g_3(0.5, 0.5) &= -0.5 < 0\end{aligned}$$

Slater 条件满足, KKT 条件是最优性的充要条件。

6.2.4 凸优化的 KKT 充分性

定理 6.2.3 (KKT 充分条件). 若问题 (6.1) 是凸优化问题 (f 凸, g_i 凸, h_j 仿射), 且 (x^*, λ^*, μ^*) 满足 KKT 条件, 则 x^* 是全局最优解。

证明概要:

$$\begin{aligned}f(x) &\geq f(x^*) + \nabla f(x^*)^T (x - x^*) \\ &= f(x^*) - \sum_i \lambda_i^* \nabla g_i(x^*)^T (x - x^*) - \sum_j \mu_j^* \nabla h_j(x^*)^T (x - x^*)\end{aligned}$$

利用凸性 $g_i(x) \geq g_i(x^*) + \nabla g_i(x^*)^T (x - x^*)$ 和互补松弛条件即可得证。

6.2.5 拉格朗日函数与鞍点

定义 6.2.4 (一般拉格朗日函数).

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^l \mu_j h_j(x)$$

其中 $\lambda \geq 0$ 。

定义 6.2.5 (鞍点). (x^*, λ^*, μ^*) 称为 $\mathcal{L}$ 的鞍点, 如果:

$$\mathcal{L}(x^*, \lambda, \mu) \leq \mathcal{L}(x^*, \lambda^*, \mu^*) \leq \mathcal{L}(x, \lambda^*, \mu^*), \quad \forall x, \forall \lambda \geq 0, \forall \mu$$

定理 6.2.4 (鞍点定理). 若 (x^*, λ^*, μ^*) 是 $\mathcal{L}$ 的鞍点, 则 x^* 是原问题最优解, (λ^*, μ^*) 是对偶问题最优解, 且无对偶间隙。

6.3 二阶最优性条件

6.3.1 临界锥 (Critical Cone)

二阶条件需要限制在临界锥上, 因为积极约束之外的方向不影响局部最优性。

定义 6.3.1 (临界锥). 设 x^* 是满足 KKT 条件的点, 对应乘子 (λ^*, μ^*) 。临界锥定义为:

$$\mathcal{C}(x^*) = \left\{ d \in \mathbb{R}^n : \begin{array}{l} \nabla g_i(x^*)^T d = 0, \quad i \in \mathcal{A}(x^*) \text{ 且 } \lambda_i^* > 0 \\ \nabla g_i(x^*)^T d \leq 0, \quad i \in \mathcal{A}(x^*) \text{ 且 } \lambda_i^* = 0 \\ \nabla h_j(x^*)^T d = 0, \quad j = 1, \dots, l \end{array} \right\}$$

临界锥的含义

- 对于强积极约束 ($\lambda_i^* > 0$): 方向必须与之正交 (不能进一步违反)
- 对于弱积极约束 ($\lambda_i^* = 0$): 方向可以非正交 (约束仍然允许移动)
- 临界锥是线性化可行方向集的子集

6.3.2 二阶必要条件

定理 6.3.1 (二阶必要条件). 设 f, g_i, h_j 二阶连续可微, x^* 是局部极小点, LICQ 成立, (λ^*, μ^*) 为对应 KKT 乘子。则:

$$d^T \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*, \mu^*) d \geq 0, \quad \forall d \in \mathcal{C}(x^*)$$

即 Lagrange 函数 Hessian 在临界锥上半正定。

6.3.3 二阶充分条件

定理 6.3.2 (二阶充分条件). 设 f, g_i, h_j 二阶连续可微, x^* 满足 KKT 条件, 对应乘子 (λ^*, μ^*) . 若:

$$d^T \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*, \mu^*) d > 0, \quad \forall d \in \mathcal{C}(x^*) \setminus \{0\}$$

则 x^* 是严格局部极小点。

例题 6.3.1 (二阶条件验证).

$$\min x_1^2 + x_2^2, \quad s.t. \quad x_1 + x_2 \geq 1$$

改写为 $g(x) = 1 - x_1 - x_2 \leq 0$. KKT 解得 $x^* = (0.5, 0.5)$, $\lambda^* = 1$.

$$\mathcal{L} = x_1^2 + x_2^2 + \lambda(1 - x_1 - x_2)$$

$$\nabla_{xx}^2 \mathcal{L} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

积极约束 g , $\lambda^* = 1 > 0$, 临界锥: $\nabla g^T d = (-1, -1) \cdot (d_1, d_2) = 0$, 即 $d_1 + d_2 = 0$.

对 $d = (t, -t)$: $d^T \nabla^2 \mathcal{L} d = 2t^2 + 2t^2 = 4t^2 > 0$ ($t \neq 0$)

二阶充分条件满足, $x^* = (0.5, 0.5)$ 是严格局部极小点。

6.3.4 例子: 区分不同 KKT 点

例题 6.3.2 (多个 KKT 点的判断).

$$\min x_1^3 + x_2^3, \quad s.t. \quad x_1 + x_2 = 1$$

$$\mathcal{L} = x_1^3 + x_2^3 + \mu(x_1 + x_2 - 1)$$

KKT 条件:

$$3x_1^2 + \mu = 0$$

$$3x_2^2 + \mu = 0$$

$$x_1 + x_2 = 1$$

由前两式: $x_1^2 = x_2^2 \Rightarrow x_1 = x_2$ 或 $x_1 = -x_2$

Case 1: $x_1 = x_2 = 0.5$, $\mu = -0.75$

$$\nabla^2 \mathcal{L} = \begin{bmatrix} 6x_1 & 0 \\ 0 & 6x_2 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix}$$

临界锥: $d_1 + d_2 = 0$, 即 $d = (t, -t)$

$d^T \nabla^2 \mathcal{L} d = 3t^2 + 3t^2 = 6t^2 > 0 \Rightarrow$ 严格局部极小!

Case 2: $x_1 = -x_2$, 结合 $x_1 + x_2 = 1$ 无解。

但若考虑 $x_1 = 0, x_2 = 1$ (需验证是否为 KKT 点): $\mu = 0$, 但 $3(1)^2 + 0 \neq 0$, 不是 KKT 点。

例题 6.3.3 (KKT 条件的几何验证).

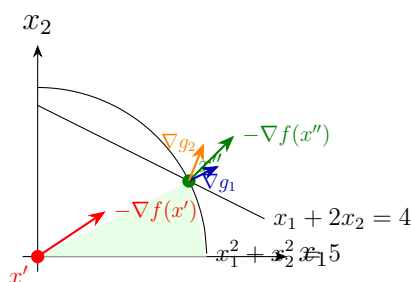
$$\min f(x) = (x_1 - 3)^2 + (x_2 - 2)^2, \quad s.t. \quad x_1^2 + x_2^2 \leq 5, \quad x_1 + 2x_2 \leq 4, \quad x_1 \geq 0, \quad x_2 \geq 0$$

判断 $x' = (0, 0)$: 起作用约束为 $g_3 = -x_1$, $g_4 = -x_2$. $\nabla f(0, 0) = (-6, -4)^T$, 平稳性要求 $(-6, -4)^T + \lambda_3(-1, 0)^T + \lambda_4(0, -1)^T = 0$, 得 $\lambda_3 = -6 < 0$, 违反对偶可行性。故 x' 不是 KKT 点。

判断 $x'' = (2, 1)$: 起作用约束为 $g_1 = x_1^2 + x_2^2 - 5$, $g_2 = x_1 + 2x_2 - 4$. $\nabla f(2, 1) = (-2, -2)^T$, $\nabla g_1 = (4, 2)^T$, $\nabla g_2 = (1, 2)^T$. 平稳性:

$$(-2, -2)^T + \lambda_1(4, 2)^T + \lambda_2(1, 2)^T = 0 \Rightarrow \lambda_1 = \frac{1}{3}, \quad \lambda_2 = \frac{2}{3}$$

均非负, 互补松弛成立。故 x'' 是 KKT 点。



几何解释: x' 处 $-\nabla f$ 指向可行域内部, 不可能是最优点; x'' 位于两个约束边界交点, $-\nabla f(x'')$ 恰好可由起作用约束法向量的非负组合表示。

例题 6.3.4 (二阶条件判定鞍点).

$$\min x_1^2 - x_2^2, \quad s.t. \quad x_1 + x_2 = 1$$

$$\mathcal{L} = x_1^2 - x_2^2 + \mu(x_1 + x_2 - 1)$$

$$\text{KKT 条件: } 2x_1 + \mu = 0, \quad -2x_2 + \mu = 0, \quad x_1 + x_2 = 1$$

$$\text{解得: } x_1 = 0.5, x_2 = 0.5, \quad \mu = -1$$

$$\nabla^2 \mathcal{L} = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}$$

$$\text{临界锥: } d_1 + d_2 = 0, \quad d = (t, -t)$$

$$d^T \nabla^2 \mathcal{L} d = 2t^2 - 2t^2 = 0$$

二阶条件无法判定。实际上这不是局部极小点 (目标函数在约束线上无下界)。

6.4 对偶理论

6.4.1 拉格朗日对偶问题

定义 6.4.1 (对偶函数). 拉格朗日对偶函数定义为:

$$q(\lambda, \mu) = \inf_{x \in \mathbb{R}^n} \mathcal{L}(x, \lambda, \mu) = \inf_{x \in \mathbb{R}^n} \left\{ f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^l \mu_j h_j(x) \right\}$$

定义 6.4.2 (拉格朗日对偶问题).

$$\begin{aligned} \max \quad & q(\lambda, \mu) \\ \text{s.t.} \quad & \lambda \geq 0 \end{aligned} \quad (6.4)$$

对偶函数的性质

- $q(\lambda, \mu)$ 是凹函数 (即使原问题非凸)
- 对偶问题是凸优化问题 (最大化凹函数)
- 若 $\inf$ 在某点达到, 则 $q(\lambda, \mu) = \mathcal{L}(x^*, \lambda, \mu)$
- 若 $\inf$ 未达到, $q(\lambda, \mu)$ 可取 $-\infty$

6.4.2 弱对偶定理

定理 6.4.1 (弱对偶定理). 设 $\bar{x}$ 是原问题可行解, (λ, μ) 是对偶问题可行解 ($\lambda \geq 0$), 则:

$$q(\lambda, \mu) \leq f(\bar{x})$$

特别地, 若 p^* 和 d^* 分别是原问题和对偶问题的最优值, 则:

$$d^* \leq p^*$$

证明: 对任意可行 $\bar{x}$ 和 $\lambda \geq 0$:

$$\begin{aligned} q(\lambda, \mu) &= \inf_x \mathcal{L}(x, \lambda, \mu) \leq \mathcal{L}(\bar{x}, \lambda, \mu) \\ &= f(\bar{x}) + \underbrace{\sum_i \lambda_i g_i(\bar{x})}_{\leq 0} + \underbrace{\sum_j \mu_j h_j(\bar{x})}_{=0} \leq f(\bar{x}) \end{aligned}$$

定义 6.4.3 (对偶间隙). 对偶间隙定义为 $p^* - d^* \geq 0$. 若 $p^* = d^*$, 称强对偶成立。

6.4.3 强对偶定理

定理 6.4.2 (强对偶定理). 对于凸优化问题, 若 Slater 条件成立 (存在严格内点), 则强对偶成立, 即 $p^* = d^*$ 。

若 p^* 有限, 则对偶最优解 (λ^*, μ^*) 可达。

例题 6.4.1 (强对偶成立).

$$\min x_1^2 + x_2^2, \quad \text{s.t. } x_1 + x_2 \geq 1$$

改写为 $g(x) = 1 - x_1 - x_2 \leq 0$ 。

$$\mathcal{L} = x_1^2 + x_2^2 + \lambda(1 - x_1 - x_2)$$

对 $\lambda \geq 0$ 求 $\inf$:

$$\frac{\partial \mathcal{L}}{\partial x_1} = 2x_1 - \lambda = 0 \Rightarrow x_1 = \lambda/2$$

同理 $x_2 = \lambda/2$ 。

$$q(\lambda) = \frac{\lambda^2}{4} + \frac{\lambda^2}{4} + \lambda(1 - \lambda) = \lambda - \frac{\lambda^2}{2}$$

对偶问题: $\max_{\lambda \geq 0} \lambda - \frac{\lambda^2}{2}$

$$q'(\lambda) = 1 - \lambda = 0 \Rightarrow \lambda^* = 1$$

$d^* = q(1) = 1 - 0.5 = 0.5 = p^*$ 。强对偶成立。

例题 6.4.2 (存在对偶间隙)。

$$\min -x, \quad s.t. \quad x^3 \leq 0, \quad x \in \mathbb{R}$$

等价于 $x \leq 0$, 最优解 $x^* = 0$, $p^* = 0$ 。

$$q(\lambda) = \inf_x \{-x + \lambda x^3\}$$

当 $\lambda > 0$: 令 $x \rightarrow -\infty$, $\lambda x^3 - x \rightarrow -\infty$, $q(\lambda) = -\infty$

当 $\lambda = 0$: $q(0) = \inf_x (-x) = -\infty$

$d^* = \sup_{\lambda \geq 0} q(\lambda) = -\infty < p^* = 0$, 存在对偶间隙!

原因: 非凸约束 ($x^3 \leq 0$ 等价于 $x \leq 0$, 但写成 x^3 非凸), Slater 条件不满足。

6.4.4 鞍点定理

定理 6.4.3 (鞍点定理). (x^*, λ^*, μ^*) 是 $\mathcal{L}$ 的鞍点当且仅当:

1. x^* 是原问题最优解
2. (λ^*, μ^*) 是对偶问题最优解
3. $p^* = d^*$ (无对偶间隙)

6.4.5 扰动函数与灵敏度分析

定义 6.4.4 (扰动函数). 定义 $v(u) = \inf\{f(x) : g(x) \leq u, h(x) = 0\}$, 其中 u 是约束右端项的扰动。

定理 6.4.4 (灵敏度分析). 若强对偶成立, 则对偶最优乘子 λ^* 满足:

$$\lambda_i^* = -\frac{\partial v(0)}{\partial u_i}$$

即 λ_i^* 表示约束右端项增加 1 单位时, 最优值的减小率 (影子价格)。

例题 6.4.3 (灵敏度分析).

$$\min x_1^2 + x_2^2, \quad s.t. \quad x_1 + x_2 \geq 1$$

扰动约束为 $x_1 + x_2 \geq 1 + u$ 。 $v(u) = \frac{(1+u)^2}{2}$ (对称解)

$\lambda^* = 1$ (前面已求得)

$v'(0) = (1+u)|_{u=0} = 1 = -\lambda^*$ (验证一致)

6.5 惩罚函数法

6.5.1 二次惩罚函数法

基本思想

将约束 violation 以惩罚项形式加入目标函数, 通过增大惩罚参数 $\rho \rightarrow \infty$, 使无约束问题的解逼近原约束问题的解。

定义 6.5.1 (二次惩罚函数). 对于问题 (6.1), 定义惩罚函数:

$$P(x, \rho) = f(x) + \frac{\rho}{2} \left[\sum_{i=1}^m \max(0, g_i(x))^2 + \sum_{j=1}^l h_j(x)^2 \right]$$

其中 $\rho > 0$ 为惩罚参数。

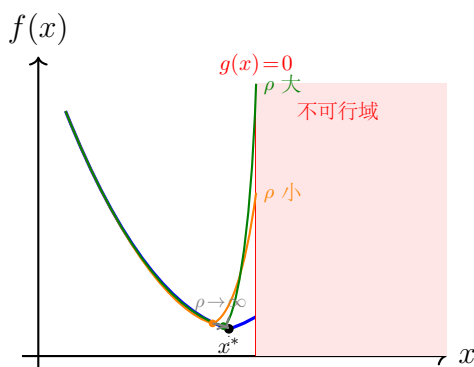


图 6.2: 二次惩罚函数法: 增大 ρ 使惩罚函数的极小点逼近约束最优解 x^*

二次惩罚法算法

初始化: 选取 $x^{(0)}$, $\rho_0 > 0$, 放大因子 $\beta > 1$, 容差 $\varepsilon > 0$

For $k = 0, 1, 2, \dots$:

1. 以 $x^{(k)}$ 为初始点, 求解 $\min_x P(x, \rho_k)$, 得 $x^{(k+1)}$
2. 若约束 violation $< \varepsilon$, 停止
3. 更新 $\rho_{k+1} = \beta \rho_k$

定理 6.5.1 (二次惩罚法的收敛性). 设 f, g_i, h_j 连续, $\rho_k \rightarrow \infty$ 。若 $x^{(k)}$ 是 $P(x, \rho_k)$ 的全局极小点, 则 $\{x^{(k)}\}$ 的任何聚点都是原问题的全局最优解。

二次惩罚法的缺点

- 当 ρ 很大时, Hessian 条件数恶化, 数值困难
- 需要求解一系列无约束问题, 计算量大
- 对非凸问题可能收敛到局部极小

例题 6.5.1 (二次惩罚法——迭代两次).

$$\min x^2, \quad s.t. \ x \geq 1$$

惩罚函数: $P(x, \rho) = x^2 + \frac{\rho}{2} \max(0, 1 - x)^2$

第 1 次迭代 ($\rho_0 = 1$, 初值 $x^{(0)} = 0$):

$$P(x, 1) = x^2 + 0.5(1 - x)^2 \quad (\text{假设 } x < 1)$$

$$\frac{dP}{dx} = 2x - (1 - x) = 3x - 1 = 0 \Rightarrow x^{(1)} = \frac{1}{3} \approx 0.333$$

约束 violation: $1 - 1/3 = 2/3 > \varepsilon$, 继续。

第 2 次迭代 ($\rho_1 = 10$):

$$P(x, 10) = x^2 + 5(1 - x)^2$$

$$\frac{dP}{dx} = 2x - 10(1 - x) = 12x - 10 = 0 \Rightarrow x^{(2)} = \frac{10}{12} = \frac{5}{6} \approx 0.833$$

可见随着 ρ 增大, $x^{(k)} \rightarrow 1$ (约束最优解)。

6.5.2 精确惩罚函数法

精确惩罚

二次惩罚法需要 $\rho \rightarrow \infty$ 。精确惩罚函数法通过 ℓ_1 惩罚, 对有限 ρ 即可得到精确解。

定义 6.5.2 (ℓ_1 精确惩罚函数).

$$\Phi(x, \rho) = f(x) + \rho \left[\sum_{i=1}^m \max(0, g_i(x)) + \sum_{j=1}^l |h_j(x)| \right]$$

定理 6.5.2 (精确性). 设 x^* 是原问题的局部最优解, LICQ 和严格互补条件成立。则存在 $\bar{\rho} > 0$, 使得对所有 $\rho \geq \bar{\rho}$, x^* 也是 $\Phi(x, \rho)$ 的局部极小点。

例题 6.5.2 (精确惩罚法——迭代两次).

$$\min x^2 + 2x, \quad s.t. \ x \geq 1$$

精确惩罚: $\Phi(x, \rho) = x^2 + 2x + \rho \max(0, 1 - x)$

Case A ($x \geq 1$): $\Phi = x^2 + 2x$, $\Phi' = 2x + 2 > 0$ ($x \geq 1$), 最小在 $x = 1$ 。

Case B ($x < 1$): $\Phi = x^2 + 2x + \rho(1 - x) = x^2 + (2 - \rho)x + \rho$

$$\Phi' = 2x + 2 - \rho = 0 \Rightarrow x = \frac{\rho - 2}{2}$$

第 1 次迭代 ($\rho = 3$): $x = 0.5 < 1$, Case B. $\Phi(0.5) = 0.25 + 1 + 1.5 = 2.75$ 。

与 Case A 的边界值 $\Phi(1) = 3$ 比较, 最小值在 $x = 0.5$ (不精确)。

第 2 次迭代 ($\rho = 5$): $x = 1.5 \geq 1$, 转入 Case A。

在 Case A 中, $\Phi' = 2x + 2 > 0$ ($x \geq 1$), 故 $x^* = 1$ 是最小点。

$\Phi(1) = 3$, 验证了 $x = 1$ 是精确解。

6.5.3 增广拉格朗日法

增广拉格朗日法结合了拉格朗日函数和惩罚函数的优点, 对有限惩罚参数即可收敛。

定义 6.5.3 (增广拉格朗日函数). 对于等式约束问题:

$$\mathcal{L}_A(x, \mu, \rho) = f(x) + \mu^T h(x) + \frac{\rho}{2} \|h(x)\|^2$$

增广拉格朗日法

初始化: $x^{(0)}, \mu^{(0)}, \rho_0 > 0$

For $k = 0, 1, 2, \dots$:

1. $x^{(k+1)} = \arg \min_x \mathcal{L}_A(x, \mu^{(k)}, \rho_k)$
2. $\mu^{(k+1)} = \mu^{(k)} + \rho_k h(x^{(k+1)})$
3. 若 $\|h(x^{(k+1)})\| < \varepsilon$, 停止
4. 可选: $\rho_{k+1} = \min(\beta \rho_k, \rho_{\max})$

乘子更新原理

由 KKT 条件 $\nabla f + \mu^T \nabla h = 0$ 和增广拉格朗日函数的平稳性:

$$\nabla f + (\mu^{(k)})^T \nabla h + \rho_k h^T \nabla h = 0$$

对比得: $\mu^{(k+1)} = \mu^{(k)} + \rho_k h(x^{(k+1)})$ 是对真实乘子的估计。

例题 6.5.3 (增广拉格朗日法——迭代两次).

$$\min x^2, \quad s.t. \quad x = 2$$

$$\mathcal{L}_A = x^2 + \mu(x - 2) + \frac{\rho}{2}(x - 2)^2$$

$$\frac{\partial \mathcal{L}_A}{\partial x} = 2x + \mu + \rho(x - 2) = 0 \Rightarrow x = \frac{2\rho - \mu}{2 + \rho}$$

第 1 次迭代 ($\mu^{(0)} = 0, \rho_0 = 1$):

$$x^{(1)} = \frac{2-0}{3} = \frac{2}{3} \approx 0.667$$

$$\mu^{(1)} = 0 + 1 \cdot (2/3 - 2) = -4/3 \approx -1.333$$

第 2 次迭代 ($\rho_1 = 2$):

$$x^{(2)} = \frac{4 - (-4/3)}{4} = \frac{16/3}{4} = \frac{4}{3} \approx 1.333$$

$$\mu^{(2)} = -4/3 + 2(4/3 - 2) = -4/3 - 4/3 = -8/3 \approx -2.667$$

继续迭代, $x^{(k)} \rightarrow 2$, $\mu^{(k)} \rightarrow -4$ (真实乘子: $\nabla f = 2x = 4$, $\mu = -4$)。

6.5.4 Python 代码实现

二次惩罚法

```

1 import numpy as np
2 from scipy.optimize import minimize
3
4 def quadratic_penalty(f, grad_f, g_funcs, h_funcs, x0,
5                       rho0=1.0, beta=10.0, eps=1e-6,
6                       max_iter=50):
7     """
8     二次惩罚函数法
9
10    Parameters:
11        f: 目标函数
12        grad_f: 目标函数梯度
13        g_funcs: 不等式约束函数列表 (g_i <= 0)
14        h_funcs: 等式约束函数列表
15        x0: 初始点
16        rho0: 初始惩罚参数
17        beta: 放大因子
18        eps: 约束容差
19
20    Returns:
21        x_opt: 最优解
22        history: 迭代历史
23    """
24    x = x0.copy()
25    rho = rho0
26    history = []
27
28    for k in range(max_iter):
29        # 构造惩罚函数

```

```

30     def P(x_val):
31         penalty = 0.0
32         for g in g_funcs:
33             gv = g(x_val)
34             if gv > 0:
35                 penalty += gv**2
36         for h in h_funcs:
37             penalty += h(x_val)**2
38         return f(x_val) + 0.5 * rho * penalty
39
40     # 求解无约束问题
41     res = minimize(P, x, method='BFGS')
42     x = res.x
43
44     # 计算约束违反
45     violation = 0.0
46     for g in g_funcs:
47         violation += max(0, g(x))**2
48     for h in h_funcs:
49         violation += h(x)**2
50     violation = np.sqrt(violation)
51
52     history.append((x.copy(), f(x), violation, rho))
53
54     if violation < eps:
55         break
56
57     rho *= beta
58
59     return x, history
60
61 # 示例:  $\min x^2, s.t. x \geq 1$ 
62 f = lambda x: x[0]**2
63 g = [lambda x: 1 - x[0]] #  $1 - x \leq 0$ , i.e.,  $x \geq 1$ 
64
65 x0 = np.array([0.0])
66 x_opt, hist = quadratic_penalty(f, None, g, [], x0,
67                                rho0=1.0, beta=10.0)
68 for i, (x, fx, v, rho) in enumerate(hist):

```

```

69     print(f"Iter {i}: x={x[0]:.4f}, f={fx:.4f}, "
70           f"viol={v:.4f}, rho={rho:.1f}")

```

增广拉格朗日法

```

1  def augmented_lagrangian(f, h_funcs, x0, mu0, rho0=1.0,
2                          beta=2.0, eps=1e-6, max_iter=50):
3
4      """
5      增广拉格朗日法（等式约束）
6
7      Parameters:
8          f: 目标函数
9          h_funcs: 等式约束函数列表
10         x0: 初始点
11         mu0: 初始乘子
12         rho0: 初始惩罚参数
13         beta: 放大因子
14
15     Returns:
16         x_opt: 最优解
17         mu_opt: 最优乘子
18         history: 迭代历史
19     """
20     x = np.array(x0, dtype=float)
21     mu = np.array(mu0, dtype=float)
22     rho = rho0
23     history = []
24
25     for k in range(max_iter):
26         # 构造增广拉格朗日函数
27         def L_A(x_val):
28             h_vals = np.array([h(x_val) for h in h_funcs])
29             return f(x_val) + np.dot(mu, h_vals) + \
30                    0.5 * rho * np.sum(h_vals**2)
31
32         res = minimize(L_A, x, method='BFGS')
33         x = res.x
34
35         h_vals = np.array([h(x) for h in h_funcs])

```

```

35     h_norm = np.linalg.norm(h_vals)
36
37     history.append((x.copy(), mu.copy(), rho, h_norm))
38
39     if h_norm < eps:
40         break
41
42     mu = mu + rho * h_vals
43     rho = min(beta * rho, 1e6)
44
45     return x, mu, history
46
47 # 示例: min x^2, s.t. x = 2
48 f = lambda x: x[0]**2
49 h = [lambda x: x[0] - 2]
50
51 x0 = np.array([0.0])
52 mu0 = np.array([0.0])
53 x_opt, mu_opt, hist = augmented_lagrangian(f, h, x0, mu0)
54 for i, (x, mu, rho, h_n) in enumerate(hist):
55     print(f"Iter {i}: x={x[0]:.4f}, mu={mu[0]:.4f}, "
56           f"rho={rho:.1f}, ||h||={h_n:.4f}")

```

6.6 障碍函数法与内点法

6.6.1 对数障碍函数法

内点法思想

内点法通过在可行域内部迭代，用障碍函数阻止迭代点靠近边界。只适用于不等式约束。

定义 6.6.1 (对数障碍函数). 对于不等式约束 $g_i(x) \leq 0$ (即 $-g_i(x) \geq 0$)，定义对数障碍函数：

$$B(x, \mu) = f(x) - \mu \sum_{i=1}^m \ln(-g_i(x))$$

其中 $\mu > 0$ 为障碍参数。当 x 接近边界时 $B \rightarrow +\infty$ 。

对数障碍法

初始化: 严格可行点 $x^{(0)}$ ($g_i(x^{(0)}) < 0$), $\mu_0 > 0$, 缩减因子 $\sigma \in (0, 1)$

For $k = 0, 1, 2, \dots$:

1. 以 $x^{(k)}$ 为初始点, 求解 $\min_x B(x, \mu_k)$, 得 $x^{(k+1)}$
2. 若 $\mu_k < \varepsilon$, 停止
3. $\mu_{k+1} = \sigma \mu_k$

定理 6.6.1 (收敛性). 设 f, g_i 连续, $\mu_k \rightarrow 0$ 。若 $x^{(k)}$ 是 $B(x, \mu_k)$ 的全局极小点, 则 $\{x^{(k)}\}$ 的任何聚点都是原问题的全局最优解。

障碍函数法的特点

- 只需要一个初始严格可行点
- 迭代点始终保持在可行域内部
- 当 $\mu \rightarrow 0$ 时, 条件数恶化 (类似惩罚法)
- 适用于凸优化问题效果良好

例题 6.6.1 (对数障碍法——迭代两次).

$$\min x, \quad s.t. \quad x \geq 0, \quad x \leq 1$$

改写: $g_1(x) = -x \leq 0$, $g_2(x) = x - 1 \leq 0$

障碍函数: $B(x, \mu) = x - \mu \ln(x) - \mu \ln(1 - x)$ (要求 $0 < x < 1$)

$$\frac{dB}{dx} = 1 - \frac{\mu}{x} + \frac{\mu}{1-x} = 0$$

$$\Rightarrow x(1-x) - \mu(1-x) + \mu x = 0$$

$$\Rightarrow x - x^2 - \mu + \mu x + \mu x = 0$$

$$\Rightarrow x^2 - (1 + 2\mu)x + \mu = 0$$

第 1 次迭代 ($\mu_0 = 0.1$):

$$x^2 - 1.2x + 0.1 = 0$$

$$x = \frac{1.2 - \sqrt{1.44 - 0.4}}{2} = \frac{1.2 - 1.02}{2} \approx 0.09 \quad (\text{取在 } (0, 1) \text{ 内的根})$$

$$\text{实际解: } x^{(1)} = \frac{1.2 - \sqrt{1.04}}{2} \approx \frac{1.2 - 1.0199}{2} \approx 0.090$$

第 2 次迭代 ($\mu_1 = 0.01$):

$$x^2 - 1.02x + 0.01 = 0$$

$$x = \frac{1.02 - \sqrt{1.0404 - 0.04}}{2} = \frac{1.02 - 1.0}{2} \approx 0.010$$

可见 $x^{(k)} \rightarrow 0$ (最优解)。

6.6.2 中心路径

定义 6.6.2 (中心路径). 对于凸优化问题, 当 μ 从 ∞ 变到 0 时, $B(x, \mu)$ 的极小点 $x(\mu)$ 形成一条轨迹, 称为**中心路径** (Central Path)。

定理 6.6.2 (中心路径的性质). 1. $x(\mu)$ 在可行域严格内部

2. $\mu \rightarrow 0$ 时, $x(\mu) \rightarrow x^*$ (最优解)

3. $\mu \rightarrow \infty$ 时, $x(\mu) \rightarrow$ 解析中心

6.6.3 原始-对偶内点法

原始-对偶方法

同时更新原始变量 x 和对偶变量 λ , 沿着牛顿方向逼近 KKT 条件。

考虑问题: $\min f(x)$ s.t. $g_i(x) \leq 0$ 。

KKT 条件改写 (引入松弛变量 $s_i = -g_i(x) \geq 0$):

$$\nabla f(x) + \sum_i \lambda_i \nabla g_i(x) = 0$$

$$\lambda_i s_i = \mu \quad (\text{扰动互补})$$

$$s_i + g_i(x) = 0$$

$$\lambda_i, s_i \geq 0$$

当 $\mu \rightarrow 0$ 时, 逼近精确 KKT 条件。

原始-对偶内点法

初始化: 严格可行点 $x^{(0)}$, $\lambda^{(0)} > 0$, $s^{(0)} > 0$, $\mu_0 > 0$

For $k = 0, 1, 2, \dots$:

1. 求解牛顿方程组得到方向 $(\Delta x, \Delta \lambda, \Delta s)$

2. 线搜索确定步长 α , 保持 $\lambda, s > 0$

3. 更新: $(x^{(k+1)}, \lambda^{(k+1)}, s^{(k+1)}) = (x^{(k)}, \lambda^{(k)}, s^{(k)}) + \alpha(\Delta x, \Delta \lambda, \Delta s)$

4. $\mu_{k+1} = \sigma \mu_k$ ($\sigma \in (0, 1)$)

5. 若 duality gap $< \varepsilon$, 停止

6.6.4 线性规划的内点法

例题 6.6.2 (LP 内点法——迭代两次).

$$\min -x_1 - 2x_2, \quad \text{s.t. } x_1 + x_2 \leq 2, \quad x_1, x_2 \geq 0$$

改写: $\min c^T x \text{ s.t. } Ax + s = b, x, s \geq 0$

其中 $c = (-1, -2)^T$, $A = [1, 1]$, $b = 2$ 。

障碍函数: $B = c^T x - \mu(\ln x_1 + \ln x_2 + \ln s_1)$

约束: $x_1 + x_2 + s_1 = 2$

消去 $s_1 = 2 - x_1 - x_2$, 对 x_1, x_2 求极小:

$$\frac{\partial B}{\partial x_1} = -1 - \frac{\mu}{x_1} + \frac{\mu}{s_1} = 0$$

$$\frac{\partial B}{\partial x_2} = -2 - \frac{\mu}{x_2} + \frac{\mu}{s_1} = 0$$

由前两式相减: $1 - \frac{\mu}{x_1} + \frac{\mu}{x_2} = 0$

第 1 次迭代 ($\mu_0 = 0.5$):

设 $x_1 = x_2 = t$, 则 $1 = 0$ 矛盾。尝试数值求解:

设 $x_1 \approx 0.4, x_2 \approx 0.8, s_1 = 0.8$

验证: $-1 - 1.25 + 0.625 = -1.625 \neq 0$, 需迭代调整。

经数值求解 (如牛顿法), $x^{(1)} \approx (0.35, 0.90)$, $s_1^{(1)} \approx 0.75$

第 2 次迭代 ($\mu_1 = 0.1$):

类似求解, $x^{(2)} \approx (0.10, 1.35)$, 更接近边界 $x_1 = 0$ 。

最优解为 $(0, 2)$, 最优值 -4 。

6.6.5 Python 代码实现

对数障碍法

```

1 import numpy as np
2 from scipy.optimize import minimize
3
4 def log_barrier_method(f, g_funcs, x0, mu0=1.0,
5                        sigma=0.1, eps=1e-6, max_iter=50):
6
7     """
8
9     Parameters:
10         f: 目标函数
11         g_funcs: 不等式约束 g_i(x) <= 0 列表
12         x0: 严格可行初始点
13         mu0: 初始障碍参数
14         sigma: 缩减因子
15         eps: 精度
16
17     Returns:
18         x_opt: 最优解

```

```

19     history: 迭代历史
20     """
21     x = np.array(x0, dtype=float)
22     mu = mu0
23     history = []
24
25     for k in range(max_iter):
26         def B(x_val):
27             barrier = 0.0
28             for g in g_funcs:
29                 gv = g(x_val)
30                 if gv >= 0:
31                     return 1e10
32             barrier -= np.log(-gv)
33             return f(x_val) + mu * barrier
34
35         res = minimize(B, x, method='BFGS',
36                       options={'gtol': 1e-8})
37         x = res.x
38
39         history.append((x.copy(), mu, f(x)))
40
41         if mu < eps:
42             break
43
44         mu *= sigma
45
46     return x, history
47
48 # 示例:  $\min x, s.t. x \geq 0, x \leq 1$ 
49 # Rewrite:  $g1(x) = -x \leq 0, g2(x) = x - 1 \leq 0$ 
50 f = lambda x: x[0]
51 g = [lambda x: -x[0], lambda x: x[0] - 1]
52
53 x0 = np.array([0.5])
54 x_opt, hist = log_barrier_method(f, g, x0, mu0=0.1, sigma=0.1)
55 for i, (x, mu, fx) in enumerate(hist):
56     print(f"Iter {i}: x={x[0]:.6f}, mu={mu:.4f}, f={fx:.6f}")

```

使用 CVXOPT 的原始-对偶内点法

```

1 import numpy as np
2
3 def primal_dual_lp(c, A, b, x0, s0, lam0,
4                   mu0=1.0, sigma=0.1, eps=1e-8,
5                   max_iter=100):
6
7     """
8     原始-对偶内点法求解标准形式LP:
9      $\min c^T x, \text{ s.t. } Ax = b, x \geq 0$ 
10
11     Parameters:
12         c: 目标函数系数
13         A, b: 等式约束
14         x0, s0, lam0: 初始点 ( $x > 0, s > 0$ )
15         mu0: 初始中心参数
16         sigma: 缩减因子
17
18     Returns:
19         x_opt, history
20     """
21
22     n = len(c)
23     m = len(b)
24
25     x = np.array(x0, dtype=float)
26     s = np.array(s0, dtype=float)
27     lam = np.array(lam0, dtype=float)
28     mu = mu0
29     history = []
30
31     for k in range(max_iter):
32         # 残差
33         r_b = A @ x - b
34         r_c = A.T @ lam + s - c
35         r_mu = x * s - mu
36
37         gap = np.dot(x, s) / n
38         history.append((x.copy(), gap, mu))
39
40         if gap < eps and np.linalg.norm(r_b) < eps:

```

```

39         break
40
41     # 牛顿方程
42     X_inv = 1.0 / x
43     S = s
44
45     # 简化：解正规方程
46     D = x / s
47     M = A @ np.diag(D) @ A.T
48
49     rhs = -r_b - A @ (D * (r_c - r_mu / x))
50     d_lam = np.linalg.solve(M, rhs)
51     d_s = -r_c - A.T @ d_lam
52     d_x = -(r_mu + x * d_s) / s
53
54     # 最大步长保持正性
55     alpha_x = min(1.0, 0.99 * min(
56         [-xi/dxi for xi, dxi in zip(x, d_x) if dxi < 0],
57         default=1.0))
58     alpha_s = min(1.0, 0.99 * min(
59         [-si/dsi for si, dsi in zip(s, d_s) if dsi < 0],
60         default=1.0))
61
62     x += alpha_x * d_x
63     s += alpha_s * d_s
64     lam += alpha_s * d_lam
65     mu = sigma * gap
66
67     return x, history
68
69 # 示例：min -x1 - 2*x2, s.t. x1 + x2 + x3 = 2, x >= 0
70 c = np.array([-1.0, -2.0, 0.0])
71 A = np.array([[1.0, 1.0, 1.0]])
72 b = np.array([2.0])
73
74 x0 = np.array([0.5, 0.5, 1.0])
75 s0 = np.array([1.0, 1.0, 1.0])
76 lam0 = np.array([0.0])
77

```

```

78 x_opt, hist = primal_dual_lp(c, A, b, x0, s0, lam0)
79 for i, (x, gap, mu) in enumerate(hist[:5]):
80     print(f"Iter {i}: x=({x[0]:.4f},{x[1]:.4f}), "
81           f"gap={gap:.6f}, mu={mu:.6f}")

```

6.7 线性规划对偶与 Fenchel 对偶

6.7.1 线性规划的对偶

标准形式的线性规划：

$$\begin{aligned}
 \min \quad & c^T x \\
 \text{s.t.} \quad & Ax = b \\
 & x \geq 0
 \end{aligned} \tag{6.5}$$

定理 6.7.1 (LP 对偶). 线性规划 (6.5) 的对偶问题为：

$$\begin{aligned}
 \max \quad & b^T y \\
 \text{s.t.} \quad & A^T y \leq c
 \end{aligned} \tag{6.6}$$

推导：

拉格朗日函数： $\mathcal{L}(x, y, s) = c^T x - y^T (Ax - b) - s^T x = (c - A^T y - s)^T x + b^T y$

对偶函数： $q(y, s) = \inf_x \mathcal{L}(x, y, s) = \begin{cases} b^T y & \text{if } c - A^T y - s = 0 \\ -\infty & \text{otherwise} \end{cases}$

对偶问题： $\max_{y, s \geq 0} \{b^T y : A^T y + s = c\}$, 即 $\max_y \{b^T y : A^T y \leq c\}$ 。

例题 6.7.1 (LP 对偶). 原问题：

$$\min 3x_1 + 2x_2, \quad \text{s.t. } x_1 + x_2 = 2, x_1, x_2 \geq 0$$

对偶问题：

$$\max 2y, \quad \text{s.t. } y \leq 3, y \leq 2$$

对偶最优： $y^* = 2, d^* = 4$ 。

原问题最优： $x^* = (0, 2), p^* = 4 = d^*$ 。

6.7.2 LP 对偶的基本定理

定理 6.7.2 (LP 对偶定理). 对于线性规划原问题和对偶问题：

1. 若原问题和对偶问题都有可行解，则都有最优解，且 $p^* = d^*$

2. 若一方无界, 则另一方不可行
3. 若一方不可行, 另一方可能无界或不可行

互补松弛条件 (LP)

对于 LP, KKT 互补松弛条件简化为:

$$x_i \cdot (c_i - a_i^T y) = 0, \quad i = 1, \dots, n$$

即: 若 $x_i > 0$, 则对应的对偶约束取等号 ($a_i^T y = c_i$); 若 $a_i^T y < c_i$, 则 $x_i = 0$ 。

6.7.3 共轭函数

定义 6.7.1 (Fenchel 共轭函数). 函数 $f: \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ 的共轭函数定义为:

$$f^*(y) = \sup_{x \in \mathbb{R}^n} \{y^T x - f(x)\}$$

定理 6.7.3 (共轭函数的性质). 1. f^* 是凸函数 (闭的下半连续凸包络)

2. 若 f 是闭的真凸函数, 则 $f^{**} = f$
3. (Fenchel 不等式) $f(x) + f^*(y) \geq x^T y$
4. 等号成立 $\Leftrightarrow y \in \partial f(x)$

例题 6.7.2 (常见共轭函数). 1. $f(x) = \frac{1}{2}x^2: f^*(y) = \frac{1}{2}y^2$

$$2. f(x) = |x|: f^*(y) = \begin{cases} 0 & |y| \leq 1 \\ +\infty & |y| > 1 \end{cases}$$

$$3. f(x) = \begin{cases} 0 & x \in C \\ +\infty & x \notin C \end{cases} \quad (\text{示性函数}): f^*(y) = \sup_{x \in C} y^T x \quad (\text{支撑函数})$$

$$4. f(x) = \frac{1}{2}x^T Q x \quad (Q \succ 0): f^*(y) = \frac{1}{2}y^T Q^{-1}y$$

6.7.4 Fenchel 对偶

定理 6.7.4 (Fenchel 对偶). 考虑问题:

$$\min f(x) + g(Ax)$$

其 Fenchel 对偶为:

$$\max -f^*(A^T y) - g^*(-y)$$

若 f 和 g 为闭真凸函数, 且存在相对内点条件, 则强对偶成立。

例题 6.7.3 (Fenchel 对偶). $\min \frac{1}{2}\|x\|^2 + \|Ax - b\|_1$

令 $f(x) = \frac{1}{2}\|x\|^2$, $g(z) = \|z - b\|_1$

$$f^*(y) = \frac{1}{2}\|y\|^2, \quad g^*(w) = \begin{cases} b^T w & \|w\|_\infty \leq 1 \\ +\infty & \text{otherwise} \end{cases}$$

Fenchel 对偶:

$$\max_{\|y\|_\infty \leq 1} \left\{ -\frac{1}{2}\|A^T y\|^2 + b^T y \right\}$$

6.7.5 正则化问题的对偶

例题 6.7.4 (L2 正则化的对偶). 原问题 (岭回归):

$$\min \frac{1}{2}\|Ax - b\|^2 + \frac{\lambda}{2}\|x\|^2$$

令 $z = Ax - b$, 则问题变为:

$$\min \frac{1}{2}\|z\|^2 + \frac{\lambda}{2}\|x\|^2 \quad s.t. \quad z = Ax - b$$

拉格朗日函数: $\mathcal{L} = \frac{1}{2}\|z\|^2 + \frac{\lambda}{2}\|x\|^2 + y^T(z - Ax + b)$

对 z 求极小: $z + y = 0 \Rightarrow z = -y$

对 x 求极小: $\lambda x - A^T y = 0 \Rightarrow x = \frac{1}{\lambda} A^T y$

代入得对偶:

$$\max_y \left\{ b^T y - \frac{1}{2}\|y\|^2 - \frac{1}{2\lambda}\|A^T y\|^2 \right\}$$

例题 6.7.5 (L1 正则化 (LASSO) 的对偶). 原问题:

$$\min \frac{1}{2}\|Ax - b\|^2 + \lambda\|x\|_1$$

令 $f(x) = \frac{1}{2}\|Ax - b\|^2$, $g(x) = \lambda\|x\|_1$

$$g^*(y) = \begin{cases} 0 & \|y\|_\infty \leq \lambda \\ +\infty & \text{otherwise} \end{cases}$$

Fenchel 对偶:

$$\max_y \{ -f^*(A^T y) : \|y\|_\infty \leq \lambda \}$$

其中 $f^*(y) = \frac{1}{2}\|y + b\|^2 - \frac{1}{2}\|b\|^2$ 。

等价于:

$$\min_y \frac{1}{2}\|A^T y - b\|^2 \quad s.t. \quad \|y\|_\infty \leq \lambda$$

6.8 本章小结

6.8.1 方法体系总览

表 6.1: 有约束最优化理论方法总结			
类别	内容	关键方法	适用场景
最优性条件	KKT 一阶条件	平稳性 + 原始/对偶可行性 + 互补松弛	一般 NLP
	二阶条件	临界锥 + Hessian 判定	严格局部最优
约束规范	LICQ, MFCQ	积极约束梯度线性无关	KKT 必要性保证
	Slater 条件	凸问题严格可行性	凸优化强对偶
对偶理论	拉格朗日对偶	弱对偶 + 强对偶定理	问题转换、下界估计
	LP 对偶	原始-对偶关系	线性规划
	Fenchel 对偶	共轭函数	复合凸优化
惩罚函数法	二次惩罚	大参数趋近精确解	简单约束
	精确惩罚	有限参数精确解	一般约束
	增广拉格朗日	乘子更新	等式约束
障碍/内点法	对数障碍	中心路径	不等式约束
	原始-对偶	牛顿方向 + 步长控制	LP, QP, SDP

6.8.2 KKT 条件速查

标准形式 NLP 的 KKT 条件
问题: $\min f(x)$ s.t. $g_i(x) \leq 0, h_j(x) = 0$
平稳性: $\nabla f(x) + \sum_i \lambda_i \nabla g_i(x) + \sum_j \mu_j \nabla h_j(x) = 0$
原始可行: $g_i(x) \leq 0, h_j(x) = 0$
对偶可行: $\lambda_i \geq 0$
互补松弛: $\lambda_i g_i(x) = 0$

6.8.3 数值方法选择指南

约束优化方法选择

1. 等式约束:

- 少量约束: 拉格朗日乘子法 + 牛顿法
- 大量约束: 增广拉格朗日法

2. 不等式约束:

- 凸问题: 内点法 (推荐)
- 非凸问题: SQP (序列二次规划) 或积极集法

3. 线性和二次规划:

- LP: 单纯形法或内点法
- QP: 积极集法或内点法

4. 大规模问题:

- 一阶方法: ADMM, 投影梯度法
- 二阶方法: 截断牛顿 + 内点

6.8.4 关键定理回顾

1. **Farkas 引理**: 两个互斥系统的择一性, KKT 理论的基石
2. **KKT 必要条件**: 在 CQ 下, 局部最优解必满足 KKT 条件
3. **KKT 充分条件**: 凸优化中, KKT 条件等价于全局最优
4. **弱对偶**: $d^* \leq p^*$ 恒成立
5. **强对偶**: 凸 + Slater $\Rightarrow d^* = p^*$
6. **鞍点定理**: 鞍点 $\Leftrightarrow$ 无对偶间隙的最优解对

6.8.5 收敛性比较

表 6.2: 数值方法收敛特性

方法	收敛条件	收敛速度	参数策略
二次惩罚	$\rho \rightarrow \infty$	线性（慢）	递增
精确惩罚	$\rho \geq \bar{\rho}$ （有限）	有限步精确	需估计
增广拉格朗日	ρ 固定或有限	超线性	乘子更新
对数障碍	$\mu \rightarrow 0$	线性（慢）	递减
原始-对偶内点	综合策略	超线性	自适应

学习建议

1. 首先掌握 KKT 条件的推导和验证

2. 理解约束规范的必要性

3. 掌握对偶理论的弱对偶和强对偶

4. 了解惩罚函数法和障碍函数法的基本思想

5. 熟悉内点法的中心路径概念

6. 能够求解简单的对偶问题

Chapter 7

约束最优化方法

7.1 非线性最小二乘问题

7.1.1 问题描述

定义 7.1.1 (非线性最小二乘问题). 给定训练集 $S = \{(x_i, y_i)\}_{i=1}^m$, 需要估计近似函数 $f(x, \theta)$ 来模拟数据变化趋势。定义残量:

$$r_i(\theta) = f(x_i, \theta) - y_i, \quad i = 1, 2, \dots, m$$

非线性最小二乘问题为:

$$\min_{\theta \in \mathbb{R}^n} F(\theta) = \frac{1}{2} \sum_{i=1}^m r_i(\theta)^2 = \frac{1}{2} \|r(\theta)\|^2$$

其中 $r: \mathbb{R}^n \rightarrow \mathbb{R}^m$ 是残量向量函数。若 r 为线性函数, 则退化为线性最小二乘。

7.1.2 梯度和 Hessian 矩阵

设 $J(\theta)$ 是 $r(\theta)$ 的 Jacobi 矩阵:

$$J(\theta) = \begin{pmatrix} \frac{\partial r_1}{\partial \theta_1} & \cdots & \frac{\partial r_1}{\partial \theta_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial r_m}{\partial \theta_1} & \cdots & \frac{\partial r_m}{\partial \theta_n} \end{pmatrix} \in \mathbb{R}^{m \times n}$$

性质 7.1.1 (梯度和 Hessian).

$$\nabla F(\theta) = J(\theta)^T r(\theta) = \sum_{i=1}^m r_i(\theta) \nabla r_i(\theta)$$

$$\nabla^2 F(\theta) = J(\theta)^T J(\theta) + S(\theta)$$

其中 $S(\theta) = \sum_{i=1}^m r_i(\theta) \nabla^2 r_i(\theta)$ 包含残量的二阶信息。

7.1.3 高斯-牛顿法 (Gauss-Newton)

高斯-牛顿法原理

在 Hessian 中忽略 $S(\theta)$ 项（假设残量较小或接近线性），得到近似 Hessian $J^T J$ 。Newton 迭代变为：

$$\theta_{k+1} = \theta_k - (J_k^T J_k)^{-1} J_k^T r_k$$

高斯-牛顿法算法

Step 1: 解线性方程组 $J_k^T J_k d_k = -J_k^T r_k$ 得到方向 d_k

Step 2: 更新 $\theta_{k+1} = \theta_k + d_k$ （或加入线搜索）

高斯-牛顿法的特点

- 只需残量的一阶导数信息（Jacobi 矩阵）
- $J^T J$ 至少半正定，保证下降方向
- 收敛速度取决于 $S(\theta)$ 的重要性：
 - $S(\theta^*) = 0$ ：二阶收敛
 - $S(\theta)$ 较小：Q-线性收敛
 - $S(\theta)$ 较大：可能不收敛
- 若 J 不满秩，方法失效
- 带线搜索的版本称为**阻尼高斯-牛顿法**

7.1.4 Levenberg-Marquardt 方法**LM 方法原理**

结合高斯-牛顿法和梯度下降法，引入信赖域思想：

$$\min_d \frac{1}{2} \|J_k d + r_k\|^2 \quad \text{s.t.} \quad \|d\| \leq \Delta_k$$

等价于求解：

$$(J_k^T J_k + \lambda_k I) d = -J_k^T r_k$$

其中 $\lambda_k > 0$ 为正则化参数。

LM 方法的特点

- λ_k 大时：接近梯度下降法（最速下降）
- λ_k 小时：接近高斯-牛顿法
- $J^T J + \lambda I$ 正定，保证下降方向

- 是最常用的非线性最小二乘算法
- 通过调整 λ 自适应地切换行为

例题 7.1.1 (高斯-牛顿法迭代两次). **问题**: 用高斯-牛顿法拟合 $f(x, \theta) = \theta_1 e^{\theta_2 x}$, 数据点 $(0, 2), (1, 3)$ 。初始 $\theta_0 = (1, 0)^T$ 。

解: 残量 $r_1 = \theta_1 - 2, r_2 = \theta_1 e^{\theta_2} - 3$

Jacobi 矩阵:

$$J = \begin{pmatrix} 1 & 0 \\ e^{\theta_2} & \theta_1 e^{\theta_2} \end{pmatrix}$$

第 0 轮: $\theta_0 = (1, 0)^T$

$$r_0 = \begin{pmatrix} -1 \\ -2 \end{pmatrix}, \quad J_0 = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

$$J_0^T J_0 = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}, \quad J_0^T r_0 = \begin{pmatrix} -3 \\ -2 \end{pmatrix}$$

解 $J_0^T J_0 d_0 = -J_0^T r_0$ 得 $d_0 = (1, 1)^T$

$$\theta_1 = \theta_0 + d_0 = (2, 1)^T$$

第 1 轮: $\theta_1 = (2, 1)^T$

$$r_1 = \begin{pmatrix} 0 \\ 2e - 3 \end{pmatrix} \approx \begin{pmatrix} 0 \\ 2.437 \end{pmatrix}, \quad J_1 = \begin{pmatrix} 1 & 0 \\ e & 2e \end{pmatrix} \approx \begin{pmatrix} 1 & 0 \\ 2.718 & 5.437 \end{pmatrix}$$

解 $J_1^T J_1 d_1 = -J_1^T r_1$ 得新的方向 d_1 , 更新到 θ_2 。

```

1 import numpy as np
2 from scipy.optimize import least_squares
3
4 # === 非线性最小二乘: 指数拟合 ===
5 def model(theta, x):
6     """f(x) = theta1 * exp(theta2 * x)"""
7     return theta[0] * np.exp(theta[1] * x)
8
9 def residuals(theta, x, y):
10     """残量"""
11     return model(theta, x) - y
12
13 # 数据
14 x_data = np.array([0.0, 1.0, 2.0, 3.0])

```

```

15 y_data = np.array([2.0, 3.2, 5.1, 8.3])
16
17 # 使用 scipy 的 LM 方法
18 result = least_squares(residuals, x0=[1.0, 0.5],
19                        args=(x_data, y_data), method='lm')
20 print(f"最优参数: theta* = {result.x}")
21 print(f"残量范数: ||r|| = {np.linalg.norm(result.fun):.6f}")
22 print(f"迭代次数: {result.nfev}")
23
24 # 手动实现高斯-牛顿法
25 def gauss_newton(residuals, jac, theta0, x, y, max_iter=50, tol=1e-6)
26 :
27     theta = theta0.copy()
28     for k in range(max_iter):
29         r = residuals(theta, x, y)
30         J = jac(theta, x)
31         # 解正规方程
32         d = np.linalg.solve(J.T @ J, -J.T @ r)
33         theta = theta + d
34         if np.linalg.norm(d) < tol:
35             break
36     return theta, k+1
37
38 def jac_exp(theta, x):
39     """Jacobi 矩阵"""
40     n = len(x)
41     J = np.zeros((n, 2))
42     J[:, 0] = np.exp(theta[1] * x)
43     J[:, 1] = theta[0] * x * np.exp(theta[1] * x)
44     return J
45
46 theta_gn, iters = gauss_newton(residuals, jac_exp,
47                                np.array([1.0, 0.5]), x_data, y_data)
48 print(f"\n高斯-牛顿: theta* = {theta_gn}, 迭代{iters}次")

```

Listing 7.1: Levenberg-Marquardt 实现

7.2 最优性条件回顾

7.2.1 无约束问题

定理 7.2.1 (无约束一阶必要条件). 设 f 在 x^* 处可微, 若 x^* 是局部最优解, 则 $\nabla f(x^*) = 0$ 。

定理 7.2.2 (无约束二阶必要条件). 设 f 在 x^* 处二阶可微, 若 x^* 是局部最优解, 则 $\nabla f(x^*) = 0$ 且 $\nabla^2 f(x^*) \succeq 0$ 。

定理 7.2.3 (无约束二阶充分条件). 设 f 在 x^* 处二阶可微, 若 $\nabla f(x^*) = 0$ 且 $\nabla^2 f(x^*) \succ 0$, 则 x^* 是严格局部最优解。

7.2.2 有约束问题——基本概念

考虑约束优化问题:

$$\min f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, \quad i \in \mathcal{I}; \quad h_j(x) = 0, \quad j \in \mathcal{E}$$

定义 7.2.1 (可行域). $\mathcal{X} = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0, \quad h_j(x) = 0, \quad \forall i, j\}$

定义 7.2.2 (起作用约束 (Active Constraint)). 在可行点 x 处, 满足 $g_i(x) = 0$ 的不等式约束称为起作用约束。起作用约束集记为:

$$\mathcal{A}(x) = \{i \in \mathcal{I} \mid g_i(x) = 0\}$$

定义 7.2.3 (可行方向). 方向 d 在点 x 处是可行方向, 若存在 $\bar{\alpha} > 0$ 使得:

$$x + \alpha d \in \mathcal{X}, \quad \forall \alpha \in [0, \bar{\alpha}]$$

定义 7.2.4 (可行下降方向). 方向 d 在点 x 处是可行下降方向, 若:

- (1) d 是可行方向
- (2) $\nabla f(x)^T d < 0$ (下降方向)

7.2.3 等式约束问题

定理 7.2.4 (等式约束的一阶条件). 设 x^* 是等式约束问题的局部最优点, 约束梯度线性无关 (LICQ), 则存在 λ^* 使得:

$$\nabla f(x^*) + \sum_{j \in \mathcal{E}} \lambda_j^* \nabla h_j(x^*) = 0$$

即 $\nabla f(x^*)$ 是约束梯度的线性组合。

例题 7.2.1 (验证最优性条件). **问题:** $\min f(x) = x_1 + x_2, \text{ s.t. } x_1^2 + x_2^2 = 2$. 验证 $x^* = (-1, -1)$ 是否满足一阶必要条件。

解:

$$\nabla f = (1, 1)^T, \quad h(x) = x_1^2 + x_2^2 - 2 = 0, \quad \nabla h = (2x_1, 2x_2)^T$$

在 $x^* = (-1, -1)$:

$$\nabla h(x^*) = (-2, -2)^T$$

需要找 λ 使 $\nabla f + \lambda \nabla h = 0$:

$$(1, 1)^T + \lambda(-2, -2)^T = 0 \Rightarrow 1 - 2\lambda = 0 \Rightarrow \lambda = \frac{1}{2}$$

满足一阶必要条件。

7.3 KKT 条件回顾

本章讨论约束优化的数值方法，这些方法通常需要判断当前迭代点是否满足最优性条件。

KKT 条件速查

KKT 条件的详细推导与证明见第 6 章第 6.2 节。

- 平稳性: $\nabla f(x^*) + \sum \mu_i^* \nabla g_i(x^*) + \sum \lambda_j^* \nabla h_j(x^*) = 0$
- 原始可行性: $g_i(x^*) \leq 0, \quad h_j(x^*) = 0$
- 对偶可行性: $\mu_i^* \geq 0$
- 互补松弛: $\mu_i^* g_i(x^*) = 0$

7.4 可行方向法

可行方向法的理论基础是最优性条件（见第 6 章第 6.1.2 节）：在局部最优点处不存在可行下降方向。因此，通过系统地寻找可行下降方向来迭代逼近最优解。

7.4.1 基本思想

可行方向法 (Feasible Direction Method) 在每次迭代中寻找一个可行下降方向，然后沿该方向进行一维搜索。

可行方向法框架

迭代格式: $x_{k+1} = x_k + \alpha_k d_k$

要求方向 d_k 满足:

(1) 可行性: $x_k + \alpha d_k \in \mathcal{X}$ 对充分小的 $\alpha > 0$

(2) 下降性: $\nabla f(x_k)^T d_k < 0$

7.4.2 线性约束情况

考虑问题:

$$\min f(x) \quad \text{s.t.} \quad Ax \leq b, \quad Ex = e$$

定理 7.4.1 (可行方向判定的充要条件). 设 x 是可行解, $A_1 x = b_1$ (起作用约束), $A_2 x < b_2$ (不起作用约束). 则:

(1) d 是可行方向的充要条件: $A_1 d \leq 0, \quad Ed = 0$

(2) 若还满足 $\nabla f(x)^T d < 0$, 则 d 是可行下降方向

求解可行下降方向: 转化为线性规划问题

$$\min_{d,z} z \quad \text{s.t.} \quad \nabla f(x)^T d \leq z, \quad A_1 d \leq 0, \quad Ed = 0, \quad -1 \leq d_j \leq 1$$

判定准则

设最优解为 (d^*, z^*) :

- 若 $z^* < 0$: d^* 是可行下降方向
- 若 $z^* = 0$: x 是 KKT 点 (找不到可行下降方向)

7.4.3 非线性约束情况

对于非线性约束 $g_i(x) \leq 0$, 可行方向条件变为:

$$\nabla g_i(x)^T d \leq 0, \quad \forall i \in \mathcal{A}(x)$$

同样转化为线性规划求解 d 和 z :

$$\min_{d,z} z \quad \text{s.t.} \quad \nabla f(x)^T d \leq z, \quad \nabla g_i(x)^T d \leq 0 \quad (i \in \mathcal{A}), \quad -1 \leq d_j \leq 1$$

7.4.4 计算步骤

可行方向法计算步骤

Step 1: 给定初始可行点 x_0 , 允许误差 $\varepsilon > 0$, $k = 0$

Step 2: 确定起作用约束集 $\mathcal{A}(x_k)$

Step 3: 若 $\mathcal{A} = \emptyset$ (内点):

- 若 $\|\nabla f(x_k)\| < \varepsilon$, 停
- 否则取 $d_k = -\nabla f(x_k)$ (最速下降), 转 Step 6

Step 4: 若 $\mathcal{A} \neq \emptyset$ (边界点), 求解方向 LP 得 (d_k^*, z^*)

Step 5: 若 $|z^*| < \varepsilon$, 停 (x_k 近似 KKT 点); 否则令 $d_k = d_k^*$

Step 6: 沿 d_k 线搜索求 α_k , 更新 $x_{k+1} = x_k + \alpha_k d_k$

Step 7: $k \leftarrow k + 1$, 转 Step 2

例题 7.4.1 (可行方向法——迭代两次). **问题:** $\min f(x) = 2x_1^2 + 2x_2^2 - 2x_1x_2 - 4x_1 - 6x_2$
s.t. $x_1 + x_2 \leq 2$, $x_1 + 5x_2 \leq 5$, $x_1 \geq 0$, $x_2 \geq 0$. 取 $x_0 = (0, 0)^T$, $\varepsilon = 0.001$.

解: $\nabla f = (4x_1 - 2x_2 - 4, 4x_2 - 2x_1 - 6)^T$

第 0 轮: $x_0 = (0, 0)$

- 约束值: $g_1 = 0$, $g_2 = 0$ (前两个约束都起作用)
- $\nabla f(x_0) = (-4, -6)^T$
- 求解方向 LP: $\min z$ *s.t.* $-4d_1 - 6d_2 \leq z$, $d_1 + d_2 \leq 0$, $d_1 + 5d_2 \leq 0$, $-1 \leq d \leq 1$
- 解得 $d_0 = (1, -0.2)^T$, $z = -2.8 < 0$, 是下降方向
- 线搜索求 α_0 : $\min_{0 \leq \alpha \leq \alpha_{\max}} f(x_0 + \alpha d_0)$
- 由约束 $x_1 + 5x_2 \leq 5$: $\alpha + 5(-0.2\alpha) \leq 5$ 恒成立
- 求稳定点得 $\alpha_0 \approx 0.5$, $x_1 = (0.5, -0.1)$

第 1 轮: 从 x_1 继续迭代, 确定新的起作用约束集, 求解新的方向 LP, 直到收敛。

最优解: $x^* \approx (0.5, 0.9)$, $f^* \approx -3.57$ 。

```

1 import numpy as np
2 from scipy.optimize import linprog
3
4 def feasible_direction_method(f, grad_f, A, b, x0,
5                               eps=1e-6, max_iter=100):
6     """
7     可行方向法 (线性约束 Ax <= b)
8     """

```

```

9      x = x0.copy()
10     n = len(x)
11     history = [x.copy()]
12
13     for k in range(max_iter):
14         g = grad_f(x)
15
16         # 确定起作用约束
17         slack = b - A @ x
18         active = np.where(np.abs(slack) < 1e-8)[0]
19
20         if len(active) == 0:
21             # 内点：梯度下降
22             if np.linalg.norm(g) < eps:
23                 break
24             d = -g
25         else:
26             # 边界点：求解方向LP
27             # min z s.t. g^T d <= z, A_active d <= 0, -1 <= d <= 1
28             A_active = A[active] if len(active) > 0 else np.zeros((1,
29                 n))
30
31             # 变量 [d; z]
32             c = np.zeros(n + 1)
33             c[-1] = 1 # 最小化 z
34
35             A_ub = np.zeros((1 + len(active) + 2*n, n + 1))
36             A_ub[0, :n] = g # g^T d <= z 即 g^T d - z <= 0
37             A_ub[0, -1] = -1
38
39             for i, idx in enumerate(active):
40                 A_ub[1+i, :n] = A[idx] # A_i d <= 0
41
42             # -1 <= d <= 1
43             for i in range(n):
44                 A_ub[1+len(active)+i, i] = 1
45                 A_ub[1+len(active)+n+i, i] = -1
46
47             b_ub = np.zeros(1 + len(active) + 2*n)
48             b_ub[1+len(active):1+len(active)+n] = 1

```

```

48         b_ub[1+len(active)+n:] = 1
49
50         res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=[(None, None)
51                 ]*n+[(None, None)])
52
53         if not res.success:
54             break
55
56         d = res.x[:n]
57         z = res.x[-1]
58
59         if abs(z) < eps:
60             break
61
62         # 一维搜索
63         alpha_max = 1.0
64         slack = b - A @ x
65         Ad = A @ d
66         for i in range(len(Ad)):
67             if Ad[i] > 1e-10:
68                 alpha_max = min(alpha_max, slack[i] / Ad[i])
69
70         # 精确线搜索
71         alphas = np.linspace(0, alpha_max, 100)
72         best_alpha = alphas[np.argmin([f(x + a*d) for a in alphas])]
73
74         x = x + best_alpha * d
75         history.append(x.copy())
76
77         return x, f(x), k+1, history
78
79 # === 测试 ===
80 def f(x):
81     return 2*x[0]**2 + 2*x[1]**2 - 2*x[0]*x[1] - 4*x[0] - 6*x[1]
82
83 def grad_f(x):
84     return np.array([4*x[0] - 2*x[1] - 4,
85                     4*x[1] - 2*x[0] - 6])
86

```

```

87 A = np.array([[1, 1], [1, 5], [-1, 0], [0, -1]])
88 b = np.array([2, 5, 0, 0])
89
90 x_opt, f_opt, iters, hist = feasible_direction_method(
91     f, grad_f, A, b, np.array([0.0, 0.0]))
92 print(f"最优解: x* = {x_opt}")
93 print(f"最优值: f* = {f_opt:.6f}")
94 print(f"迭代次数: {iters}")

```

Listing 7.2: 可行方向法实现

7.5 制约函数法（罚函数法与内点法）

罚函数法与内点法的理论基础见第 6 章第 6.5 节。本章侧重算法的具体实现与数值计算。

7.5.1 外点罚函数法

外点罚函数法思想

通过构造惩罚函数 $P(x, \sigma)$ ，将约束问题转化为一系列无约束问题：

$$\min_x P(x, \sigma) = f(x) + \sigma \cdot \text{penalty}(x)$$

其中惩罚项在可行域内为零，在可行域外取很大的正值。

常用惩罚函数：

等式约束： $h_j(x) = 0$ ：

$$P(x, \sigma) = f(x) + \frac{\sigma}{2} \sum_{j \in \mathcal{E}} h_j(x)^2$$

不等式约束： $g_i(x) \leq 0$ ：

$$g_i^+(x) = \max\{0, g_i(x)\}, \quad P(x, \sigma) = f(x) + \frac{\sigma}{2} \sum_{i \in \mathcal{I}} g_i^+(x)^2$$

一般形式：

$$P(x, \sigma) = f(x) + \frac{\sigma}{2} \left[\sum_{j \in \mathcal{E}} h_j(x)^2 + \sum_{i \in \mathcal{I}} (g_i^+(x))^2 \right]$$

外点罚函数法算法

Step 1: 给定初始点 x_0 , 初始罚因子 $\sigma_0 > 0$, 放大系数 $c > 1$, $\varepsilon > 0$, $k = 0$

Step 2: 以 x_k 为初值, 求解 $\min_x P(x, \sigma_k)$, 得 x_{k+1}

Step 3: 若 $\sigma_k \cdot \text{penalty}(x_{k+1}) < \varepsilon$, 停

Step 4: $\sigma_{k+1} = c\sigma_k$, $k \leftarrow k + 1$, 转 Step 2

定理 7.5.1 (外点罚函数法收敛性). 设 $\sigma_k \rightarrow +\infty$, x_k 是 $P(x, \sigma_k)$ 的全局极小点, 则 $\{x_k\}$ 的每个极限点都是原 COP 问题的全局极小点。

外点罚函数法的优缺点

优点: 不要求初始点为可行点

缺点:

- $\sigma \rightarrow \infty$ 时 Hessian 条件数恶化, 数值不稳定
- 中间迭代点不可行
- 需要求解一系列越来越困难的无约束问题

7.5.2 内点法（障碍函数法）**内点法思想**

在可行域**内部**设置**障碍函数** (Barrier Function), 使迭代点始终保持在可行域内部。当点接近边界时, 障碍函数趋向 $+\infty$ 。

障碍问题:

$$\min_x B(x, \mu) = f(x) + \mu \cdot b(x) \quad \text{s.t.} \quad x \in \text{int}(\mathcal{X})$$

其中 $\mu > 0$ 为障碍参数, 随迭代减小至 0。

常用障碍函数:

倒数障碍函数:

$$b(x) = \sum_{i=1}^m \frac{-1}{g_i(x)}$$

对数障碍函数 (更常用):

$$b(x) = - \sum_{i=1}^m \ln(-g_i(x))$$

内点法算法

Step 1: 给定**严格内点** x_0 ($g_i(x_0) < 0$), $\mu_0 > 0$, 缩减因子 $\beta \in (0, 1)$, $\varepsilon > 0$, $k = 0$

Step 2: 以 x_k 为初值, 求解 $\min_x B(x, \mu_k)$, 得 x_{k+1}

Step 3: 若 $\mu_k \cdot m < \varepsilon$ (m 为约束数), 停

Step 4: $\mu_{k+1} = \beta\mu_k$, $k \leftarrow k + 1$, 转 Step 2

定理 7.5.2 (内点法收敛性). 设 $\mu_k \rightarrow 0$, x_k 是障碍问题的解, 则 $\{x_k\}$ 的每个极限点都是原问题的最优解。

例题 7.5.1 (内点法——迭代两次). **问题:** $\min f(x) = x^2$, $s.t. x \geq 2$. 用对数障碍函数, $x_0 = 3$, $\mu_0 = 1$ 。

解: 约束改写为 $g(x) = 2 - x \leq 0$ 。

障碍函数: $B(x, \mu) = x^2 - \mu \ln(x - 2)$

求导: $\frac{\partial B}{\partial x} = 2x - \frac{\mu}{x-2} = 0 \Rightarrow 2x(x-2) = \mu$

第 0 轮: $\mu_0 = 1$, 解 $2x^2 - 4x - 1 = 0$:

$$x_1 = \frac{4 + \sqrt{16 + 8}}{4} = \frac{4 + 2\sqrt{6}}{4} \approx 2.2247$$

第 1 轮: $\mu_1 = 0.5$, 解 $2x^2 - 4x - 0.5 = 0$:

$$x_2 = \frac{4 + \sqrt{16 + 4}}{4} = \frac{4 + 2\sqrt{5}}{4} \approx 2.1180$$

随着 $\mu \rightarrow 0$, $x_k \rightarrow 2 = x^*$ 。

```

1 import numpy as np
2 from scipy.optimize import minimize
3
4 def penalty_method(f, h_eq, g_ineq, x0, sigma0=1.0, c=10.0,
5                   eps=1e-6, max_iter=20):
6     """外点罚函数法"""
7     x = x0.copy()
8     sigma = sigma0
9     history = []
10
11     for k in range(max_iter):
12         # 构造罚函数
13         def P(x):
14             penalty = 0
15             if h_eq is not None:
16                 for h in h_eq:
17                     penalty += h(x)**2
18             if g_ineq is not None:
19                 for g in g_ineq:
20                     penalty += max(0, g(x))**2
21             return f(x) + 0.5 * sigma * penalty

```

```

22
23     res = minimize(P, x, method='BFGS')
24     x = res.x
25     history.append(x.copy())
26
27     # 计算约束违反度
28     violation = 0
29     if h_eq is not None:
30         for h in h_eq:
31             violation += abs(h(x))
32     if g_ineq is not None:
33         for g in g_ineq:
34             violation += max(0, g(x))
35
36     if violation < eps:
37         break
38     sigma *= c
39
40     return x, f(x), k+1, history
41
42
43 def interior_point_method(f, g_ineq, x0, mu0=1.0, beta=0.1,
44                           eps=1e-6, max_iter=50):
45     """对数障碍函数内点法（简化版）"""
46     x = x0.copy()
47     mu = mu0
48     history = [x.copy()]
49
50     for k in range(max_iter):
51         # 障碍函数
52         def B(x):
53             barrier = 0
54             for g in g_ineq:
55                 val = g(x)
56                 if val >= 0:
57                     return np.inf
58             barrier -= np.log(-val)
59             return f(x) + mu * barrier
60
61     res = minimize(B, x, method='BFGS')

```

```

62     x = res.x
63     history.append(x.copy())
64
65     if mu < eps:
66         break
67     mu *= beta
68
69     return x, f(x), k+1, history
70
71
72 # === 测试 ===
73 print("=" * 50)
74 print("罚函数法")
75 print("=" * 50)
76 x_opt, f_opt, iters, hist = penalty_method(
77     lambda x: x[0]**2 + x[1]**2,
78     [lambda x: x[0] + x[1] - 1], #  $h(x) = 0$ 
79     None,
80     np.array([0.5, 0.5])
81 )
82 print(f"最优解: x* = {x_opt}")
83 print(f"最优值: f* = {f_opt:.6f}")
84
85 print("\n" + "=" * 50)
86 print("内点法")
87 print("=" * 50)
88 x_opt2, f_opt2, iters2, hist2 = interior_point_method(
89     lambda x: x[0]**2,
90     [lambda x: 2 - x[0]], #  $g(x) \leq 0$  means  $x \geq 2$ 
91     np.array([3.0])
92 )
93 print(f"最优解: x* = {x_opt2}")
94 print(f"最优值: f* = {f_opt2:.6f}")
95 print(f"迭代次数: {iters2}")

```

Listing 7.3: 罚函数法与内点法实现

7.6 增广拉格朗日函数法与乘子法

7.6.1 罚函数的困难

外点罚函数法要求 $\sigma \rightarrow \infty$ 才能收敛到可行解，这导致：

- Hessian 矩阵条件数恶化
- 子问题求解越来越困难
- 数值不稳定

关键观察

在罚函数最优解 $x^*(\sigma)$ 处：

$$\nabla P(x^*, \sigma) = \nabla f(x^*) + \sigma \sum_i g_i^+(x^*) \nabla g_i(x^*) = 0$$

对比 KKT 条件： $\nabla f(x^*) + \sum_i \mu_i^* \nabla g_i(x^*) = 0$

要使两者一致，需要 $\sigma g_i^+(x^*) \approx \mu_i^*$ ，即约束违反度 $g_i^+(x^*) \approx \mu_i^*/\sigma$ 。当 $\sigma \rightarrow \infty$ 时违反度趋于 0，但中间过程约束不被精确满足。

7.6.2 增广拉格朗日函数法

增广拉格朗日函数

将罚函数与拉格朗日函数结合，对等式约束问题 $h_j(x) = 0$ ：

$$\mathcal{L}_A(x, \lambda, \sigma) = f(x) + \sum_j \lambda_j h_j(x) + \frac{\sigma}{2} \sum_j h_j(x)^2$$

其中 λ 是拉格朗日乘子估计。

增广拉格朗日法（乘子法）

Step 1: 给定 $x_0, \lambda_0, \sigma_0 > 0, k = 0$

Step 2: 求解 $\min_x \mathcal{L}_A(x, \lambda_k, \sigma_k)$ 得 x_{k+1}

Step 3: 更新乘子：

$$\lambda_{k+1} = \lambda_k + \sigma_k h(x_{k+1})$$

Step 4: 若 $\|h(x_{k+1})\| < \varepsilon$ ，停

Step 5: 可适当增大 σ_{k+1} ， $k \leftarrow k + 1$ ，转 Step 2

增广拉格朗日法的优势

- 若 λ 接近最优乘子 λ^* ，有限 σ 即可使约束精确满足
- 避免了 $\sigma \rightarrow \infty$ 导致的数值困难

- 乘子更新使得约束违反度与 μ 成正比，接近最优时自动减小
- 收敛速度比纯罚函数法快得多

7.6.3 不等式约束的处理

引入松弛变量 $s_i \geq 0$ 将不等式转为等式： $g_i(x) + s_i = 0$ 。

增广拉格朗日函数变为：

$$\mathcal{L}_A(x, s, \mu, \sigma) = f(x) + \sum_i \mu_i (g_i(x) + s_i) + \frac{\sigma}{2} \sum_i (g_i(x) + s_i)^2$$

例题 7.6.1 (增广拉格朗日法——迭代两次). **问题：** $\min f(x) = x^2, \text{ s.t. } x = 1$ 。

增广拉格朗日函数： $\mathcal{L}_A = x^2 + \lambda(x - 1) + \frac{\sigma}{2}(x - 1)^2$

参数： $\lambda_0 = 0, \sigma = 2$

第 0 轮： $\min_x x^2 + (x - 1)^2 = 2x^2 - 2x + 1$

$$\frac{d}{dx} = 4x - 2 = 0 \Rightarrow x_1 = 0.5$$

更新： $\lambda_1 = \lambda_0 + \sigma(x_1 - 1) = 0 + 2(-0.5) = -1$

第 1 轮： $\mathcal{L}_A = x^2 - (x - 1) + (x - 1)^2 = x^2 - x + 1 + x^2 - 2x + 1 = 2x^2 - 3x + 2$

$$\frac{d}{dx} = 4x - 3 = 0 \Rightarrow x_2 = 0.75$$

更新： $\lambda_2 = -1 + 2(-0.25) = -1.5$

第 2 轮： $\mathcal{L}_A = x^2 - 1.5(x - 1) + (x - 1)^2 = 2x^2 - 3.5x + 2.5$

$$x_3 = \frac{3.5}{4} = 0.875$$

可见 $x_k \rightarrow 1, \lambda_k \rightarrow -2 = \lambda^*$ 。

7.7 线性规划的内点法

7.7.1 解析中心

定义 7.7.1 (解析中心 (Analytic Center)). 设集合 S 由不等式定义： $S = \{x \in \mathbb{R}^n \mid a_i^T x \geq b_i, i = 1, \dots, m\}$ ，内点集非空。定义**势函数**：

$$\psi(x) = - \sum_{i=1}^m \ln(a_i^T x - b_i)$$

集合 S 的**解析中心**定义为：

$$x_{AC} = \arg \min_{x \in \text{int}(S)} \psi(x)$$

性质 7.7.1 (解析中心性质). • 解析中心是唯一的 (因为 ψ 严格凸)

- 解析中心与不等式的表示方式有关 (冗余约束会改变中心)
- 势函数是松弛变量对数和的负数, 等价于松弛变量乘积的倒数的对数

例题 7.7.1 (单位立方体的解析中心). $S = \{x \in \mathbb{R}^n \mid x_i \geq 0, 1 - x_i \geq 0, i = 1, \dots, n\}$

势函数: $\psi(x) = -\sum_{i=1}^n [\ln(x_i) + \ln(1 - x_i)]$

求导: $\frac{\partial \psi}{\partial x_i} = -\frac{1}{x_i} + \frac{1}{1-x_i} = 0 \Rightarrow x_i = \frac{1}{2}$

解析中心为立方体中心 $x_{AC} = (\frac{1}{2}, \dots, \frac{1}{2})^T$ 。

7.7.2 中心路径

考虑 LP 问题: $\min c^T x$ s.t. $Ax = b, x \geq 0$ 。

定义 7.7.2 (中心路径). 对 $\mu > 0$, 定义障碍问题:

$$\min c^T x - \mu \sum_{j=1}^n \ln(x_j) \quad \text{s.t.} \quad Ax = b, x > 0$$

当 μ 从 ∞ 变化到 0 时, 障碍问题最优解的轨迹形成**中心路径** (Central Path), 连接解析中心 ($\mu \rightarrow \infty$) 和 LP 最优面 ($\mu \rightarrow 0$)。

7.7.3 KKT 条件与原始-对偶系统

引入拉格朗日乘子 y (对偶变量), 障碍问题的 KKT 条件:

$$c - \mu X^{-1}e - A^T y = 0$$

$$Ax - b = 0$$

令 $s = \mu X^{-1}e$ (即 $Xs = \mu e$), 条件变为:

$$\begin{cases} Xs = \mu e \\ Ax = b \\ A^T y + s = c \end{cases}$$

定义 7.7.3 (原始-对偶内点法). 求解一系列 $\mu_k \rightarrow 0$ 的扰动 KKT 系统, 用 Newton 法求解每个子问题。

7.7.4 Newton 步的求解

对当前点 (x, y, s) (满足 $x > 0, s > 0$), 求 Newton 步 $(\Delta x, \Delta y, \Delta s)$:

$$\begin{pmatrix} S & 0 & X \\ A & 0 & 0 \\ 0 & A^T & I \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \\ \Delta s \end{pmatrix} = \begin{pmatrix} \mu e - Xs \\ b - Ax \\ c - A^T y - s \end{pmatrix}$$

简化计算：

$$AS^{-1}XA^T\Delta y = b - Ax + AS^{-1}(X(c - A^Ty - s) - \mu e + Xs)$$

预测-校正方法 (Predictor-Corrector)

Mehrotra 预测-校正是最常用的原始-对偶内点法：

1. **预测步**（仿射步）：求解 $\mu = 0$ 的 Newton 系统，预测中心路径方向
2. **计算步长**：确定最大可行步长 α_P, α_D
3. **更新 μ** ：根据互补性减少程度调整
4. **校正步**：加入中心性校正，求解新的 Newton 系统

```

1 import numpy as np
2
3 def interior_point_lp(c, A, b, x0, max_iter=50, tol=1e-8):
4     """
5     原始-对偶内点法求解 LP: min c^T x s.t. Ax=b, x>=0
6     """
7     m, n = A.shape
8     x = x0.copy()
9     y = np.zeros(m)
10    s = np.ones(n)
11
12    history = []
13
14    for k in range(max_iter):
15        # 计算互补间隙
16        mu = (x @ s) / n
17        history.append((x.copy(), mu))
18
19        if mu < tol:
20            break
21
22        # 构造并求解 Newton 系统
23        X = np.diag(x)
24        S = np.diag(s)
25
26        # 右侧
27        r1 = mu * np.ones(n) - X @ s

```

```

28     r2 = b - A @ x
29     r3 = c - A.T @ y - s
30
31     # 简化系统:  $(A S^{-1} X A^T) dy = rhs$ 
32     SX_inv = np.diag(x / s)
33     M = A @ SX_inv @ A.T
34
35     rhs = r2 + A @ (r1 / s) + A @ (X @ r3 / s)
36
37     try:
38         dy = np.linalg.solve(M, rhs)
39     except np.linalg.LinAlgError:
40         dy = np.linalg.lstsq(M, rhs, rcond=None)[0]
41
42     ds = r3 + A.T @ dy
43     dx = (r1 - X @ ds) / s
44
45     # 线搜索保持正性
46     alpha_p = min(0.99 / max(-dx[x > 0] / x[x > 0], default=0,
47                        key=abs), 1) if np.any(dx < 0) else 0.99
48     alpha_d = min(0.99 / max(-ds[s > 0] / s[s > 0], default=0,
49                        key=abs), 1) if np.any(ds < 0) else 0.99
50
51     x += alpha_p * dx
52     y += alpha_d * dy
53     s += alpha_d * ds
54
55     return x, c @ x, k+1, history
56
57 # === 测试 ===
58 print("=" * 50)
59 print("原始-对偶内点法求解LP")
60 print("=" * 50)
61
62 # min -x1 - 2*x2 s.t. x1 + x2 <= 2, x1 >= 0, x2 >= 0
63 # 标准型: min c^T x s.t. Ax = b, x >= 0
64 # 引入松弛: x1 + x2 + x3 = 2
65 c = np.array([-1.0, -2.0, 0.0])
66 A = np.array([[1.0, 1.0, 1.0]])

```

```

66 b = np.array([2.0])
67
68 x_opt, obj, iters, hist = interior_point_lp(c, A, b, np.array([0.5,
    0.5, 1.0]))
69 print(f"最优解: x* = {x_opt[:2]} (前两个变量)")
70 print(f"最优值: c^T x = {obj:.6f}")
71 print(f"迭代次数: {iters}")
72 print(f"最终互补间隙: {hist[-1][1]:.8f}")

```

Listing 7.4: 原始-对偶内点法（简化版）

7.8 ADMM 与邻近算法简介

7.8.1 ADMM 方法

定义 7.8.1 (交替方向乘子法 (ADMM)). 考虑结构化优化问题:

$$\min f(x) + g(z) \quad s.t. \quad Ax + Bz = c$$

ADMM 迭代格式:

$$\begin{aligned} x_{k+1} &= \arg \min_x \mathcal{L}_\rho(x, z_k, y_k) \\ z_{k+1} &= \arg \min_z \mathcal{L}_\rho(x_{k+1}, z, y_k) \\ y_{k+1} &= y_k + \rho(Ax_{k+1} + Bz_{k+1} - c) \end{aligned}$$

其中增广拉格朗日函数:

$$\mathcal{L}_\rho(x, z, y) = f(x) + g(z) + y^T(Ax + Bz - c) + \frac{\rho}{2}\|Ax + Bz - c\|^2$$

ADMM 的特点

- 将复杂问题分解为两个（或多个）子问题分别求解
- x 更新和 z 更新可以独立进行，适合并行计算
- 广泛用于统计学习、图像处理、分布式优化
- 在凸问题上有收敛保证

例题 7.8.1 (LASSO 的 ADMM 求解). **问题:** $\min \frac{1}{2}\|Ax - b\|^2 + \lambda\|z\|_1 \quad s.t. \quad x - z = 0$

ADMM 迭代:

$$\begin{aligned} x_{k+1} &= (A^T A + \rho I)^{-1}(A^T b + \rho(z_k - u_k)) \\ z_{k+1} &= S_{\lambda/\rho}(x_{k+1} + u_k) \quad (\text{软阈值}) \\ u_{k+1} &= u_k + x_{k+1} - z_{k+1} \end{aligned}$$

其中 $S_\kappa(v) = \text{sign}(v) \max\{|v| - \kappa, 0\}$ 是软阈值算子。

7.8.2 邻近算子与邻近算法

定义 7.8.2 (邻近算子 (Proximity Operator)). 函数 f 的邻近算子定义为:

$$\text{prox}_f(x) = \arg \min_y \left\{ f(y) + \frac{1}{2} \|y - x\|^2 \right\}$$

性质 7.8.1 (邻近算子性质). (1) 若 f 为闭凸函数, 则 $\text{prox}_f(x)$ 对任意 x 有唯一解

(2) $\text{prox}_f(x) = x$ 当且仅当 $x \in \arg \min f$

(3) Moreau 分解: $x = \text{prox}_f(x) + \text{prox}_{f^*}(x)$

例题 7.8.2 (常见邻近算子). • $f(x) = I_C(x)$ (指示函数): $\text{prox}_f = \Pi_C$ (投影到 C)

• $f(x) = \lambda \|x\|_1$: $\text{prox}_f(x) = S_\lambda(x)$ (软阈值)

• $f(x) = \frac{\lambda}{2} \|x\|^2$: $\text{prox}_f(x) = \frac{x}{1+\lambda}$

邻近梯度法 (Proximal Gradient Method)

用于求解 $F(x) = f(x) + g(x)$, 其中 f 光滑、 g 可能非光滑:

$$x_{k+1} = \text{prox}_{\alpha_k g}(x_k - \alpha_k \nabla f(x_k))$$

即先沿梯度方向走一步, 再做邻近投影。

```

1 import numpy as np
2
3 def soft_threshold(x, kappa):
4     """ 软阈值算子 """
5     return np.sign(x) * np.maximum(np.abs(x) - kappa, 0)
6
7 def admm_lasso(A, b, lam, rho=1.0, max_iter=100, tol=1e-4):
8     """
9     ADMM 求解 LASSO: min 0.5*||Ax-b||^2 + lam*||x||_1
10    """
11    n = A.shape[1]
12    x = np.zeros(n)
13    z = np.zeros(n)
14    u = np.zeros(n)
15
16    # 预计算 (A^T A + rho I)^{-1}

```

```
17     ATA = A.T @ A
18     L = np.linalg.cholesky(ATA + rho * np.eye(n))
19
20     history = []
21
22     for k in range(max_iter):
23         # x更新
24         q = A.T @ b + rho * (z - u)
25         x = np.linalg.solve(L.T, np.linalg.solve(L, q))
26
27         # z更新 (软阈值)
28         z_old = z.copy()
29         z = soft_threshold(x + u, lam / rho)
30
31         # u更新
32         u = u + x - z
33
34         # 检查收敛
35         r_norm = np.linalg.norm(x - z)
36         s_norm = np.linalg.norm(-rho * (z - z_old))
37         history.append((r_norm, s_norm))
38
39         if r_norm < tol and s_norm < tol:
40             break
41
42     return z, k+1, history
43
44
45 # === 测试 ===
46 np.random.seed(42)
47 n = 100
48 m = 50
49 A = np.random.randn(m, n)
50 x_true = np.zeros(n)
51 x_true[:5] = np.array([1, -2, 3, -4, 5])
52 b = A @ x_true + 0.1 * np.random.randn(m)
53
54 print("=" * 50)
55 print("ADMM 求解 LASSO")
56 print("=" * 50)
```

```

57 x_opt, iters, hist = admm_lasso(A, b, lam=0.5, rho=1.0)
58 nnz = np.sum(np.abs(x_opt) > 0.01)
59 print(f"非零元素: {nnz}")
60 print(f"迭代次数: {iters}")
61 print(f"残量范数: {hist[-1][0]:.6f}")
62 print(f"前10个分量: {x_opt[:10].round(4)}")

```

Listing 7.5: ADMM 与邻近算法实现

7.9 本章小结

7.9.1 本章知识框架

约束问题的最优化方法——核心内容

1. 非线性最小二乘

- 高斯-牛顿法：忽略二阶项，利用 $J^T J$
- Levenberg-Marquardt 法： $J^T J + \lambda I$ ，最常用

2. 最优性条件

- 无约束： $\nabla f = 0$, $\nabla^2 f \succeq 0$
- 等式约束： $\nabla f + \lambda^T \nabla h = 0$
- 一般约束：KKT 条件（平稳性 + 可行性 + 对偶性 + 互补性）

3. KKT 条件的应用

- 一阶必要条件和二阶充分条件
- 凸规划中 KKT 是充要条件

4. 可行方向法

- 线性约束：求解 LP 找可行下降方向
- 非线性约束：利用梯度近似

5. 制约函数法

- 外点罚函数法： $\sigma \rightarrow \infty$ ，不要求初始可行
- 内点法（障碍函数法）： $\mu \rightarrow 0$ ，要求严格内点

6. 增广拉格朗日函数法（见第 6 章第 6.5 节理论）

- 结合罚函数和拉格朗日函数
- 有限 σ 即可精确满足约束
- 乘子更新保证收敛

7. 线性规划的内点法

- 解析中心、中心路径
- 原始-对偶方法、预测-校正

8. ADMM 与邻近算法

- ADMM: 分解复杂问题，交替求解
- 邻近算子: 软阈值、投影等
- 邻近梯度法: $x_{k+1} = \text{prox}_{\alpha g}(x_k - \alpha \nabla f)$

7.9.2 方法特性对比

表 7.1: 约束优化方法对比				
方法	约束类型	需可行点	收敛性	特点
可行方向法	线性/非线性	是	保证	需解方向 LP
外点罚函数	等式 + 不等式	否	$\sigma \rightarrow \infty$	数值不稳定
内点法	不等式	严格内点	$\mu \rightarrow 0$	路径跟踪
增广拉格朗日	等式 + 不等式	否	有限 σ	最好平衡
ADMM	可分结构	否	凸保证	分布式
邻近梯度	非光滑项	否	次线性到线性	稀疏优化

7.9.3 关键公式汇总

7.9.4 关键术语中英对照

```
1 import numpy as np
2 from scipy.optimize import minimize, least_squares
3 import time
4
5 def solve_qp_comparison():
6     """综合求解二次规划问题，比较各种方法"""
7
```

表 7.2: 核心公式速查

方法/概念	公式
高斯-Newton 步	$(J^T J)d = -J^T r$
LM 步	$(J^T J + \lambda I)d = -J^T r$
KKT 平稳性	$\nabla f + \mu^T \nabla g + \lambda^T \nabla h = 0$
互补松弛	$\mu_i g_i(x) = 0$
外点罚函数	$P = f + \frac{\sigma}{2} \sum h_j^2 + \frac{\sigma}{2} \sum (g_i^+)^2$
对数障碍函数	$B = f - \mu \sum \ln(-g_i)$
增广拉格朗日	$\mathcal{L}_A = f + \lambda^T h + \frac{\sigma}{2} \ h\ ^2$
乘子更新	$\lambda_{k+1} = \lambda_k + \sigma h(x_{k+1})$
中心路径	$Xs = \mu e, Ax = b, A^T y + s = c$
ADMM x 更新	$x^+ = \arg \min_x \mathcal{L}_\rho(x, z, y)$
ADMM z 更新	$z^+ = \arg \min_z \mathcal{L}_\rho(x^+, z, y)$
邻近算子	$\text{prox}_f(x) = \arg \min_y \{f(y) + \frac{1}{2} \ y - x\ ^2\}$
近端梯度	$x^+ = \text{prox}_{\alpha g}(x - \alpha \nabla f)$

中文	英文
非线性最小二乘	Nonlinear Least Squares
Jacobi 矩阵	Jacobian Matrix
高斯-牛顿法	Gauss-Newton Method
Levenberg-Marquardt	Levenberg-Marquardt Method
起作用约束	Active Constraint
可行方向	Feasible Direction
KKT 条件	Karush-Kuhn-Tucker Conditions
互补松弛性	Complementary Slackness
罚函数法	Penalty Function Method
障碍函数法	Barrier Function Method
内点法	Interior Point Method
增广拉格朗日法	Augmented Lagrangian Method
解析中心	Analytic Center
中心路径	Central Path
ADMM	Alternating Direction Method of Multipliers
邻近算子	Proximity/Proximal Operator
近端梯度法	Proximal Gradient Method
软阈值	Soft Thresholding

```

8      # min 0.5*x^T Q x + c^T x s.t. A x = b
9      Q = np.array([[2, 0], [0, 2]])
10     c = np.array([-2, -6])
11     A = np.array([[1, 1]])
12     b = np.array([2.0])
13
14     # 消元法: x2 = 2 - x1
15     # f = x1^2 + (2-x1)^2 - 2*x1 - 6*(2-x1)
16     #     = 2*x1^2 - 4*x1 + 4 - 2*x1 - 12 + 6*x1
17     #     = 2*x1^2 + 0*x1 - 8
18     # df/dx1 = 4*x1 = 0 -> x1 = 0, x2 = 2
19
20     # 使用 scipy
21     def obj(x):
22         return 0.5 * x @ Q @ x + c @ x
23
24     def grad(x):
25         return Q @ x + c
26
27     constraints = {'type': 'eq', 'fun': lambda x: A @ x - b}
28
29     res = minimize(obj, [0, 0], jac=grad, method='SLSQP',
30                   constraints=constraints)
31     print(f"Scipy SLSQP: x* = {res.x}, f* = {res.fun:.6f}")
32
33     # 解析解验证
34     x_analytical = np.array([0.0, 2.0])
35     print(f"解析解: x* = {x_analytical}, f* = {obj(x_analytical):.6f}")
36
37     solve_qp_comparison()

```

Listing 7.6: 综合比较代码

Chapter 8

进阶专题

8.1 复合优化问题

8.1.1 问题形式

定义 8.1.1 (复合优化问题). 一般将目标函数写为多个函数之和的形式, 每个函数可能具有不同的性质:

$$\min_{x \in \mathbb{R}^n} \theta(x) = f(x) + g(x)$$

其中 $f(x)$ 可微 (通常光滑), $g(x)$ 可能不可微 (如正则项、指示函数等)。

典型应用

- 正则化问题: f 为数据拟合项 (如最小二乘), g 为正则项 (如 L1 范数)
- 约束优化: g 为集合的指示函数 $I_{\mathcal{X}}(x)$
- 深度学习: 各种优化器的改进版本 (Adagrad, 动量法, Adam 等) 都适用于此类问题

8.1.2 求解方法

可用方法汇总

1. 梯度类方法: 各种梯度下降法的改进版本 (理论基础见第 5 章第 5.3 节)
 - Adagrad, 动量法, Nesterov 动量法, RMSProp, AdaDelta, Adam
2. 约束最优化方法: 罚函数法、增广拉格朗日法、ADMM、坐标轮换下降法 (详见第 7 章)
3. 邻近点方法 (PPA, Proximal Point Algorithm, 见第 5.7.5 节)
4. 交替方向乘子法 (ADMM) —— 特别适合可分离结构 (见第 7.8 节)

例题 8.1.1 (LASSO 作为复合优化).

$$\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$$

其中 $f(x) = \frac{1}{2} \|Ax - b\|^2$ (可微), $g(x) = \lambda \|x\|_1$ (不可微)。

可用方法:

- 次梯度法
- 近端梯度法 (*ISTA/FISTA*)
- *ADMM*

```

1 import numpy as np
2
3 def prox_l1(x, lam):
4     """L1范数的邻近算子 = 软阈值"""
5     return np.sign(x) * np.maximum(np.abs(x) - lam, 0)
6
7 def ista(A, b, lam, max_iter=100, L=None):
8     """
9     近端梯度法(ISTA)求解 LASSO
10    min 0.5*||Ax-b||^2 + lam*||x||_1
11    """
12    if L is None:
13        L = np.linalg.norm(A.T @ A, 2) # Lipschitz常数
14
15    x = np.zeros(A.shape[1])
16    alpha = 1.0 / L
17
18    history = []
19    for k in range(max_iter):
20        # 梯度步 + 邻近算子
21        grad = A.T @ (A @ x - b)
22        x = prox_l1(x - alpha * grad, alpha * lam)
23
24        obj = 0.5 * np.linalg.norm(A @ x - b)**2 + lam * np.sum(np.
25                           abs(x))
26        history.append(obj)
27
28    return x, history

```

```

29
30 # === 测试 ===
31 np.random.seed(42)
32 n = 100
33 m = 50
34 A = np.random.randn(m, n)
35 x_true = np.zeros(n)
36 x_true[:5] = [1, -2, 3, -4, 5]
37 b = A @ x_true + 0.01 * np.random.randn(m)
38
39 x_opt, hist = ista(A, b, lam=0.1, max_iter=200)
40 print(f"ISTA 求解 LASSO")
41 print(f"非零元素: {np.sum(np.abs(x_opt) > 0.01)}")
42 print(f"最终目标值: {hist[-1]:.4f}")

```

Listing 8.1: 复合优化求解示例

8.2 变分不等式与邻近点算法

8.2.1 单调变分不等式

定义 8.2.1 (变分不等式 (VI)). 设 $\Omega \subset \mathbb{R}^n$ 是非空闭凸集, $F: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 是算子映射。变分不等式问题为: 求 $u^* \in \Omega$ 使得

$$(u - u^*)^T F(u^*) \geq 0, \quad \forall u \in \Omega$$

若 F 满足 $(u - v)^T (F(u) - F(v)) \geq 0$, 则称 F 为单调算子, 上述不等式称为单调变分不等式。

性质 8.2.1 (凸函数与变分不等式). 可微凸函数 f 的梯度算子 ∇f 是单调的。因此凸优化问题

$$\min_{x \in \Omega} f(x)$$

等价于变分不等式: 求 $x^* \in \Omega$ 使得 $(x - x^*)^T \nabla f(x^*) \geq 0, \forall x \in \Omega$ 。

8.2.2 线性约束凸优化的变分不等式

考虑线性约束凸优化问题:

$$\min_{x \in \mathcal{X}} \theta(x) \quad \text{s.t.} \quad Ax = b$$

其拉格朗日函数 $\mathcal{L}(x, \lambda) = \theta(x) - \lambda^T(Ax - b)$ 的鞍点 (x^*, λ^*) 满足:

$$\begin{cases} x^* \in \mathcal{X}, & (x - x^*)^T(\nabla\theta(x^*) - A^T\lambda^*) \geq 0, \quad \forall x \in \mathcal{X} \\ \lambda^* \in \mathbb{R}^m, & (\lambda - \lambda^*)^T(Ax^* - b) \geq 0, \quad \forall \lambda \in \mathbb{R}^m \end{cases}$$

定理 8.2.1 (等价变分不等式). 上述鞍点条件可合并为单调混合变分不等式: 求 $w^* = (x^*, \lambda^*) \in \Omega = \mathcal{X} \times \mathbb{R}^m$ 使得

$$(w - w^*)^T F(w^*) \geq 0, \quad \forall w \in \Omega$$

其中 $w = \begin{pmatrix} x \\ \lambda \end{pmatrix}$, $F(w) = \begin{pmatrix} \nabla\theta(x) - A^T\lambda \\ Ax - b \end{pmatrix}$ 。

8.2.3 可分离结构

对于两个算子的可分离结构:

$$\min_{x \in \mathcal{X}, y \in \mathcal{Y}} \theta_1(x) + \theta_2(y) \quad \text{s.t.} \quad Ax + By = b$$

对应的变分不等式: 求 $(x^*, y^*, \lambda^*) \in \mathcal{X} \times \mathcal{Y} \times \mathbb{R}^m$ 使得

$$(w - w^*)^T F(w^*) \geq 0$$

其中 $w = \begin{pmatrix} x \\ y \\ \lambda \end{pmatrix}$, $F(w) = \begin{pmatrix} \nabla\theta_1(x) - A^T\lambda \\ \nabla\theta_2(y) - B^T\lambda \\ Ax + By - b \end{pmatrix}$ 。

ADMM 的收敛性

两个算子的可分离结构可用 ADMM 有效求解。但三个及以上算子的直接 ADMM 推广不一定收敛, 需要基于变分不等式的预测-校正方法处理。

8.2.4 邻近点算法 (PPA)

定义 8.2.2 (邻近点算法). 对于凸优化问题 $\min_{x \in \Omega} \theta(x)$, 给定 $r > 0$, PPA 迭代格式为:

$$x_{k+1} = \arg \min_{x \in \Omega} \left\{ \theta(x) + \frac{r}{2} \|x - x_k\|^2 \right\}$$

性质 8.2.2 (PPA 性质). (1) PPA 可写成 $x_{k+1} = \text{prox}_{\theta/r}(x_k)$

(2) 对闭凸函数, PPA 生成的序列 $\{x_k\}$ 收敛到最优解

(3) PPA 是求解凸优化和单调变分不等式的基本算法

例题 8.2.1 (PPA 的迭代). **问题:** $\min f(x) = x^2 \text{ s.t. } x \in [1, 3], x_0 = 3, r = 1$ 。

解: PPA 子问题:

$$x_{k+1} = \arg \min_{x \in [1, 3]} \left\{ x^2 + \frac{1}{2}(x - x_k)^2 \right\}$$

对内部无约束极小点: $2x + (x - x_k) = 0 \Rightarrow x = \frac{x_k}{3}$

第 0 轮: $x_0 = 3$, 候选 $x = 1$, 但 $1 \in [1, 3]$, $f(1) + 0.5(1 - 3)^2 = 1 + 2 = 3$ 与边界比较: $x_{k+1} = \max(1, \min(3, x_k/3)) = \max(1, 1) = 1$

第 1 轮: $x_1 = 1$, 候选 $x = 1/3 < 1$, 故 $x_2 = 1$ 。

已收敛到最优解 $x^* = 1$ 。

8.3 启发式算法概述

8.3.1 优化方法分类

优化方法的多维分类

- 全局优化 vs 局部优化
- 单目标 vs 多目标
- 无约束 vs 有约束
- 基于梯度 vs 无梯度

梯度方法的限制

梯度方法（如第 5 章介绍的最速下降法、Newton 法、共轭梯度法等）假设目标函数是单峰（单谷）函数，否则容易陷入局部极小点。对于复杂的多变量组合优化问题（大规模、非线性、非凸），需要**无梯度方法**——启发式方法。

8.3.2 启发式算法 vs 近似算法

定义 8.3.1 (启发式算法). 一种技术，使得在可接受的计算费用内寻找到最好的解，但不一定能保证所得解的可行性和最优性，甚至在多数情况下无法阐述所得解同最优解的近似程度。

定义 8.3.2 (ε -近似算法). 设 A 是一个问题， $OPT(\pi)$ 和 $H(\pi)$ 分别为实例 π 的最优解和启发式算法解。若

$$|H(\pi) - OPT(\pi)| \leq \varepsilon \cdot |OPT(\pi)|, \quad \forall \pi \in A$$

则称该算法为 A 的 ε -近似算法。

性质 8.3.1 (启发式与近似的区别). • 启发式算法集合包含近似算法集合

- 近似算法强调最坏情况误差界，启发式算法不考虑
- 最坏情况误差界估计需要较好的数学技巧，往往只能通过启发式解决

8.3.3 组合优化问题

经典组合优化问题
• 划分问题 (Partitioning) • 调度问题 (Scheduling) • 装箱问题 (Bin Packing) • 背包问题 (Knapsack) • 指派问题 (Assignment) • 旅行商问题 (TSP) • 斯坦纳最小树问题 (Steiner Minimal Tree) • 最小生成树 (MST) • 最大流/最小费用流问题

例题 8.3.1 (TSP 问题的计算复杂度). 城市数为 n 时，解空间为 $(n - 1)!/2$ 。

城市数	枚举时间（每秒评估 1000 次）
24	1 秒
25	24 秒
26	10 分钟
27	4.3 小时
28	4.9 天
29	136.5 天
30	10.8 年
31	325 年

8.3.4 独立系统与拟阵

定义 8.3.3 (独立系统). 集合系统 $(E, \mathcal{F})$ 称为独立系统，若：

- (1) $\emptyset \in \mathcal{F}$
- (2) 继承性: $Y \subseteq X \in \mathcal{F} \Rightarrow Y \in \mathcal{F}$

集合 $\mathcal{F}$ 的元素称为**独立集**， $2^E \setminus \mathcal{F}$ 的元素称为**相关集**。

定义 8.3.4 (基与秩). • 最小相关集称为**环** (*circuits*)

- 最大独立集称为**基** (*bases*)
- $X \subseteq E$ 的**秩**: $r(X) = \max\{|Y| : Y \subseteq X, Y \in \mathcal{F}\}$

8.3.5 启发式算法的特点

优点:

- 适用于 NP-hard 问题的近似求解
- 简单易行，直观易被接受
- 速度快，适合实时应用
- 程序简单，易于修改
- 可用在最优算法中估界（如分支定界）

缺点:

- 不能保证求得最优解
- 表现不稳定
- 算法好坏依赖于实际问题

8.3.6 智能优化算法一览

概率算法与墨菲定律

现代优化算法大都具有一定的随机性，能以概率跳出局部最优点。正如墨菲定律所说：“If something can go wrong, it will.”——有概率跳出局部最优，就一定有这种可能（前提是尝试次数足够多）。

8.4 模拟退火算法

8.4.1 物理背景

模拟退火 (Simulated Annealing, SA) 模拟金属退火过程:

1. **加热**: 将固体加热至高温，原子高速运动，内能增加
2. **缓慢冷却**: 温度下降，原子运动减慢，内能下降

表 8.1: 现代智能优化算法

算法	英文名	灵感来源	跟随最优
遗传算法	GA	进化论	否
粒子群优化	PSO	鸟群觅食	是
差分进化	DE	进化论	否
人工蜂群	ABC	蜜蜂觅食	是
蚁群优化	ACO	蚂蚁觅食	是
人工鱼群	AFSA	鱼群觅食	是
杜鹃搜索	CS	杜鹃产卵	是
萤火虫算法	FA	萤火虫求偶	是
灰狼算法	GWO	狼群觅食	是
鲸鱼算法	WOA	鲸鱼觅食	是
烟花算法	FWA	烟花爆炸	是
菌群优化	BFO	菌群生长	是

3. **热平衡**: 有时内能以 Boltzmann 概率随机增加, 即温度 t 时能量增加 ΔE 的概率密度为 $\exp(-\Delta E/kt)$
4. **最终状态**: 足够慢地冷却, 最终达到内能最低的稳定状态

8.4.2 Metropolis 准则

定义 8.4.1 (Metropolis 接受准则). 设当前解为 x , 邻域候选解为 x^* , 温度参数为 T :

- 若 $\Delta E = E(x^*) - E(x) < 0$ (更优), **接受** x^*
- 若 $\Delta E \geq 0$ (更差), 以概率 $P = \exp(-\Delta E/T)$ **接受** x^*

模拟退火的关键机制

- 温度高时, 更可能接受劣解, 有助于跳出局部最优
- 温度低时, 接受劣解概率小, 趋向于稳定在最优解附近
- 随着温度慢慢降至 0, 算法以相当高的概率收敛到全局最小值

8.4.3 算法流程

模拟退火算法

输入：初始解 x_0 ，初始温度 T_0 ，降温系数 $\alpha \in (0, 1)$

Step 0: $k = 0, x = x_0, T = T_0$

Step 1 (内循环)：在当前温度 T 下重复 L 次

1. 从 x 的邻域随机选取候选解 x^*

2. 计算 $\Delta E = E(x^*) - E(x)$

3. 若 $\Delta E < 0$ ，令 $x = x^*$ ；否则以概率 $\exp(-\Delta E/T)$ 令 $x = x^*$

Step 2: 降温 $T_{k+1} = \alpha \cdot T_k$

Step 3: 若满足终止条件 ($T < T_{\min}$ 或迭代次数超限)，输出最优解；否则 $k \leftarrow k + 1$ ，转 Step 1

8.4.4 关键参数

表 8.2: 模拟退火的关键参数		
参数	含义	常用设置
T_0	初始温度	足够高，使接受率约 80%
α	降温系数	0.95 ~ 0.995
L	Markov 链长度	每个温度下迭代次数
$T_{\min}$	终止温度	接近 0 的正值

例题 8.4.1 (模拟退火求解函数极小值). 问题：用 SA 求解 $\min f(x) = x^2 - 4x + 4 = (x - 2)^2, x \in [0, 5]$ 。初始 $x_0 = 0, T_0 = 10, \alpha = 0.9$ ，每个温度迭代 2 次。

解：

温度 $T = 10$:

- 第 1 次：候选 $x^* = 1$ (邻域随机选取)， $f(1) = 1, f(0) = 4, \Delta E = -3 < 0$ ，接受 $x = 1$
- 第 2 次：候选 $x^* = 0.5, f(0.5) = 2.25, \Delta E = 1.25 > 0$ ，概率 $\exp(-1.25/10) \approx 0.88$ ，假设接受

温度 $T = 9$:

- 第 1 次：候选 $x^* = 1.5, f(1.5) = 0.25, \Delta E < 0$ ，接受 $x = 1.5$

- 第 2 次：候选 $x^* = 2.5$, $f(2.5) = 0.25$, $\Delta E = 0$, 概率 $\exp(0) = 1$, 接受

继续迭代直到温度足够低，最终收敛到 $x^* = 2$ 。

8.4.5 玻尔兹曼机

定义 8.4.2 (Boltzmann 机). *G.E. Hinton* 等于 1983–1986 提出。神经元只有两种输出状态 (0 或 1)，状态的取值根据概率统计法则决定，该法则的表达形式与 Boltzmann 分布类似。

节点输出为 1 的概率：

$$P(x_j = 1) = \frac{1}{1 + \exp(-net_j/T)}$$

其中 $net_j = \sum_i w_{ij}x_i - \theta_j$ 。

网络能量函数：

$$E = -\frac{1}{2} \sum_{i,j} w_{ij}x_i x_j + \sum_j \theta_j x_j$$

性质 8.4.1 (Boltzmann 分布). 网络中任意两个状态出现的概率与能量之间的关系：

$$\frac{P_A}{P_B} = \frac{\exp(-E_A/T)}{\exp(-E_B/T)} = \exp\left(-\frac{E_A - E_B}{T}\right)$$

温度 T 越高，不同状态出现的概率越接近；温度越低，低能量状态概率越大。

```

1 import numpy as np
2
3 def simulated_annealing(f, x0, T0=100, alpha=0.95, L=100,
4                          T_min=1e-4, bounds=None):
5     """
6     模拟退火算法
7     f: 目标函数
8     x0: 初始解
9     bounds: [(lb, ub), ...]
10    """
11    x = np.array(x0, dtype=float)
12    T = T0
13    n = len(x)
14
15    # 确定邻域搜索步长
16    if bounds is not None:
17        step = np.array([(ub - lb) * 0.1 for lb, ub in bounds])
18    else:

```

```

19     step = np.ones(n) * 0.1
20
21     best_x = x.copy()
22     best_f = f(x)
23     history = [(T, best_f)]
24
25     while T > T_min:
26         for _ in range(L):
27             # 邻域候选解
28             x_new = x + np.random.randn(n) * step
29             if bounds is not None:
30                 x_new = np.clip(x_new, [b[0] for b in bounds],
31                                 [b[1] for b in bounds])
32
33             delta_E = f(x_new) - f(x)
34
35             # Metropolis 准则
36             if delta_E < 0 or np.random.rand() < np.exp(-delta_E / T):
37                 x = x_new.copy()
38                 if f(x) < best_f:
39                     best_f = f(x)
40                     best_x = x.copy()
41
42             T *= alpha
43             history.append((T, best_f))
44
45     return best_x, best_f, history
46
47
48 # === 测试：多峰函数 ===
49 def multimodal_f(x):
50     """多峰函数:  $f(x) = x \sin(5x) + x^2 \cos(x)$ """
51     return x[0] * np.sin(5*x[0]) + x[0]**2 * np.cos(x[0])
52
53 print("=" * 50)
54 print("模拟退火 - 多峰函数优化")
55 print("=" * 50)
56
57 x_opt, f_opt, hist = simulated_annealing(

```

```

58     multimodal_f, x0=[-2.0], T0=50, alpha=0.95, L=50,
59     bounds=[(-5, 5)])
60
61 print(f"最优解: x* = {x_opt[0]:.6f}")
62 print(f"最优值: f* = {f_opt:.6f}")
63 print(f"温度迭代次数: {len(hist)}")
64
65 # 多次运行统计
66 results = []
67 for _ in range(10):
68     x, fx, _ = simulated_annealing(multimodal_f, x0=[-2.0],
69                                   T0=50, alpha=0.95, L=30,
70                                   bounds=[(-5, 5)])
71     results.append(fx)
72 print(f"10次运行最优值: min={min(results):.4f}, mean={np.mean(results):.4f}")

```

Listing 8.2: 模拟退火算法实现

8.5 遗传算法

8.5.1 生物启发

遗传算法（Genetic Algorithm, GA）借助生物学中**适者生存**的规律，通过模拟自然选择和遗传机制来搜索最优解。

遗传算法的核心特点

1. **编码**：解的编码称为**染色体**（Chromosome），优化问题的一切性质通过编码研究
2. **适应度**：自然选择决定哪些染色体产生超过平均数的后代，通过**适应函数**评价
3. **交叉**：染色体结合时，双亲遗传基因结合使子女保持父母特征
4. **变异**：染色体结合后，随机变异造成子代与父代的不同

8.5.2 编码方式

例题 8.5.1（二进制编码）。问题： $\max f(x) = x^2$, $x \in [0, 31]$, x 为整数。

采用 5 位二进制编码：

- $x = 16$ ：染色体 10000

- $x = 31$: 染色体 11111
- $x = 9$: 染色体 01001
- $x = 2$: 染色体 00010

每个分量称为基因，两种状态 0 和 1。

8.5.3 基本遗传算法

遗传算法

Step 1: 选择编码方式，产生初始群体 $POP(0)$ (N 个染色体)

Step 2: 对 $POP(t)$ 中每个染色体 $x_i(t)$ 计算适应函数 $f(x_i)$

Step 3: 若停止准则满足，终止；否则计算选择概率：

$$p_i = \frac{f(x_i)}{\sum_{j=1}^N f(x_j)}, \quad i = 1, 2, \dots, N$$

以该概率从 $POP(t)$ 中选择染色体构成种群（轮盘赌选择）

Step 4: 通过交叉（概率 p_c ）得到 $CROSS(t+1)$

Step 5: 以较小概率 p_m 使染色体基因变异，形成 $MUT(t+1)$ ，令 $POP(t+1) = MUT(t+1)$ ，转 Step 2

8.5.4 遗传算子

选择 (Selection) —— 轮盘赌

适应度越大的个体入选概率越大。第 i 个个体被选中的概率：

$$p_i = \frac{\text{fitness}(x_i)}{\sum_j \text{fitness}(x_j)}$$

交叉 (Crossover)

随机选择两个父代染色体，按交叉点交换部分基因。例如：

- 父代 1: 0110|1, 父代 2: 1010|0
- 交叉后: 子代 1 = 01100, 子代 2 = 10101

变异 (Mutation)

以小概率随机翻转某个基因。例如 01101 $\rightarrow$ 11101 (第 1 位变异)。

例题 8.5.2 (遗传算法——迭代两次). **问题:** $\max f(x) = x^2$, $x \in [0, 31]$ 整数。群体大小 $N = 4$, 5 位二进制编码。

初始群体: $x_1 = 01101$ (13), $x_2 = 10101$ (21), $x_3 = 01001$ (9), $x_4 = 10000$ (16)

适应度: $f(x_1) = 169$, $f(x_2) = 441$, $f(x_3) = 81$, $f(x_4) = 256$, 总和 = 947

选择概率: $p_1 = 0.178$, $p_2 = 0.466$, $p_3 = 0.086$, $p_4 = 0.270$

轮盘赌选择 (假设选中 x_1, x_2, x_2, x_4):

第 1 轮——交叉 (设 $p_c = 0.7$, 交叉点在第 3 位后):

- 父代 $x_1 = 011|01$ 与 $x_2 = 101|01$ 交叉
- 子代 $y_1 = 01101$, $y_2 = 10101$
- 父代 $x_2 = 101|01$ 与 $x_4 = 100|00$ 交叉
- 子代 $y_3 = 10100$ (20), $y_4 = 10001$ (17)

第 2 轮——变异 (设 $p_m = 0.01$):

- 假设 y_3 第 2 位变异: $10100 \rightarrow 00100$ (4)

新一代群体: 计算适应度, 继续迭代。

```

1 import numpy as np
2
3 def genetic_algorithm(fitness_func, n_bits, n_pop, n_gen,
4                       pc=0.8, pm=0.01, bounds=(0, 31)):
5     """
6     遗传算法 (二进制编码)
7     fitness_func: 适应度函数
8     n_bits: 编码位数
9     n_pop: 群体大小
10    n_gen: 迭代代数
11    """
12    lb, ub = bounds
13
14    # 初始化群体
15    pop = np.random.randint(0, 2, (n_pop, n_bits))
16    best_hist = []
17
18    def decode(chrom):
19        val = np.sum(chrom * (2 ** np.arange(n_bits-1, -1, -1)))
20        return lb + val % (ub - lb + 1)
21
22    for gen in range(n_gen):
23        # 评估适应度
24        fitness = np.array([fitness_func(decode(c)) for c in pop])

```

```

25     best_hist.append(np.max(fitness))
26
27     # 选择概率
28     p = fitness / fitness.sum()
29
30     # 轮盘赌选择
31     selected_idx = np.random.choice(n_pop, n_pop, p=p)
32     selected = pop[selected_idx].copy()
33
34     # 交叉
35     offspring = []
36     for i in range(0, n_pop, 2):
37         p1, p2 = selected[i], selected[(i+1) % n_pop]
38         if np.random.rand() < pc:
39             pt = np.random.randint(1, n_bits)
40             c1 = np.concatenate([p1[:pt], p2[pt:]])
41             c2 = np.concatenate([p2[:pt], p1[pt:]])
42         else:
43             c1, c2 = p1.copy(), p2.copy()
44         offspring.extend([c1, c2])
45
46     pop = np.array(offspring)
47
48     # 变异
49     for i in range(n_pop):
50         for j in range(n_bits):
51             if np.random.rand() < pm:
52                 pop[i, j] = 1 - pop[i, j]
53
54     # 最终最优
55     fitness = np.array([fitness_func(decode(c)) for c in pop])
56     best_idx = np.argmax(fitness)
57     return decode(pop[best_idx]), fitness[best_idx], best_hist
58
59
60 # === 测试 ===
61 print("=" * 50)
62 print("遗传算法: max x^2, x in [0, 31]")
63 print("=" * 50)
64

```

```

65 x_opt, f_opt, hist = genetic_algorithm(
66     lambda x: x**2, n_bits=5, n_pop=20, n_gen=30,
67     pc=0.8, pm=0.01)
68
69 print(f"最优解: x* = {x_opt}, f* = {f_opt}")
70 print(f"迭代过程最优值: {hist[:10]}")

```

Listing 8.3: 遗传算法实现

8.6 粒子群优化与差分进化

8.6.1 粒子群优化算法 (PSO)

PSO 原理

模拟鸟群、鱼群的觅食行为。每个粒子代表一个潜在解，通过跟踪个体最优 (pbest) 和全局最优 (gbest) 来更新速度和位置。

定义 8.6.1 (PSO 迭代公式). 粒子 i 在第 t 代的速度和位置更新:

$$\begin{aligned}
 v_i^{t+1} &= w \cdot v_i^t + c_1 r_1 (pbest_i - x_i^t) + c_2 r_2 (gbest - x_i^t) \\
 x_i^{t+1} &= x_i^t + v_i^{t+1}
 \end{aligned}$$

其中:

- w : 惯性权重
- c_1, c_2 : 认知系数和社会系数
- $r_1, r_2 \sim U(0, 1)$: 随机数

PSO 算法

Step 1: 初始化粒子群 (位置和速度)

Step 2: 评估每个粒子的适应度, 更新 $pbest$ 和 $gbest$

Step 3: 按迭代公式更新每个粒子的速度和位置

Step 4: 若满足终止条件, 停; 否则转 Step 2

例题 8.6.1 (PSO 迭代两次). **问题:** $\min f(x) = (x - 3)^2$, 1 维, 2 个粒子。

初始: $x_1 = 0, v_1 = 0.5$; $x_2 = 5, v_2 = -0.3$; $w = 0.8, c_1 = c_2 = 1.5$

第 0 轮 (初始化)

- 计算适应度: $f(0) = 9, f(5) = 4$

- 个体最优: $pbest_1 = 0$ (粒子 1 的历史最优位置), $pbest_2 = 5$ (粒子 2 的历史最优位置)
- 全局最优: $gbest = 5$ (因为 $f(5) = 4 < f(0) = 9$, 取适应度最好的位置)

第 1 轮 (设随机数 $r_1 = 0.3, r_2 = 0.7$)

更新粒子 1:

$$\begin{aligned} v_1 &= 0.8 \times 0.5 + 1.5 \times 0.3 \times (0 - 0) + 1.5 \times 0.7 \times (5 - 0) \\ &= 0.4 + 0 + 5.25 = 5.65 \\ x_1 &= 0 + 5.65 = 5.65 \end{aligned}$$

更新粒子 2:

$$\begin{aligned} v_2 &= 0.8 \times (-0.3) + 1.5 \times 0.5 \times (5 - 5) + 1.5 \times 0.4 \times (5 - 5) \\ &= -0.24 + 0 + 0 = -0.24 \\ x_2 &= 5 + (-0.24) = 4.76 \end{aligned}$$

第 2 轮: 更新 $pbest$ 和 $gbest$ ($f(5.65) = 7.0225 > f(4.76) = 1.5376$, 故 $gbest$ 更新为 4.76), 继续迭代直至收敛到 $x^* = 3$ 。

8.6.2 差分进化算法 (DE)

DE 原理

模拟群体进化, 通过个体的合作与竞争过程进行优化。特点是使用差分向量生成变异个体。

定义 8.6.2 (DE 的变异操作). 对当前个体 x_i , 随机选择三个不同个体 x_{r1}, x_{r2}, x_{r3} , 生成变异向量:

$$v_i = x_{r1} + F \cdot (x_{r2} - x_{r3})$$

其中 $F \in (0, 2)$ 为缩放因子。

定义 8.6.3 (DE 的交叉操作). 将变异向量 v_i 与目标向量 x_i 进行交叉, 生成试验向量 u_i :

$$u_{i,j} = \begin{cases} v_{i,j}, & \text{if } rand_j \leq CR \text{ or } j = j_{rand} \\ x_{i,j}, & \text{otherwise} \end{cases}$$

其中 $CR \in [0, 1]$ 为交叉概率。

DE 算法**Step 1:** 初始化种群**Step 2:** 对每个个体 x_i

- **变异:** 随机选 3 个不同个体, 生成 v_i
- **交叉:** v_i 与 x_i 交叉生成 u_i
- **选择:** 若 $f(u_i) < f(x_i)$, 则 $x_i \leftarrow u_i$

Step 3: 若满足终止条件, 停; 否则转 Step 2

```

1 import numpy as np
2
3 def pso(f, dim, n_particles=30, max_iter=100,
4         w=0.8, c1=1.5, c2=1.5, bounds=None):
5     """粒子群优化"""
6     if bounds is None:
7         bounds = [(-5, 5)] * dim
8
9     lb = np.array([b[0] for b in bounds])
10    ub = np.array([b[1] for b in bounds])
11
12    # 初始化
13    x = np.random.uniform(lb, ub, (n_particles, dim))
14    v = np.random.randn(n_particles, dim) * 0.1
15
16    pbest = x.copy()
17    pbest_val = np.array([f(xi) for xi in x])
18    gbest_idx = np.argmin(pbest_val)
19    gbest = pbest[gbest_idx].copy()
20
21    history = []
22
23    for t in range(max_iter):
24        for i in range(n_particles):
25            r1, r2 = np.random.rand(2)
26            v[i] = (w * v[i] + c1 * r1 * (pbest[i] - x[i])
27                  + c2 * r2 * (gbest - x[i]))
28            x[i] = x[i] + v[i]
29            x[i] = np.clip(x[i], lb, ub)

```

```

30
31         val = f(x[i])
32         if val < pbest_val[i]:
33             pbest_val[i] = val
34             pbest[i] = x[i].copy()
35             if val < f(gbest):
36                 gbest = x[i].copy()
37
38         history.append(f(gbest))
39
40     return gbest, f(gbest), history
41
42
43 def de(f, dim, n_pop=50, max_iter=100, F=0.8, CR=0.9,
44       bounds=None):
45     """差分进化算法"""
46     if bounds is None:
47         bounds = [(-5, 5)] * dim
48
49     lb = np.array([b[0] for b in bounds])
50     ub = np.array([b[1] for b in bounds])
51
52     pop = np.random.uniform(lb, ub, (n_pop, dim))
53     fitness = np.array([f(x) for x in pop])
54
55     history = []
56
57     for t in range(max_iter):
58         for i in range(n_pop):
59             # 选择3个不同个体
60             idxs = [j for j in range(n_pop) if j != i]
61             r1, r2, r3 = np.random.choice(idxs, 3, replace=False)
62
63             # 变异
64             v = pop[r1] + F * (pop[r2] - pop[r3])
65             v = np.clip(v, lb, ub)
66
67             # 交叉
68             j_rand = np.random.randint(dim)
69             u = np.array([v[j] if np.random.rand() < CR or j ==

```

```

        j_rand
70         else pop[i, j] for j in range(dim)])
71
72     # 选择
73     fu = f(u)
74     if fu < fitness[i]:
75         pop[i] = u
76         fitness[i] = fu
77
78     best_idx = np.argmin(fitness)
79     history.append(fitness[best_idx])
80
81     best_idx = np.argmin(fitness)
82     return pop[best_idx], fitness[best_idx], history
83
84
85 # === 测试 ===
86 def rosenbrock(x):
87     return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2
88
89 print("=" * 50)
90 print("PSO - Rosenbrock")
91 print("=" * 50)
92 x_pso, f_pso, _ = pso(rosenbrock, dim=2, n_particles=50,
93                       max_iter=200, bounds=[(-2, 2), (-1, 3)])
94 print(f"PSO: x* = {x_pso}, f* = {f_pso:.6f}")
95
96 print("\n" + "=" * 50)
97 print("DE - Rosenbrock")
98 print("=" * 50)
99 x_de, f_de, _ = de(rosenbrock, dim=2, n_pop=50,
100                   max_iter=200, bounds=[(-2, 2), (-1, 3)])
101 print(f"DE: x* = {x_de}, f* = {f_de:.6f}")

```

Listing 8.4: PSO 与 DE 实现

8.7 其他群智能算法

8.7.1 人工蜂群算法（ABC）

ABC 原理
模拟蜜蜂的觅食行为，分为三类蜜蜂： • 雇佣蜂（Employed Bees）：在已知食物源（解）附近搜索 • 观察蜂（Onlooker Bees）：根据雇佣蜂的信息选择食物源 • 侦察蜂（Scout Bees）：当某个食物源枯竭时随机搜索新食物源
ABC 算法
Step 1：初始化食物源（解）及其适应度 Step 2（雇佣蜂阶段）：每个雇佣蜂在对应食物源邻域生成新解，贪心选择 Step 3（观察蜂阶段）：按适应度选择食物源，在邻域搜索新解 Step 4（侦察蜂阶段）：若某食物源迭代次数超过限制，放弃并随机生成新食物源 Step 5：记录最优解，若满足终止条件则停；否则转 Step 2

8.7.2 蚁群优化算法（ACO）

ACO 原理
模拟蚂蚁觅食时通过信息素（Pheromone）通信的行为。蚂蚁在路径上释放信息素，后续蚂蚁倾向于选择信息素浓度高的路径，形成正反馈。

- 关键机制：
- 1. 信息素更新：优质路径上信息素增加，所有路径信息素随时间蒸发
 - 2. 转移概率：蚂蚁从节点 i 到节点 j 的概率：

$$P_{ij} = \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_k \tau_{ik}^\alpha \cdot \eta_{ik}^\beta}$$

其中 τ_{ij} 为信息素浓度， η_{ij} 为启发信息（如距离的倒数）， α, β 为参数。

8.7.3 灰狼算法（GWO）

GWO 原理
模拟灰狼的社会等级和狩猎行为： • α 狼：群体首领，最优解 • β 狼：次优解，辅助 α

- δ 狼：第三优解
- ω 狼：普通成员，跟随前三者

位置更新：

$$\vec{x}(t+1) = \frac{\vec{x}_\alpha + \vec{x}_\beta + \vec{x}_\delta}{3}$$

参数 a 从 2 线性递减到 0，控制探索和开发平衡。

8.7.4 鲸鱼算法 (WOA)

WOA 原理

模拟座头鲸的**泡泡网捕食**行为，有两种主要策略：

1. **收缩包围**：向最优解（猎物）移动

$$\vec{x}(t+1) = \vec{x}^*(t) - A \cdot |C \cdot \vec{x}^*(t) - \vec{x}(t)|$$

2. **螺旋更新**：模拟螺旋形泡泡网

$$\vec{x}(t+1) = |\vec{x}^*(t) - \vec{x}(t)| \cdot e^{bl} \cdot \cos(2\pi l) + \vec{x}^*(t)$$

其中 A, C 为随机参数， $l \in [-1, 1]$ 。

8.7.5 烟花算法 (FWA)

FWA 原理

每个烟花的位置代表一个可行解。烟花**爆炸**产生两类火星：

- **正常火星**：在当前振幅内随机均匀分布
- **特别火星**：在当前位置附近以正态分布产生

振幅与适应度有关：适应度越好（越亮），振幅越小；适应度越差，振幅越大（探索更广区域）。

火星保留：每代产生多于应有数量的火星，通过适应度和距离选择保留指定数量。

```

1 import numpy as np
2
3 def gwo(f, dim, n_wolves=30, max_iter=100, bounds=None):
4     """灰狼算法"""
5     if bounds is None:
6         bounds = [(-5, 5)] * dim

```

表 8.3: 群智能算法对比

算法	灵感来源	信息传递	探索/开发	特点
PSO	鸟群	全局 + 局部最优	惯性权重平衡	简单高效
DE	进化	差分向量	变异 + 交叉	全局搜索强
ABC	蜂群	雇佣蜂舞蹈	三角色分工	局部搜索好
ACO	蚁群	信息素	正反馈	适合组合优化
GWO	狼群	等级制	线性递减	收敛快
WOA	鲸鱼	螺旋 + 包围	概率切换	平衡好
FWA	烟花	爆炸振幅	振幅自适应	多样性保持

```

7
8 lb = np.array([b[0] for b in bounds])
9 ub = np.array([b[1] for b in bounds])
10
11 # 初始化
12 wolves = np.random.uniform(lb, ub, (n_wolves, dim))
13 fitness = np.array([f(w) for w in wolves])
14
15 # 排序找 alpha, beta, delta
16 idx = np.argsort(fitness)
17 alpha, beta, delta = wolves[idx[0]], wolves[idx[1]], wolves[idx
    [2]]
18
19 history = []
20
21 for t in range(max_iter):
22     a = 2 - 2 * t / max_iter # a从2线性减到0
23
24     for i in range(n_wolves):
25         for d in range(dim):
26             # 对 alpha, beta, delta 分别计算
27             r1, r2 = np.random.rand(2)
28             A1 = 2 * a * r1 - a
29             C1 = 2 * r2
30             D_alpha = abs(C1 * alpha[d] - wolves[i, d])
31             X1 = alpha[d] - A1 * D_alpha
32
33             r1, r2 = np.random.rand(2)

```

```

34         A2 = 2 * a * r1 - a
35         C2 = 2 * r2
36         D_beta = abs(C2 * beta[d] - wolves[i, d])
37         X2 = beta[d] - A2 * D_beta
38
39         r1, r2 = np.random.rand(2)
40         A3 = 2 * a * r1 - a
41         C3 = 2 * r2
42         D_delta = abs(C3 * delta[d] - wolves[i, d])
43         X3 = delta[d] - A3 * D_delta
44
45         wolves[i, d] = (X1 + X2 + X3) / 3
46
47         wolves[i] = np.clip(wolves[i], lb, ub)
48         fitness[i] = f(wolves[i])
49
50         # 更新 alpha, beta, delta
51         idx = np.argsort(fitness)
52         alpha, beta, delta = wolves[idx[0]], wolves[idx[1]], wolves[
            idx[2]]
53         history.append(fitness[idx[0]])
54
55     return alpha, f(alpha), history
56
57
58 # === 测试 ===
59 print("=" * 50)
60 print("灰狼算法 - Sphere函数")
61 print("=" * 50)
62
63 sphere = lambda x: np.sum(x**2)
64 x_gwo, f_gwo, _ = gwo(sphere, dim=5, n_wolves=30, max_iter=100,
65                        bounds=[(-5, 5)] * 5)
66 print(f"GWO: f* = {f_gwo:.10f}")

```

Listing 8.5: 灰狼算法实现

8.8 应用实例

8.8.1 试飞任务规划

问题描述

民用航空飞机的试飞阶段包含上千个试飞科目，各科目之间具有严格的逻辑关系、优先级约束、时间窗口约束、备选飞机约束等。高效的试飞任务调度是 NP-hard 问题。

决策变量：

- $x_{i,k,h} \in \{0, 1\}$: 第 i 号任务分配给第 k 架飞机，且为该飞机第 h 项任务
- $t_{i,k}$: 第 i 号任务在第 k 号飞机上的开始时刻
- $T_{i,k}$: 第 i 号任务在第 k 号飞机上的完成时刻

目标函数：

$$\min \max_k \{T_k\} \quad (\text{最小化最大完成时间})$$

其中 T_k 为第 k 架飞机的试飞完成天数。

关键约束：

- (1) 优先级约束: $t_{i,k} \geq \max_{j \in P_i} T_{j,l}$, 任务 i 必须在其所有前置任务完成后开始
- (2) 唯一分配: $\sum_{k \in C_i} \sum_h x_{i,k,h} = 1$, 每项任务恰好分配给一架飞机
- (3) 执行时长: $T_{i,k} - t_{i,k} = \pi_i$ (固定执行时长)
- (4) 时间窗口: $Ta_i \leq t_{i,k} \leq Tb_i$
- (5) 试飞强度: μ 控制单位时间内完成的任务量

8.8.2 柔性作业车间调度

问题描述

建立多个工件和多个机器之间的匹配关系。每个工件有不同工序，工序之间有确定顺序，每道工序可与多台机器匹配。目标是建立工件、工序和机器的最优匹配。

优化目标：最小化订单完成时间 (Makespan)

求解方法：遗传算法、粒子群优化、模拟退火等智能优化算法。

8.8.3 旅行商问题 (TSP)

TSP 的 SA 求解

问题：给定 n 个城市，求访问每个城市恰好一次并返回起点的最短路径。

邻域定义 (2-opt)：任选路径中的两条边，反转它们之间的子路径。

能量函数：路径总长度 $E(\pi) = \sum_{i=1}^n d(\pi_i, \pi_{i+1})$

例题 8.8.1 (TSP 的 2-opt 邻域). 当前路径: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 1$

选择边 (2,3) 和 (5,6), 反转中间子路径:

新路径: $1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 6 \rightarrow 1$

2-opt 邻域大小为 $\frac{n(n-3)}{2}$, 远小于完全解空间 $n!$ 。

8.8.4 深度学习的优化

深度学习中的优化特点

- 大多数优化问题是**非凸的**, 甚至是 NP-hard 的
- 可控梯度下降: $x_{t+1} = x_t - \eta \nabla f(x_t)$
- 需要确保: 步长足够小以保证下降, 同时 Hessian 大时允许梯度有较大波动
- 挑战: 局部最优、梯度消失/爆炸、学习率选择、初始化敏感
- 随机网络 (如 Boltzmann 机) 通过概率机制跳出局部最优

```

1 import numpy as np
2 import time
3
4 # === 测试函数集 ===
5 def sphere(x):
6     """Sphere函数 (凸)"""
7     return np.sum(x**2)
8
9 def rastrigin(x):
10    """Rastrigin函数 (多峰)"""
11    return 10 * len(x) + np.sum(x**2 - 10 * np.cos(2 * np.pi * x))
12
13 def ackley(x):
14    """Ackley函数 (多峰)"""
15    a, b, c = 20, 0.2, 2 * np.pi
16    n = len(x)
17    return -a * np.exp(-b * np.sqrt(np.sum(x**2)/n)) \

```

```

18         - np.exp(np.sum(np.cos(c * x))/n) + a + np.e
19
20
21 def compare_algorithms(func, dim=10, bounds=(-5, 5)):
22     """对比多种智能优化算法"""
23     results = {}
24
25     # 模拟退火
26     from scipy.optimize import dual_annealing
27     start = time.time()
28     res = dual_annealing(func, [(bounds, bounds)] * dim, maxiter=200)
29     results['SA'] = {'f*': res.fun, 'time': time.time() - start}
30
31     # 差分进化
32     from scipy.optimize import differential_evolution
33     start = time.time()
34     res = differential_evolution(func, [(bounds, bounds)] * dim,
35                                 maxiter=200, tol=1e-6)
36     results['DE'] = {'f*': res.fun, 'time': time.time() - start}
37
38     return results
39
40
41 print("=" * 60)
42 print("智能优化算法对比 (dim=10)")
43 print("=" * 60)
44
45 for name, f in [('Sphere', sphere), ('Rastrigin', rastrigin),
46               ('Ackley', ackley)]:
47     print(f"\n--- {name} ---")
48     results = compare_algorithms(f, dim=10)
49     for alg, info in results.items():
50         print(f"    {alg:6s}: f* = {info['f*']:.6e}, "
51               f"time = {info['time']:.3f}s")

```

Listing 8.6: 多种智能算法对比测试

8.9 支持向量机 (SVM)

支持向量机 (Support Vector Machine, SVM) 是凸优化理论在机器学习中的经典应用, 其核心思想是通过求解一个凸二次规划问题来构造最大间隔分类器。SVM 的求解过程涉及第 6 章中的多个核心概念: KKT 条件 (第 6.2.2 节)、对偶理论 (第 6.4 节) 和互补松弛条件。

8.9.1 线性可分与硬间隔 SVM

给定训练样本 $\{(x_i, y_i)\}_{i=1}^N$, 其中 $x_i \in \mathbb{R}^p$, $y_i \in \{-1, +1\}$ 。线性分类器用超平面 $w^T x + b = 0$ 将两类分开, 预测规则为

$$\hat{y} = \text{sign}(w^T x + b).$$

若样本线性可分, 则 $y_i(w^T x_i + b) > 0$ 。由于 (w, b) 同时乘以正数不改变分类面, 可将函数间隔规范化为

$$y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, N.$$

样本点到超平面的几何间隔为 $y_i(w^T x_i + b)/\|w\|_2$ 。在上述规范化下, 离分类面最近的样本的间隔为 $1/\|w\|_2$, 两类支持超平面之间的总间隔为 $2/\|w\|_2$ 。因此最大间隔等价于:

定义 8.9.1 (硬间隔 SVM)。

$$\min_{w, b} \frac{1}{2} \|w\|_2^2, \quad s.t. \ y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, N.$$

这是一个凸二次规划问题 (目标函数为凸二次型, 约束为仿射不等式)。

8.9.2 拉格朗日对偶与支持向量

将约束写成 $1 - y_i(w^T x_i + b) \leq 0$, 引入拉格朗日乘子 $\alpha_i \geq 0$:

$$L(w, b, \alpha) = \frac{1}{2} \|w\|_2^2 + \sum_{i=1}^N \alpha_i [1 - y_i(w^T x_i + b)].$$

对 w, b 求驻点:

$$\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^N \alpha_i y_i x_i, \quad \frac{\partial L}{\partial b} = 0 \Rightarrow \sum_{i=1}^N \alpha_i y_i = 0.$$

代回消去 w, b , 得到对偶问题:

定义 8.9.2 (硬间隔 SVM 对偶)。

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j, \quad s.t. \ \alpha_i \geq 0, \quad \sum_{i=1}^N \alpha_i y_i = 0.$$

KKT 互补松弛条件为 $\alpha_i[1 - y_i(w^T x_i + b)] = 0$ 。因此只有满足 $y_i(w^T x_i + b) = 1$ 且 $\alpha_i > 0$ 的样本会决定超平面，这些点称为**支持向量**。分类函数可写成

$$f(x) = \text{sign} \left(\sum_{i=1}^N \alpha_i y_i x_i^T x + b \right).$$

8.9.3 软间隔 SVM

若数据不可完全线性分离，引入松弛变量 $\xi_i \geq 0$ ：

定义 8.9.3 (软间隔 SVM).

$$\min_{w, b, \xi} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^N \xi_i, \quad \text{s.t. } y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0,$$

其中 $C > 0$ 是惩罚参数，控制间隔最大化与误分类惩罚之间的权衡。

其对偶问题在硬间隔基础上多出上界约束：

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^T x_j, \quad \text{s.t. } 0 \leq \alpha_i \leq C, \sum_{i=1}^N \alpha_i y_i = 0.$$

8.9.4 核方法

当线性超平面不足以分离数据时，可通过核技巧将数据映射到高维特征空间。设映射为 $\phi: \mathbb{R}^p \rightarrow \mathcal{H}$ ，定义核函数

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j).$$

只需把对偶问题中的内积 $x_i^T x_j$ 替换为 $K(x_i, x_j)$ ，即可在特征空间中构造非线性最大间隔分类器，而无需显式计算高维映射 ϕ 。

常见核函数包括：

- 线性核： $K(x_i, x_j) = x_i^T x_j$
- 多项式核： $K(x_i, x_j) = (x_i^T x_j + c)^d$
- 高斯径向基 (RBF) 核： $K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$

SVM 与凸优化的联系

SVM 的整个求解框架紧密依赖凸优化理论：

- (1) 原问题是凸二次规划（目标凸、约束仿射），保证全局最优
- (2) 通过对偶转化，将高维 w 的优化转化为关于 α 的问题，且对偶也是凸的

- (3) KKT 条件刻画了最优解的结构：只有支持向量起作用，体现了稀疏性
- (4) 核技巧利用了仿射映射复合保持凸性的性质

8.10 本章小结

8.10.1 本章知识框架

复合优化与智能优化——核心内容

1. 复合优化问题： $\min f(x) + g(x)$ (f 可微, g 不可微)
 - 梯度方法：Adagrad, Adam, 动量法等
 - 邻近点方法 (PPA)
 - ADMM
2. 变分不等式 (VI)
 - 单调变分不等式
 - 线性约束凸优化等价于 VI
 - 可分离结构与 ADMM 收敛性
3. 启发式算法概述
 - 组合优化与 NP-hard 问题
 - 独立系统与拟阵
 - 启发式 vs 近似算法
4. 模拟退火算法 (SA)
 - Metropolis 准则、Boltzmann 分布
 - 温度参数控制探索与开发
 - 玻尔兹曼机
5. 遗传算法 (GA)
 - 编码、选择、交叉、变异
 - 轮盘赌、二进制编码
6. 粒子群优化 (PSO) 与差分进化 (DE)
 - PSO: 速度-位置更新, 个体/全局最优

- DE: 差分向量变异, 贪心选择

7. 其他群智能算法

- ABC: 雇佣蜂、观察蜂、侦察蜂
- ACO: 信息素机制
- GWO: 灰狼等级制
- WOA: 泡泡网捕食
- FWA: 烟花爆炸

8.10.2 智能优化算法特性对比

表 8.4: 智能优化算法综合对比				
算法	来源	跟随最优	关键参数	适合问题
GA	进化论	否	p_c, p_m	组合/连续
PSO	鸟群觅食	是	w, c_1, c_2	连续优化
DE	进化论	否	F, CR	连续优化
ABC	蜜蜂觅食	是	limit	连续优化
ACO	蚂蚁觅食	是	α, β, ρ	组合优化
SA	物理退火	–	T_0, α	通用
GWO	狼群	是	a	连续优化
WOA	鲸鱼	是	a, b	连续优化
FWA	烟花	是	振幅	连续优化

8.10.3 选择算法的建议

算法选择指南
• 连续单峰: 梯度法、Newton 法 (最快) • 连续多峰: DE、PSO、SA (全局搜索) • 组合优化: GA、ACO、SA • 高维问题: DE、PSO (可扩展性好) • 快速求解: GWO、PSO (参数少, 收敛快) • 鲁棒性要求: SA、DE (稳定, 不易陷入局部最优)

8.10.4 关键公式汇总

表 8.5: 核心公式速查

概念/算法	核心公式
PPA	$x_{k+1} = \arg \min_x \{\theta(x) + \frac{r}{2} \ x - x_k\ ^2\}$
变分不等式	$(u - u^*)^T F(u^*) \geq 0, \forall u \in \Omega$
Metropolis 准则	$P = \exp(-\Delta E/T)$
Boltzmann 分布	$P_A/P_B = \exp(-(E_A - E_B)/T)$
GA 选择概率	$p_i = f(x_i) / \sum_j f(x_j)$
PSO 速度	$v = wv + c_1 r_1 (pbest - x) + c_2 r_2 (gbest - x)$
DE 变异	$v = x_{r1} + F(x_{r2} - x_{r3})$
GWO 位置	$x = (x_\alpha + x_\beta + x_\delta)/3$

8.10.5 关键术语中英对照

```

1 import numpy as np
2
3 def unified_metaheuristic(f, dim, method='de', bounds=None,
4                             max_iter=100, **kwargs):
5     """
6     统一元启发式优化接口
7     """
8     if bounds is None:
9         bounds = [(-5, 5)] * dim
10
11     if method == 'de':
12         from scipy.optimize import differential_evolution
13         result = differential_evolution(f, bounds, maxiter=max_iter,
14                                         tol=1e-6, seed=42)
15         return result.x, result.fun
16
17     elif method == 'sa':
18         from scipy.optimize import dual_annealing
19         result = dual_annealing(f, bounds, maxiter=max_iter,
20                                 seed=42)
21         return result.x, result.fun
22

```

中文	英文
复合优化	Composite Optimization
变分不等式	Variational Inequality
邻近点算法	Proximal Point Algorithm
启发式算法	Heuristic Algorithm
近似算法	Approximation Algorithm
模拟退火	Simulated Annealing
遗传算法	Genetic Algorithm
粒子群优化	Particle Swarm Optimization
差分进化	Differential Evolution
人工蜂群	Artificial Bee Colony
蚁群优化	Ant Colony Optimization
灰狼算法	Grey Wolf Optimizer
鲸鱼算法	Whale Optimization Algorithm
烟花算法	Fireworks Algorithm
玻尔兹曼机	Boltzmann Machine
拟阵	Matroid
独立系统	Independent System

```

23     elif method == 'pso':
24         # 简化PSO
25         n = kwargs.get('n_particles', 30)
26         lb, ub = np.array([b[0] for b in bounds]), np.array([b[1] for
27             b in bounds])
28         x = np.random.uniform(lb, ub, (n, dim))
29         pbest, pbest_f = x.copy(), np.array([f(xi) for xi in x])
30         gbest_idx = np.argmin(pbest_f)
31         gbest = pbest[gbest_idx].copy()
32         v = np.zeros((n, dim))
33
34         for _ in range(max_iter):
35             for i in range(n):
36                 r1, r2 = np.random.rand(2)
37                 v[i] = (0.8 * v[i] + 1.5 * r1 * (pbest[i] - x[i])
38                     + 1.5 * r2 * (gbest - x[i]))
39                 x[i] = np.clip(x[i] + v[i], lb, ub)
40                 fi = f(x[i])
41                 if fi < pbest_f[i]:
42                     pbest_f[i], pbest[i] = fi, x[i].copy()
43                     if fi < f(gbest):
44                         gbest = x[i].copy()
45
46             return gbest, f(gbest)
47
48         else:
49             raise ValueError(f"Unknown method: {method}")
50
51 # === 测试 ===
52 print("=" * 60)
53 print("统一接口测试 (Rastrigin函数)")
54 print("=" * 60)
55
56 rastrigin = lambda x: 10 * len(x) + np.sum(x**2 - 10*np.cos(2*np.pi*x
57     ))
58
59 for method in ['de', 'sa', 'pso']:
60     x_opt, f_opt = unified_metaheuristic(rastrigin, dim=5,
61         method=method, max_iter
62         =200)

```

60

```
print(f"{method.upper():4s}: f* = {f_opt:.6f}")
```

Listing 8.7: 统一智能优化接口

Chapter 9

课程总结与展望

本章对组合优化与凸优化课程的八章核心知识进行系统梳理，建立完整的知识框架，提供方法选择的实用指南，并展望优化领域的前沿研究方向。

章节内容速查

本教材共计八章，其逻辑关系如下：第 1 章（绪论）奠定数学基础（凸集、凸函数、仿射函数）；第 2 章（线性规划）介绍经典的单纯形法与对偶理论；第 3 章（凸分析）深化凸性理论与最优性条件；第 4 章（一维搜索）提供多维优化中不可或缺的步长子程序；第 5 章（无约束优化）涵盖从梯度法到次梯度法的完整方法体系；第 6 章（约束优化理论）建立 KKT 条件与对偶理论的系统框架；第 7 章（约束优化方法）介绍罚函数法、内点法等实用算法；第 8 章（进阶专题）探讨智能优化算法与深度学习中的优化。

9.1 优化问题的统一框架

最优化问题的一般形式

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, l \end{aligned}$$

分类维度：

- 目标函数：线性 / 非线性 / 凸 / 非凸 / 光滑 / 非光滑
- 约束：无约束 / 等式约束 / 不等式约束 / 线性约束 / 非线性约束
- 变量：连续 / 离散 / 混合整数

9.2 核心知识体系

9.2.1 一维搜索与线搜索方法

表 9.1: 一维搜索与线搜索方法核心要点 (详见第 4 章)			
信息	方法	收敛速度	核心特征与选择依据
零阶	Dichotomous 搜索	线性 (≈ 0.5)	对称取点, 区间减半
	黄金分割法	线性 ($\tau = 0.618$)	每步只需一个新函数值
	Fibonacci 法	线性 (最优)	固定次数下最少试验点
导数	二分法 (导数)	线性 (0.5)	每次一个导数计算
	Newton 法	二次	需二阶导, 初始点要接近极小点
	插值法	超线性	不需二阶导, 实用性强
非精确	Armijo 回溯	保证下降	最常用, 仅需充分下降条件
	Wolfe 条件	保证下降	充分下降 + 曲率控制
	Goldstein 条件	保证下降	双边控制, 防止步长过小

核心原则: 多维优化算法中的步长搜索首选**非精确线搜索** (Armijo 或 Wolfe), 计算效率远高于精确线搜索。

9.2.2 凸性：优化的基石

凸集与凸函数

凸集: $\forall x, y \in \mathcal{X}, \theta \in [0, 1]$, 有 $\theta x + (1 - \theta)y \in \mathcal{X}$

凸函数: $f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$

核心结论: 凸优化问题的局部极小 = 全局极小

判断凸性的常用工具:

- 一阶条件: $f(y) \geq f(x) + \nabla f(x)^T(y - x)$
- 二阶条件: $\nabla^2 f(x) \succcurlyeq 0$ (半正定)
- 保凸运算: 非负加权和、仿射变换、逐点最大值、复合函数

9.2.3 最优性条件体系

表 9.2: 最优性条件对照表（详见第 6 章第 6.2 节和第 6.3 节）

问题类型	一阶条件	二阶必要条件	二阶充分条件
无约束	$\nabla f(x^*) = 0$	$\nabla^2 f(x^*) \succcurlyeq 0$	$\nabla^2 f(x^*) \succ 0$
等式约束	$\nabla f + \lambda^T \nabla h = 0$	$d^T \nabla_{xx}^2 L d \geq 0$	$d^T \nabla_{xx}^2 L d > 0$
一般约束	KKT 条件	临界锥上 $\nabla^2 L \succcurlyeq 0$	临界锥上 $\nabla^2 L \succ 0$

KKT 条件——约束优化的核心

对于一般约束优化问题，KKT 条件包含：

1. 平稳性： $\nabla f + \sum \mu_i \nabla g_i + \sum \lambda_j \nabla h_j = 0$

2. 原始可行性： $g_i(x) \leq 0, h_j(x) = 0$

3. 对偶可行性： $\mu_i \geq 0$

4. 互补松弛： $\mu_i g_i(x) = 0$

充分性：在凸问题 + Slater 条件下，KKT $\Leftrightarrow$ 全局最优。

9.2.4 线性规划：经典与对偶

表 9.3: 线性规划核心方法（详见第 2 章）

方法	要点
单纯形法	在极点 (BFS) 之间迭代；入基选最负检验数，出基用最小比值
大 M 法	人工变量系数 $-M$ ， M 足够大；适用于没有明显初始可行基
两阶段法	第一阶段最小化人工变量和，第二阶段求解原问题
对偶单纯形法	保持对偶可行性，恢复原始可行性；适用于灵敏度分析
内点法	多项式时间复杂度；原始-对偶路径跟踪

对偶理论核心：

- 弱对偶： $c^T x \leq b^T y$ （任意可行对）

- 强对偶：最优值相等（凸 + Slater 条件）
- 互补松弛： $y_i^*(b_i - a_i^T x^*) = 0$
- 影子价格：对偶变量表示约束右端项变化对目标的影响

9.2.5 无约束优化方法

表 9.4: 无约束优化方法对比（详见第 5 章）				
方法	信息需求	收敛速度	存储	适用场景
最速下降	梯度	Q-线性	$O(n)$	教学演示，条件数小
牛顿法	梯度 + Hessian	Q-二次	$O(n^2)$	小规模，Hessian 易求
共轭梯度	梯度	n 步终止 (二次)	$O(n)$	大规模稀疏问题
BFGS	梯度	Q-超线性	$O(n^2)$	中大规模光滑问题
SGD	随机梯度	$O(1/\sqrt{k})$	$O(n)$	机器学习大规模数据

关键公式：

- 牛顿步： $x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$
- CG 方向： $d_{k+1} = -g_{k+1} + \beta_k d_k$ ，其中 $\beta_k^{FR} = \frac{\|g_{k+1}\|^2}{\|g_k\|^2}$
- BFGS 更新： $B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$

9.2.6 约束优化方法

表 9.5: 约束优化方法对比（详见第 7 章）		
方法	关键参数	核心特征与适用场景
外点罚函数	$\sigma \rightarrow \infty$	不要求初始可行，但数值不稳定
内点法	$\mu \rightarrow 0$	多项式复杂度，要求严格内点
增广 Lagrange	有限 σ	乘子更新 $\lambda_{k+1} = \lambda_k + \sigma h(x_{k+1})$
可行方向法	无	每次迭代求解 LP，保证可行性
ADMM	$\rho > 0$	分解为子问题交替求解，适合分布式

9.2.7 非光滑与复合优化

近端梯度法

对于 $\min f(x) + g(x)$ (f 光滑, g 非光滑):
$$x^+ = \text{prox}_{\alpha g}(x - \alpha \nabla f(x))$$

其中近端算子: $\text{prox}_f(x) = \arg \min_y \{f(y) + \frac{1}{2}\|y - x\|^2\}$
ISTA: 收敛 $O(1/k)$ **FISTA**: 收敛 $O(1/k^2)$

9.2.8 智能优化算法

表 9.6: 智能优化算法概要 (详见第 8 章第 8.3 节至第 8.6 节)

算法	核心机制	特点
模拟退火 (SA)	Metropolis 准则	概率接受劣解, 逃离局部最优
遗传算法 (GA)	选择 + 交叉 + 变异	群体进化, 全局搜索能力强
粒子群 (PSO)	个体最优 + 全局最优引导	参数少, 收敛快, 易早熟
差分进化 (DE)	差分变异 + 贪心选择	连续优化高效, 参数鲁棒

9.3 收敛性与复杂度

表 9.7: 典型收敛率

问题类别	方法	收敛率
凸 +L-光滑	梯度下降	$O(1/k)$
凸 +L-光滑	Nesterov 加速	$O(1/k^2)$ (最优下界)
强凸 +L-光滑	梯度下降	$O((1 - \mu/L)^k)$ (线性)
强凸 +L-光滑	Nesterov 加速	$O((1 - 1/\sqrt{\kappa})^k)$
一般凸	次梯度法	$O(1/\sqrt{k})$
复合凸 (光滑 + 非光滑)	近端梯度	$O(1/k)$
复合凸	FISTA	$O(1/k^2)$

收敛速度定义:

- Q-线性: $\|e_{k+1}\| \leq c\|e_k\|$, $c \in (0, 1)$
- Q-超线性: $\|e_{k+1}\|/\|e_k\| \rightarrow 0$
- Q-二次: $\|e_{k+1}\| \leq c\|e_k\|^2$

9.4 方法选择决策树

方法选择指南

Step 1: 判断问题类型

- 目标函数和约束均为线性 → **线性规划**（单纯形法 / 内点法）
- 目标或约束含非线性 → 进入 Step 2

Step 2: 判断凸性

- 凸问题 → KKT 条件充分，可用高效算法
- 非凸问题 → 可能多局部极小，考虑全局/启发式方法

Step 3: 判断约束与光滑性

- 无约束 + 光滑 + 小规模 → **牛顿法**（二次收敛）
- 无约束 + 光滑 + 大规模 → **共轭梯度**或 **BFGS**
- 无约束 + 非光滑 → **近端梯度法** / **次梯度法**
- 约束 + 需保证可行性 → **可行方向法** / **内点法**
- 约束 + 不要求初始可行 → **罚函数法** / **增广拉格朗日**
- 大规模机器学习 → **SGD** / **Adam**
- 离散/组合/NP-hard → **启发式算法**（SA, GA, PSO, DE）

9.5 课程核心公式速查

表 9.8: 核心公式速查表	
名称	公式
Taylor 展开	$f(x + d) = f(x) + \nabla f(x)^T d + \frac{1}{2} d^T \nabla^2 f(x) d + o(\ d\ ^2)$
Armijo 条件	$f(x_k + \alpha d_k) \leq f(x_k) + c_1 \alpha \nabla f_k^T d_k$
Wolfe 条件	$f(x_k + \alpha d_k) \leq f(x_k) + c_1 \alpha \nabla f_k^T d_k$ 且 $\phi'(\alpha) \geq c_2 \phi'(0)$
强凸性	$f(y) \geq f(x) + \nabla f(x)^T (y - x) + \frac{\mu}{2} \ y - x\ ^2$
L-光滑	$\ \nabla f(x) - \nabla f(y)\ \leq L \ x - y\ $
弱对偶	$p^* - d^* \geq 0$ (原问题最优 - 对偶问题最优)
强对偶	$p^* = d^*$ (凸问题 + Slater 条件)
KKT 条件	$\nabla f + \sum \mu_i \nabla g_i + \sum \lambda_j \nabla h_j = 0, \quad g_i(x) \leq 0, \quad h_j(x) = 0, \\ \mu_i \geq 0, \quad \mu_i g_i(x) = 0$
近端算子	$\text{prox}_f(x) = \arg \min_y \{f(y) + \frac{1}{2} \ y - x\ ^2\}$
ADMM	$x^+ = \arg \min_x L_\rho(x, z, y); \quad z^+ = \arg \min_z L_\rho(x^+, z, y); \\ y^+ = y + \rho(Ax^+ + Bz^+ - c)$
黄金分割	收缩比 $\tau = (\sqrt{5} - 1)/2 \approx 0.618$

9.6 本章小结

9.6.1 核心知识框架

本课程共八章，形成完整的优化理论与方法体系：

表 9.9: 课程知识体系概览		
章	主题	核心内容
1	绪论	运筹学概述，凸集/凸函数/仿射函数
2	线性规划	单纯形法，对偶理论，灵敏度分析
3	凸分析	Taylor 展开，最优性条件，凸规划
4	一维搜索	精确/非精确线搜索，步长选择准则
5	无约束优化	梯度法，Newton 法，CG，BFGS，次梯度
6	约束优化理论	KKT 条件，对偶理论，Farkas 引理
7	约束优化方法	罚函数法，内点法，增广 Lagrange 法
8	进阶专题	SA，GA，PSO，DE，深度学习优化

9.6.2 方法选择总诀

面对实际优化问题的选择流程：**先判断问题类型**（线性/非线性 → 凸/非凸 → 光滑/非光滑 → 有无约束）→ **再匹配信息能力**（能算几阶导数）→ **最后根据规模选算法**（小/中/大规模）。

9.7 研究前沿与展望

优化领域正在多个方向快速发展，以下列出与本课程内容密切相关的几个前沿方向：

前沿研究方向
1. 大规模分布式优化：随着数据量和模型规模的爆发式增长，ADMM、联邦优化（Federated Optimization）等方法在分布式机器学习中得到广泛应用，核心挑战是通信效率与收敛速度的平衡。 2. 非凸优化理论突破：深度学习中的优化问题本质是非凸的，近期研究在过参数化（Over-parameterization）、隐式正则化（Implicit Regularization）等方面取得重要进展，解释了为什么梯度法能在高维非凸 landscape 中找到好的解。 3. 自动微分与可微编程：PyTorch、JAX 等框架实现了高效的自动微分，使得任意复杂模型的梯度计算变得简单，推动了”优化即编程”的新范式。 4. 随机优化的方差缩减技术：从 SGD 到 SVRG、SAGA，再到近年来结合了动量和方差缩减的先进方法，大幅度提升了大规模机器学习中的收敛效率。 5. 混合整数规划与组合优化：结合深度学习与传统优化方法（如 Learning to Optimize），利用神经网络预测分支策略或初始解，正在革新运筹学中 NP-hard 问题的求解方式。 6. 量子优化算法：量子退火（Quantum Annealing）和变分量子本征求解器（VQE）为组合优化问题提供了全新的计算范式，虽然仍处于早期阶段，但已展现出在某些问题上的潜力。
进一步学习建议
• 理论基础：Boyd & Vandenberghe 《Convex Optimization》（经典教材，免费在线） • 算法实现：熟练运用 scipy.optimize、CVXPY、JuMP 等优化建模工具 • 前沿追踪：关注 NeurIPS、ICML、COLT 等顶级会议中的优化相关论文

- 实践结合：在深度学习、信号处理、运筹学等应用中尝试不同优化方法的对比实验